کتاب و جزوه آموزش پایتون

منطبق بر دوره مسترکدر پایتون با تدریس دکتر بابک نوری مقدم



جمع آوری کننده : علیرضا احمدی

نویسنده : دکتر بابک نوری مقدم

بستر انتشار : وبسایت دانشجویار

با تشکر از یاسین مریدی

پایتون چیست؟

پایتون یک زبان برنامه نویسی سطح بالا، شیء گرا و همه منظوره است که توسط خیدو فان روسوم طراحی شده است. این زبان در سال ۱۹۹۱ منتشر شد و به دلیل خوانایی بالا، سادگی و انعطاف پذیری، به یکی از محبوب ترین زبان های برنامه نویسی در جهان تبدیل شده است.

برخی از ویژگی های کلیدی پایتون عبارتند از:

- خوانایی بالا: کد پایتون به زبان انگلیسی ساده شباهت دارد و به همین دلیل یادگیری و خواندن آن آسان است.
- سادگی: پایتون از نحو ساده ای استفاده می کند و به همین دلیل برای مبتدیان مناسب است.
- انعطاف پذیری: پایتون از طیف گسترده ای از کتابخانه ها و ابزارها پشتیبانی می کند که می توان از آنها برای انجام انواع مختلف وظایف استفاده کرد.
- قدرتمندی: پایتون یک زبان قدرتمند است که می توان از آن برای توسعه برنامه های پیچیده استفاده کرد.

کاربردهای پایتون

- توسعه وب: پایتون برای توسعه وب سایت ها و برنامه های وب قدرتمند مانند Django و Flaskاستفاده می شود.
- علم داده: پایتون به دلیل کتابخانه های قدرتمندی مانندNumPy ، Pandasو SciPy برای علم داده و یادگیری ماشین بسیار محبوب است.
- هوش مصنوعی: پایتون برای توسعه برنامه های هوش مصنوعی و یادگیری عمیق مانند TensorFlowو PyTorch استفاده می شود.
- اتوماسیون: پایتون برای خودکارسازی وظایف مختلف مانند پردازش اسناد، تست نرم افزار و مدیریت سیستم استفاده می شود.
- آموزش: پایتون به دلیل سادگی و خوانایی بالا، برای آموزش برنامه نویسی به مبتدیان بسیار مناسب است.

پایتون یک زبان برنامه نویسی قدرتمند، انعطاف پذیر و همه منظوره است که می توان از آن برای انجام انواع مختلف وظایف استفاده کرد. این زبان به دلیل خوانایی بالا، سادگی و یادگیری آسان، به یکی از محبوب ترین زبان های برنامه نویسی در جهان تبدیل شده است.

اعداد در پایتون

در این جلسه درباره اعداد در پایتون و نحوه استفاده از آنها خواهیم آموخت

در اینجا در مورد موضوعات زیر آموزش خواهیم دید:

1.انواع اعداد در پایتون

2.حساب اعمال ابتدایی

3.تفاوت بین تقسیم کلاسیک و تقسیم به طور قسمتی

4.اختصاص شی در پایتون

انواع اعداد

پایتون دارای انواع مختلفی از اعداد (حروف عددی) است. اصلی ترین نوعها اعداد صحیح و اعداد اعشاری (اعداد ممیز) شناور هستند.

اعداد صحیح شامل اعداد مثبت و منفی غیر اعشاری مثل ۲ و ۲- میشود.

اعداد اعشاری در پایتون به دلیل داشتن علامت اعشاری با استفاده از نمایش عددی علمی (e) قابل توجه هستند. به عنوان مثال ۲,۰ و ۲,۱ نمونه هایی از اعداد اعشاری هستند ،همچنین E2٤یا همان (٤ضربدر ۱۰ به توان ۲)نیز نمونه ای از عدد اعشاری در پایتون است. در طول این دوره اصلی ترین کار ما با اعداد صحیح و اعداد اعشاری ساده خواهد بود.

در زیر جدولی از دو نوع اصلی که بیشترین زمان خود را برای کار با آنها صرف خواهیم کرد همراه با برخی مثالها آمده است:

نوع عددی	مثال
integers	1,-5,1000
Floating-point numbers	1.2,2E3

In[2] :

```
1 2E6
```

Out[2] :

۲۰۰۰۰۰۰,۰

In[3] :

```
1 ۰,۰۰۰۰۰۰٦
```

Out[3]:

6e-07

در ادامه یکسری مثالهای مربوط به محاسبات ساده ریاضی را با هم مشاهده کنیم.

ریاضیات پایه

In [4]:

```
1 # Addition
2 2+۱
```

Out[4]:

۳

In[5]:

```
1 # Subtraction
2 5 - 7
```

Out[5]:

-2

In[6]:

```
1 # Multiplication
2 5 * ٥
```

Out[6]:

25

In[7]:

```
1 # Division
2 14/2
```

Out[7]:

7.0

In[9]:

```
1 # Floor Division
2 15// 2
```

Out[9]:

7

عجب چه اتفاقی افتاد؟ آخرین بار که چک ،کردم تقسیم۱۵ بر ۲ برابر ۷,۵ بود نه !!

دلیل این نتیجه این است که ما از تقسیم "بدون تقریب" استفاده میکنیم عامل // (دو خط کشیده به سمت جلو) اعشاری را بدون تقریب قطع میکند و نتیجهی یک عدد صحیح را برمیگرداند.

خب اگه فقط باقیمانده تقسیم رو بخواهیم چی؟

In[11]:

```
1 # Modulo
2 7 % 5
```

Out[11]:

2

In[12]:

```
1 5%13
```

Out[12]:

5

۴ یکبار در ۷ وجود دارد بنابراین باقیمانده معادل3 خواهد بود بنابراین عملگر % باقیمانده را پس از تقسیم برمیگرداند

ادامه اعمال ریاضی

In[13]:

```
۱ # Powers
۲ ۲ ** ۵
```

Out[13]:

۲۵

In[14]:

```
1 5**7
```

Out[14]:

۷۸۱۲۵

In[15]:

```
1 # Can also do roots this way
2 ۲۵ **۰/۵
```

Out[۱۵]:

۵/۰

In[19]:

```
1 ۲۷ **(۳/۱)
```

Out[15]:

۳/۰

اولویت عملگرها

Python Operator Precedence

Precedence	Operator Sign	Operator Name
Highest	**	Exponentiation
↓	+x,-x,~x	Unary positive, unary negative, bitwise negation
↓	%,//,/,*	Multiplication, division, floor, division, modulus
↓	-,+	Addition, subtraction
↓	<<,>>	Left-shift, right-shift
↓	&	Bitwise AND
↓	^	Bitwise XOR
↓	|	Bitwise OR
↓	is,is not,=<,<,=>,>,=!,==	Comparison, Identity
↓	not	Boolean NOT
↓	and	Boolean AND
Lowest	or	Boolean OR

In[20]:

```
1 #Order of Operations followed in Python
2  2 +10* 10+ 3
```

out[20] :

۱۰۵

In [21]:

```
1 #Can use parentheses to specify orders
2 2+ (10*10) + 3
```

out[21] :

۱۰۵

In [22] :

```
1 (۱۰+۲)* 10 +3
```

Out [22] :

۱۲۳

In [23]:

```
1 (۳ +۱۰) *(۱۰+۲)
```

out[23] :

۱۵۶

In [24] :

```
1 5*12/6*2
```

out[24] :

۲۰/۰

انتساب به متغییرها

حالا که دیدیم چگونه میتوانیم از اعداد به عنوان یک ماشین حساب در پایتون استفاده کنیم بیایید ببینیم چگونه میتوانیم نام ها را به متغیرها اختصاص دهیم و متغیرها ایجاد کنیم.

ما از یک علامت تک مساوی برای اختصاص برچسب به متغیرها استفاده میکنیم چند نمونه از این کار را بیایید ببینیم.

In [26] :

```
1 #Let's create an object called "a" and assign it the number
2 a = 5
```

In [27] :

```
1 a
```

Out [27]:

۵

In [28] :

```
1 id(a)
```

Out[28] :

۱۹۳۰۵۸۵۵۷۱۶۹۶

لطفا توجه کنید که در صورتی که a را در اسکریپت پایتون خود فراخوانی نید پایتون آن را به عنوان عدد 5 در نظر خواهد گرفت

In [29] :

```
۱ # Adding the objects
۲ a ** a
```

Out [29] :

۳۱۲۵

در صورت انتساب مجدد متغیر چه اتفاقی میافتد؟ آیا پایتون به ما اجازه میدهد آن را دوباره نوشته شود؟

In [30]:

```
۱ # Reassignment
۲ a = 15
```

In [31]:

```
1 # Check
```

```
2 a
```

out[31]:

۱۵

بله پایتون به شما اجازه میدهد مقادیر متغیر را مجدداً با مقدار جدید بازنویسی کنید همچنین میتوانید همان متغیرها را در هنگام انجام بازنویسی استفاده کنید در ادامه یک مثال برای توضیح این موضوع آورده شده است:

In [32]:

```
۱ # Check
۲ a
```

out[32] :

15

In [33]:

```
۱ # Use A to redefine A
۲ a = a + a
```

In [34] :

```
۱ # Check
۲ a
```

out[34] :

30

نامهایی که هنگام ایجاد این برچسبها استفاده میکنید باید اصول خاصی را رعایت کنند :

۱.نامها نمیتوانند با یک عدد شروع شوند

۲.نمیتوانید در نامها از فاصله استفاده کنید؛ به جای آن از خط تیره (_) استفاده کنید

۳.نمیتوانید از هیچ یک از علائم زیر استفاده کنید :

<>/?|()!@#$%^&*~+

۴.توصیه میشود(براساس PEP8) نام ها با حروف کوچک نوشته شوند.

۵. از استفاده از کاراکترهای "l"(حرف کوچک ال)،"O"(حرف بزرگ او) و یا "I"(حرف بزرگ آی) به عنوان نام متغیر با یک حرف اجتناب کنید.

۶. از استفاده از کلماتی که در پایتون معنای خاصی دارند مانند "list"و "str" خودداری کنید.

در برنامه نویسی روشهای متنوعی برای نام گذاری متغییرهای چند کلمه ای مرسوم می باشد :

۱.روش پاسکال

۲. روش camel case

۳. روش(python)==>snake case

۴.روش kebab case

In [ ]:

```
1  MyFamilyHouse        #Pascal Case
2
3  myFamilyHouse        #Camel Case
4
5  my_family_house      #Snake Case
6
7  my-family-hous       #Kebab Case ==> invalid in python
```

In [36]:

```
1  a1 = 15
```

پایتون به بزرگ و کوچک بودن حروف حساس می باشد

In [37]:

```
1  chair = 4
```

In [38]:

```
1  Chair = 25
```

In [39]:

```
1 CHAIR = -50
```

In [42] :

```
1 CHAIR
```

out[42] :

-۵۸

استفاده از نامهای متغیر میتواند روشی بسیار مفید برای پیگیری متغیرهای مختلف در پایتون باشد. به عنوان مثال:

In [43] :

```
1 #Use object names to keep better track of what's going on in your code!
2 my_incone = 100
3
4 tax_rate = 0.1
5
6 my_taxes = my_incone * tax_rate
```

In [44] :

```
1 #Show my taxes!
2  my_taxes
```

Out[44] :

۱۰,۰

پویایی نوع داده ها (Dynamic Typing)

پایتون از پویایی نوع داده ها استفاده میکند به این معنی که شما میتوانید متغیرها را به نوع داده های مختلف دوباره اختصاص دهید. این ویژگی باعث میشود پایتون در اختصاص داده های نوع بسیار انعطاف پذیر باشد؛ این در مقایسه با زبانهای دیگری که نوعهای استاتیک دارند تفاوت دارد.

In [45] :

```
1 my_age = 15
```

In [46] :

```
1 my_age
```

Out[46] :

۱۵

In [48] :

```
1 my_age = [1,2,3]
```

In [49] :

```
1 my_age
```

Out[49] :

[1,2,3]

مزایا و معایب پویایی نوع داده ها(Dynamic Typing)

مزایا پویایی نوع داده ها

۱. مزایا پویایی نوع دادهها
۲. زمان توسعه سریعتر

معایب پویایی نوع داده ها

۱. ممکن است باعث بروز خطاهای غیر منتظره شود
۲. باید از TYPE() آگاه باشید
۳. کندی احتمالی اجرا نسبت به زبانهای strict

چک کردن نوع داده با استفاده از تابع type()

میتوانید با استفاده از تابع داخلی type()در پایتون نوع شی که به یک متغیر اختصاص داده شده است را بررسی کنید. نوع های داده های متداول عبارتند از:

- int (for integer)
- float
- str (for string)
- list
- tuple
- dict (for dictionary)
- set
- bool (for Boolean True/False)

In [50] :

```
۱ type(my_age)
```

Out[50]:

List

In [51] :

```
1 a=(۱’۲)
```

In [52] :

```
1 type(a)
```

out[52] :

tuple

در نتیجه چه چیزهایی یاد گرفتیم؟ ما به برخی از مبانی اعداد در پایتون پرداختیم. همچنین یاد گرفتیم چگونه عملیات حسابی را انجام داده و از پایتون به عنوان یک ماشین حساب ساده استفاده کنیم ،همچنین با مفهوم اختصاص متغیر در پایتون آشنا شدیم /

در ادامه به مبحث رشته ها (Strings) خواهیم پرداخت!

رشته ها

رشته ها در پایتون برای ثبت اطلاعات متنی مانند نامها استفاده میشوند رشته ها در پایتون در واقع یک دنباله هستند که به اصطلاح به این معناست که پایتون هر عنصر را در رشته به عنوان یک دنباله در نظر میگیرد به عنوان مثال، پایتون رشته "hello" را به عنوان یک دنباله از حروف با ترتیب خاص درک میکند این بدان معناست که ما قادر خواهیم بود با استفاده از ایندکس گذاری حروف خاصی را دریافت کنیم (مانند اولین حرف یا آخرین حرف)

این ایده از دنباله یک مفهوم مهم در پایتون است و ما در آینده به آن خواهیم پرداخت.

در این درس درباره موارد زیرخواهیم آموخت :

۱ .ایجاد رشته ها

۲ چاپ رشته ها

۳ ایندکس گذاری و برش رشته ها

4.ویژگی های رشته ها

۵ متدهای رشته ها

۶ قالب بندی چاپ

ایجاد رشته

برای تعریف یا ایجاد رشته ها در پایتون باید متن مورد نظر خود را داخل دابل کوتیشن قرار بدهید

In [15]:

```
# Single word
'Hello'
```

out[15] :

'Hello'

In [16]:

```
type("Hello")
```

out[16] :

str

In [17]:

```
# Entire phrase
'This is also a string!'
```

Out[17]:

'This is also a string!'

In [18]:

```
#We can also use double quote
"String built with double qoutes"
```

out[18] :

'String built with double qoutes'

In [19]:

```
# Be careful with quotes!
'I'm using single quotes, but this will create an error
```

Cell In[19], line 2

'I'm using single quotes, but this will create an error

SyntaxError: unterminated string literal (detected at line 2)

دلیل بروز خطا در بالا به این دلیل است که نقطه تکی کوتیشن در I'm رشته را متوقف .کرد شما میتوانید ترکیبی از نقطه دوتایی و نقطه تکی را برای به دست آوردن عبارت کامل استفاده کنید.

In [20]:

```
1 "I'm using single quotes, but this will create an error"
```

out[20] :

"I'm using single quotes, but this will create an error"

In [21]:

```
1 'I\'m using single quotes, but this will create an error'
```

Out[21] :

" I'm using single quotes, but this will create an error"

In [23]:

```
1 print("I\tm using single quotes, but this will create an error")
```

I m using single quotes, but this will create an error

حالا در ادامه درباره نحوه چاپ کردن (print) یاد بگیریم

چاپ یا print کردن رشته ها

با استفاده از دفترچه Jupyter با فقط یک رشته در یک سلول به طور خودکار رشته ها را در خروجی نمایش میدهد اما روش صحیح برای نمایش رشته ها در خروجی شما استفاده از تابع چاپ (print)است.

In [24]:

```
1 # We can simply declare a string
2 'Hello world!'
```

Out [24]:

'Hello world!'

In [25]:

```
1 # Note that we can't output multiple strings this way
2 'Hello world! 1'
3 'Hello world! 2'
```

Out[25]:

'Hello world! 2'

ما از دستور print میتوان چاپ چندین رشته استفاده نماییم.

In [28]:

```
1 print('Hello world! 1')
2 2 print('Hello world! 2')
3
4 4 print('Use \n to print a new line')
5 5 print('\n')
6 6 print('See what i mean?')
```

Hello world! 1

Hello world! 2

Use

to print a new line

See what i mean?

In [30]:

```
1 print('Hello', 'World', 555)
```

Hello World 555

In [32]:

```
1 a = 5
2 2
3 A = a**/
4 4 2
```

مفاهیم پایه رشته ها

استفاده از تابع len()برای بدست آوردن تعداد کاراکترهای (طول) رشته

In [33] :

```
1 len('Hello world!')
```

Out[33] :

۱۲

تابع len()یکی از توابع داخلی پایتون میباشد که تمامی کاراکترها را در رشته شمارش میکند شامل فاصله ها و علائم نگارشی.

ایندکس گذاری رشته ها

ما میدانیم رشته ها یک دنباله هستند به این معنا که پایتون میتواند از ایندکسها برای فراخوانی بخشهای مختلف دنباله استفاده کند. بیایید ببینیم این عملیات چگونه کار میکند.

در پایتون ما از براکتها [ ] پس از یک شیء استفاده میکنیم تا ایندکس آن را فراخوانی .کنیم همچنین باید توجه کنیم که در پایتون شماره گذاری از ۰ شروع میشود بیایید یک شی جدید به نام s ایجاد کنیم و سپس از چند مثال از ایندکس گذاری استفاده کنیم.

In [34] :

```
1 # Assign s as a string
2 s = "Hello world!"
```

In [35] :

```
1 #Check
2 s
```

out[35] :

'Hello world!'

In [36] :

```
1 # Print the object
2 print(s)
```

out[36] :

Hello world!

In [37] :

```
1 s = 'foobar'
```

In [ ] :

```
1
```

حالا بیایید از ایندکس گذاری در رشته ها استفاده کنیم!

-6 -5 -4 -3 -2 -1

f	0	0	b	a	r

In [38] :

```
1 # Show first element (in this case a letter)
2 s[0]
```

Out [38]:

'f'

In [39]:

```
1 s[2]
```

Out [39]:

'o'

In [40]:

```
1 s[5]
```

Out [40]:

'r'

In [41]:

```
1 s[5]
```

IndexError

Traceback (most recent call last)

Cell In[41], line 1

--->1 s[10]

IndexError: string index out of range

ما میتوانیم از علامت : برای انجام برش (slicing) استفاده کنیم که همه چیز را تا یک نقطه مشخص انتخاب کنیم به عنوان مثال:

In [52]:

```
1 s[0]
```

Out [52]:

'f'

In [53]:

```
1 # Let's try to change the first letter to 'x'
2  s[0] = 'X'
```

TypeError

Traceback (most recent call last)

Cell In[53], line 2

```
1    #Let's try to change the first letter to 'x'
----> 2 s[0] = 'X'
```

TypeError: 'str' object does not support item assignment

In [54]:

```
1 s = 'Hello'
```

In [55]:

```
1 s
```

Out[55]:

'Hello'

توجه کنید که خطای به ما میگوید که چه کاری نمیتوانیم انجام دهیم، تغییر دادن عنصر!

اما چیزی که میتوانیم انجام دهیم، اتصال (concatenate)رشته هاست!

In [56]:

```
1 S = "hello world!"
```

In [57]:

```
1 # Concatenate strings!
2 s+”Concatenate Me!!"
```

Out[57]:

'hello world! Concatenate Me!!! '

In [58]:

```
1 S + 5
```

TypeError

Traceback (most recent call last)

Cell In[58], line 1

-----> 1 s + 5

TypeError: can only concatenate str (not "int") to str

In [59] :

```
1 # We can reassign s completely though!
2 s
```

Out[59]:

'hello world!'

In [60] :

```
1 s= s + "Concatenate Me!!"
```

In [61] :

```
1 print(s)
```

hello world! Concatenate Me!!

In [62] :

```
1 letter = 'z'
```

In [66] :

```
1 letter * 100
```

Out[66] :

'zzzz

Out[67]:

'H'

In [69]:

```
1 "Hello World!" [0::2]
```

Out [69]:

'Hlowrd'

In [72]:

```
1 # Upper Case a string
2 x = s. upper()
3 x
```

Out [72]:

'HELLO WORLD!'

In [71]:

```
1 s
```

Out [71]:

'Hello World!'

In [73]:

```
1 # Lower case
2 s.lower()
```

Out [73]:

'hello world!'

In [74]:

```
1 s.capitalize()
```

Out [74]:

'Hello world!'

In [76]:

```
1 s
```

Out [76]:

'Hello World!'

In [80]:

```
1 s.count('11")
```

Out [80]:

١

In [81]:

```
1 s.replace('1', '3')
```

Out [81]:

'He33o Wor3d!'

In [82]:

```
1 s
```

Out [82]:

'Hello World!'

In [83]:

```
1 s1 = '          string number one          '
```

In [84]:

```
1 s1.strip()
```

Out [84]:

'string number one'

In [85]:

```
1 s1.1strip()
```

Out [85]:

' string number one '

In [86]:

```
1 s1.rstrip()
```

Out[86]:

'string number one'

In [88]:

```
1 s
```

Out[88]:

'Hello World!'

In [87]:

```
1 # Split a string by blank space (this is the default)
2 s.split()
```

Out[87]:

['Hello', 'World!']

In [90]:

```
# Split by a specific element (doesn't include the element that was split on)
s.split('11')
```

Out[90]:

['He', 'o World!']

قالب بندی رشته ها(string fromat)

رشته ها را میتوانید با استفاده از فرمت بندی ،رشته اقدام به قرار دادن موارد درون رشته کنید به جای اینکه سعی کنید با استفاده از کاماها یا اتصال رشته ها آیتمها را به هم پیوند دهید برای مقایسه سریع در نظر بگیرید

player = 'Thomas'

points = 33

'Last night, 'player+' scored '+str(points)+' points. ' #اتصال

f'Last night, {player} scored {points} points.' #فرمت بندی رشته

سه روش برای انجام فرمت بندی رشته وجود دارد.

- قدیمی ترین روش شامل استفاده از جایگزینها با استفاده از علامت مدولو % است.
- یک تکنیک بهبود یافته از روش format(). رشته استفاده میکند.
- جدیدترین روش که با پایتون ۳.۶ معرفی شده است از رشته های قالب بندی شده با استفاده از f-strings استفاده میکند.

از آنجا که احتمالاً با سه نسخه در کد شخص دیگری روبه رو میشوید هر کدام را در اینجا توضیح میدهیم

قالب بندی با استفاده از جا نگه دارها(Placeholders)

یتوانید از %S برای درج رشته ها در دستورات چاپ خود استفاده کنید علامت مدولو به عنوان "عملگر فرمت بندی رشته" شناخته میشود.

In [1]:

```
1 print("I'm going to inject %s here."%' something')
```

I'm going to inject something here.

میتوانید چندین آیتم را با قرار دادن آنها درون یک تاپل پس از عملگر % ارسال کنید

In [۲]:

```
1 print("I'm going to inject %s text here, and %s text here." %('some', 'more'))
```

I'm going to inject some text here, and more text here.

همچنین میتوانید متغییرها را نیز استفاده نمایید.

In [1]:

```
1 x, y = 'some', 'more'
۲ print("I'm going to inject %s text here, and %s text here. "%(x,y))
```

I'm going to inject some text here, and more text here.

روشهای تبدیل قالب

باید توجه شود که دو روش %s و %r با استفاده از دو متد جداگانه str() و repr() هر شی پایتونی را به یک رشته تبدیل میکنند ما در دوره ی بعدی بیشتر درباره این توابع یاد خواهیم گرفت اما باید توجه داشت که %r و repr() نمایش رشته مربوط به شیء را ارائه میدهند که شامل نقل قول و هر کاراکتر خروجی است.

In [4]:

```
1 print('He said his name was %s.' %' Fred')
۲ print('He said his name was %r.' %'Fred')
```

He said his name was Fred.

He said his name was 'Fred'.

به عنوان مثال دیگر \t یک تب در یک رشته وارد میکند.

In [5]:

```
1 print('I once caught a fish %s.' %'this \tbig')
۲ print('I once caught a fish %r. %'this \tbig')
```

I once caught a fish this	big.

I once caught a fish 'this \tbig'.

عامل %s هر آنچه را که میبیند به رشته تبدیل میکند از جمله اعداد صحیح و اعشاری، عامل %d ابتدا اعداد را به صورت اعداد صحیح تبدیل میکند بدون گرد کردن تفاوت را در زیر مشاهده کنید.

In [6]:

```
1 print('I wrote %s programs today. %3.75)
2 print('I wrote %d programs today.' %3.75)
```

I wrote 3.75 programs today.

I wrote 3 programs today.

فاصله گذاری و مشخص کردن دقت در اعداد اعشاری

اعداد اعشاری از فرمت5.2f % استفاده میکنند در اینجا عدد ۵ حداقل تعداد کاراکترهایی است که رشته باید شامل شود؛ این اعداد میتوانند با فضای سفید پر شوند اگر کل عدد به اندازه این تعداد رقم نداشته باشد. در کنار این ۲f . برای نشان دادن چند رقم اعشاری پس از نقطه اعشاری استفاده میشود بیایید چند مثال ببینیم

In [7]:

```
1 print('Floating point numbers: %5.2f %(13.144))
```

Floating point numbers: 13.14

In [8]:

```
1 print('Floating point numbers: %1.0f' %(13.144))
```

Floating point numbers: 13

In [9]:

```
1 print('Floating point numbers: %1.5f' % (13.144))
```

Floating point numbers: 13.14400

In [10] :

```
1 print('Floating point numbers: %10.2f' %(13.144))
```

Floating point numbers: 13.14

In [11] :

```
1 print('Floating point numbers: %25.2f' %(13.144))
```

Floating point numbers: 13.14

برای کسب اطلاعات بیشتر در مورد قالب بندی رشته با استفاده از جایگزینها به لینک زیر مراجعه کنید:

https://docs.python.org/3/library/stdtypes.html#old-string-formatting

(https://docs.python.org/3/library/stdtypes.html#old-string-formatting)

In [12] :

```
1 print('First: %s, Second: %5.2f, Third: %r' ('hi! ',3.1415, 'bye! ' ))
```

First: hi!, Second: 3.14, Third: 'bye!'

قالب بندی رشته ها با متد .format()

یک روش بهتر برای قالب بندی اشیاء در رشته هایتان برای دستورهای چاپ استفاده از روش format() . است. سینتکس آن به شرح زیر است:

'String here {} then also {}'.format('something1','something')

In [13] :

```
1 print('This is a string with an {}'.format('insert'))
```

This is a string with an insert

روش format() چندین مزیت نسبت به روش placeholders دارد:

۱. ترتیب بندی: با استفاده از روش format(). میتوانید ترتیب جایگشت placeholderها را به صورت صریح با استفاده از اندیسها یا نامها مشخص کنید. این امکان را به شما میدهد تا به طور انعطاف پذیرتری آرگومان ها را درون رشته ترتیب دهید.

مقادیر اشیا ورودی را میتوانیم با ایندکس گذاری با ترتیبهای متفاوتی در رشته قرار دهیم

In [14] :

```
1 print('The {2} {1} {0}'.format('fox', 'brown', 'quick' ))
```

The quick brown fox

مقادیر اشیا ورودی را میتوانیم با کلمات کلیدی خاص استفاده نماییم

In [15] :

```
1 ('First Object: {a}, Second Object: {b}, Third Object: {c}'.format(a=1,b='Two', c=12.3))
```

First Object: 1, Second Object: Two, Third Object: 12.3

مقادیر اشیا ورودی را میتوانیم چندین بار مورد استفاده قرار دهیم

In [16] :

```
1 print('A %s saved is a %s earned.' %('penny', 'penny'))
2  # vs.
3 print('A {p} saved is a {p} earned.'.format(p='penny'))
```

A penny saved is a penny earned.

A penny saved is a penny earned.

جهت گذاری ، فاصله گذاری و نمایش دقت با متد forma().

در داخل آکولادهای میتوانید طول فیلد ترازبندی چپ راست پارامترهای گرد کردن و سایر موارد را تعیین کنید.

In [17] :

```
1 print('{0:8} | {1:9}'.format('Fruit', 'Quantity' ))
2 2 print('(0:8) | (1:9)".format("Apples", 3.))
3 3 print('(0:8) | (1:9}'.format('Oranges', 10))
```

Fruit | Quantity

Apples | 3.0

Oranges | 10

به طور پیش فرض format(). متن را به چپ و اعداد را به راست تراز می.کند میتوانید از < ، ^ یا > به عنوان اختیاری استفاده کنید تا ترازبندی ،چپ مرکز یا راست را تعیین کنید.

In [18]:

```
1 print('{0:<8} | {1:^8} | {2:8}'.format('Left', 'Center', 'Right'))
2 print('{0:<8) | {1:^8) | (2:>8}'.format(11,22,33))
```

Left 11 | Center | Right

11 | 22 | 33

میتوانید قبل از عملگر ترازبندی یک کاراکتر پر کننده قرار دهید.

In [19] :

```
1 print('{0:=<8) | {1:-^8) | {2:.>8}'.format('Left', 'Center', 'Right'))
2 print('(0:-<8} | (1:-^8) | (2:28. format(11,22,33))
```

Left==== | -Center- | ... Right

11======|---22--- | 33

عرض فیلدها و دقت اعداد اعشاری به یک روش مشابه با placeholderها مدیریت میشوند دو دستور چاپ زیر معادل هم هستند:

In [20]:

```
1 print('This is my ten-character, two-decimal number:%10.2f' %13.579)
2 print('This is my ten-character, two-decimal number: (0:10.2f}'.format(13.579))
```

This is my ten-character, two-decimal number: 13.58

This is my ten-character, two-decimal number: 13.58

توجه کنید که پس از علامت دو نقطه ۵ فاصله وجود دارد و عدد ۱۳.۵۸ پنج کاراکتر را به خود اختصاص داده است بنابراین مجموعاً ده کاراکتر تشکیل میدهد.

برای کسب اطلاعات بیشتر در مورد روش format()، رشته به لینک زیر مراجعه کنید:

https://docs.python.org/3/library/string.html#formatstrings

(https://docs.python.org/3/library/string.html#formatstrings)

قالب بندی رشته ها با Literals(f-strings)

در Python ۳٫۶ معرفی شدهاند f-strings چندین مزیت نسبت به روش قدیمی format(). رشته که در بالا توضیح داده شده است .دارند یکی از مزایای آنها این است که میتوانید متغیرهای بیرونی را به طور مستقیم درون رشته قرار دهید بدون اینکه آنها را به عنوان آرگومان ها از طریق format(var) ارسال کنید.

In [2]:

```
1 name = 'Fred'
2
3 print (f"He said his name is {name}.")
```

He said his name is Fred.

برای دریافت نمایش رشتهای یک شیء میتوانید را درون f-string استفاده کنید:

In [22]:

```
1 print(f"He said his name is {name!r}")
```

He said his name is 'Fred'

قالب بندی اعداد اعشاری "result:{value:{width}.{precision}}"

در روش format(). شما ممکن است بنویسید{ value: ۱۰/۴۴ }، اما با استفاده از f-strings این میتواند به صورت { value: {10}.{6} } تبدیل شود.

In [23]:

```
num = 23.45678
print ("My 10 character, four decimal number is: {0:10.4f}".format(num))
print (f"My 10 character, four decimal number is: {num:{10}.{6}}")
```

My 10 character, four decimal number is : ۲۳/۴۵۶۸

My 10 character, four decimal number is : ۲۳/۴۵۶۸

توجه داشته باشید که با استفاده از f-strings، دقت به تعداد کل ارقام اعشاری اشاره میکند نه فقط آنچه که پس از ممیز قرار میگیرد. این مطابقت بیشتری با نمایش علمی و تجزیه و تحلیل آماری دارد.
متأسفانه در f-stringsحتی اگر دقت این امکان را فراهم کند پدینگ به سمت راست اعشاری انجام نمی شود.

In [24]:

```
num = 23.45
print("My 10 character, four decimal number is: {0:10.4f}".format(num))
print (f"My 10 character, four decimal number is: {num: {10}.{6}}")
```

My 10 character, four decimal number is: ۲۳/۴۵۰۰

My 10 character, four decimal number is: 23.45

اگر این موضوع مهم هست همیشه میتوانید از سینتکس روش format(). درون یک f-string استفاده کنید.

In [25]:

```
num = 23.45
print("My 10 character, four decimal number is: {0:10.4f}".format(num))
print (f"My 10 character, four decimal number is: {num: 10.4f}")
```

My 10 character, four decimal number is: ۲۳٫۴۵۰۰

My 10 character, four decimal number is: ۲۳٫۴۵۰۰

برای مطالب بیشتر به لینک زیر مراجعه کنید :

https://docs.python.org/3/reference/lexical_analysis.html#f: sloj plyo João Sid as jy lbs 5'xt strings (https://docs.python.org/3/reference/lexical analysis.html#f-strings)

Template Strings

In [۳]:

```
1 from string import Template
```

In [۴]:

```
1 t =Template('Hey, $name!')
```

In [۵]:

```
1 s = 'John'
۲
۳ t.substitute (name = s)
```

Out [5]:

'Hey, John!'

اینم از بحث قالب بندی رشته ها!

لیست ها

در بحث رشته ها مفهوم دنباله را در پایتون معرفی کردیم. لیستها میتوانند به عنوان نسخه ی عمومی ترین دنباله در پایتون در نظر گرفته شوند بر خلاف رشتهها آنها قابل تغییر هستند به این معنا که عناصر درون یک لیست میتوانند تغییر کنند!

در این بخش موارد زیر را خواهیم آموخت:

۱ .ایجاد لیست ها

2.ایندکس گذاری و برش لیست ها

3.متدهای پایه ای لیست

۴ تو در تو کردن لیست ها

5.مقدمه ای بر فهم لیست ها

لیست ها با استفاده از براکتها[] و کاماها بین هر عنصر در لیست ساخته میشوند.

بیایید شروع به ساختن لیست ها کنیم!

In [3]:

```
1 # Assign a list to an variable named my_list
2 my_list - 1,2,3]
```

ما تازه یک لیست از اعداد صحیح ایجاد کردیم اما در واقع لیستها میتوانند شامل انواع مختلفی از اشیاء باشند. به عنوان مثال :

In [4]:

```
1 my_list = ['A string, 23, 100, 3.14, [55, 66]]
```

مانند رشته ها تابع len() تعداد عناصر موجود در دنباله لیست را به شما میگوید.

In [5]:

```
1 len(my list)
```

Out[5]:

5

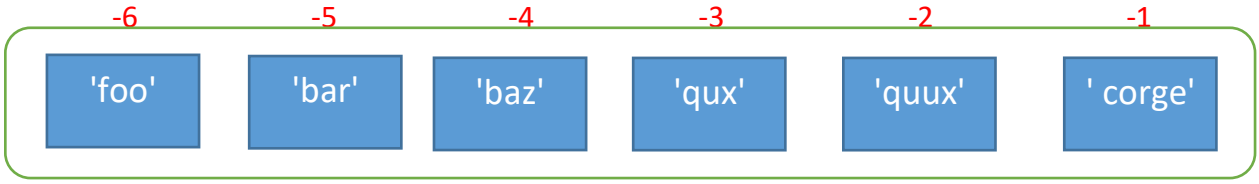
In [ ]:

```
1 len(my list)
```

ایندکس گذاری و برش

عمل ایندکس گذاری و برش مشابه با رشته ها عمل میکند بیایید یک لیست جدید بسازیم تا به یادآوری نحوه کار این عملیات بپردازیم

==========



In [6] :

```
1 my_list 'foo', 'bar', 'baz', 'qux', 'quux', 'corge']
```

In [7]:

```
1 # Grab element at index 0
2 my_list[0]
```

Out [7]:

'foo'

In [8]:

```
1 # Grab index 1 and everything past it
2 my_list [1:]
```

Out [8]:

['bar', 'baz', 'qux', 'quux', 'corge']

In [9]:

```
1 # Grab everything UP TO index 3
2 my_list[:3]
```

Out [9]:

['foo', 'bar', 'baz']

In [10]:

```
1 my_list[-1]
```

Out[10]:

['corge', 'quux', 'qux', 'baz', 'bar', 'foo']

بررسی عضویت یا عدم عضویت در لیست با عملگرin

In [11]:

```
1 'qux' in my_list
```

Out[11]:

True

In [12]:

```
1 'bob' in my_list
```

Out[12]:

False

In [13]:

```
1 'bob' not in my_list
```

Out[13]:

True

لیست قابل جهش هستند

In [14]:

```
1 my_list
```

Out[14]:

['foo', 'bar', 'baz', 'qux', 'quux', 'corge']

In [15]:

```
1 my_list[0] = 35
```

In [16]:

```
1 my_list
```

Out[16]:

[35, 'bar', 'baz', 'qux', 'quux', 'corge']

In [17]:

```
1 my_list[-1]= 66
```

In [18]:

```
1 my_list
```

Out[18]:

[35, 'bar', 'baz', 'qux', 'quux', 66]

In [19]:

```
1 1 = my_list
```

In [20]:

```
1 id(my list)
```

Out [20]:

۲۶۵۶۹۵۶۴۶۶۳۰۴

In [21]:

```
1 id(1)
```

Out[21]:

۲۶۵۶۹۵۶۴۶۶۳۰۴

In [22]:

```
1 1[0] = -555
```

In [23]:

```
1 1
```

Out[23]:

[-555, 'bar', 'baz', 'qux', 'quux', 66]

In [24]:

```
1 my list
```

Out[24]:

[-555, 'bar', 'baz', 'qux', 'quux', 66]

لیستها در پایتون ترتیب دارند

In [25]:

```
1 I = [1, 2, 3]
2 I1  = 1
3
4 I2 = [3, 2, 1]
```

In [26]:

```
1 I is I1
```

Out[26]:

True

In [27]:

```
1 I is I2
```

Out[27]:

False

In [28]:

```
1 I == I1
```

Out[28]:

True

همچنین میتوانیم از + برای اتصال لیستها استفاده کنیم درست مانند کاری که برای رشته ها انجام دادیم.

In [29]:

```
1 my_list['new item']
```

Out[29]:

[-555, 'bar', 'baz', 'qux', 'quux', 66, "new item']

توجه: این در واقع لیست اصلی را تغییر نمی دهد

In [30]:

```
1 my_list
```

Out[30]:

[-555, 'bar', 'baz', 'qux', 'quux', 66, "new item']

برای دائمی شدن این تغییر باید لیست جدید را مجدد در متغیر ذخیره نمایید!

In [31]:

```
1 # Reassign
2 my_list = my_list 'add new item permanently']
```

In [32]:

```
1 my_list
```

Out[32]:

[-555, 'bar', 'baz', 'qux', 'quux', 66, 'add new item permanently']

In [33]:

```
1 # List + number
2
3 my list +5
```

TypeError

Traceback (most recent call last)

Cell In[33], line 3

1 # list + number

---> 3 my_list + 5

TypeError: can only concatenate list (not "int") to list

In [34]:

```
1 # List + string
2 my_list + 'string'
```

TypeError

Traceback (most recent call last)

Cell In[34], line 2

1 # list + *string*

---> 2my_list + *'string'*

TypeError: can only concatenate list (not "str") to list

همچنین میتوانیم از * برای تکرار یک لیست مشابه رشته ها استفاده کنیم:

In [35]:

```
1 # Make the List double
2 my_list * 2
```

Out[35]:

[-555,

'bar',

'baz',

'qux',

'quux',

66,

'add new item permanently',

-555,

'bar',

'baz',

'qux',

'quux",

66,

'add new item permanently']

In [36]:

```
# Again doubling not permanent
my_list
```

Out[36]:

[-555, 'bar', 'baz', 'qux', 'quux', 66, 'add new item permanently']

متدهای پایه لیست ها

اگر با یک زبان برنامه نویسی دیگر آشنایی دارید ممکن است شروع کنید به تشبیه بین آرایه ها در آن زبان و لیست ها در پایتون اما لیست ها در پایتون به دو دلیل از آرایه ها در زبانهای دیگر انعطاف پذیرتر هستند آنها اندازه ثابتی ندارند(بدین معنی که نیازی نداریم اندازه یک لیست را مشخص کنیم)و محدودیت نوع ثابتی ندارند (همانطور که در بالا دیدیم).

بیایید به بررسی تعدادی از متدهای ویژه برای لیستها بپردازیم:

In [37]:

```
# Create a new list
list1= [1,2,3]
```

متد append یک و تنها یک عضو به انتهای لیست اضافه میکند:

In [38]:

```
# Append
list1.append(-100)
```

In [39]:

```
# Show
list1
```

Out[39] :

[1,2,3,-100,[55,66]]

اگر بخواهیم چند عضو را بصورت همزمان به لیست اضافه کنیم دیگر متد append پاسخگوی نیاز ما نمی باشد

In [41]:

```
1 list1.append([55, 66])
```

In [42]:

```
1 list1
```

Out[42]:

[1, 2, 3, 100, [55, 66]]

در این حالت باید از متد extend استفاده نماییم

In [43]:

```
1 # add multiple items to end of list
2 list1.extend([555, 666, 777])
```

In [44]:

```
1 list1
```

Out[44]:

[1, 2, 3, -100, [55, 66], 555, 666, 777]

درج عضو جدید در ایندکس مورد نظر با استفاده از متد insert

In [45]:

```
1 # insert an item to specific index
2 list1.insert(1, 'quux')
3
```

In [46]:

```
1 list1
```

Out[46]:

[1,'quux', 2, 3, 100, [55, 66], 555, 666, 777]

افزودن چندین عضو به ابتدا یا انتها با استفاده از عملگر+=

In [47]:

```
1 a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']
2
3 a += ['bbb', 'ccc']
```

In [48]:

```
1 a
```

Out[48]:

['foo', 'bar', 'baz', 'qux', 'quux', 'corge', 'bbb', 'ccc']

In [49]:

```
1 a = [200, 100] + a
2
3 a
```

Out[49]:

١٠٠ ,٢٠٠,' foo', 'bar', 'baz', 'qux', 'quux', 'corge', 'bbb', 'ccc']

یک نکته جالب

In [51]:

```
1  a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']
2
3  a += 'string'
```

In [52]:

```
1  a
```

Out[52]:

['foo', 'bar', 'baz', 'qux', 'quux', 'corge', 's', 't', 'r', 'i', 'n',

'g']

متد pop برای حذف و بازگرداند یک عضو از لیست مورد استفاده قرار میگیرد بصورت پیش فرض این متد آخرین ایندکس یا ۱- را حذف کرده و مقدار آن را برای استفاده در سایر محاسبات برمیگرداند

In [53]:

```
1  list1 = [1,2,3,4,5]
```

In [54]:

```
1  # Pop off the indexed item
2  list1.pop(0)
```

Out[54]:

1

In [55]:

```
1  # Show
2  list1
```

Out[55]:

[2, 3, 4, 5]

In [56]:

```
1 List1.pop(0)
```

Out[56]:

2

In [57]:

```
1 List1
```

Out[57]:

[3, 4, 5]

In [58]:

```
1 # Assign the popped element, remember default popped index is -1
2 poped_item = list1.pop()
```

In [59]:

```
1 poped_item
```

Out [59]:

5

In [60]:

```
1 # Show remaining list
2 list1
```

Out [60]:

[3, 4]

حذف اعضا از لیست به چندین روش مقدور میباشد.

1استفاده از متد remove

2.استفاده از تابعdel

3.استفاده از روش انتساب چند ایندکسی

In [61]:

```
1 # remove a first occurance of value
2 list1= [1,2,3,4,5,6,1,1]
3
4 list1.remove(1)
```

In [62]:

```
1 list1
```

Out[62]:

[2, 3, 4, 5, 6, 1, 1]

In [63]:

```
1 del list1[5]
```

In [64]:

```
1 list1
```

Out[64]:

[2, 3, 4, 5, 6, 1]

In [65]:

```
1 del list1[0:2]
```

In [66]:

```
1 list1
```

Out[66]:

[4, 5, 6, 1]

همچنین باید توجه داشت که اگر در ایندکس مورد نظر عنصری وجود نداشته باشد عمل ایندکس گذاری در لیست خطا خواهد داد. به عنوان مثال:

In [67]:

```
1 list1[100]
```

IndexError

Traceback (most recent call last)

Cell In[67], line 1

----> 1 list1[100]

IndexError: list index out of range

میتوانیم از متد sort و متد reverse نیز برای تغییر لیستهای شما استفاده کنیم:

In [68]:

```
1 new_list = ['a', 'e', 'x', 'b', 'c']
```

In [69]:

```
1 #Show
2 new_list
```

Out [69]:

['a', 'e', 'x', 'b', 'c']

In [70]:

```
1 new list[-1]
```

Out [70]:

['c', 'b', 'x', 'e', 'a']

In [71]:

```
1 # Use reverse to reverse order (this is permanent!)
2 new_list.reverse()
```

In [72]:

```
1 new_list
```

Out [72]:

['c', 'b', 'x', 'e', 'a']

In [73]:

```
1 # Use sort to sort the List (in this case alphabetical order, but for numbers it wil
2 new_list.sort()
3 new_list
```

Out [73]:

['a', 'b', 'c', 'e', 'x']

In [74]:

```
1 new_list.sort (reverse=True)
2 new_list
```

Out [74]:

['x', 'e', 'c', 'b', 'a']

In [75]:

```
1 I = new_list.copy()
2
3 I is new list
```

Out [75]:

False

In [76]:

```
1 I
```

Out [76]:

['x', 'e', 'c', 'b', 'a']

تغییر چندین مقدار در لیست با عملگر انتساب

In [77]:

```
1 a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']
```

In [78]:

```
1 a[1:4]
```

Out [78]:

['bar', 'baz', 'qux']

۱. تعداد موارد انتخاب شده با موارد جایگزین برابر باشند

In [79]:

```
1 a[1:4] = [11, 22, 33]
```

In [80]:

```
1 a
```

Out [80]:

['foo', 11, 22, 33, 'quux', 'corge']

۲. تعداد موارد انتخاب شده کمتر از موارد جایگزین باشد

In [81]:

```
1 a[0:2]= [-100, 200, -300, -400]
2
3 a
```

Out [81]:

[-100, 200, -300, -400, 22, 33, 'quux', 'corge']

In [83]:

```
1 a[2:2] = ['foo', 'bar', 'baz']
2
3 a
```

Out [83]:

foo', 'bar', 'baz', -300, -400, 22, 33, 'quux', 'corge" ,۲۰۰ ,۱۰۰-]

3.تعداد موارد انتخاب شده بیشتر از موارد جایگزین باشد

In [84]:

```
1 a[0:6] ['hello']
2
3 a
```

Out [84]:

['hello', 400, 22, 33, 'quux', 'corge']

In [85]:

```
1  a[1:3] = []
2
3  a
```

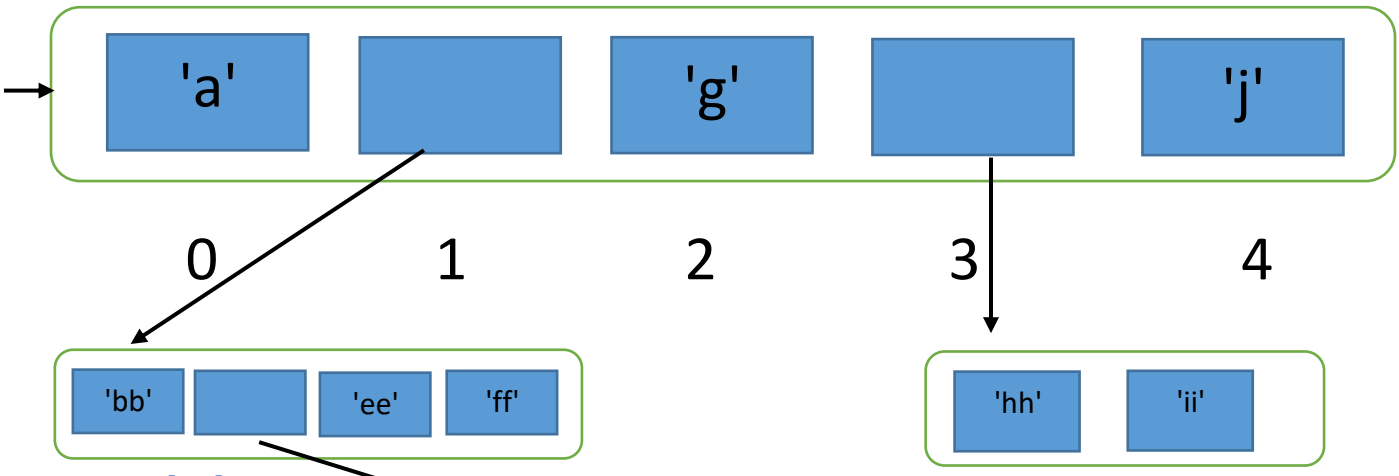
Out [85]:

' hello', 33, 'quux', 'corge']

لیستهای تو در تو

یک ویژگی بزرگ از ساختارهای داده در پایتون این است که آنها از تو در تو کردن پشتیبانی میکنند این بدان معنی است که میتوانیم ساختارهای داده را درون ساختارهای داده دیگر داشته باشیم به عنوان مثال یک لیست درون یک لیست

بیایید ببینیم چگونه کار میکند!



In [86] :

```
1  1st 1 = [
2  1st2 = [4
3  1st3 = [7,8,9]
4
5
```

```
6 matrix [1st1, 1st2, 1st3]
```

In [87] :

```
1 # Show
2 matrix
```

Out [87]:

[[۱ ,۲ ,۳], [۴ ,۵ ,۶], [۷ ,۸ ,۹]]

ما میتوانیم باز هم از عمل ایندکس گذاری استفاده کنیم تا عناصر را بگیریم اما اکنون دو سطح برای ایندکس وجود دارد. عناصر درون شی ماتریس و سپس عناصر درون آن لیست!

In [88] :

```
1 # Grab first item in matrix object
2 matrix[0]
```

Out [88]:

[1, 2, 3]

In [89]:

```
1 # Grab first item of the first item in the matrix object
2 [1, 2, 3][0]
```

Out [89]:

۱

In [90]:

```
1 matrix[0][0]
```

Out [90]:

۱

In [91]:

```
1 matrix[1]
```

Out [91]:

[4, 5, 6]

In [92]:

```
1 matrix[1][1][:: -1]
```

Out [92]:

[۴ ,۵ ,۶]

In [112]:

```
1 x = ['a', ['bb', ['ccc', 'ddd'], 'ee', 'ff'], 'g', ['hh', 'ii'], 'j']
```

In [113]:

```
1 x [1]
```

Out [113]:

['bb', ['ccc', 'ddd'], 'ee', 'ff']

In [114]:

```
1 x [1][1]
```

Out [114]:

['ccc', 'ddd']

In [115]:

```
1 x [1][1][1]
```

Out [115]:

'ddd'

In [116]:

```
1 x [1][1][1][2]
```

Out [116]:

'd'

In [117]:

```
1 x [1][1][1] .upper()
```

Out [117]:

'DDD'

In [118]:

```
1 'ddd'.upper()
```

Out [118]:

'DDD'

بریم سراغ توپل ها

تاپل ها

در پایتون تاپل ها بسیار شبیه به لیست ها هستند با این تفاوت که برخلاف لیست ها آنها غیر قابل تغییر هستند و یعنی نمیتوان آنها را تغییر داد شما از تاپل ها برای نمایش اشیاءی که نباید تغییر کنند مانند روزهای هفته یا تاریخ ها در تقویم استفاده خواهید کرد.

در این بخش مروری کوتاهی بر موارد زیر خواهیم داشت:

۱. ساخت تاپل ها
۲. متدهای پایه ای تاپل
۳. غیر قابل تغییر بودن
۴. زمان استفاده از تاپل ها

بر اساس آنچه درباره لیستها یاد گرفته اید شما به نحوی به تاپلها آشنا خواهید بود. ما میتوانیم با رفتار بسیار مشابهی از آنها استفاده کنیم با این تفاوت که تاپلها غیر قابل تغییر هستند.

ساخت تاپل ها

ساخت تاپل ها با استفاده از پرانتز () و عناصری که توسط کاما از هم جدا شدهاند انجام میشود. به عنوان مثال

In [5]:

```
1 # Create a tuple
2 t = (1, 2, 3, 4, 5)
```

In [6]:

```
1 type(t)
```

Out [6]:

Tuple

In [7]:

```
1 # Check Len just like a list
2 len(t)
```

out[7]:

۵

In [22]:

```
1 # Can also mix object types
2 t = ('one', 2, 'one')
3
4 # Show
5 t
```

Out [22]:

('one', 2, 'one')

In [9]:

```
1 # Use indexing just Like we did in Lists
2 t[0]
```

Out [9]:

'one'

In [10]:

```
1 # Slicing just Like a List
2 t[-1]
```

Out[10]:

2

تعریف تاپل تک عضوی

In [11]:

```
1 x = (1)
```

In [12]:

```
1 x
```

Out[12]:

1

In [13]:

```
1 type(x)
```

Out[13]:

Int

In [17]:

```
1 x = (1,)
```

In [18]:

```
1 type(x)
```

Out[18]:

tuple

In [19]:

```
1 x
```

Out[19]:

(۱)

متدهای پایه تاپل ها

تاپل ها دارای متدهای داخلی هستند اما تعداد آنها به اندازه لیستها نیست بیایید به دو متد آنها نگاهی بیندازیم:

In [21]:

```
1 # Use .index to enter a value and return the index
2 t.index(2)
```

Out[21]:

1

In [23]:

```
1 t
```

Out[23]:

('one', 2, 'one')

In [24]:

```
1 # Use .count to count the number of times a value appears
2 t.count('one')
```

Out[24]:

2

غیر قابل جهش بودن Immutability

اهمیت غیر قابل تغییر بودن تاپل ها را نمیتوان به اندازه کافی تاکید کرد برای بیان این نکته:

In [25]:

```
1 t[0] = 'change'
```

TypeError

Traceback (most recent call last)

Cell In[25], line

-----> 1 t[0]= 'change'

TypeError: 'tuple' object does not support item assignment

به دلیل این غیر قابل تغییری ، تاپل ها قادر به رشد .نیستند یکباری که یک تاپل ساخته شد نمیتوانیم به آن عناصر اضافه کنیم

In [26]:

```
1 t.append('nope')
```

AttributeError

Traceback (most recent call last)

Cell In[26], line 1

----> 1 t.append(' nope ')

AttributeError: 'tuple' object has no attribute 'append'

نکته بسیار مهم ،تاپل ها فقط در سطح اول خود غیرقابل جهش می باشند عملکرد سطوح بعدی به نوع داده آن سطح بستگی دارد

In [2^]:

```
1 t = (1, 2, 3, (5,6), [10, 11, 12])
```

In [30]:

```
1 t[0] = 55
```

TypeError

Traceback (most recent call last)

Cell In[30], line 1

------> 1 t[0] = 55

TypeError: 'tuple' object does not support item assignment

In [31]:

```
1 t [3]
```

Out[31] :

(5, 6)

In [33]:

```
1 t [3][1] = 55
```

TypeError

Traceback (most recent call last)

Cell In[33], line 1

------> 1 t[3][1] = 55

TypeError: 'tuple' object does not support item assignment

In [34]:

```
1 t [3] = (555,666)
```

TypeError

Traceback (most recent call last)

Cell In[34], line 1

------> 1 t[3] = (555,666)

TypeError: 'tuple' object does not support item assignment

In [35]:

```
1 t
```

Out[35]:

(1, 2, 3, (5, 6), [10, 11, 12])

In [36]:

```
1 t [4]
```

Out[36]:

[10, 11, 12]

In [38]:

```
1 t [4] = [55, 66]
```

TypeError

Traceback (most recent call last)

Cell In[38], line 1

------> 1 t[4] = (55,66)

TypeError: 'tuple' object does not support item assignment

In [40]:

```
1 t [4][0] = -100
```

In [41]:

```
1 t
```

Out [41]:

(1, 2, 3, (5, 6), [-100, 11, 12])

بسته بندی و بازکردن بسته در تاپل ها

In [42]:

```
1 # tuple packing
2 x = 1, 'one', 2, 'three'
```

In [43]:

```
1 x
```

Out[43]:

(1, 'one', 2, 'three')

In [4۴]:

```
1 type(x)
```

Out[4۴]:

Tuple

In [4۵]:

```
1 # tuple unpacking
2 x1, x2, x3, x4 = x
۳
۴ print(x1, x2, x3, x4)
```

1 one 2 three

In [46]:

```
1 # be careful
2
۳  x1, x2, x3 = x
```

ValueError

Traceback (most recent call last)

Cell In[46], line 3

۱ be careful

------> 3 x1, x2, x3 = x

ValueError: too many values to unpack (expected 3)

In [47]:

1	*x1, x2, x3, x4, x5 = X*

ValueError

Traceback (most recent call last)

Cell In[47], line 1

------> x1, x2, x3, x4, x5 = x

ValueError: not enough values to unpack (expected 5, got 4)

چه زمانی از تاپل ها

شاید بپرسید "چرا باید از تاپل ها استفاده کنیم وقتی که تعداد کمتری از متدها در دسترس هستند؟"
صادقانه بگوییم تاپل ها به همان اندازه که لیست ها در برنامه نویسی استفاده میشوند اما زمانی که غیر قابل تغییر بودن ضروری است تاپل ها مورد استفاده قرار میگیرند اگر در برنامه خود شما یک شیء را انتقال میدهید و نیاز دارید مطمئن شوید که آن تغییر نمیکند، در این صورت تاپل بهترین گزینه .است آنها منبع مناسبی از سلامت داده را فراهم میکنند.

اکنون باید قادر باشید تاپل ها را ایجاد و استفاده کنید و درکی از غیر قابل تغییری آنها داشته باشید.

در ادامه دیکشنری ها!

Dictionaries

تا اینجا ما درباره توالی ها" در پایتون یاد گرفتیم اما حالا قصد داریم به مبحث نگاشتها" در پایتون بپردازیم. اگر با زبانهای برنامه نویسی دیگر آشنایی دارید میتوانید این دیکشنریها را به عنوان جدولهای هش در نظر بگیرید.

این بخش به عنوان یک مقدمه مختصر به دیکشنریها خدمت میکند و شامل موارد زیر است

1. ساختن یک دیکشنری

2.دسترسی به اشیاء در یک دیکشنری

3. تو در تو کردن دیکشنری ها

۴ . متدهای پایه ای دیکشنری

پس چه چیزی است نگاشت؟ نگاشت ها مجموعه ای از اشیاء هستند که توسط یک کلید ذخیره میشوند، به عکس توالی ها که اشیاء را بر اساس موقعیت نسبی ذخیره می.کنند این تفاوت مهم است زیرا نگاشت ها ترتیب را حفظ نمیکنند زیرا اشیاء توسط یک کلید تعریف شده هستند.

یک دیکشنری پایتون از یک کلید و سپس یک مقدار مرتبط تشکیل شده است این مقدار میتواند تقریباً هر شیء پایتونی باشد.

ساختن یک دیکشنری

بیایید ببینیم چگونه میتوانیم دیکشنریها را بسازیم تا درک بهتری از کارکرد آنها داشته باشیم

برای تعریف دیکشینری ها سه روش وجود دارد

روش اول:

```
d = {
key: value,
key: value,
.
.
.
key: value
}
```

In [۱]:

```
1  MLB_team = {
2  'Colorado'              : 'Rockies',
3  'Boston'                : 'Red Sox'
4  "Minnesota'             : 'Twins',
5  'Milwaukee'             : 'Brewers',
6  'Seattle'               : 'Mariners'
7  }
```

In [۱]:

```
1  MLB_team
```

Out[2]:

{'Colorado': 'Rockies',

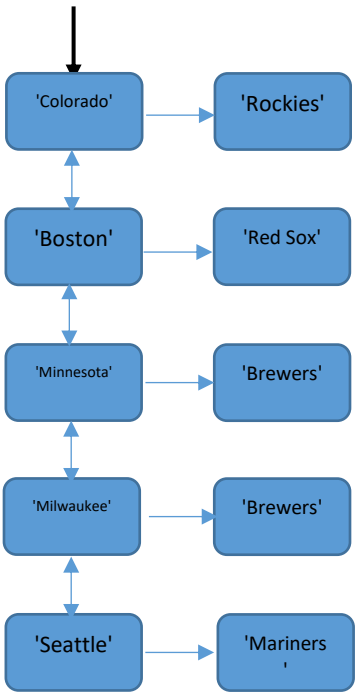
'Boston': 'Red Sox',

'Minnesota': 'Twins',

'Milwaukee': 'Brewers',

'Seattle': 'Mariners' }

MLB team



روش دوم استفاده از تابعdict

In [ ]:

```
d = dict([
(<key>, <value>),
(<key>, <value),
        .
        .
        .
(<key>, <value>)
])
```

In [3]:

```
MLB team = dict([
('Colorado', 'Rockies'),
('Boston', 'Red Sox'),
('Minnesota', 'Twins'),
('Milwaukee', 'Brewers'),
('Seattle', 'Mariners')
])
```

In [4]:

```
MLB team
```

Out[4]:

{'Colorado': 'Rockies',

'Boston': 'Red Sox',

'Minnesota': 'Twins',

'Milwaukee': 'Brewers',

'Seattle': 'Mariners'}

حالت استثنا روش دوم در صورتیکه تمام کلیدها از جنس رشته باشد میتوانیم بصورت خلاصه تر عمل کنیم:

In [5]:

```
1  MLB team = dict(
2  Colorado Rockies',
3  Boston='Red Sox',
4  Minnesota Twins',
5  Milwaukee='Brewers',
6  Seattle="Mariners'
7  )
```

In [6]:

```
1  MLB team
```

Out[6]:

{'Colorado': 'Rockies',

'Boston': 'Red Sox',

'Minnesota': 'Twins',

'Milwaukee': 'Brewers',

'Seattle': 'Mariners'}

روش سوم را پس از یادگیری نحوه دسترسی به اعضای دیکشینری یاد میگیریم

نکته مهم: از پایتون ۳/۶ به بعد دیکشینریها مانند لیستها دارای ترتیب در اعضا می باشند ولی این ترتیب در مقایسه برابر بودن لحاظ نمی شود.

In [7]:

```
1  I = [1,2,3]
2
3  I1 = [2,1,3]
4
5  I== 11
```

Out[7]:

False

In [8]:

```
1 d = {'k1': 'v1', 'k2': 'v2'}
2 d2 = {'k2': 'v2', 'k1': 'v1'}
3
4 d == d2
```

Out[8]:

True

دسترسی به مقادیر اعضا در دیکشینری

In [9]:

```
1 MLB_team
```

Out[9]:

{'Colorado': 'Rockies',

'Boston Red' :' Sox',

'Minnesota': 'Twins',

'Milwaukee': 'Brewers',

'Seattle': 'Mariners'}

In [10]:

```
1 MLB _team[0]
```

KeyError

Traceback (most recent call last)

Cell In[10], line 1

----> 1 MLB _team[0]

KeyError: 0

In [11]:

```
1 MLB _team['Colorado']
```

Out[11]:

'Rockies'

In [12]:

```
1 MLB _team['Seattle']
```

Out[12]:

'Mariners'

In [13]:

```
1 MLB _team['Toronto']
```

KeyError

Traceback (most recent call last)

Cell In[13], line 1

----> 1 *MLB _team['Toronto']*

KeyError: *'Toronto'*

دیکشینری ها قابل تغییر هستند.

In [14]:

```
1 MLB team['Seattle'] = 'Seahawks'
```

In [15]:

```
1 MLB team
```

Out [15]:

{'Colorado': 'Rockies',

'Boston': 'Red Sox',

Minnesota': 'Twins','

'Milwaukee': 'Brewers',

'Seattle': 'Seahawks'}

In [16]:

```
1 MLB_team['Kansas City'] = 'Royals'
```

In [17]:

```
1 MLB_team
```

Out [17]:

{'Colorado': 'Rockies',

'Boston': 'Red Sox',

'Minnesota': 'Twins',

'Milwaukee': 'Brewers',

'Seattle': 'Seahawks',

'Kansas City': 'Royals'}

این مهم است که توجه کنید دیکشنری ها در نوع داده هایی که میتوانند نگه دارنده باشند بسیار انعطاف پذیر هستند. به عنوان مثال:

In [18]:

```
1 my_dict = {'key1" :123, 'key2' : [12, 23,33], "key3":["iteme','item1','item2']}
```

In [19]:

```
1 # Let's call items from the dictionary
2 my_dict['key3']
```

out[19] :

['iteme0', item1', 'item2']

In [20]:

```
1 # Can call an index on that value
2 my_dict['key3'][0]
```

out[20] :

'iteme0'

In [21]:

```
1 # Can then even call methods on that value
2 my_dict['key3'][0].upper()
```

out[21] :

'ITEME0'

همانگونه که مشاهده نمودین تمام انواع داده ها میتوانن بعنوان مقادیر یک کلید در دیکیشنری استفاده شود

اما چه نوع دادههایی میتوانند بعنوان کلید استفاده شوند؟

بصورت ساده میتوان گفت که فقط داده های غیر قابل تغییر یا جهش میتوانند بعنوان کلید استفاده شوند، یعنی اعداد رشته ها، تاپل ها، بولین ها

اما دلیل اصلی این ویژگی عدم امکان محاسبه مقدار hash برای انواع دادههای قابل تغییر مانند لیست ها، دیکشینری ها و مجموعه ها می باشد

In [22]:

```
1 d = {0: 'v1', 5: 3.14}
```

In [23]:

```
1 d
```

out[23] :

{0: 'v1', 5: 3.14}

In [24]:

```
1 d[0]
```

out[24] :

'v1'

In [25]:

```
1 d={(-1,-4): 'Tabriz', (0, 0): 'Tehran', (0, -3): 'Shiraz'}
```

In [26]:

```
1 d
```

out[26] :

{(1, 4): 'Tabriz', (0, 0): 'Tehran', (0, -3): 'Shiraz'}

In [27]:

```
1 d[(0, -3)]
```

out[27] :

'Shiraz'

In [28]:

```
1 { d [1,1]: 'not possible'}
```

TypeError

Traceback (most recent call last)

Cell In[28], line 1

----> 1 d = {[1,1]: 'not possible'}

TypeError: unhashable type: 'list

روش سوم ساخت دیکشینری بصورت افزایشی میباشد

In [29]:

```
# Create a new dictionary
d = {}
```

In [30]:

```
# Create a new key through assignment
d['animal'] = 'Dog'
```

In [31]:

```
# Can do this with any object
d['answer'] = 42
```

In [32]:

```
#Show
 d
```

Out[32] :

{'animal': 'Dog', 'answer' : 42}

دیکشینری های تو در تو

امیدوارم شروع به درک قدرتمندی که پایتون با انعطاف پذیری در تو در تو کردن اشیاء و فراخوانی متدها در آن دارد کنید بیایید یک دیکشنری را درون یک دیکشنری ببینیم

In [33]:

```
# Dictionary nested inside a dictionary nested inside a dictionary
d = {'key1':{'nestkey':{'subnestkey' : 'value'}}}
```

خیلی جالب هست !این یک تو در توی شگفت انگیز از دیکشنری هاست !ببایید ببینیم چگونه میتوانیم آن مقدار را دریافت کنیم

In [۳۴]:

```
# Keep calling the keys
d['key1']['nestkey']['subnestkey']
```

Out[34] :

'value'

In [۳۵]:

```
d['key1']['nestkey']['subnestkey'].upper()
```

Out[۳۵] :

'VALUE'

برخی از متدهای دیکشینری ها

اینجا چندین روش برای متد یک دیکشنری وجود دارد بیایید با چندتایی از آنها آشنا شویم:

In [۳۶]:

```
# Create a typical dictionary
D ='key1':1, 'key2' :2,"key3":3}
```

In [۳۷]:

```
# Method to return a list of all keys
d.keys()
```

Out [37]:

dict_keys(['key1', key2', 'key3'])

In [38]:

```
# Method to grab all values
d. values()
```

Out [38]:

dict_values([1, 2, 3])

In [39]:

```
# Method to return tuples of all items (we'LL Learn about tuples soon)
 d.items()
```

Out [39]:

Dict_items([('key1', 1), ('key2', 2), ('key3', 3)])

روش امن دریافت مقادیر یک کلید از دیکشینری با متد get

In [40]:

```
d['key5']
```

KeyError

Traceback (most recent call last)

Cell In[40], line 1

----> 1 d['key5']

KeyError: 'keys'

In [41]:

```
d.get('key5')
```

In [42]:

```
1 d.get('key1')
```

Out [42]:

۱

در دیکشینری ها ما دو نوع روش pop داریم.

In [43]:

```
1 # must give key
2  d.pop('key1')
```

Out [43]:

1

In [44]:

```
1 d
```

Out [44]:

('key2' : 2, 'key3': 3)

In [45]:

```
1 d.popiten()
```

Out [45]:

('key', 3)

In [46]:

```
1 d
```

Out [46]:

{'key2':2}

بروزرسانی یک دیکشینری با استفاده از اعضای دیکشینری دیگر با استفاده متد update

In [47]:

```
1 d1 = {'a' 1, 'b': 5}
2
3 d2 = {'b': 777, 'c': -15}
```

In [48]:

```
1 d1.update(d2)
```

In [49]:

```
1 d1
```

Out[49] :

{'a': 1, 'b': 777, 'c': -15}

در این مباحث سعی شد تا با مفاهیم پایه و کاربردی دیکشینری ها در پایتون بخوبی آشنا شویم در ادامه به بررسی نوع داده مجموعه و بولین ها خواهیم پرداخت.

مجموعه و بولینها

دو نوع دیگر از اشیاء در پایتون وجود دارند که باید به طور سریع به آنها اشاره کنیم: مجموعه ها و بولینها.

مجموعه ها

مجموعه ها مجموعه های نامرتب از عناصر منحصربفرد هستند. ما میتوانیم آنها را با استفاده از تابع set()بسازیم. بیایید از این تابع برای ساختن یک مجموعه استفاده کنیم تا نحوه عملکرد آن را ببینیم

دو نکته مهم در مجموعه های ریاضی که در مجموعههای پایتون نیز صادق می باشند

۱. تکرار در مجموعههای بی معنا می باشد.

۲. ترتیب در مجموعه ها مهم نیست

نوع داده مجموعه در پایتون بیشتر برای ذخیره داده ها و بررسی عضویت مقادیر در یک مجموعه کاربرد دارد. ما در مجموعه امکان بازیابی مقادیر از طریق ایندکس گذاری یا کلید و ..... نداریم.

In [۱]:

```
1 x = set()
```

In [2]:

```
1 type(x)
```

Out[2]:

Set

In [3]:

```
1 # We add to sets with the add() method
2 x.add(1)
```

In [4]:

```
1 #Show
2 x
```

Out[4]:

{1}

توجه کنید که از پرانتزهای آکولادی استفاده شده است. این نشان دهنده یک دیکشنری نیست !با این حال میتوانید بگویید مجموعه یک دیکشنری است که فقط دارای کلیدهاست.

ما میدانیم که یک مجموعه تنها ورودی های منحصربفرد دارد پس چه اتفاقی میافتد وقتی سعی میکنیم چیزی را به مجموعه اضافه کنیم که از قبل در آن وجود دارد؟

In [5]:

```
1 # Add a different element
2 x.add(2)
```

In [6]:

```
1 #Show
2 x
```

Out[6]:

{1, 2}

In [7]:

```
1 # Try to add the same element
2 x.add(1)
```

In [8]:

```
1 #Show
2 x
```

Out[8]:

{1, 2}

In [9]:

```
1 # membership check with in operator
2 1 in x
```

Out[9]:

True

In [10]:

```
1 3 in x
```

Out[10]:

False

توجه کنید که مجموعه مجدداً یک عدد ۱ را در آن قرار نمیدهد این به این دلیل است که مجموعه تنها در ارتباط با عناصر منحصر بفرد اهمیت دارد ما میتوانیم یک لیستی که شامل عناصر تکراری است را به یک مجموعه تبدیل کنیم تا عناصر منحصر بفرد را بدست آوریم. به عنوان مثال:

In [13]:

```
1 # Create a List with repeats
2 List1 = [1,1,1,2,2,3,4,4,4,5,6,6,7,7,7,7,7]
```

In [14]:

```
1 # Cast as set to get unique values
2 Unique = set(list1)
3
4
5 unique
```

Out[14]:

{1, 2, 3, 4, 5, 6, 7}

In [15]:

```
1 list(unique)
```

Out[15]:

[1, 2, 3, 4, 5, 6, 7]

In [16]:

```
1 tuple (unique)
```

Out[16]:

(1, 2, 3, 4, 5, 6, 7)

In [17]:

```
1 set('Hello wollll')
```

Out[17]:

{'', 'H', 'e', '1', 'o', 'w'}

اعمال مجموعه ها

۱. اشتراک

In [18]:

```
1 x = {1, 2, 3, 4, 5, 6, 7}
2 y = {4, 6, 10, 11, 12}
```

In [19]:

```
1 x.intersection(y)
```

Out[19]:

{4,6}

۲. اجتماع

In [20]:

```
1 x.union(y)
```

Out[20]:

{۱, ۲, ۳, ۴, ۵, ۶, ۷, ۱۰, ۱۱, ۱۲}

۳. تفاضل مجموعه ها

In [21]:

```
1 x.difference(y)
```

Out[21]:

{1, 2, 3, 5, 7}

۴. تفاضل متقارن مجموعه ها

In [۲۲]:

```
1 x.symmetric_difference (y)
```

Out[۲۲]:

{1, 2, 3, 5, 7, 10, 11, 12}

بولین ها

پایتون دارای بولین ها است(بانمایش های از پیش تعیین شده True و False که در واقع فقط عددهای صحیح ۱ و ۰ هستند). همچنین یک شی پوشاننده به نام None نیز دارد. بیایید از طریق چند مثال سریع از بولین ها بگذریم (بعدها در این دوره به طور عمیق تر به آنها خواهیم پرداخت).

In [۲۳]:

```
1 # Set object to be a boolean
۲ a = True
```

In [۲۴]:

```
1 #Show
۲ a
```

Out[۲۴]:

True

In [۲۵]:

```
1 type(a)
```

Out[۲۵]:

Bool

ما همچنین میتوانیم از عملگرهای مقایسه برای ایجاد بولین ها استفاده کنیم. در طول دوره به تمامی عملگرهای مقایسه خواهیم پرداخت.

In [۲۶]:

```
1 # Output is boolean
۲ 1 > 2
```

Out[۲۶]:

False

In [۲۷]:

```
1 2 < 5
```

Out[۲۷]:

True

ما میتوانیم از None به عنوان یک نمادگذار برای یک شیء که هنوز نمیخواهیم آن را مجدداً اختصاص دهیم استفاده کنیم.

In [۲۹]:

```
1 # None placeholder
۲ b = None
```

In [۳۰]:

```
1 # Show
۲ b
```

In [۳۱]:

```
۱ type(b)
```

Out[۳۱]:

NoneType

درسته اکنون باید درک اولیه ای از اشیا و نوع ساختار داده های پایتون داشته باشید. حالا بروید و آزمون ارزیابی را انجام دهید!

فایل ها

پایتون از اشیای فایل برای تعامل با فایلهای خارجی در رایانه تان استفاده میکند. این اشیای فایل میتوانند هر نوع فایلی که در رایانه تان دارید باشند مانند فایل صوتی فایل متنی ،ایمیل ها اسناد اکسل و غیره توجه شاید نیاز داشته باشید کتابخانه ها یا ماژول های خاصی را نصب کنید تا با این انواع مختلف فایل تعامل ،کنید اما آنها به راحتی در دسترس هستند. (دردوره بعدی درباره دانلود کردن ماژول ها صحبت خواهیم کرد).

پایتون یک تابع باز کننده داخلی دارد که به ما اجازه میدهد با انواع پایهٔ فایل بازی کنیم اما ابتدا به یک فایل نیاز داریم ما قصد داریم از یکی از قدرتهای IPython برای ایجاد یک فایل متنی استفاده کنیم

نوشتن یک فایل درIPython

این تابع ویژه نوت بوک های Jupyter است به طور جایگزین میتوانید به سرعت یک فایل ساده txt. را با ویرایشگرsublime ایجاد کنید

In [1]:

```
1 %%writefile test.txt
۲ Hello, this is a quick test file
```

Overwriting test.txt

باز کردن یک فایل در پایتون

بیایید با شروع فایل test.txt را که در همان دایرکتوری این نوت بوک قرار دارد باز کنیم. در حال حاضر ما با فایل هایی که در همان دایرکتوری نوت بوک یا اسکریپت py. قرار دارند کار خواهیم کرد.

در این مرحله بسیار آسان است که خطایی دریافت کنید:

In [1]:

```
1 myfile = open( ' whoops.txt')
```

FileNotFoundError

Traceback (most recent call last)

<ipython-input-1-dafe28ee473f> in <module> ()

-----> 1 myfile = open('whoops.txt')

FileNotFoundError: [Errno 2] No such file or directory: 'whoops.txt'

برای جلوگیری از این ،خطا مطمئن شوید که فایل txt. شما در همان محلی که نوت بوک شما قرار دارد ذخیره شده باشد. برای بررسی مکان نوت بوک خود از pwd استفاده کنید:

In [1]:

```
1 pwd
```

Out[1] :

'C:\\Users\\babak \ \Complete-Python-3-Bootcamp-master\\00-Python Object and Data Structure Basics'

به عنوان یک روش جایگزین برای دریافت فایل ها از هر محلی در رایانه تان به سادگی مسیر کامل فایل را به عنوان ورودی ارسال کنید.

برای ویندوز باید دو بار علامت \ استفاده کنید تا پایتون علامت \ دوم را به عنوان یک کاراکتر فرار متصور نکند مسیر فایل به صورت زیر است:

myfile = open("C:\\Users\\پوشه نام کاربری خانه \ \myfile.txt")

برای مک و لینوکس از خط کج به سمت معکوس استفاده میکنید:

myfile = open("/users/پوشه/نام کاربری/myfile.txt")

In [2]:

```
# Open the text.txt we made earlier
my_file = open('test.txt')
```

In [3]:

```
# We can now read the file
my file.read()
```

Out[3] :

'Hello, this is a quick test file.'

In [4]:

```
# But what happens if we try to read it again?
my_file.read()
```

Out[4] :

''

این اتفاق می افتد زیرا میتوانید تصور کنید که "مکان نمایشگر" خواندنی در انتهای فایل قرار دارد بعد از آنکه فایل را خوانده اید ،بنابراین چیزی برای خواندن باقی نمی ماند. میتوانیم "مکان نمایشگر" را مجدداً تنظیم کنیم به این شکل:

In [۵]:

```
# Seek to the start of file (index 0)
my_file.seek(0)
```

Out[۵] :

۰

In [6]:

```
# Now read again
```

```
2 my_file.read()
```

Out[6] :

'Hello, this is a quick test file.'

شما میتوانید با استفاده از متد readlines یک فایل را به صورت خط به خط .بخوانید با فایلهای بزرگ با احتیاط عمل کنید زیرا همه چیز در حافظه نگهداری خواهد شد. در دوره بعدی خواهیم آموخت که چگونه بر روی فایل های بزرگ حلقه زنی کنیم.

In [7]:

```
1 # Readlines returns a list of the Lines in the file
2 my_file.seek(0)
3 my_file.readlines()
```

Out[7] :

['Hello, this is a quick test file. ']

وقتی که از یک فایل استفاده کردید همیشه عمل خوبی است که آن را ببندید.

In [8]:

```
1 my_file.close()
```

نوشتن در یک فایل

به طور پیش فرض ، تابع open() فقط به ما اجازه میدهد فایل را بخوانیم برای اینکه بتوانیم در فایل بنویسیم باید آرگومان را به آن بدهیم. به عنوان مثال:

In [9]:

```
1 # Add a second argument to the function, 'w' which stands for write.
٢ # Passing 'w+' Lets us read and write to the file
٣
٤ my_file = open('test.txt', 'w+')
```

با احتیاط عمل کنید

با باز کردن یک فایل با استفاده از ' w ' یا'w+' ، محتوای اصلی فایل بریده میشود به این معنی که هر چیزی که در فایل اصلی بود **پاک میشود!**

In [10]:

```
# Write to the file
my_file.write('This is a new line')
```

Out[10]:

18

In [11]:

```
# Read the file
my_file.seek(0)
my file.read()
```

Out[11]:

'This is a new line'

In [12]:

```
my_file.close()    # always do this when you're done with a file
```

افزودن به یک فایل

ارسال آرگومان 'a' فایل را باز میکند و نشانگر را در انتها قرار میدهد بنابراین هر چیزی که نوشته میشود به فایل اضافه میشود مانند 'w+' و 'a+' به ما اجازه میدهد برای یک فایل بخوانیم و بنویسیم اگر فایل وجود نداشته باشد یک فایل ساخته میشود.

In [13]:

```
my_file = open('test.txt', 'a+')
my file.write("\nThis is text being appended to test.txt')
my_file.write('\nAnd another line here.')
```

Out[13]:

23

In [14]:

```
my_file.seek(0)
print(my_file.read())
```

This is a new line.

This is text being appended to test.txt

And another line here.

In [15]:

```
my file.close()
```

افزودن با استفاده ازwritefile%%

میتوانیم همین کار را با استفاده از قدرتهای خاص IPython انجام دهیم:

In [۱۶]:

```
%%writefile -a test.txt

This is text being appended to test.txt
And another line here.
```

Appending to test.txt

اگر میخواهید خط اول بر روی خط خودش آغاز شود یک فضای خالی اضافه ،کنید زیرا Jupyter ترتیبهای فرار مانند\n را تشخیص نمی دهد.

حلقه ها در یک فایل

بیایید با استفاده از یک حلقه for بر روی یک فایل متنی یک نمایش سریع از عملکرد آن را .ببینیم ابتدا بیایید یک فایل متنی جدید با استفاده از برخی از قدرتهای IPython ایجاد کنیم:

In [۱۷]:

```
%%writefile test.txt
2 First Line
3 Second Line
```

Overwriting test.txt

حالا میتوانیم از یک کمی روند استفاده کنیم تا به برنامه بگوییم که در هر خط از فایل حلقه بزند و عملی انجام دهد :

In [18]:

```
for line in open('test.txt'):
    print(line)
```

First Line

Second Line

فعلاً نگران جزئیات کامل نباشید حلقه های for در ادامه قرار دارند اما ما در بالا آنچه انجام دادیم را تجزیه کردیم ما گفتیم که برای هر خط در این فایل متنی ادامه دهید و آن خط را چاپ کنید در اینجا چند نکته مهم وجود دارد:

۱. ما میتوانستیم هرچه که بخواهیم به اشیای" "خط" نامیده شود (نمونه زیر را ببینید).

۲. با فراخوانی ()read بر روی فایل کل محتوای فایل متنی در حافظه ذخیره نشده است.

۳. توجه کنید که برای دستور ،چاپ فاصله درونی در خط دوم لازم است این فاصله سفید در پایتون الزامی است

In [19]:

```
# Pertaining to the first point above
for asdf in open('test.txt'):
    print (asdf)
```

First Line

Second Line

آزمون ارزیابی اشیا و ساختارهای داده

دانش خود را آزمایش کنید.

** به سوالات زیر پاسخ دهید**

برای همه انواع اشیا و ساختارهای داده ای که دربارۀ آنها یاد گرفته ایم یک توصیف کوتاه بنویسید(یا به آرامی به خودتان بگویید) میتوانید با دابل کلیک بر روی این سلول آن را ویرایش کنید در واقع این فقط برای آزمایش است که ببینید آیا تفاوت بین این ها را میدانید یا نه بنابراین احساس راحتی کنید و فقط درباره آن فکر ،کنید زیرا پاسخهای شما ارزیابی خودی است.

دابل کلیک کنید تا این سلول markdown را ویرایش کنید و پاسخهایتان را بنویسید.

اعداد:

رشته ها:

لیست ها:

تاپل ها

دیکشنری ها:

اعداد

یک معادله بنویسید که از عملیات ،ضرب تقسیم ،توان جمع و تفریق استفاده کند و مساوی ۱۰۰/۲۵ باشد.

راهنمایی :این فقط برای آزمایش حافظه ی شما در مورد دستورات حسابی پایه است. از ۱۰۰/۲۵ به سمت عقب کار کنید.

In [ ]:

1	

به این ۳ سوال بدون نوشتن کد پاسخ دهید سپس کد را بنویسید تا پاسخ خود را بررسی کنید.

۱. مقدار عبارت ۴ *(۶ + ۵) چیست؟
۲. مقدار عبارت ۴ * ۶ + ۵ چیست؟
۳. مقدار عبارت ۴ + ۶ * ۵ چیست؟

In [ ]:

```
1
```

نوع نتیجه عبارت ۳+ ۱٫۵ + ۴ چیست؟

از چه چیزی برای یافتن جذر مربع یک عدد و همچنین مربع آن استفاده میکنید؟

In [ ]:

```
1 # Square root:
۲
```

In [ ]:

```
1 # Square root:
۲
```

رشته ها

با توجه به رشته hello یک دستور فهرست (index) را بنویسید که 'e' را برگرداند کد خود را در سلول زیر وارد کنید

In [ ]:

```
1 s ='hello'
۲ # Print out 'e' using indexing
۳
۴
```

با استفاده از دستورات ایندکس گذاری و برش رشته hello را معکوس نمایید:

In [ ]:

```
1 s ='hello'
۲ # Reverse the string using slicing
۳
۴
```

با استفاده از ایندکس گذاری با دو روش متفاوت مقدار " را در رشته 'hello' استخراج نمایید.

In [ ]:

```
s ='hello'
# Print out the 'o'

# Method 1:


```

In [ ]:

```
# Method 2:


```

لیست ها

لیست [۰،۰،۰] را با دو روش متفاوت تولید نمایید:

In [ ]:

```
# Method 1:

```

In [ ]:

```
# Method 2:

```

در لیست تو در توی زیر، مقدار رشته hello را به رشته goodbye تغییر دهید.

In [ ]:

```
list3 [1,2,[3,4, 'hello']]
```

لیست زیر را با دو روش متفاوت مرتب نمایید.

In [ ]:

```
list4 [5,3,4,6,1]
```

دیکشینری ها

با استفاده از کلیدها و فهرست (indexing) ’hello’ , را از دیکشنری های زیر بیابید:

In [ ]:

```
d = {'simple_key' : 'hello'}
# Grab 'hello'
```

In [ ]:

```
d = {'k1':{'k2': 'hello'}}
# Grab 'hello'
```

In [ ]:

```
# Getting a little tricker
d('k':[{'nest_key': ['this is deep', ['hello']]}]}

#Grab hello
```

In [ ]:

```
# This will be hard and annoying!
d = {'k1' :[1,2,{ 'k2' : [ ' this is tricky', 'tough' : [1, 2, [ 'hello']]}]}]}
```

میتوانید یک دیکشنری را مرتب کنید؟ چرا؟ یا چرا نه؟

تاپل ها

بزرگترین تفاوت بین تاپل ها و لیست ها چیست؟

چگونه یک تاپل بسازید؟

مجموعه ها

چه چیز منحصربه فردی دربارهٔ یک مجموعه وجود دارد؟

از یک مجموعه استفاده کنید تا مقادیر یکتای لیست زیر را پیدا کنید:

In [ ]:

```
list5 [1,2,2,33,4,4,11,22,3,3,2]
```

آزمون ارزیابی اشیا و ساختارهای داده

دانش خود را آزمایش کنید.

** به سوالات زیر پاسخ دهید**

برای همه انواع اشیا و ساختارهای داده ای که دربارهٔ آنها یاد گرفته ایم یک توصیف کوتاه بنویسید(یا به آرامی به خودتان بگویید) میتوانید با دابل کلیک بر روی این سلول آن را ویرایش کنید در واقع این فقط برای آزمایش است که ببینید آیا تفاوت بین این ها را میدانید یا نه بنابراین احساس راحتی کنید و فقط درباره آن فکر ،کنید زیرا پاسخهای شما ارزیابی خودی است.

دابل کلیک کنید تا این سلول markdown را ویرایش کنید و پاسخهایتان را بنویسید.

اعداد:

رشته ها:

لیست ها:

تاپل ها

دیکشنری ها:

اعداد

یک معادله بنویسید که از عملیات ،ضرب تقسیم ،توان جمع و تفریق استفاده کند و مساوی ۱۰۰/۲۵ باشد.

راهنمایی :این فقط برای آزمایش حافظه ی شما در مورد دستورات حسابی پایه است. از ۱۰۰/۲۵ به سمت عقب کار کنید.

In [1]:

```
1 ( 60 + (10**2)/4*7) -134.75
```

Out[1]:

100.25

به این ۳ سوال بدون نوشتن کد پاسخ دهید سپس کد را بنویسید تا پاسخ خود را بررسی کنید.

۱. مقدار عبارت ۴ *(۶ + ۵) چیست؟
۲. مقدار عبارت ۴ * ۶ + ۵ چیست؟
۳. مقدار عبارت ۴ + ۶ * ۵ چیست؟

In [2]:

```
1 4*(6+5)
```

Out[2]:

44

In [3]:

```
1 4*6+5
```

Out[3]:

29

In [4]:

```
1 4+6*5
```

Out[4]:

34

In [6]:

```
1 3+1.0+4
```

Out[6]:

8.0

نوع نتیجه عبارت ۳+ ۱٫۵ + ۴ چیست؟

از چه چیزی برای یافتن جذر مربع یک عدد و همچنین مربع آن استفاده میکنید؟

In [7]:

```
1 # Square root:
۲ 100 ** 0.5
```

Out[7]:

10.0

In [8]:

```
# Square root:
10**3
```

Out[8]:

1000

رشته ها

با توجه به رشته hello یک دستور فهرست (index) را بنویسید که 'e' را برگرداند کد خود را در سلول زیر وارد کنید

In[9]:

```
s ='hello'
# Print out 'e' using indexing

S[1]
```

Out[9]:

'e'

با استفاده از دستورات ایندکس گذاری و برش رشته hello را معکوس نمایید:

In [10]:

```
s ='hello'
# Reverse the string using slicing

s[::-1]
```

Out[9]:

'olleh'

با استفاده از ایندکس گذاری با دو روش متفاوت مقدار " را در رشته 'hello' استخراج نمایید.

In [11]:

```
s ='hello'
# Print out the 'o'

# Method 1:

S[4]
```

Out[11]:

'o'

In [12]:

```
# Method 2:

S[-1]
```

Out[12]:

'o'

لیست ها

لیست [۰،۰،۰] را با دو روش متفاوت تولید نمایید:

In [13]:

```
# Method 1:
[0]*3
```

Out[13]:

[0,0,0]

In [15]:

```
# Method 2:
List([0,0,0])
```

Out[15]:

[0,0,0]

در لیست تو در توی زیر، مقدار رشته hello را به رشته goodbye تغییر دهید.

In [18:

```
list3 [1,2,[3,4, 'hello']]

list3[2][2] = 'Goodbye'

list3
```

Out[18]:

[1,2,[3,4, *'Goodbye']]*

لیست زیر را با دو روش متفاوت مرتب نمایید.

In [19]:

```
list4 [5,3,4,6,1]

sorted(list4)

```

Out[19]:

[1,3,4,5,6]

In [20]:

```
1 list4
```

Out[20]:

[5,3,4,6,1]

In [21]:

```
1 list4 .s()
2
3 list4
```

Out[21]:

[1,3,4,5,6]

دیکشینری ها

با استفاده از کلیدها و فهرست (indexing) ,'hello' را از دیکشنری های زیر بیابید:

In [22]:

```
1 d = {'simple_key' : 'hello'}
٢ # Grab 'hello'
٣ d['simple_key']
```

Out[22]:

'hello'

In [24]:

```
1 d = {'k1':{'k2': 'hello'}}
٢ # Grab 'hello'
٣
4 d['k']['k2']
```

Out[24]:

'hello'

In [29]:

```
# Getting a little tricker
d('k':[{'nest_key': ['this is deep', ['hello']]}]}

#Grab hello
d['k'][0]['nest_key'][1][0]
```

Out[29]:

'hello'

In [36]:

```
# This will be hard and annoying!
d = {'k1' :[1,2,{ 'k2' : [ ' this is tricky', 'tough' : [1, 2, [ 'hello']]}]}]}

d['k'][2]['k2'][1]['tough'][2][0]
```

Out[29]:

'hello'

میتوانید یک دیکشنری را مرتب کنید؟ چرا؟ یا چرا نه؟

تاپل ها

بزرگترین تفاوت بین تاپل ها و لیست ها چیست؟

چگونه یک تاپل بسازید؟

مجموعه ها

چه چیز منحصربه فردی دربارهٔ یک مجموعه وجود دارد؟

از یک مجموعه استفاده کنید تا مقادیر یکتای لیست زیر را پیدا کنید:

In [37]:

```
1  list5 [1,2,2,33,4,4,11,22,3,3,2]
2
3  x = set(list5)
4
5  list(x)
6
```

Out[37]:

[1,2,33,4,3,11,22]

عملگرهای مقایسه

در این درس دربارهٔ عملگرهای مقایسه در پایتون آموزش خواهیم دید این عملگرها به ما امکان میدهند متغیرها را مقایسه کرده و یک مقدار منطقی (True) یا (False) را خروجی دهند.

اگر شما تا حدی تجربه در ریاضیات داشته باشید این عملگرها بسیار ساده خواهند بود.

ابتدا جدولی از عملگرهای مقایسه را ارائه میدهیم و سپس برخی مثالها را بررسی خواهیم کرد:

جدول عملگرهای مقایسه

در جدول زیر ۳ =aو 4=bاست

عملگر	توضیحات	مثال
==	اگر مقادیر دو عملوند برابر باشند شرط صحیح میشود	(a==b)درست نیست
=!	**اگر مقادیر دو عملوند برابر ،نباشند شرط صحیح میشود**	(a!=b)درست است
<	اگر مقدار عملوند چپ از مقدار عملوند راست بزرگتر باشد شرط صحیح میشود	(a<b)درست است
>	اگر مقدار عملوند چپ از مقدار عملوند راست کمتر باشد شرط صحیح میشود	(a>b)درست نیست
<=	اگر مقدار عملوند چپ از مقدار عملوند راست بزرگتر یا مساوی باشد شرط صحیح میشود	(a<=b)درست است
>=	اگر مقدار عملوند چپ از مقدار عملوند راست کوچکتر یا مساوی باشد شرط صحیح میشود	(a>=b)درست نیست
is	اگر علاوه بر برابر بودن مقدار عملوند چپ و عملوند آدرس ذخیره سازی مقادیر نیز با هم برابر باشد شرط صحیح می شود.	a = [1, , 3] b = a درست است (a is b)
in	اگر مقدار عملوند چپ داخل دنباله رشته ،لیست دیکشینری تاپل (مجموعه سمت راست ،باشد شرط صحیح میشود.	a = [1, 2, 3] b = 3 درست است (b in a)

.

بیایید چند مثال با هم ببینیم

Equal برابری:

In [۱]:

```
1 ۲==۲
```

Out[1]

True

In [2]:

```
1 1==0
```

Out[2]

False

بررسی برابری در اعداد اعشاری

In [1]:

```
1 x = 0.1 + 0.2
2
3 x =  0.3
```

Out[1]

False

In [2]:

```
1 x
```

Out[2]

۰/۳۰۰۰۰۰۰۰۰۰۰۰۰۰۰۴

عجب !!! دقت داشته باشید که محاسبات اعشاری در تمام زبانهای برنامه نویسی به دلیل تبدیل خطای تبدیل اعداد اعشاری حقیقی به مبنای دو دارای خطا خواهند بود راه صحیح مقایسه برابر بودن اعداد اعشاری بصورت زیر می باشد.

In [6]:

```
1 tolerance = 0.001
2
3 x = 0.1 + 0.2
4
5 abs(x - 0.3) < tolerance
```

Out[6]

True

توجه کنید که == یک عملگر مقایسه است در حالی که = یک عملگر تخصیص است.

Not Equal نابرابری

In [5]:

```
1 | 2 != 1
```

Out[5]:

True

In [6]:

```
1 | 2 != 2
```

Out[6]:

False

Greater Than بزرگتر

In [7]:

```
1 | 2 > 1
```

Out[7]:

True

In [8]:

```
1 | 2 > 4
```

Out[8]:

False

Less Than کوچکتر

In [9]:

```
1 2 < 4
```

Out[9]:

True

In [10]:

```
1 2 < 1
```

Out[10]:

False

Greater Than or Equal to بزرگتر مساوی

In [11]:

```
1 2 >= 2
```

Out[11]:

True

In [12]:

```
1 2 >= 1
```

Out[12]:

True

Less than or Equal to کوچکتر مساوی

In [13]:

```
1 2 <= 2
```

Out[13]:

True

In [14]:

```
1 2 <= 4
```

Out[14]:

True

عملگر عضویت در یک دنباله in

In [15]:

```
1 I = [1,3,5,7,9]
```

In [16]:

```
1 1 in I
```

Out[16]:

True

In [17]:

```
1 2 in I
```

Out[17]:

False

In [18]:

```
1 2 not in I
```

Out[18]:

True

عملگر یکسان بودن is

In [19]:

```
1 I = [1,2,3,4]
2
3 I1= [1,2,3,4]
4
5 I2 = I
```

In [20]:

```
1 Id(I)
```

Out[20]:

2797747660160

In [21]:

```
1 Id(I1)
```

Out[21]:

۲۷۹۷۷۴۶۷۱۴۲۴۰

In [22]:

```
1 Id(I2)
```

Out[22]:

۲۷۹۷۷۴۷۶۶۰۱۶۰

In [23]:

```
1 I is I2
```

Out[23]:

True

In [24]:

```
1 I is I1
```

Out[24]:

False

In [25]:

```
1 I is  not I1
```

Out[25]:

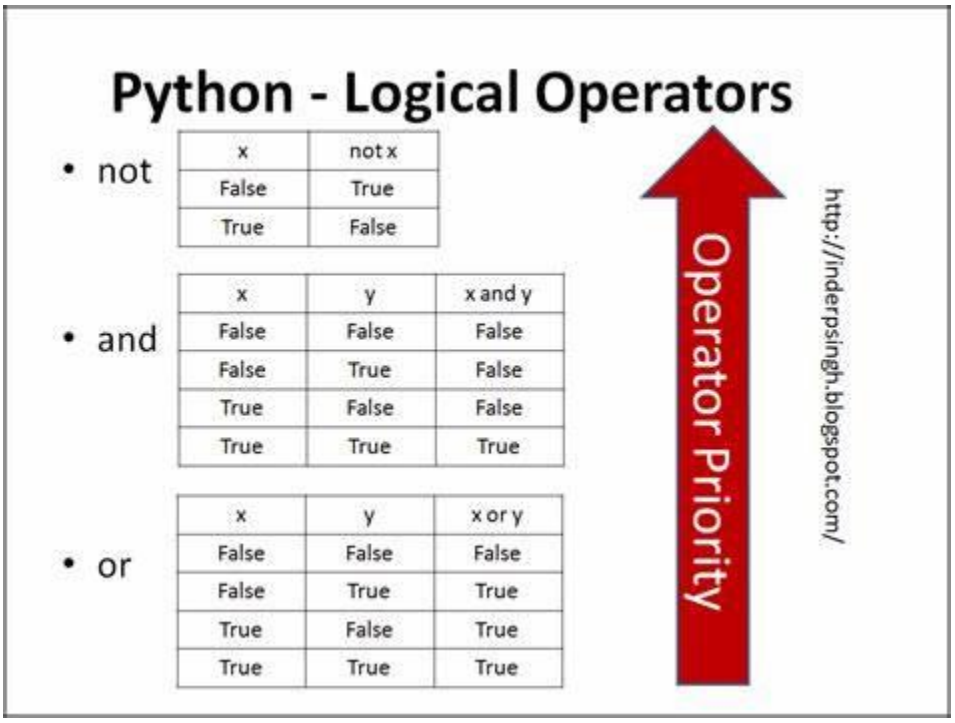
True

در ادامه درباره عملگرهای منطقی و نجیره سازی عملگرهای مقایسه ای یاد میگیریم!

عملگرهای منطقی

عملگرهای منطقی برای ترکیب جملاتی منطقی) جملاتی که نتیجه آنها True یا False (باشد بکار گرفته میشوند.

عملگر	مثال	معنی
not	not x	True اگر X یک مقدار False باشد False اگر x یک مقدار True باشد (بصورت منطقی مقدار x را برعکس میکند.)
or	x or y	Trueاگر هر کدام از x or yمقدار True را داشته باشند و در غیر اینصورت Fals
and	x and y	Trueاگر هر دوی x وy مقدار True را داشته باشند و در غیر اینصورت False



In [1]:

```
1 # not operator example
2 not True
```

Out[1]:

False

In [2]:

```
1 not False
```

Out[2]:

True

In [3]:

```
1 1 > 3
```

Out[3]:

False

In [4]:

```
1 not 1 > 3
```

Out[4]:

True

In [5]:

```
1 and operator exmaple
2
3 1<2 and 2 == 2
```

Out[5]:

True

In [6]:

```
1 2> 5 and 100 != 5
```

Out[6]:

False

In [7]:

```
1 # or operator example
2
3 2 > 5 or 100 != 5
```

Out[7]:

True

In [8]:

```
1 2 > 5 or 100 != 100
```

Out[8]:

False

مقادیرfalsy and truthy

بعضی از مقادیر وجود دارند که زمان تفسیر به عبارت بولی معادل True یا False در نظر گرفته میشوند. به این مقادیر دراصطلاح Trurhy و Falsy گفته می شود.

مقادیر Falsy در :پایتون ", 0,[],{},(),None

سایر مقادیر معادل True ارزیابی میشوند.

In [9]:

```
1 bool(0)
```

Out[9]:

False

In [10]:

```
1 bool('')
```

Out[10]:

False

In [11]:

```
1 bool([])
```

Out[11]:

False

In [12]:

```
1 bool(())
```

Out[12]:

False

In [13]:

```
1 bool({})
```

Out[13]:

False

In [14]:

```
1 bool(None)
```

Out[14]:

False

In [15]:

```
1 bool(55)
```

Out[15]:

True

In [17]:

```
1 bool('hello')
```

Out[17]:

True

In [18]:

```
1 bool([1,2])
```

Out[18]:

True

اتصال کوتاه

در اتصال کوتاه سعی بر آن است که در صورتیکه پاسخ جملات ترکیبی شرطی زودتر از به انتها رسیدن ارزیابی تمام جملات ممکن باشد سریعتر نتیجه ارزیابی را بدست آوریم تا در مصرف منابع سیستم صرف جویی نماییم ارزیابی عبارت از چپ به راست صورت میگیرد در پایتون کوتاه کردن مدار توسط چندین عملگر و تابع بولین پشتیبانی می شود.

In [19]:

```
1 # short-circuiting on 'or' operator
2
3 2 < 0 or 2 or 5 > -5 or True
```

Out[19]:

2

In [20]:

```
1 2 < 0 or 0 or 5 > -5 or True
```

Out[20]:

True

In [21]:

```
1 2 < 0 or 0 or 5 < -5 or []
```

Out[21]:

[]

In [ ]:

```
1 # short-circuiting on 'and' operator
```

In [22]:

```
1 5<17 and 4 > 8 and True
```

Out[22]:

False

In [23]:

```
1 5<17 and 4 > 8 and True
```

Out[23]:

0

In [23]:

```
1 5<17 and 4< 8 and 'Hello'
```

Out[23]:

' Hello'

یکی از مهمترین کاربردهای اتصال کوتاه در تعیین مقدار پیش فرض متغیرها می باشد

In [27]:

```
1 x = input('Enter a name:') or 'Empty'
```

Out[27]:

Enter a name: asddsa

In [28]:

```
1 x
```

Out[28]:

'asddsa'

زنجیره سازی عملگرهای مقایسه ای

In [29]:

```
1 1<2<3
```

Out[29]:

True

عبارت بالا مشابه حالتی است که جملات را با and به یکدیگر متصل می نماییم

In [30]:

```
1 1<2 and 2<3
```

Out[30]:

True

In [31]:

```
1 1 < 3 > 2
```

Out[31]:

True

In [32]:

```
1 | 1 < 3  and 3> 2
```

Out[32]:

True

مقدمه ای بر دستورات پایتون

در این سخنرانی ما به مرور سریعی بر دستورات پایتون خواهیم پرداخت. این سخنرانی تاکید بر تفاوت های پایتون با زبان های دیگر مانند C++ خواهد داشت.

دو دلیل وجود دارد که ما از این روش برای یادگیری زمینه دستورات پایتون استفاده می کنیم:

۱. اگر شما از یک زبان دیگر می آیید این روش سرعت درک شما از پایتون را به طور قابل توجهی افزایش خواهد داد
۲. یادگیری درباره دستورات به شما اجازه خواهد داد تا در آینده به راحتی زبانهای دیگر را بخوانید.

پایتون در مقابل زبان های دیگر

بیایید یک دستور ساده ایجاد کنیم که بگوید : " اگر a از b بزرگتر است ، ۲ را به a و ۴ را به b اختصاص دهید " به این دو دستور if نگاه کنید (ما به زودی درباره ساخت if statements یاد خواهیم گرفت.)

نسخه ۱ (زبان های دیگر)

```
if (a>b){
a = 2;
b = 4;
}
```

نسخه ۲ (پایتون)

```
if a>b:
۲ = 2
b = 4
```

شما متوجه خواهید شد که پایتون کمتر شلوغ و خواناتر از نسخه اول است پایتون چگونه این کار را می کند؟ بیایید از طریق تفاوت های اصلی قدم بزنیم

پایتون با استفاده از دو عامل اصلی کولن و فضای سفید ، () و {} را حذف میکند دستور با یک کولن به پایان می رسد و فضای سفید (تورفتگی)برای توصیف آنچه در صورت بیانیه اتفاق میافتد استفاده می شود.

تفاوت دیگر عمده عدم وجود نقطه ویرگول در پایتون است. نقطه ویرگول برای نشان دادن پایان دستورات در بسیاری از زبان های دیگر استفاده میشود اما در پایتون پایان یک خط همان پایان یک دستور است.

در نهایت برای پایان دادن به این مرور کوتاه از تفاوت ها بیایید به نحوه تورفتگی در پایتون در مقابل زبان های دیگر نگاه دقیق تری داشته باشیم

تورفتگی

در اینجا کد pseudo-code برای نشان دادن استفاده از فضای سفید و تورفتگی در پایتون وجود دارد:

زبان های دیگر

```
if (x)
    if(y)
        code-statement;
else
    another-code-statement;
```

پایتون

```
if x:
    If y :
        code-statement;
else:
    another-code-statement;
```

توجه داشته باشید که پایتون به طور قوی توسط تورفتگی کد و فضای سفید محرک می شود این بدان معنی است که خوانایی کد بخش اصلی طراحی زبان پایتون است.

حالا بیایید با کدنویسی این نوع دستورات در پایتون عمیق تر شویم

بریم کد بزنیم

دستوراتif elif else

در پایتون به ما اجازه میدهد تا به کامپیوتر بگوییم که بر اساس یک مجموعه نتایج خاص if دستورات اقدامات جایگزینی را انجام دهد.

به زبان ساده می توانیم تصور کنیم که به کامپیوتر می گوییم

"سلام اگر این مورد رخ داد برخی از اقدامات را انجام دهید"

ما می توانیم ایده را با استفاده از دستورات elif و else گسترش دهیم که به ما اجازه میدهد تا به کامپیوتر بگوییم:

"سلام اگر این مورد رخ داد برخی از اقدامات را انجام دهید در غیر این صورت اگر مورد دیگری رخ داد، برخی از اقدامات دیگر را انجام دهید در غیر این صورت اگر هیچ یک از موارد فوق رخ ،نداد این عمل را انجام دهید."

بیایید به سینتکس فرمت برای دستورات if نگاه کنیم تا درک بهتری از این موضوع پیدا کنیم:

```
if case1:
    perform action1
elif case2:
    perform action2
else:
    perform action3
```

اولین مثال

بیایید باهم یک مثال ساده را باهم بررسی کنیم

In [۴]:

```
1  x = 5
۲
۳  if x < -3.14:
۴      print(f'x is has value of {x} which is greater than -3.14')
۵
۶  print('outside')
```

x is has value of -5 which is greater than -3.14

outside

حالا بیایید تا یک جمله else را اضافه کنیم

In [۶]:

```
1  x = -5
۲
۳  if x < -3.14:
۴      print(f'x is has value of {x} which is greater than -3.14')
۵  else:
۶      print(fx is not less than -3.14')
۷
۸  print('outside')
```

x is has value of -5 which is greater than -3.14

outside

شاخه های چندگانه

بیایید تصویر کامل تری از اینکه چگونه if ، elifو else میتواند ما را هدایت کند، بدست آوریم!
ما این را در یک ساختار تو در تو مینویسیم توجه داشته باشید که چگونه خطوط if ، elifو else در کد مرتب می شوند این میتواند به شما کمک کند تا ببینید کدام if به کدام elif یا دستورات else مرتبط است.

ما دوباره سینتکس مقایسه ای برای پایتون را معرفی خواهیم کرد.

In [۹]:

```
1  x = -1
۲
۳  if x < -3.14:
۴      print(f'x is has value of {x} which is greater than -3.14')
۵  elif x >= 0:
۶      print (f'x is greater than or equal to ')
۷  else:
۸      print(f'x is not less than -3.14')
۹
۱۰ print('outside')
```

x is not less than -3.14

outside

In [۱۰]:

```
1  x = -1
۲
۳  if x < -3.14:
۴      print(f'x is has value of {x} which is greater than -3.14')
۵  elif x >= 0:
۶      print (f'x is greater than or equal to ۰')
۷  elif x >= -3.14 and x < 0 :
۸      print (f'x is between -3.14 and 0')
۹  else:
۱۰     print(f'x is not less than -3.14')
11
12 print('outside')
```

x is between -3.14 and 0

outside

In [ ]:

```
x = -1

if x < -3.14:
    print(f'x is has value of {x} which is greater than -3.14')
if x >= 0:
    print (f'x is greater than or equal to 0')
if x = -3.14 and x < 0:
    print(f'x is between -3.14 and 0')
else:
    print (f'x is not less than -3.14')

print('outside')
```

توجه داشته باشید که چگونه دستورات if تو در تو هر کدام تا زمانی که یک بولین True باعث اجرای کد زیر آن می شود بررسی میشوند. شما همچنین باید توجه داشته باشید که میتوانید تعداد دلخواهی از دستورات elift را قبل از بستن با یک else قرار دهید.

بیایید دو مثال ساده دیگر برای دستورات if ، elifو else ایجاد کنیم:

In [11]:

```
person = 'Sammy'

if person == 'Sammy':
    print('Welcome Sammy!')
else:
    print("Welcome, what's your name?")
```

Welcome Sammy!

In [13]:

```
person = 'george'

if person == 'Sammy':
    print('Welcome Sammy!')
elif person == 'George':
    print('Welcome George!')
else:
    print("Welcome, what's your name?")
```

تورفتگی

مهم است که درک خوبی از نحوه کار تورفتگی در پایتون داشته باشید تا ساختار و نظم کد خود را حفظ کنید. ما دوباره درباره این موضوع زمانی که شروع به ساخت توابع میکنیم صحبت خواهیم کرد!

ساختار match-case

In [16]:

```
Command = 'Hello, World!'
match command:
    case 'Hello, World!':
        print('Hello to you too!') case 'Goodbye, World!':
    case 'Goodbye, world!':
        print('See you later') case other:
    case other:
        print('No match found')
```

Hello to you too!

حلقه های while

دستور while در پایتون یکی از راههای عمومی برای انجام تکرار .است یک دستور while به طور مکرر یک دستور واحد یا گروهی از دستورات را تا زمانی که شرط درست باشد اجرا می.کند دلیل اینکه آن را حلقه مینامند این است که دستورات کد به صورت مکرر از طریق آنها تکرار میشوند تا زمانی که شرط دیگر برآورده نشود

قالب عمومی یک حلقه while به شرح زیر است:

```
while test:
    code statements
else:
    final code statements
```

بیایید به چند حلقه while ساده در عمل نگاه کنیم

In [1]:

```
x = 0

while x < 10:
    print(x)
    X = x + 1 # x += 1

print('Outside of While loop')
```

```
0
1
2
3
4
5
6
7
```

8

9

Outside of While loop

توجه کنید که چند بار دستورات چاپ اتفاق افتاده و حلقه while ادامه داشت تا زمانی که شرط درست برآورده شد. مهم است که توجه داشته باشید که هنگامی که این اتفاق افتاد کد متوقف شد بیایید ببینیم چگونه میتوانیم یک دستورelse اضافه کنیم:

In [4]:

```
1  x = 10
2
3  while x < 10:
4      print(x)
5      X = x + 1      # x += 1
6  else:
7      print('All done as expected!')
8
9  print('Outside of While loop')
```

All done as expected!

Outside of While loop.

break. continue. Pass

ما میتوانیم از دستورات break ، continueو pass در حلقه های خود استفاده کنیم تا قابلیتهای اضافی برای موارد مختلف اضافه کنیم ، سه دستور تعریف شده توسط:

break: این دستور اجرای نزدیکترین حلقه را متوقف می کند.

Continue :این دستور اجرای دور کنونی حلقه را نیمه کاره رها کرده و به ابتدای دور بعدی خواهد رفت.

Pass :هیچ کاری انجام نمیدهد

درباره دستورات break و continue ، قالب عمومی حلقه while به این شکل است

```
while test:
    code statement
if test:
    break
if test2:
    continue
else:
```

دستورات break و continue میتوانند در هر جایی از بدنه حلقه ظاهر شوند اما معمولاً آنها را در داخل یک دستور if قرار میدهیم تا بر اساس شرایط خاصی عملی را انجام دهیم.

بیایید پیش بروید و به چند مثال نگاه کنید:

In [۱۲]:

```
x = 0

while x < 10:
        if x % 2 == 0:
            print('Continue')
            x = x + 1
            continue
         print(x)
         X = X + 1 # x += 1
else:
        print('All done as expected!")

print('Outside of While loop')
```

Continue

1

Continue

3

Continue

5

Continue

7

Continue

9

All done as expected!

Outside of While loop

توجه کنید که چگونه ما یک دستور چاپی داریم وقتی x==۳ ، و یک continue در حالی که از طریق حلقه while خارجی ادامه میدهیم چاپ میشود بیایید یک break را هنگامی که X==3 قرار دهیم و ببینیم آیا نتیجه منطقی است:

In [۱۰]:

```
1   x = 0
۲
۳   while x < 10:
۴       print('x is currently: ",x)
۵      print(x is still less than 10, adding 1 to x')
۶     X+=1
7    if x==3:
8        print('Breaking because x==3')
9       break
۱۰   else:
۱۱      print('continuing...")
۱۲      continue
```

x is currently:۰

x is still less than 10, adding 1 to x

continuing...

x is currently:

x is still less than 10, adding 1 to x continuing...

continuing...

x is currently:

x is still less than 10, adding 1 to x

Breaking because x==3

نکته مهم درباره کاربرد همزمان دستورات break با وجود عبارت else برای حلقه

توجه داشته باشید که دستورات داخل else زمانی اجرا خواهند شد که حلقه بصورت طبیعی به اتمام برسد و در صورتیکه حلقه با دستور break متوقف شود دیگر دستورات else اجرا نخواهند شد!

In [۱۱]:

```
1  x = 0
۲
۳  while x < 10:
۴      print('x is currently: ',x)
۵      X+=1
۶      if x==3:
7          print('Breaking because x==3')
8          break
9  else:
۱۰     print('The while loop has been ended naturally')
۱۱
۱۲ print('outside loop')
```

x is currently:۰

x is currently:۱

x is currently:۲

```
Breaking because x==3

outside loop
```

توجه کنید که دستور else دیگری به آن نرسیده و continue هرگز چاپ نشده است!

پس از این مثال های ساده اما مختصر شما باید در استفاده از دستورات while در کد خود راحت باشید.

اما یک کلمه احتیاط! امکان دارد که با دستورات while یک حلقه بی پایان در حال اجرا ایجاد کنید به عنوان مثال

In [ ]:

```
# DO NOT RUN THIS CODE!!!!
while True:
    print("I'm stuck in an infinite loop!")
```

یک یادداشت سریع: اگر شما سلول بالا را اجرا کردید روی منوی Kernel بالا کلیک کنید تا Kernel را مجدداً راه اندازی کنید!

یک مثال دیگر

در مثالهایی که تاکنون بیان کردیم تعداد تکرارهای حلقه از پیش قابل تعیین بود اما با حلقه while می توان حلقه هایی را نوشت که تعداد تکرارها از پیش مشخص نباشد.

برای مثال اگر منتظر ورودی خاصی از طرف کاربر باشیم و تا زمانی که کاربر ورودی مورد نظر را وارد نکرده است به او اجازه ندهیم به بخش دیگری مراجعه نماید.

In [۱۶]:

```
x = 1

while x != '5':
    X = input('Please enter number FIVE in the inpute:') 5 else:
else:
    print('you have entered number FIVE')
```

```
Please enter number FIVE in the inpute:0

Please enter number FIVE in the inpute:8

Please enter number FIVE in the inpute:4
```

Please enter number FIVE in the inpute:5

you have entered number FIVE

حلقه های for

یک حلقه for به عنوان یک تکرار کننده در پایتون عمل میکند؛ این به مواردی که در یک دنباله یا هر مورد قابل تکرار دیگری هستند میرود اشیاء ای که ما درباره آنها یاد گرفته ایم که میتوانیم روی آنها تکرار کنیم شامل رشته ها لیست ها، تاپل ها و حتی تکرار کننده های داخلی برای دیکشنری ها مانند کلیدها یا مقادیر است.

ما قبلاً دستور For را در سخنرانی های گذشته کمی دیده ایم اما حالا بیایید درک خود را رسمی کنیم. در اینجا فرمت عمومی برای یک حلقه for در پایتون است

for item in object:

    statements to do stuff

اسم متغیر استفاده شده برای مورد کاملاً به برنامه نویس بستگی دارد بنابراین از قضاوت خود برای انتخاب نامی استفاده کنید که منطقی باشد و شما قادر به درک آن باشید وقتی به کد خود باز میگردید این نام مورد سپس میتواند در داخل حلقه شما ارجاع داده شود به عنوان مثال اگر میخواستید از دستورات if برای انجام بررسی ها استفاده کنید. بزارید پیش بروید و چندین نمونه از حلقه های for را با استفاده از انواع مختلف داده های شئ بسط دهید. ما ساده شروع خواهیم کرد و بعداً پیچیدگی را افزایش خواهیم داد.

مثال ۱

تکرار در لیست

In [۳]:

```
# We'll Learn how to automate this sort of list in the next Lecture
list1= [1,2,3,4,5,6,7,8,9,10]
```

In [۷]:

```
1 for num in list1:
2     print(num)
```

۱

۲

۳

۴

۵

۶

۷

۸

۹

۱۰

تکرار شونده ها یا itrators

In [8]:

```
1 X = iter ([1,2,3,4,5])
```

In [9]:

```
1 x
```

Out[9]:

<list_iterator at 0x22c6bf7ae30>

In [10]:

```
1 type(x)
```

Out[10]:

list_iterator

In [11]:

```
1 next(x)
```

Out[11]:

1

In [12]:

```
1 next(x)
```

Out[12]:

2

In [13]:

```
1 next(x)
```

Out[13]:

3

In [14]:

```
1 Y = iter('hi')
```

Out[14]:

In [15]:

```
1 next(y)
```

Out[15]:

'h'

In [16]:

1	*next(y)*

Out[16]:

'i'

In [17]:

1	*next(y)*

StopIteration

Traceback (most recent call last)

Cell In[17], line 1

-----> 1 next(y)

StopIteration:

In [18]:

1	*z = iter({'k1: 100, 'k2': 200})*

In [19]:

1	*next(z)*

Out[19]:

'k1'

In [20]:

1	*next(z)*

Out[20]:

'k2'

برگردیم به حلقه های for !

مثال ۲

بیایید فقط اعداد زوج از آن لیست را چاپ کنیم !

In [21]:

```
1 list1
```

Out[21]:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

In [22]:

```
1 for num in list1:
2     if num % 2 == 0:
3         print(num)
```

Out[22]:

2

4

6

8

10

می توانیم برای اعداد فرد هم از عبارت else استفاده کنیم

In [23]:

```
1 for num in list1:
2     if num % 2 == 0:
3         print(num)
4 else:
5     print('Odd Number')
```

Out[23]:

Odd Number

Odd Number

۴

Odd Number

۶

Odd Number

۸

Odd Number

۱۰

مثال ۳

یک ایده دیگر متداول در حلقه for نگهداری یک مقدار در حین تکرارهای یک حلقه .است به عنوان مثال بیایید یک حلقهfor ایجاد کنیم که مجموع لیست را بدست آورد:

In [۲۶]:

```
1  # Start sum at zero
2  list_sum = 0
3
4  for num in list1:
5      list_sum = list_sum + num
۶
۷  print(list_sum)
```

عالی بالای سلول را بخوانید و مطمئن شوید که کاملاً متوجه هستید که چه اتفاقی میافتد همچنین ما میتوانستیم از عملگر =+ را برای انجام جمع به سمت مجموع پیاده سازی کنیم. به عنوان مثال:

In [۲۷]:

```
1  # Start sum at zero
```

```
2 list_sum = 0
3
4 for num in list1:
5     list_sum += num
۶
۷ print(list_sun)
```

۵۵

مثال ۴

ما از حلقه for با لیستها استفاده کردیم چطور با رشته ها؟ به یاد داشته باشید که رشته ها یک دنباله هستند بنابراین وقتی از آنها تکرار میکنیم به هر مورد در آن رشته دسترسی خواهیم داشت.

In [۲۷]:

```
1 for letter in 'This is a string' :
2     print(letter)
```

T

h

i

s

i

s

a

s

t

r

i

n

g

مثال ۵

حالا به این نگاه کنیم که چگونه یک حلقه for میتواند با یک tuple استفاده شود

In [29]:

```
1  tup = (1,2,3,4,5)
2
3  fort in tup:
4      print(t**2)
```

1

4

9

16

25

مثال ۶

تاپل ها یک کیفیت خاص در مورد حلقه های for دارند. اگر در حال تکرار از طریق یک دنباله که شامل تاپل ها باشید، هر عضو میتواند خود تاپل ،باشد این یک مثال از. Tuple Packing است. در طول حلقه forما در حال باز کردن تاپل درون یک دنباله هستیم و میتوانیم به موارد فردی درون آن تاپل دسترسی پیدا کنیم!

In [۳۶]:

```
1  # Now with packing!
2  x = 1, 2, 3
```

In [۳۱]:

```
1 # Now with packing!
2 x = 1, 2, 3
```

Out[31]:

(1, 2, 3)

In [۳2]:

```
1 type(x)
```

Out[32]:

tuple

In [۳3]:

```
1 x = (1,)
```

In [۳4]:

```
1 type(x)
```

Out[34]:

Tuple

In [۳7]:

```
1 x = 1, 2, 3
2 x1, x2, x3 = x
```

In [۳8]:

```
1 x1
```

Out[38]:

1

In [۳9]:

```
1 X3
```

Out[39]:

3

In [41]:

```
1 list2 = [(2,4), (6,8), (10,12)]
```

In [42]:

```
1 for tup in list2:
2     print(tup)
```

(2, 4)
(6, 8)

(10,12)

In [43]:

```
1 # Now with unpacking in for Loop!
2 for t1, t2 in list2:
3     print(t1, t2)
```

2 4
6 8
10 12

عالی با تاپل ها در یک دنباله میتوانیم به موارد درون آنها از طریق باز کردن دسترسی پیدا کنیم دلیل اینکه این مهم است این است که بسیاری از اشیاء قابل تکرار خود را از طریق تاپل ها تحویل میدهند بیایید شروع به کاوش در تکرار از طریق دیکشنری ها برای بررسی بیشتر این موضوع کنیم!

مثال ۷

همانگونه که در ساخت تکرار شونده ها مشاهده کردیم در زمان ساخت تکرار شونده از یک دیکشنری تنها به کلیدها دسترسی خواهیم داشت حالا با هم مثالهایی از حالات مختلف برای دیکشینری ها حل میکنیم

In [۴]:

```
1 d = {'k1': 100, 'k2' : 200, 'k3' : 300}
```

In [۵]:

```
1 for item in d:
۲     print(item)
```

k1

k2

k3

In [46]:

```
1 for item in d:
۲ print(iten, d[item])
```

k1 100

k2 200
k3 300

توجه کنید که این فقط کلیدها را تولید می.کند پس چگونه میتوانیم مقادیر را بدست آوریم؟ یا هر دو کلید و مقدار؟ ما در مورد متدهای keys() و values() و items() قبلا یاد گرفتیم و در این بخش در حلقه های forاز این متدها استفاده خواهیم کرد.

در پایتون هر یک از این روش ها یک dictionary view object برمیگرداند این عملیات مانند آزمایش عضویت و تکرار را پشتبیانی می.کند اما محتوای آن مستقل از دیکشنری اصلی نیست فقط یک نمایش است. آن را در عمل ببینید:

In [۴۷]:

```
1 # Create a dictionary view object
۲ d.items()
```

Out[47]:

dict items([('k1', 108), ('k2', 200), ('k3', 300)])

از آنجا که روش items() تکرار را پشتیبانی میکند ما میتوانیم بسته بندی دیکشنری را انجام دهیم تا کلیدها و مقادیر را جدا کنیم همانطور که در مثالهای قبلی انجام دادیم.

In [48]:

```
1 # Dictionary unpacking
2 for k, v in d.items():
3 print(k, v)
```

k1 100

k2 200

k3 300

اگر میخواهید یک لیست واقعی از کلیدها مقادیر یا تاپل های کلید مقدار بدست آورید میتوانید نمایش را به عنوان یک لیست cast کنید

In [49]:

```
1 list(d.keys())
```

Out[49]:

['k1', 'k2', 'k3']

به یاد داشته باشید که دیکشتری ها بدون ترتیب هستند و کلیدها و مقادیر به ترتیب دلخواه بر میگردند.
میتوانید یک لیست مرتب شده با استفاده از sorted() بدست آورید:

In [50]:

```
1 sorted(d.values())
```

Out[50]:

[100, 200, 300]

عبارت else در حلقهfor

همانگونه که در حلقه while مشاهده نمودیم پایتون به امکان میدهد تا در حلقه ها از عبارت else پس از حلقه استفاده نماییم. همچنین دقت داشته باشید که دستورات این عبارت تنها زمانی اجرا خواهند شد که حلقه بصورت طبیعی به اتمام برسد !

In [5۱]:

```
1  tup (1,2,3,4,5)
۲
۳  for t in tup:
۴      print(t)
۵  else:
۶      print('the for loop ended')
۷
۸  print('outside')
```

Out[5۱]:

۱

۲

۳

۴

۵

the for loop ended

outside

In [5١]:

```
1  tup  = (1,2,3,4,5)
٢
٣  for t in tup:
٤      print(t)
٥      if(t == 3):
٦          break
٧  else:
٨      print('the for loop ended')
9
10 print('outside')
```

1

2

3

Outside

تابعrange

در اکثر موارد در حلقه for امکان تعریف یک دنباله بصورت دستی فراهم نیست. برای مثال اگر نیاز باشد تا اعداد زوج در بازه ۱ تا ۱۰۰۰۰۰۰۰ را بدست اوریم، در این حالت استفاده از یک لیست یا دنباله که بصورت دستی نوشته شده است به دو دلیل تقریبا غیر ممکن است:

١. نوشتن دستی اعداد این بازه واقعا خسته کننده و عملا غیر ممکن است.
٢. حتی اگر بصورت دستی هم چنین کاری انجام دهید این تعداد داده به فضای بسیار زیادی در ramرایانه نیاز خواهند داشت.

از این رو از تابع range استفاده می نماییم این تابع بصورت خودکار یک تکرار شونده ایجاد میکند و در هر مرحله تنها یک عضو را برای ما تولید میکند و همچنین در تکرار بعدی عضو بعدی را تولید خواهد کرد در نتیجه در میزان مصرف رم بشدت صرفه جویی خواهد نمود.

range(start, stop, step)

In [53]:

```
1  Range(10)
```

Out[53]:

range(0,10)

In [54]:

```
1 # cast to list
2 list(range(10))
```

Out[54]:

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

In [58]:

```
1 for i in range(1, 100) :
2     if i % 20 == 0:
3         print(i)
```

20

40

60

80

نتیجه گیری

ما یاد گرفتیم که چگونه از حلقه های for برای تکرار از طریق تاپل ها لیست ها رشته ها و دیکشنری ها استفاده کنیم. این یک ابزار مهم برای ما خواهد بود بنابراین مطمئن شوید که آن را خوب میدانید و مثالهای فوق را درک کرده اید.

عملگرهای مفید

چند تابع و "عملگر" در پایتون وجود دارد که به خوبی در هیچ یک از دسته بندی ها جا نمیگیرند بنابراین ما در این سخنرانی به آنها میپردازیم بیایید شروع کنیم

range

تابع range به شما اجازه میدهد تا به سرعت یک لیست از اعداد صحیح را تولید کنید این خیلی کاربردی است، بنابراین توجه کنید که چگونه از آن استفاده کنید سه پارامتر وجود دارد که میتوانید منتقل کنید یک شروع یک توقف و یک اندازه گام بیایید چند مثال را ببینیم

In [۱]:

```
1 range(0,11)
```

Out[1]:

range(0,11)

توجه داشته باشید که این یک تابع generator است بنابراین برای دریافت یک لیست از آن ما نیاز داریم که آن را با list() به یک لیست تبدیل کنیم چه چیزی یک تولید کننده است؟ این یک نوع خاص از تابع است که اطلاعات را تولید میکند و نیازی به ذخیره آن در حافظه ندارد ما هنوز درباره توابع یا تولید کننده ها صحبت نکرده ایم بنابراین فعلاً این را در یادداشت های خود نگه دارید ما در جزئیات بسیار بیشتری در آینده در آموزش شما به این موضوع خواهیم پرداخت!

In [3]:

```
1 Notice how 11 is not included, up to but not including 11, just like slice notation
2 list(range(0,11))
```

Out[3]:

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

In [4]:

```
1 list(range(0,12))
```

Out[4]:

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11]

In [6]:

```
1 # Third parameter is step size!
2 # step size just means how big of a jump/leap/step you
3 # take from the starting number to get to the next number.
4
5 list(range(0,11,2))
```

Out[6]:

[0, 2, 4, 5, 6, 8, 10]

In [7]:

```
1 list(range(0,101,10))
```

Out[7]:

[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

enumerate

enumerateیک تابع بسیار مفید برای استفاده با حلقه های for است. بیایید این وضعیت را تصور کنیم

In [8]:

```
1 index_count = 0
2
3 for letter in 'abcde':
4     print("At index () the letter is ()".format(index_count,letter))
5     index_count += 1
```

At index the letter is a

At index 1 the letter is b

At index 2 the letter is c

At index 3 the letter is d

At index 4 the letter is e

ردیابی اینکه چند حلقه از طریق رفته اید به قدری متداول است که enumerate ایجاد شده است تا نیازی نباشد نگران ایجادو به روزرسانی این متغیر index_count یا loop_count باشید.

In [۱۰]:

```
1 # Notice the tuple unpacking!
2
3 for i, letter in enumerate('abcde'):
4     print("At index () the letter is ()".format(i,letter))
```

At index the letter is a

At index 1 the letter is b

At index 2 the letter is c

At index 3 the letter is d

At index 4 the letter is e

ZIP

توجه داشته باشید که فرمت enumerate در واقع چه چیزی را برمی گرداند، بیایید با تبدیل آن به list نگاهی به آن بیندازیم

In [12]:

```
1 list (enumerate('abcde'))
```

Out[12]:

[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd'), (4, 'e')]

این لیست از tuple ها بود، به این معنی که میتوانستیم از tuple unpacking در حلقه for ما استفاده کنیم. این ساختار داده در پایتون واقعاً بسیار رایج است ، به خصوص هنگام کار با کتابخانه های خارجی شما میتوانید از تابع zip() برای ایجاد سریع لیست tupleها با" زیپ کردن" دو لیست با هم استفاده کنید.

In [13]:

```
1 mylist1 = [1,2,3,4,5]
2 mylist2 = ['a','b','c', 'd','e']
```

In [15]:

```
1 # This one is also a generator! We will explain this later, but for now Let's transfer
2 zip (mylist1, mylist2)
```

Out[15]:

<zip at 0x1d205086f08

In [۱7]:

```
1 list (zip(mylist1, mylist2))
```

Out[17]:

[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')]

برای استفاده از generator ، ما می توانستیم فقط از یک حلقه for استفاده کنیم

In [20]:

```
1 for iteni, item2 in zip(mylisti, mylist2):
2 print('For this tuple, first item was {} and second item was {}'.format(item1, it
```

For this tuple, first item was 1 and second item was a

For this tuple, first item was 2 and second item was b

For this tuple, first item was 3 and second item was c

For this tuple, first item was 4 and second item was d

For this tuple, first item was 5 and second item was e

min and max

برای بدست آوردن حداقل مقدار یا حداکثر مقدار داخل یک لیست یا تاپل میتوانیم از توابع min() و max() استفاده کنیم

In [۱]:

```
1 mylist [10, 20, 30, 40, 100]
```

In [۲۷]:

```
1 min (mylist)
```

Out[27]:

10

In [44]:

```
1 max (mylist)
```

Out[44]:

100

sum

برای بدست آوردن مجموع اعضای یک لیست یا تاپل میتوانیم از تابع sum() استفاده نماییم.

In [۲]:

```
1 mylist [10,20,30,40,50]
```

In [3]:

```
1 sum(mylist)
```

Out[3]:

150

random

پایتون با یک کتابخانه تصادفی داخلی همراه است تعداد زیادی از توابع در این کتابخانه تصادفی وجود دارد ، بنابراین ما فقط دو تابع مفید را برای حال نشان می دهیم

In [29]:

```
1 from random import shuffle
```

In [35]:

```
1 # This shuffles the list "in-place" meaning it won't return
2 # anything, instead it will effect the list passed
3 shuffle (mylist)
```

In [36]:

```
1 mylist
```

Out [36]:

[۴۰ ,۱۰ ,۱۰۰ ,۳۰ ,۲۰]

In [۳۹]:

```
1 from random import randint
```

In [۴۱]:

```
1 # Return random integer in range [a, b], including both end points.
۲ randint(0,100)
```

Out [۴۱]:

۲۵

In [۴۲]:

```
1 # Return random integer in range [a, b], including both end points.
۲ randint(0,100)
```

Out [۴۲]:

۹۱

input

همانگونه که در بخش قبل مشاهده ،نمودیم این تابع برای دریافت ورودی از کاربر استفاده می شود

In [۴۳]:

```
1 input('Enter Something into this box: ')
```

Enter Something into this box: great job!

Out [۴۳]:

'great job!'

توجه داشته باشید که تمام مقادیری که توسط این تابع دریافت میشوند به رشته تبدیل میشوند

In [۴]:

```
1 x =input('Enter Something into this box: ')
```

Enter Something into this box: ۶

In [۵]:

```
1 Type(x)
```

Out [۵]:

Str

In [۶]:

```
1 x
```

Out [۶]:

'۶'

List Comprehensions

علاوه بر عملیات توالی و روشهای لیست ، پایتون عملیات پیشرفته تری را با نام List Comprehensionsشامل میشود List Comprehensions به ما اجازه می دهد لیست ها را با استفاده از یک نماد متفاوت بسازیم. شما می توانید آن را به عنوان یک حلقه for در یک خط در داخل براکت ها در نظر بگیرید برای یک مثال ساده:

مثال ۱

In [۱]:

```
1  cubic_list = [ ]
2
3  for i in range(1,11):
4      result = i ** 3
5      cubic_list.append(result)
6
7  cubic_list
```

Out [۱]:

[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]

In [۲]:

```
1
2  I =  [x for x in range(1, 11)]
3
4  I
```

Out [۲]:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

In [3]:

```
1  I = [x**3 for x in range(1, 11)]
2
3  I
```

Out [3]:

[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]

In [4]:

```
1  # Grab every Letter in string
2
3  Ist [x for x in 'Hello world! ']
```

In [5]:

```
1 # Check
2 lst
```

Out [5]:

['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '']

این ایده اصلی یک List Comprehensions است اگر با نمادهای ریاضی آشنا باشید، این فرمت باید برای شما آشنا باشد

به عنوان مثال: x : x^2+ در{ ۰،۱،۲ ... ۱۰} بیایید چند مثال دیگر از فهم لیست در پایتون را ببینیم

مثال ۲

In [۶]:

```
1 #Square numbers in range and turn into list
2 lst = [x**2 for x in range(0,11)]
```

In [۷]:

```
1 lst
```

Out [۷]:

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

مثال ۳

حالا اگر نیاز به دستور ۱۴ داشتیم چگونه عمل کنیم؟

In [۸]:

```
1 # Check for even numbers in a range
2 lst = [x**2 for x in range(0,11) if x % 2 == 0]
```

In [۹]:

```
1  lst
```

Out [۹]:

[0, 4, 16, 36, 64, 100]

مثال ۴

همچنین میتوانیم محاسبات پیچیده تری نیز داشته باشیم

In [10]:

```
1  # Convert Celsius to Fahrenheit
2  celsius [0, 10, 20.1, 34.6]
3
4  fahrneheit = [(((9 / 5) * temp) + 32) for temp in celsius]
5
6  fahrneheit
```

Out [10]:

[32.0, 50.0, 68.18, 94.28]

مثال ۵

علاوه بر مثال های ،بالا ما میتوانیم عملیاتی تودرتو از List Comprehensions را داشته باشیم

In [11]:

```
1  lst = [x  2 for x in [i 2 for i in range(11)]]
2
3  lst
```

Out [11]:

[0, 1, 16, 81, 256, 625, 1296, 2401, 4096, 6561, 10000]

آزمون ارزیابی عبارات پایتون

بیایید دانش شما را آزمایش کنیم!

از for و split و if برای ایجاد یک بیانیه استفاده کنید که کلماتی را چاپ کند که با 's' شروع میشوند:

In [ ]:

```
1 st = 'Print only the words that start with s in this Sentence'
```

In [ ]:

```
1
```

از ()range برای چاپ همه اعداد زوج از ۰ تا ۱۰ استفاده کنید.

In [ ]:

```
1 #Code Here
```

از List Comprehension برای ایجاد لیستی از همه اعداد بین ۱ و ۵۰ که بر ۳ قابل تقسیم هستند استفاده کنید

In [ ]:

```
1 #Code in this cell
2
```

رشته زیر را پیمایش نمایید و اگر طول یک کلمه زوج باشد "leven " را چاپ کنید

In [ ]:

```
1 st = 'Print every word in this sentence that has an even number of letters'
```

In [ ]:

```
1 #Code in this cell
۲
```

برنامه ای بنویسید که اعداد صحیح از ۱ تا ۱۰۰ را چاپ می.کند اما برای مضارب سه به جای عدد "Fizz" را چاپ کنید و برای مضارع پنج "Buzz" را چاپ کنید برای اعدادی که مضارع هر دو سه و پنج هستند "FizzBuzz" را چاپ کنید.

In [ ]:

```
1 #Code in this cell
2
```

از List Comprehension برای ایجاد لیستی از حروف اول هر کلمه در رشته زیر استفاده کنید:

In [ ]:

```
1 st = 'Create a list of the first letters of every word in this string'
```

In [ ]:

```
1 #Code in this cell
2
```

آزمون ارزیابی عبارات پایتون

بیایید دانش شما را آزمایش کنیم!

از for و split و if برای ایجاد یک بیانیه استفاده کنید که کلماتی را چاپ کند که با 's' شروع میشوند:

In [۴]:

```
1 st = 'Print only the words that start with s in this Sentence'
```

In [۶]:

```
1 #Code here
2
3 for word in st.split():
4     if word[0].lower()
5         print (word)
```

---

start

s

Sentence

از ()range برای چاپ همه اعداد زوج از ۰ تا ۱۰ استفاده کنید.

In [7]:

```
1 #Code Here
2
3 list(range(0,11,2))
```

Out[7] :

[0, 2, 4, 6, 8, 10]

از List Comprehension برای ایجاد لیستی از همه اعداد بین ۱ و ۵۰ که بر ۳ قابل تقسیم هستند استفاده کنید

In [8]:

```
1 #Code in this cell
2 [x for x in range(1, 51) if x % 3 == 0]
```

Out[8]:

[3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48]

رشته زیر را پیمایش نمایید و اگر طول یک کلمه زوج باشد "leven " را چاپ کنید

In [9]:

```
1 st = 'Print every word in this sentence that has an even number of letters'
```

In [11]:

```
1 #Code in this cell
۲
3 for word in st.split():
4     if len(word) % 2 == 0:
5         print (word, "<-- hqas an even length!")
```

word <-- hqas an even length!

in <--hqas an even length!

this <-- hqas an even length!

sentence <- hqas an even length!

that <-- hqas an even length!

an <hqas an even length!

even <-- hqas an even length!

number <hqas an even length!

of <hqas an even length!

برنامه ای بنویسید که اعداد صحیح از ۱ تا ۱۰۰ را چاپ می.کند اما برای مضارب سه به جای عدد "Fizz" را چاپ کنید و برای مضارع پنج "Buzz" را چاپ کنید برای اعدادی که مضارع هر دو سه و پنج هستند "FizzBuzz" را چاپ کنید.

In [ ]:

```
1  #Code in this cell
٢
3  for num in range(1, 101):
4      if num % 3 == 0 and num % 5 == 0:
5          print('FizzBuzz')
6      elif num % 3 == 0:
7          print("Fizz')
8      elif num % 5 == 0:
9          print("Buzz')
10     else:
11         print(num)
```

از List Comprehension برای ایجاد لیستی از حروف اول هر کلمه در رشته زیر استفاده کنید:

In [16]:

```
1 st = 'Create a list of the first letters of every word in this string'
```

In [ ]:

```
1 #Code in this cell
٢ [word[0] for word in st.split()]
```

Out [17]:

['C', 'a', '1', 'o', 't', 'f', '1', 'o', 'e', 'w', 'i', 't', 's']

عالی!

چالش بازی حدس زدن

بیایید از حلقه while برای ایجاد یک بازی حدس زدن استفاده کنیم.

چالش:

برنامه ای بنویسید که یک عدد صحیح تصادفی از ۱ تا ۱۰۰ انتخاب کند و بازیکنان عدد را حدس بزنند.
قوانین به شرح زیراست:

۱. اگر حدس بازیکن کمتر از ۱ یا بیشتر از ۱۰۰ باشد بگویید "out of bound"
۲. در نوبت اول بازیکن، اگر حدس آنها
- در فاصله ۱۰ واحد از عدد ،باشد برگردانید "warm!"
- بیشتر از ۱۰ واحد دورتر از عدد ،باشد. برگردانید cold
۳. در تمام نوبت های بعدی اگر حدس
- نزدیک تر به عدد از حدس قبلی باشد برگردانید warmer دورتر از عدد نسبت به حدس قبلی باشد برگردانید colder
۴. وقتی حدس بازیکن برابر با عددهاست به آنها بگویید که به درستی حدس زدهاند و چند حدس طول کشید!

شما می توانید این را از صفر شروع کنید با مراحل ذکر شده در زیر را دنبال کنید یک دفترچه جلوه جداگانه فراهم شده است. موفق باشید!

ابتدا با استفاده از ماژول تصادفی یک عدد صحیح تصادفی از ۱ تا ۱۰۰ انتخاب کنید و آن را به یک متغیر اختصاص دهید

توجه random.randint(a,b) یک عدد صحیح تصادفی در بازه a b برمی گرداند از جمله هر دو نقطه پایانی.

In [ ]:

```
1
```

در مرحله بعد پیامد خوش آمدگویی و قوانین را چاپ کنید

In [ ]:

```
1  print("WELCOME TO GUESS ME!")
2  print("I'm thinking of a number between 1 and 100")
3  print("If your guess is more than 10 away from my number, I'll tell you you're
   COLD"
4  print("If your guess is within 10 of my number, I'll tell you you're WARM")
5  print("If your guess is farther than your most recent guess, I'll say you're getting 6
6  print("If your guess is closer than your most recent guess, I'll say you're getting 7
7  print("LET'S PLAY!")
```

یک لیست برای ذخیره حدس ها ایجاد کنید

راهنمایی: صفر یک مقدار جایگزین خوب است. این مفید است زیرا به "غلط" ارزیابی می شود

In [ ]:

```
1
```

یک حلقه while بنویسید که برای یک حدس معتبر درخواست کند آن را چند بار تست کنید تا مطمئن شوید که کار می کند

In [ ]:

```
1  while True:
٢    pass
```

یک حلقه while بنویسید که حدس بازیکن را با عدد ما مقایسه کند. اگر بازیکن به درستی حدس زده باشد، از حلقه خارج شوید. در غیر این صورت به بازیکن بگویید که آنها گرم تر یا سرد تر هستند و ادامه دهید تا حدس های جدید درخواست شود.

چند راهنمایی:

- ممکن است اولین بار کمک کند تا تمام ترکیبات ممکن را روی کاغذ نقش ببندید
- شما میتوانید از تابع ( )abs برای پیدا کردن تفاوت مثبت دو عدد استفاده کنید
- اگر شما همه حدس های جدید را به لیست اضافه کنید پس حدس قبلی به عنوان guesses [۲] داده میشود

In [ ]:

```
while True:
  pass
```

این همه است! شما فقط بازی اول خود را برنامه نویسی کرده اید

در بخش بعد ما چگونگی تبدیل بعضی از این اعمال تکرار شونده به توابع را خواهیم آموخت که هروقت نیاز داریم فراخوانده شود.

عالی!

چالش بازی حدس زدن

بیایید از حلقه while برای ایجاد یک بازی حدس زدن استفاده کنیم.

چالش:

برنامه ای بنویسید که یک عدد صحیح تصادفی از ۱ تا ۱۰۰ انتخاب کند و بازیکنان عدد را حدس بزنند. قوانین به شرح زیراست:

۱. اگر حدس بازیکن کمتر از ۱ یا بیشتر از ۱۰۰ باشد بگویید "out of bound"
۲. در نوبت اول بازیکن، اگر حدس آنها
- در فاصله ۱۰ واحد از عدد ،باشد برگردانید "warm!"
- بیشتر از ۱۰ واحد دورتر از عدد ،باشد. برگردانید cold

۳. در تمام نوبت های بعدی اگر حدس
- نزدیک تر به عدد از حدس قبلی باشد برگردانید warmer
- دورتر از عدد نسبت به حدس قبلی باشد برگردانید colder

۴. وقتی حدس بازیکن برابر با عددهاست به آنها بگویید که به درستی حدس زدهاند و چند حدس طول کشید!

شما می توانید این را از صفر شروع کنید با مراحل ذکر شده در زیر را دنبال کنید یک دفترچه جلوه جداگانه فراهم شده است. موفق باشید!

ابتدا با استفاده از ماژول تصادفی یک عدد صحیح تصادفی از ۱ تا ۱۰۰ انتخاب کنید و آن را به یک متغیر اختصاص دهید

توجه random.randint(a,b) یک عدد صحیح تصادفی در بازه a b برمی گرداند از جمله هر دو نقطه پایانی.

In [ ]:

```
1 from random import randint
2
3 num = randint(1, 100)
4
5 num
```

Out[1]:

۵۱

در مرحله بعد پیامد خوش آمدگویی و قوانین را چاپ کنید

In [ ]:

```
1 print("WELCOME TO GUESS ME!")
2 print("I'm thinking of a number between 1 and 100")
3 print("If your guess is more than 10 away from my number, I'll tell you you're
  COLD"
4 print("If your guess is within 10 of my number, I'll tell you you're WARM")
5 print("If your guess is farther than your most recent guess, I'll say you're getting 6
6 print("If your guess is closer than your most recent guess, I'll say you're getting 7
7 print("LET'S PLAY!")
```

WELCOME TO GUESS ME!

I'm thinking of a number between 1 and 100

If your guess is more than 10 away from my number, I'll tell you you're CO

LD

If your guess is within 10 of my number, I'll tell you you're WARM

If your guess is farther than your most recent guess, I'll say you're gett ing COLDER

If your guess is closer than your most recent guess, I'll say you're getti ng WARMER

LET'S PLAY!

یک لیست برای ذخیره حدس ها ایجاد کنید

راهنمایی: صفر یک مقدار جایگزین خوب است. این مفید است زیرا به "غلط" ارزیابی می شود

In [3]:

```
1 guesses = [0]
```

یک حلقه while بنویسید که برای یک حدس معتبر درخواست کند آن را چند بار تست کنید تا مطمئن شوید که کار می کند

In [ ]:

```
1 while True:
2     guess = int(input("I'm thinking of a number between 1 and 100. \nWhat is your gue:
3
4     if guess 1 or guess > 100:
5        print('OUT OF BOUNDS! please try again: ')
6       continue
7     break
```

یک حلقه while بنویسید که حدس بازیکن را با عدد ما مقایسه کند. اگر بازیکن به درستی حدس زده باشد، از حلقه خارج شوید. در غیر این صورت به بازیکن بگویید که آنها گرم تر یا سرد تر هستند و ادامه دهید تا حدس های جدید درخواست شود.

چند راهنمایی:

- ممکن است اولین بار کمک کند تا تمام ترکیبات ممکن را روی کاغذ نقش ببندید
- شما میتوانید از تابع ( )abs برای پیدا کردن تفاوت مثبت دو عدد استفاده کنید

- اگر شما همه حدس های جدید را به لیست اضافه کنید پس حدس قبلی به عنوان guesses [۲] داده میشود

In [4]:

```
while True:

    # we can copy the code from above to take an input
    guess = int(input("I'm thinking of a number between 1 and 100. \n What is your g

    if guess < 1 or guess > 100:
        print('OUT OF BOUNDS! Please try again: ')
        continue

    # here we compare the player's guess to our number
    if guess == num:
      print (f'CONGRATULATIONS, YOU GUESSED IT IN ONLY {len (guesses) } GUESSES!!')
      break

    # if guess is incorrect, add guess to the List
    guesses.append(guess)

    # when testing the first guess, guesses [-2] ==0, which evaluates to False
   # and brings us down to the second section
  # [0, 65, 57, 59]
  if guesses[-2]:
     if abs (num-guess) < abs (num-guesses[-2]):
        print('WARMER!')
    else:
        print('COLDER!')

 else:
     if abs (num-guess) <= 10:
         print('WARM!')
     else:
         print('COLD!')
```

I'm thinking of a number between 1 and 100.

What is your guess? 65

!COLD

I'm thinking of a number between 1 and 100.

What is your guess? 57

!WARMER

.I'm thinking of a number between 1 and 100

What is your guess? 59

!COLDER

.I'm thinking of a number between 1 and 100

What is your guess? 51

!!CONGRATULATIONS, YOU GUESSED IT IN ONLY 3 GUESSES!!

این همه است! شما فقط بازی اول خود را برنامه نویسی کرده اید

در بخش بعد ما چگونگی تبدیل بعضی از این اعمال تکرار شونده به توابع را خواهیم آموخت که هروقت نیاز داریم فراخوانده شود.

عالی!

توابع

مقدمه ای بر توابع

این سخنرانی شامل توضیح این است که یک تابع در پایتون چیست و چگونه میتوان آن را ایجاد کرد.
توابع یکی از اصلی ترین بلوک های ساختمانی ما خواهند بود زمانی که ما مقدار بیشتر و بیشتری از کد را برای حل مشکلات بسازیم.

یک تابع چیست؟

به طور رسمی، یک تابع یک دستگاه مفید است که مجموعه ای از عبارات را در کنار هم قرار میدهد تا بتوانند بیش از یک بار اجرا شوند. آنها همچنین به ما اجازه میدهند که پارامترهایی را مشخص کنیم که به عنوان ورودی های توابع عمل کنند.

در سطح بنیادی تر توابع به ما اجازه میدهند که نیاز نباشد همان کد را دوباره و دوباره تکرار کنیم اگر به درس های قبل درباره رشته ها و لیست ها ،برگردید به خاطر داشته باشید که ما از یک تابع len() برای دریافت طول یک رشته استفاده کردیم. از آنجایی که بررسی طول یک دنباله یک وظیفه رایج است شما مایلید یک تابع بنویسید که این کار را در هر فرمان تکرار کند.

توابع یکی از سطوح پایه ای استفاده مجدد از کد در پایتون خواهند بود و همچنین به ما اجازه میدهد تا شروع به فکر کردن در مورد طراحی برنامه کنیم(ما بسیار عمیق تر در مورد ایده های طراحی وقتی درباره برنامه نویسی شی گرا یاد می گیریم غوطه ور خواهیم شد).

چرا حتی از توابع استفاده کنید؟

به طور ساده شما باید از توابع استفاده کنید وقتی قصد دارید چندین بار از یک بلوک کد استفاده کنید. تابع به شما اجازه می دهد تا همان بلوک کد را فراخوانده شود بدون اینکه نیاز باشد آن را چندین بار بنویسید این نوبت به شما اجازه می دهد تا اسکریبت های پایتون پیچیده تری ایجاد کنید برای درک واقعی ،این ما واقعاً باید توابع خودمان را بنویسیم

مباحث تابع

- کلمه کلید def
- نمونه ساده از یک تابع
- فراخوانی یک تابع با ()
- قبول پارامتر
- فراخوانی بصورت keyword argument
- مقدار پیش فرض برای پارامتر ورودی
- ارسال مقدار vs ارسال رفرنس
- چاپ در مقابل بازگشت
- بازگشت همزمان چند مقدار
- اضافه کردن منطق در داخل یک تابع
- اضافه کردن حلقه ها در داخل یک تابع
- توابع با تعداد ووردی دلخواه *args
- توابع با تعداد و نام ورودی دلخواه kwargs
- تعاملات بین توابع

In [ ]:

```
1  s='foobar'
2
3  len(s)
```

In [ ]:

```
1  s.upper()
```

In [ ]:

```
1  id(s)
```

کلمه کلیدیdef

بیایید ببینیم چگونه ساختار یک تابع در پایتون را ایجاد کنیم این شکل زیر را دارد:

In [ ]:

```
1  def name_of_function (arg1, arg2):
2      '''
3      This is where the function's Document String (docstring) goes.
4      When you call help() on your function it will be printed out.
5      '''
6      # Do stuff here
7      # Return desired result
```

ما با def شروع میکنیم سپس یک فضای خالی پس از نام تابع سعی کنید نام ها را مرتبط نگه دارید به عنوان مثال ()len یک نام خوب برای یک تابع length() .است. همچنین با نام ها مراقب باشید شما نمیخواهید یک تابع را با همان نام یک تابع داخلی در پایتون (https://docs.python.org/3/library/functions.html) مانند (len) نامگذاری کنید.

سپس یک جفت پرانتز با تعدادی آرگومان جدا شده توسط یک کاما می آید این آرگومان ها ورودی های تابع شما هستند. شما قادر خواهید بود از این ورودی ها در تابع خود استفاده کنید و به آنها ارجاع دهید پس از این شما یک کولن قرار میدهید.

حالا مرحله مهم شما باید برای شروع به درستی کد درون تابع خود را indent .کنید پایتون از فضای سفید برای سازماندهی کد استفاده میکند بسیاری از زبان های برنامه نویسی دیگر این کار را انجام نمیدهند بنابراین به آن فکر کنید. شما docstring را مشاهده خواهید کرد این جایی است که شما یک توضیحات ابتدایی از تابع را می نویسید با استفاده از Jupyter و Juyter، Notebooks شما قادر خواهید بود با فشار دادن Shift+Tab پس از نام تابع این doctrings را .بخوانید Doctrings برای توابع ساده ضروری نیستند اما تمرین خوبی است که آنها را درون قرار دهید تا شما یا دیگران به راحتی کدی را که نوشته اید درک کنید

پس از همه این ها شما شروع به نوشتن کدی میکنید که مایلید اجرا شود.

بهترین راه برای یادگیری ،توابع عبور از مثال هاست پس بیایید سعی کنیم از مثال هایی عبور کنیم که به اشیاء و ساختارهای دادهای که قبلاً یاد گرفته ایم مرتبط باشد.

نمونه ساده ای از یک تابع

In [3]:

```
1 def f():
2   S='__Inside of f()'
3   print(s)
```

فراخوانی یک تابع call

In [4]:

```
1 f()
```

__Inside of f()

اگر شما پرانتز () را فراموش کنید آن را به سادگی نشان میدهد که say hello یک تابع است. بعداً ما یاد خواهیم گرفت که در واقع میتوانیم توابع را در داخل توابع دیگر قرار دهیم اما در حال حاضر فقط به خاطر داشته باشید که توابع را با() فراخوانی کنید.

In [5]:

```
1 f()
```

__Inside of f()

In [6]:

```
1 f()
```

Out[6]:

<function _main__.f()>

In [7]:

```
1 for i in range(10):
2     f()
```

__Inside of f()

__Inside of f()

__Inside of f()

__Inside of f()

__Inside of f()

__Inside of f()

__Inside of f()

__Inside of f()

__Inside of f()

__Inside of f()

1.2)ارسال ارگومان ورودی

In [8]:

```
1 def add(a, b, c):
2     result = a+b+c
3     print (f'Result: (result)")
```

In [9]:

```
1 add(3, 7, 18)
```

Result: 28

نکته مهم:

به متغیرهایی همچون a.b.c که در امضا تابع مشخص میشوند پارامتر ورودی میگویم و به مقادیر ۳’۷’۸ که در زمان فراخوانی تابع بعنوان ورودی به آن ارسال میشود آرگومان ورودی می گوییم.

In [10]:

```
1 add(5, 24, 109)
```

Result: 138

In [13]:

```
1 def add1(a: int, b: int, c: int) -> int:
2     result = a+b+c
3     print (f"Result: {result}")
```

In [14]:

```
1 add(5, 24, 109)
```

Result: 138

In [15]:

```
1 add1('a', 'b', 'c')
```

Result: abc

In [16]:

```
1 def f(qty, item, price):
2      per_item price/ qty
3     print (f'{qty} of {item} cost ${price: .2f}')
```

در فراخوانی زیر ما فقط مقادیر را به تابع ارسال میکنیم و هیچ گونه اطلاعات دیگری را به در مورد فراخوانی به پایتون نمی دهیم از این رو پایتون مقادیر را بصورت موقعیت مکانی به ترتیب از سمت چپ داخل پارامترها قرار می دهد.

In [17]:

```
1 f(6, 'bananas', 1.74)
```

۶ of bananas cost $ 1.74

Function Call → Function Definition

f(6, 'bananas', 1.74) → def f(qty, item, price) :

arguments (actual parameters) — parameters (formal parameters)

In [18]:

```
1 f('bananas', 6, 1.74)
```

TypeError

Traceback (most recent call last)

Cell In[18], line 1

----> 1 f('bananas', 6, 1.74)

Cell In[16], line 2, in f(qty, item, price)

1 def f(qty, item, price):

----> 2 per item price/ qty

3 print(f'{qty} of {item} cost ${price: .2f}')

TypeError: unsupported operand type(s) for /: 'float' and 'str'

In [19]:

```
1 (6, 1.74)
```

TypeError

Traceback (most recent call last)

Cell In[19], line 1

----> 1 f(6,1.74)

TypeError: f() missing 1 required positional argument: 'price'

In [20]:

```
1 f("bananas, 6, 1.74, 55)
```

TypeError

Traceback (most recent call last)

Cell In[20], line 1

----> 1 f('bananas', 6, 1.74,55)

TypeError: f() takes 3 positional arguments but 4 were given

۱٫۲٫۲) ارسال ارگومان های ورودی بصورت نام دار یا keyword

In [۲۱]:

```
1 f(qty = 11, item = 'Orange', price = 17)
```

11 of Orange cost $17.00

In [۲۲]:

```
1 f(item 'Orange', qty = 11, price = ۱۷)
```

11 of Orange cost $17.00

In [23]:

```
1 f(item'Orange', qty = 11, cost = 17)
```

TypeError

Traceback (most recent call last)

Cell In[23], line 1

----> 1 f(item'Orange', qty = 11, cost = 17)

TypeError: f() got an unexpected keyword argument 'cost'

۱٫۲٫۳) مقادیر پیش فرض

گاهی برای انعطاف پذیری بیشتر یک تابع در فراخوانی امکانی را فراهم میکنیم تا اگر مقدار یک پارامتر در زمان فراخوانی مشخص نشود، آن پارامتر از یک مقدار پیش فرض استفاده نماید.

In [24]:

```
1 def f(qty, item, price = 5):
2         per_item = price/ qty
3         print(f'{qty} of {item} cost ${price: .2f}')
```

In [25]:

```
1 f(qty = 6, item = 'bananas', price = 1.74)
```

6 of bananas cost $ 1.74

In [26]:

```
1 f(qty = 6, item = 'bananas')
```

6 of bananas cost $ 5.00

In [ ]:

```
1 f(6, 'orange')
```

In [27]:

```
1 def f(qty = 6, item = 'Oranges', price = 2)
2  print('{qty} {item} cost ${price: .2f}')
```

In [28]:

```
1 f()
```

6 Oranges cost $2.00

In [29]:

```
1 f()
```

10 Oranges cost $2.00

In [30]:

```
1 f(10, 'bananas')
```

10 bananas cost $2.00

In [31]:

```
1 f(10, 'bananas',15)
```

10 bananas cost $15.00

In [32]:

```
1 f(price = 20)
```

6 Oranges cost $20.00

In [33]:

```
1 f(price = 20)
```

5 Oranges cost $20.00

نکته مهم: زمانی که یک مقدار از نوع داده قابل جهش یعنی لیست دیکشینری و یا مجموعه بعنوان مقدار پیش فرض استفاده می نمایید هر تغییری که در دفعات پیشین استفاده از مقدار پیش فرض اعمال شده باشد اثر خود را بر مقدار پیش فرض حفظ خواهد کرد.

In [36]:

```
1 def f(my_list = []):
2     my_list.append('###')
3     return my_list
```

In [37]:

```
1 f('foo', 'bar', 'baz'])
```

Out [37]:

['foo', 'bar', 'baz', '###']

In [38]:

```
1 f([1,2,3,4,5])
```

Out [38]:

[1, 2, 3, 4, 5, '###']

In [39]:

```
1 f()
```

Out [39]:

['###']

In [40]:

```
1 f()
```

Out [40]:

['###', '###']

In [41]:

```
1 f()
```

Out [41]:

['###', '###', '###']

In [42]:

```
1 f([1,2,3,4,5])
```

Out [42]:

[1, 2, 3, 4, 5, '###']

In [43]:

```
1 f('foo', 'bar', 'baz'])
```

Out [43]:

['###', '###', '###', '###']

1.3 Pass-By-Value vs Pass-By-Reference in Python

1.3)ارسال ارگومان ها به توابع بصورت مقدار در مقابل ارسال با ارجاع

آیا پارامترها در پایتون با ارجاع یا با مقدار ارسال میشوند؟ جواب این است که هیچ کدام دقیقاً نیستند.
زیرا ارجاع در پایتون به معنای همان چیز در پاسکال نیست.

In [44]:

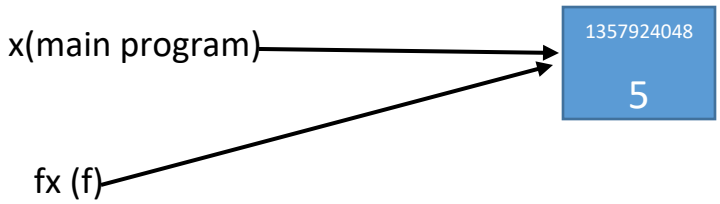
```
1 def f(fx):
2     fx = 10
```

In [45]:

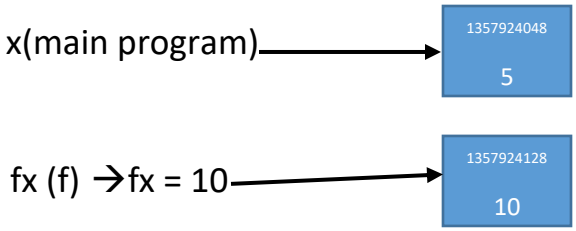
```
1  x = 5
2
3  f(x)
4
5  print(x)
```

5

۱. در لحظه فراخوانی تابع"f"



۲. بعد از فراخوانی تابع "f"



In [47]:

```
1  def f(fx):
2      print('fx, fx, '/ id(fx) = ', id(fx))
3      fx = 10
4      print('fx, fx, / id(fx), id(fx))
5
6  x = 5
7  print('1) ===> x=', x, '/ id(x) = ', id(x))
8
9  f(x)
10
11 print('2) ===> x=', x, '/ id(x) = ', id(x))
```

1) ===> x = 5 / id(x) = 2078305960752

fx = 5 / id(fx) = 2078305960752

fx = 10 / id (fx)= 2078311418608

2) ===> x = 5 / id(x) = 2078305960752

In [48]:

```
1  def my_list(11):
2       print('l1=', l1, '/ id(l1)= ', id(l1))
3      l1 [11, 12, 13, 14]
4      print('l1, l1, / id(11), id(l1))
5
6
7  l0 = [5, 6, 7, 8]
8  print('1) ===> ,l0=',l0, '/ id(10) = ', id(10))
9
10 my list(10)
11
12
13 print('2) ===> l0=',l0, '/ id(10) = ', id(10))
```

1) ===> l0 = [5, 6, 7, 8] / id(l0) = 2078413703360

l1 = [5, 6, 7, 8] / id(l1) =2078413703360

l1 = [11, 12, 13, 14] / id(l1) = 2078413712704

2) ===> l0 = [5, 6, 7, 8] / id(l0) = 2078413703360

In [49]:

```
1  def my list(l1):
2        print('l1=', l1, '/ id(l1)= ', id(l1))
3       l1[0] = 11
4      print('11, 11, / id(11) = ', id(11))
5
6
7  l0 = [5, 6, 7, 8]
8  print('1) ===> l0=',l0, '/ id(l0)= ', id(l0))
9
10 my list(10)
11
12
13 print('2) ===> l0', 10, '/id(10)= ', id(10))
```

1) ===> l0 = [5, 6, 7, 8] / id(l0) = 2078413388864

11 = [5, 6, 7, 8] / id(l1) = 2078413388864

l1 = [11, 6, 7, 8] / id(l1) = 2078413388864

2) ===> l0 = [11, 6, 7, 8] id(l0) = 2078413388864

In [50]:

```
1  l = [1,2,3]
2  l1 = l
3
4  l1[0] = -100
5
6  print(1, 11)
```

[-100, 2, 3] [-100, 2, 3]

In [51]:

```
1  def my list(11):
2        print('l1= ', l1, ' / id(l1) = ', id(l1))
3        l1[0] = 11
4        print('11, 11, / id(11) = ', id(11))
5
6
7  l0 = [5, 6, 7, 8]
8  print('1) ===> l0=',l0,/ id(l0)= ', id(l0))
9
10 my list(l0.copy())
11
12
13 print('2) ===> l0=', l0, 'id(l0)= ', id(l0))
```

1) ===>l0 = [5, 6, 7, 8] / id(l0) = 2078413718720

l1 = [5, 6, 7, 8] / id(l1) = 2078384502720

l1 = [11, 6, 7, 8] / id(l1) = 2078384502720

2) ===> l0 = [5, 6, 7, 8] id(l0) = 2078413718720

1.4)دستور return برای بازگرداندن مقادیر از تابع

In [54]:

```
1  def f():
2      print('foo')
3      print('bar')
4      return
5
6  type(f())
```

foo

bar

Out[54]:

None Type

In [58]:

```
def f(x):
  if x < 0:
      return
  if x 100:
      return
  print(x)


f(55)
```

55

In [60]:

```
def f():
    return 'foo'

s = f()

print(s)

type(f())
```

foo

Out [60]:

str

In [63]:

```
def add(a, b, c):
     result = a+b+c
    return result

r = add (10, 5, 9)

print (r**3)
```

13824

In [ ]:

```
def add(a, b, c):
    summation = a+b+ C
    avg summation / 3
   return summation

r = add(10, 5, 9)

print(r ** 3)
```

1.4.1)بازگرداندن چند مقدار بصورت همزمان از تابع

توجه نمایید که دستور return همواره یک مقدار یا یک بسته را میتواند بعنوان خروجی تابع برگرداند، از این رو برای بازگرداند چندین مقدار بصورت همزمان در پایتون باید مقادیر مد نظر را در قالب یک بسته بندی مانند لیست دیکشینری ویا تاپل ها انجام دهیم.

In [۷۲]:

```
# dictionary

def f():
    return dict(foo = 1, bar = 2, baz = 3)

f()

type(f())

#d= f()

#d[ 'baz ']**3

f()['baz'] ** 3
```

Out[72]:

27

In [۷۳]:

```
1  # return List
2
3  def f():
4      return ['foo', 'bar', 'baz', 'qux']
5
۶  f()
```

Out [73]:

['foo', 'bar', 'baz', 'qux']

In [۷۴]:

```
۱  f()[2]
```

Out [۷۴]:

'baz'

In [۷۵]:

```
۱  f()[: : -1]
```

Out [۷۵]:

['qux', 'baz', 'bar', 'foo']

In [ ]:

```
1  x = 1, 2, 3, 4
```

In [76]:

```
1  # return tuple with packing
2
3  def f():
4      return 'foo', 'bar', 'baz', 'qux'
5
6  f()
7
```

Out[76]:

('foo', 'bar', 'baz', 'qux')

In [77]:

```
1 a, b, c, d = f()
2
3 print (f'a {a}, b = {b}, c = {c}, d = {d}')
```

a=foo, b=bar, c = baz, d= qux

1.5)توابع با تعداد آرگومان ورودی دلخواهargs

In [80]:

```
1 def avg(a, b, c):
2     return (a+b+c) / 3
```

In [81]:

```
1 avg(1,2,3)
```

Out[81]:

2.0

In [83]:

```
1 avg(1,5,10,4,6)
```

TypeError

Traceback (most recent call last)

Cell In[83], line 1

----> 1 avg(1, 5, 10, 4, 6)

TypeError: avg() takes 3 positional arguments but 5 were given

In [ ]:

```
1  def avg(a, b=0, c=0, d=0, e=0):
2      .
3      .
4     .
```

In [ ]:

```
1  avg(1)
2  avg(1, 2)
3  avg(1, 2, 3)
4  avg(1, 2, 3, 4)
5  avg(1, 2, 3, 4, 5)
```

In [ ]:

```
1  def avg(a, b=0, c=0, d=0, e=0):
2      num = 1
3     if b != 0:
4         num += 1
5
6    return (a+b+c+d+e) / # Divided by what???
```

راه حل اول:

In [84]:

```
1  def avg(a):
2
3      total = 0
4
5    for v in a:
6        total= total + v
7
8    return total / len(a)
```

In [86]:

```
1  avg([1,2,3])
```

Out[86]:

2.0

In [87]:

```
1  avg([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Out[87]:

5.0

In [88]:

```
1  t= (1, 2, 3, 4, 5)
2  avg(t)
```

Out[88]:

3.0

1.5.1)استفاده از بسته بندی تاپل ها برای ساخت توابع با تعداد آرگومان ورودی دلخواه

In [89]:

```
1 def f(args):
2     print(args)
3     print(type (args), len(args))
4    for x in args:
5         print(x)
```

In [90]:

```
1 f(1, 2, 3, 4, 5)
```

(1, 2, 3, 4, 5)

<class 'tuple'> 5

1

2

3

4

5

In [91]:

```
1 f('foo', 'bar', 'baz', 'qux', 'quux')
```

('foo', 'bar', 'baz', 'qux', 'quux') <class 'tuple'> 5

foo

bar

baz

qux

quux

In [92]:

```
def avg(*args):
    total = 0
    for i in args:
        total= total + i
    return total / len(args)
```

In [93]:

```
avg(1, 2, 3)
```

Out[93]:

2.0

In [94]:

```
avg(1, 2, 3,4,5)
```

Out[94]:

3.0

In [103]:

```
def avg(*args):
    return sum(args) / len(args)

avg(1, 2, 3, 4, 5)
```

Out[103]:

3.0

1.5.2)استفاده از بازگشایی تاپل ها لیستها در فراخوانی توابع

In [96]:

```
def f(x, y, z):
    print (f'x = {x}')
    print (f'y={y}')
    print (f'z = {z}')
```

In [97]:

```
f(1,2,3)
```

X = 1

y = 2

Z = 3

In [99]:

```
T= ('foo', 'bar', 'baz')

f(*t)
```

x = foo

y = bar

z = baz

In [100]:

```
a= ['foo', 'bar', 'baz']
f(*a)
```

x = foo

y= bar

z = baz

In [101]:

```
1  s = {1, 2, 3}
2  print(type(s))
3
4  f(*s)
```

<class 'set'>

X = 1

y = 2

z = 3

۱٫۶ ساخت توابع با تعداد و نام ارگومان دلخوه به روش بسته بندی دیکیشنری **kwargs

In [104]:

```
1  def f(**kwargs):
2          print(kwargs)
3          print(type (kwargs), len(kwargs))
4
5          for key, val in kwargs.items():
6              print(key, ->', val)
7
8      print (kwargs.get('foo'))
```

In [107]:

```
1  f(zoo 1, bar 2, baz = 3)
```

{'zoo': 1, 'bar': 2, 'baz': 3} <class 'dict'> 3

ZOO -> 1

bar -> 2

baz -> 3

None

In [108]:

```
1 f(foo=1, bar=2, baz=3, qux = 351, quux = -197)
```

{'foo': 1, 'bar': 2, 'baz': 3, 'qux': 351, 'quux': -197}

<class 'dict'> 5

foo -> 1

bar -> 2

baz -> 3

qux -> 351

quux ->

-197

۱

۱/۶/۱ استفاده از بازگشایی دیکشینری در فراخوانی توابع

In [109]:

```
1 def f(a, b, c):
2 print(f'a={a}'')
3 print(f'b =(b)')
4 print(f'c=(c)')
```

In [110]:

```
1 d = {'a': 'foo', 'b': 25, 'c': 'qux'}
2
3 f(**d)
```

a = foo

b = 25

C = qux

In [111]:

```
1 f(a='foo', b=25, c='qux')
```

a = foo

b = 25

c = qux

1.7مثالی از ترکیب همه حالات تعریف تابع

In [112]:

```
1 def f(a, b, *args, **kwargs):
2     print (F'a= {a}')
3     print (F'b= {b}')
4     print (F'args = {args}')
5     print (F'kwargs = {kwargs}')
```

In [113]:

```
1 f(1, 2, 'foo', 'baz', dd = 'qux', x=100, y=200, z=300)
```

a = 1

b = 2

args = ('foo', 'baz')

kwargs = {'dd': 'qux', 'x': 100, 'y': 200, 'z': 300}

In [114]:

```
1 f(1, 2, 'foo', ff = 'bar', 'baz', dd = 'qux', x=100, y=200, z=300)
```

Cell In[114], line 1

f(1, 2, 'foo', ff 'bar', 'baz', dd = 'qux', x=100, y=200, z=300)

SyntaxError: positional argument follows keyword argument

مثال ۱

تابع بنویسید تا بررسی کند که لیست ورودی حداقل دارای یک عدد زوج می باشد یا نه

In [119]:

```
def check_even_list (num_list):
    # Go through each number
    for number in num_list:
        # Once we get a "hit" on an even number, we return True
        if number % 2 == 0:
            return True
        # Don't do anything if its not even
        else:
            pass
    # Notice the indentation! This ensures we run through the entire for Loop
    return False
```

In [124]:

```
check even_list([1,3,5,7,9])
```

Out[124]:

False

تکمیلی یک تابع بنویسید که لیست تمام اعداد زوج را از داخل لیست ورودی استخراج کند!

In [ ]:

```

```

مثال ۲

یک تابع بنویسید تا اول بودن یک عدد را بررسی نماید.

In [125]:

```
def is_prime (num):
    ...
   Naive Method of checking for primes
   ...
  for n in range(2, num):
     if num % n == 0:
          print(f' {num} is not a prime!')
         break
  else:
    print (f' {num} is a Prime Number!!')

```

In [136]:

```
is_prime (5915587277)
```

5915587277 is a Prime Number!!

In [137]:

```
import math

def is_prime (num):
     ...
    Better method of checking for primes.
    ...

   if num % 2 == 0 and num > 2:
         return False
  for i in range (3, int (math.sqrt(num)) + 1, 2):
         if num % i == 0:
                return False
   return True

```

In [138]:

```
1 is_prime (5915587277)
```

Out[138]:

True

مسائل تمرینی توابع

مسائل به ترتیب افزایش درجه سختی مرتب شده اند

- دست گرمی - اینها میتوانند با استفاده از مقایسه های پایه و روش های ساده حل شوند
- سطح ۱- این ها ممکن است شامل عبارات شرطی if / then و روش های ساده باشد
- سطح ۲ – این ها ممکن است نیاز به تکرار بر روی دنباله ها داشته باشد ، معمولاً با نوعی حلقه
- چالش برانگیز - حل این مسائل نیاز به خلاقیت دارد

دست گرمی :

کمتر از دو زوج : تابعی بنویسید که از میان دو عدد داده را بعنوان ورودی دریافت میکند و * اگر * هر دو عدد زوج باشند مقدار کوچکتر را بر میگرداند ، اما اگر حداقل یکی از اعداد فرد باشد مقدار بزرگتر را برمی گرداند

lesser_of_two_evens (24) -> 2

lesser_of_two_evens (2.5) -> 5

In [ ]:

```
1
```

In [ ]:

```
1 # Check
۲ lesser_of_two_evens(2, 4)
```

In [ ]:

```
1 # Check
۲ lesser_of_two_evens(2, ۵)
```

ANIMAL CRACKERS: تابعی بنویسید که یک رشته دو کلمه ایی از اسامی حیوانات را بعنوان ورودی دریافت کند و اگر حرف اول دو کلمات داخل رشته با یکدیگر برابر ،باشد مقدار True را برمیگرداند

animal_crackers ('Levelheaded Llama') --> True

animal_crackers ('Crazy Kangaroo')-> False

In [ ]:

```
1
```

In [ ]:

```
1 # Check
2 animal crackers ('Levelheaded Llama')
```

In [ ]:

```
1 # Check
2 animal_crackers('Crazy Kangaroo )
```

MAKES TWENTY : تابعی بنویسید که دو عدد را بعنوان ورودی دریافت میکند و اگر یکی از اعداد ۲۰ باشد یا مجموعه دو عدد ۲۰ باشد مقدار True را برگرداند.

makes twenty (20,10)-> True

makes twenty (12,8) --> True

makes twenty (2,3) --> False

In [ ]:

```
1
```

In [ ]:

```
1 # Check
2 nakes_twenty (20,10)
```

In [ ]:

```
# Check
nakes_twenty ( ۱۲,۸)
```

In [ ]:

```
# Check
nakes_twenty (2, ۳)
```

مسائل سطح ۱

OLD MACDONALD :تابعی بنویسید تا یک رشته را بعنوان ورودی دریافت کند و اگر طول رشته بزرگتر از ۳ باشد حرف اول و چهارم را با حروف بزرگ انگلیسی بنویسد

old_macdonald('macdonald') -> MacDonald

' Note: 'macdonald'.capitalize() returns 'Macdonald

In [ ]:

```

```

In [ ]:

```
# Check
old_macdonald ('macdonald')
```

In [ ]:

```
old_macdonald ('ma')
```

MASTER YODA: تابعی بنویسید تا یک رشته را دریافت نمایید و ترتیب کلمات را در آن مطابق مثالهای زیر معکوس نماید :

master_yoda('I am home') --> 'home am I'

master_yoda ('We are ready')--> 'ready are We'

In [ ]:

```
1
```

In [ ]:

```
1 # Check
2 master_yoda ('I am home')
```

In [ ]:

```
1 # Check
2 master_yoda ('We are ready')
```

ALMOST THERE: تابع بنویسید که یک عدد را دریافت کند و اگر در فاصله ۱۰ تا از ۱۰۰ یا ۲۰۰ باشد True و در غیراینصورت False را برگرداند

almost there (90) --> True

almost there (104) --> True

almost there (150) --> False

almost there (209) --> True

NOTE: abs (num) returns the absolute value of a number

In [ ]:

```
1
```

In [ ]:

```
1 # Check
2 almost there (90)
```

In [ ]:

```
# Check
almost there (104)
```

In [ ]:

```
# Check
almost there (150)
```

In [ ]:

```
# Check
almost there (209)
```

مسائل سطح ۲

:FIND 33

تابعی بنویسید که لیستی از اعداد صحیح را دریافت نماید و اگر دو عضو با مقدار ۳ پشت سرهم پیدا نماید مقدار True را برگرداند و در غیر اینصورت مقدار False

has_33([1, 3, 3]) → True

has_33([1, 3, 1, 3]) False

has_33([3, 1, 3]) False

In [ ]:

```

```

In [ ]:

```
# Check
Has_33([1, 3,3])
```

In [ ]:

```
# Check
Has_33([1, 3,1,3])
```

In [ ]:

```
# Check
Has_33([3,1,3])
```

PAPER DOLL :تابعی بنویسید که یک رشته را بعنوان ورودی دریافت نماید و یک رشته جدید بعنوان خروجی برگرداند که در آن به ازای هر حرف در رشته اصلی سه بار در رشته جدید تکرار شده باشد.

paper_doll('Hello') -> 'HHHeeel11111000'

paper_doll('Mississippi') --> 'MMMiiissssssiiipppppppiii'

In [ ]:

```

```

In [ ]:

```
# Check
paper_doll('Hello')
```

In [ ]:

```
# Check
paper_doll('Mississippi')
```

BLACK JACK :تابعی بنویسید که سه عدد صحیح در بازه ۱ تا ۱۱ دریافت کند در صورتیکه مجموعه اعداد کمتر مساوی ۲۱ باشد مقدار مجموع را برگرداند در صورتیکه مجموعه بیشتر از ۲۱ ،باشد ؛ مجموع را منهای ۱۰ کند اگر مقدار جدید کمتر از ۲۱ باشد و در میان اعداد ۱۱ ،باشد مجموع جدید آن را گزارش کند و در غیر اینصورت عبارت Bust را چاپ کند.

blackjack (5,6,7) --> 18

blackjack(9,9,9) --> 'BUST'

blackjack (9,9,11) --> 19

In [ ]:

```
1
```

In [ ]:

```
1 # Check
2 blackjack (5,6,7)
```

In [ ]:

```
1 # Check
2 blackjack (9,9,9)
```

In [ ]:

```
1 # Check
2 blackjack (9,9,11)
```

SUMMER OF ۶۹ :تابعی بنویسید که یک لیست از اعداد صحیح را بعنوان ورودی دریافت کند و مجموع اعداد را با در نظر گرفتن شرایط مقابل محاسبه نماید اگر در هنگام محاسبه مجموع به عضوی در لیست با مقدار ۶ برسیم خود آن عضو و اعضای بعدی را رسید به عضوی که مقدار آن ۹ است در مجموع لحاظ نخواهیم کرد(در لیست ورودی حتما پس از ۶ باید حداقل یک ۹ بیاید)

summer_69([1, 3, 5]) --> 9

summer_69([4, 5, 6, 7, 8, 9]) -> 9

summer_69([2, 1, 6, 9, 11]) --> 14

In [ ]:

```
1
```

In [ ]:

```
# Check
summer_69([1, 3, 5])
```

In [ ]:

```
# Check
summer_69([4, 5, 6, 7, 8, 9])
```

In [ ]:

```
# Check
summer_69([2, 1, 6, 9, 11])
```

مسائل چالش برانگیز

SPY GAME :تابعی بنویسید تا لیست از اعداد صحیح را بعنوان ورودی دریافت نماید و اگر در میان اعضا بتوان سه عضو پشت سرهم یا با فاصله پیدا نماید که الگوی ۰۰۷ را ایجاد کند مقدار True را برگرداند در غیر اینصورت False را برگرداند

spy game ([1,2,4,0,0,7,5]) --> True

spy game([1,0,2,4,0,5,7]) --> True

In [ ]:

```

```

In [ ]:

```
# Check
spy game ([1,2,4,0,0,7,5])
```

In [ ]:

```
# Check
spy game([1,0,2,4,0,5,7])
```

In [ ]:

```
# Check
spy game ([1,7,2,0,4,5,0])
```

COUNT PRIMES :تابعی بنویسید که یک عدد صحیح را بعنوان ورودی دریافت کند و لیست اعداد اول تا خود عدد ورودی را چاپ و تعداد آنها را چاپ کند

count primes (100) --> 25

.By convention, 0 and 1 are not prime

In [ ]:

```

```

In [ ]:

```
# Check
count_primes(100)
```

جایزه :راه حل سریعتر که اعداد اول را در طول مسیر پیمایش جمع آوری می نماید

In [ ]:

```

```

In [ ]:

```
count_primes2(100)
```

صرفا جهت فان(:

PRINT BIG :تابعی بنویسید که یک حرف انگلیسی را دریافت کند و آن را با حرف بزرگ و با استفاده از علامت های * چاپ کند.

```
print_big('a')
```

```
out:      *
        *   *
        *****
        *   *
        *   *
```

راهنما: در نظر داشته باشید که یک فرهنگ لغت از الگوهای ممکن ایجاد کنید و حروف الفبا را به ترکیبات خاص ۵ خطی از الگوهای مرتبط نگاشت .کنید برای اهداف این تمرین اگر فرهنگ لغت شما در "E" متوقف شود مشکل نیست.

In [ ]:

```
1
```

In [ ]:

```
1 print_big('c')
```

عالی!

مسائل تمرینی توابع

مسائل به ترتیب افزایش درجه سختی مرتب شده اند

- دست گرمی - اینها میتوانند با استفاده از مقایسه های پایه و روش های ساده حل شوند
- سطح ۱- این ها ممکن است شامل عبارات شرطی if / then و روش های ساده باشد
- سطح ۲ – این ها ممکن است نیاز به تکرار بر روی دنباله ها داشته باشد ، معمولاً با نوعی حلقه
- چالش برانگیز - حل این مسائل نیاز به خلاقیت دارد

دست گرمی :

کمتر از دو زوج : تابعی بنویسید که از میان دو عدد داده را بعنوان ورودی دریافت میکند و * اگر * هر دو عدد زوج باشند مقدار کوچکتر را بر میگرداند ، اما اگر حداقل یکی از اعداد فرد باشد مقدار بزرگتر را برمی گرداند

lesser_of_two_evens (24) -> 2

lesser_of_two_evens (2.5) -> 5

In [1]:

```
1  def lesser_of_two_evens (a, b):
۲      if a % 2 == 0 and b % 2 == 0:
۳          return min(a, b)
۴      else:
۵          return max(a, b)
```

In [2]:

```
1  # Check
۲  lesser_of_two_evens(2, 4)
```

Out[2]:

2

In [3]:

```
1  # Check
۲  lesser_of_two_evens(2, ۵)
```

Out[3]:

5

ANIMAL CRACKERS: تابعی بنویسید که یک رشته دو کلمه ایی از اسامی حیوانات را بعنوان ورودی دریافت کند و اگر حرف اول دو کلمات داخل رشته با یکدیگر برابر ،باشد مقدار True را برمیگرداند

animal_crackers ('Levelheaded Llama') --> True

animal_crackers ('Crazy Kangaroo')-> False

In [7]:

```
1  def animal crackers (text):
2      wordlist text.split()
3     return wordlist[0][0].lower()
```

In [8]:

```
1  # Check
٢  animal crackers ('Levelheaded Llama')
```

Out[8]:

True

In [9]:

```
1  # Check
٢  animal_crackers('Crazy Kangaroo )
```

Out[9]:

False

: MAKES TWENTY

تابعی بنویسید که دو عدد را بعنوان ورودی دریافت میکند و اگر یکی از اعداد ۲۰ باشد یا مجموعه دو عدد ۲۰ باشد مقدار True. را برگرداند

makes twenty (20,10)-> True

makes twenty (12,8) --> True

makes twenty (2,3) --> False

In [10]:

```
def makes twenty (a, b):
    return a 20 or b == 20 or (a + b) == 20
```

In [11]:

```
# Check
nakes_twenty (20,10)
```

Out[11]:

True

In [12]:

```
# Check
nakes_twenty ( ۱۲,۸)
```

Out[12]:

True

In [13]:

```
# Check
nakes_twenty (2, ۳)
```

Out[13]:

False

مسائل سطح ۱

OLD MACDONALD :تابعی بنویسید تا یک رشته را بعنوان ورودی دریافت کند و اگر طول رشته بزرگتر از ۳ باشد حرف اول و چهارم را با حروف بزرگ انگلیسی بنویسد

old_macdonald('macdonald') -> MacDonald

' Note: 'macdonald'.capitalize() returns 'Macdonald

In [18]:

```
def old macdonald (name):
    if len(name) > 3:
        return name[:3].capitalize()+ name [3:].capitalize()
    else:
        return 'Name is too short
```

In [19]:

```
# Check
old_macdonald ('macdonald')
```

Out[19]:

'MacDonald'

In [20]:

```
old_macdonald ('ma')
```

Out[20]:

'Name is too short!

MASTER YODA: تابعی بنویسید تا یک رشته را دریافت نمایید و ترتیب کلمات را در آن مطابق مثالهای زیر معکوس نماید :

master_yoda('I am home') --> 'home am I'

master_yoda ('We are ready')--> 'ready are We'

In [25]:

```
def master_yoda(text):
  return '   '.join(text.split()[::-1])
```

In [26]:

```
1  # Check
2  master_yoda ('I am home')
```

Out[26]:

'home am I'

In [28]:

```
1  # Check
2  master_yoda ('We are ready')
```

Out[28]:

'ready are We'

ALMOST THERE: تابع بنویسید که یک عدد را دریافت کند و اگر در فاصله ۱۰ تا از ۱۰۰ یا ۲۰۰ باشد True و در غیراینصورت False را برگرداند

almost there (90) --> True

almost there (104) --> True

almost there (150) --> False

almost there (209) --> True

NOTE: abs (num) returns the absolute value of a number

In [29]:

```
1  def almost there (num):
2      return ((abs (100 num) <= 10) or (abs (200 num) <= 10))
```

In [30]:

```
# Check
almost there (90)
```

Out[30]:

True

In [31]:

```
# Check
almost there (104)
```

Out[31]:

True

In [32]:

```
# Check
almost there (150)
```

Out[32]:

False

In [33]:

```
# Check
almost there (209)
```

Out[33]:

True

مسائل سطح ۲

FIND 33:

تابعی بنویسید که لیستی از اعداد صحیح را دریافت نماید و اگر دو عضو با مقدار ۳ پشت سرهم پیدا نماید مقدار True را برگرداند و در غیر اینصورت مقدار False

has_33([1, 3, 3]) -->True

has_33([1, 3, 1, 3])-->False

has_33([3, 1, 3])--> False

In [35]:

```
1  def has 33 (nums):
2      for i in range(0, len (nums) -1 ):
3
4          if nums[i:i+2] == [3,3]:
5              return True
6
7      return False.
```

In [36]:

```
1  # Check
٢  Has_33([1, 3,3])
```

Out[36]:

True

In [37]:

```
1  # Check
2  Has_33([1, 3,1,3])
```

Out[37]:

False

In [38]:

```
1 # Check
2 Has_33([3,1,3])
```

Out[38]:

False

تابعی بنویسید که یک رشته را بعنوان ورودی دریافت نماید و یک رشته جدید بعنوان خروجی برگرداند که در آن به ازای هر حرف در رشته اصلی سه بار در رشته جدید تکرار شده باشد. :PAPER DOLL

paper_doll('Hello') -> 'HHHeeel11111000'

paper_doll('Mississippi') --> 'MMMiiissssssiiipppppppiii'

In [39]:

```
1 def paper doll (text):
2     result = ' '
3
4    for char in text:
5        result += char *3
6   return result
```

In [40]:

```
1 # Check
٢ paper_doll('Hello')
```

Out[40]:

'HHHeeellllllooo'

In [41]:

```
1 # Check
2 paper_doll('Mississippi')
```

Out[41]:

'MMMiiissssssiiissssssiiipppppppiii'

BLACK JACK :تابعی بنویسید که سه عدد صحیح در بازه ۱ تا ۱۱ دریافت کند در صورتیکه مجموعه اعداد کمتر مساوی ۲۱ باشد مقدار مجموع را برگرداند در صورتیکه مجموعه بیشتر از ۲۱ ،باشد ؛ مجموع را منهای ۱۰ کند اگر مقدار جدید کمتر از ۲۱ باشد و در میان اعداد ۱۱ ،باشد مجموع جدید آن را گزارش کند و در غیر اینصورت عبارت Bust را چاپ کند.

blackjack (5,6,7) --> 18

blackjack(9,9,9) --> 'BUST'

blackjack (9,9,11) --> 19

In [43]:

```
1  def blackjack(a, b, c):
2
3      if sum((a, b, c)) <= 21:
4          return sum((a, b, c))
5      elif sum((a, b, c)) <= 31 and 11 in (a, b, c): return sum((a, b, c)):
6          return sum((a, b, c))  - 10
7      else:
8          return 'BUST'
```

In [44]:

```
1  # Check
٢  blackjack (5,6,7)
```

Out[44]:

18

In [45]:

```
1  # Check
2  blackjack (9,9,9)
```

Out[45]:

'BUST'

In [46]:

```
# Check
blackjack (9,9,11)
```

Out[46]:

19

SUMMER OF ۶۹: تابعی بنویسید که یک لیست از اعداد صحیح را بعنوان ورودی دریافت کند و مجموع اعداد را با در نظر گرفتن شرایط مقابل محاسبه نماید اگر در هنگام محاسبه مجموع به عضوی در لیست با مقدار ۶ برسیم خود آن عضو و اعضای بعدی را رسید به عضوی که مقدار آن ۹ است در مجموع لحاظ نخواهیم کرد(در لیست ورودی حتما پس از ۶ باید حداقل یک ۹ بیاید)

summer_69([1, 3, 5]) --> 9

summer_69([4, 5, 6, 7, 8, 9]) -> 9

summer_69([2, 1, 6, 9, 11]) --> 14

In [32]:

```
def summer_69(1st):

    total = 0
    add = True

  for num in 1st:
      while add:
          if num != 6:
              total += num
              break
          else:
              add = False

    while not add:
        if num != 9:
            break
        else:
            add = True
            break
  return total
```

In [33]:

```
# Check
summer_69([1, 3, 5])
```

Out[33]:

9

In [34]:

```
# Check
summer_69([4, 5, 6, 7, 8, 9])
```

Out[33]:

9

In[35]:

```
# Check
summer_69([2, 1, 6, 9, 11])
```

Out[33]:

14

مسائل چالش برانگیز

SPY GAME :تابعی بنویسید تا لیست از اعداد صحیح را بعنوان ورودی دریافت نماید و اگر در میان اعضا بتوان سه عضو پشت سرهم یا با فاصله پیدا نماید که الگوی ۰۰۷ را ایجاد کند مقدار True را برگرداند در غیر اینصورت False را برگرداند

spy game ([1,2,4,0,0,7,5]) --> True

spy game([1,0,2,4,0,5,7]) --> True

In [47]:

```
def spy_game (nums):

    code = [0, 0, 7, 'x']

    for num in nums:
        if num == code[0]:
            code.pop(0)
    return len(code) == 1
```

In [48]:

```
# Check
spy game ([1,2,4,0,0,7,5])
```

Out[48]:

True

In [49]:

```
# Check
spy game([1,0,2,4,0,5,7])
```

Out[49]:

True

In [50]:

```
# Check
spy game ([1,7,2,0,4,5,0])
```

Out[50]:

False

COUNT PRIMES :تابعی بنویسید که یک عدد صحیح را بعنوان ورودی دریافت کند و لیست اعداد اول تا خود عدد ورودی را چاپ و تعداد آنها را چاپ کند

count primes (100) --> 25

.By convention, 0 and 1 are not prime

In [51]:

```
def count primes (num):
    primes = [2]
    x = 3
    if num < 2:
        return 0
    while x <= num:
        for y in range(3, x, 2):
            if x % y == 0:
                x += 2
                break
        else:
            primes.append(x) #primes => [2, 3, 5, 7]
            x += 2
    print(primes)
    return len(primes)

```

In [52]:

```
# Check
count_primes(100)
```

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 7 1, 73, 79, 83, 89, 97]

Out[52]:

25

جایزه :راه حل سریعتر که اعداد اول را در طول مسیر پیمایش جمع آوری می نماید

In [53]:

```
1  def count primes2(num):
2      primes = [2]
3      x = 3
4      if num < 2:
5          return 0
6      while x <= num:
7          for y in primes: # use the primes list!
8              if x%y == 0:
9                  x += 2
10                 break
11         else:
12             primes.append(x)
13             x += 2
14     print (primes)
15     return len(primes)
```

In [43]:

```
1  count_primes2(100)
```

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 7 1, 73, 79, 83, 89, 97]

Out[43]:

25

صرفا جهت فان(:

PRINT BIG :تابعی بنویسید که یک حرف انگلیسی را دریافت کند و آن را با حرف بزرگ و با استفاده از علامت های * چاپ کند.

print_big('a')

out:

```
  *
 * *
*****
*   *
*   *
```

راهنما: در نظر داشته باشید که یک فرهنگ لغت از الگوهای ممکن ایجاد کنید و حروف الفبا را به ترکیبات خاص ۵ خطی از الگوهای مرتبط نگاشت .کنید برای اهداف این تمرین اگر فرهنگ لغت شما در "E" متوقف شود مشکل نیست.

In [67]:

```
1 def print_big(letter):
2     patterns = {1:'*',2:'**,3:'**',4:**,5:*',6:*',7: *
3     alphabet {'A':[1,2,4,3,3], 'B':[5,3,5,3,5], 'C':[4,9,9,9,4], 'D':[5,3,3,3,5], 'E':
4     for pattern in alphabet [letter.upper()]:
5     print(patterns [pattern])
```

In [70]:

```
1 print_big('c')
```

```
*****
*
*
*
*****
```

عبارات لامبدا، نگاشت (map) و فیلتر(filter)

حالا وقت آن است که در مورد دو تابع داخلی فیلتر و نقشه سریع یاد بگیریم. پس از اینکه در مورد چگونگی عملکرد آنها یاد گرفتیم میتوانیم در مورد عبارت لامبدا یاد بگیریم که هنگامی که مهارتهای خود را بیشتر توسعه می دهید، به درد میخورد

تابع نگاشت

تابع نگاشت به شما اجازه میدهد تا یک تابع را به یک شی قابل تکرار" نگاشت کنید به این معناست که شما می توانید به سرعت همان تابع را به هر آیتم در یک قابل تکرار مانند یک لیست فراخوانی کنید. به عنوان مثال:

In [١]:

```
1  def square (num):
٢      return num  ** 2
```

In [2]:

```
1  my_nums = [1,2,3,4,5]
```

In [3]:

```
1  map(square, my_nums)
```

Out[3]:

<map at 0x1d811b505e0>

In [4]:

```
1  # To get the results, either iterate through map()
2  # or just cast to a list
3  list(map(square, my_nums))
```

Out[4]:

[1, 4, 9, 16, 25]

In [6]:

```
1  [map(square, my_nuns) ]
```

Out[6]:

[1, 4, 9, 16, 25]

تابع مورد استفاده در نگاشت میتواند پیچیده تر نیز باشد

In [7]:

```
def splicer (mystring):
    if len(mystring)%2==0:
        return 'even'
    else:
        return mystring[0]
```

In [8]:

```
mynanes = [John', 'Cindy', 'Sarah', 'Kelly', "Mike"]
```

In [9]:

```
[*map(splicer, mynames)]
```

out[9] :

['even', 'C', 'S', 'K', 'even ']

تابع فیلتر

تابع فیلتر یک تکرار کننده را برمی گرداند که شامل اعضای از دنباله ورودی خواهند بود که مقدار تابع فیلتر کننده برای آنها True باشد. به این معنی که شما باید با یک تابع فیلتر کنید که یا True یا False را بازگرداند. سپس تابع مذکور و دنباله خود را بعنوان ورودی به تابع فیلتر بدهید و در مقابل فقط اعضایی را دریافت خواهید نمود که تابع برای آنها مقدار True را برگردانده است.

In [10]:

```
def check_even (num):
    return num % 2 == 0
```

In [11]:

```
nums = [0,1,2,3,4,5,6,7,8,9,10]
```

In [12]:

```
1 filter(check_even, nums)
```

out[12] :

<filter at 0x1d8130250f0>

In [13]:

```
1 [* filter(check_even, nums)]
```

out[13] :

[0, 2, 4, 6, 8, 10]

عبارت لامبدا

یکی از ابزارهای مفید (و برای مبتدیان گیج کننده) پایتون عبارت لامبدا است. عبارات لامبدا به ما امکان می دهد تا توابع "ناشناس" ایجاد کنیم. این در اصل به معنای این است که میتوانیم به سرعت توابع

ad-hoc ایجاد کنیم بدون نیاز به تعریف صحیح یک تابع با استفاده از .def

شی های تابع با اجرای عبارات لامبدا دقیقاً همانطور که توسط defs ایجاد و اختصاص داده شده است، کار می کنند تفاوت کلیدی وجود دارد که لامبدا را در نقش های تخصصی مفید می کند

بدنه لامبدا یک عبارت منفرد است نه یک بلوک از بیانات.

- بدنه لامبدا شبیه به آنچه که در بیان بازگشت der body قرار می دهیم است. ما به سادگی نتیجه را به عنوان یک عبارت تایپ می کنیم به جای بازگشت آن به صورت صریح به دلیل محدود شدن به یک دستور، استفاده از آن نسبت به توابع معمولی کمتر است. لامبدا برای پیاده سازی توابع ساده طراحی شده است و def وظایف بزرگتر را انجام میدهد بیایید به آرامی یک عبارت لامبدا را با تجزیه یک تابع بسازیم

In [14]:

```
1 def square (num):
2     result = num**2
3    return result
```

In [16]:

```
1 square (5)
```

Out[16] :

25

تابع بالا را میتوانیم خلاصه تر نماییم:

In [17]:

```
1 def square(num):
2     return num**2
```

In [18]:

```
1 square (5)
```

Out[18] :

25

جالبه که می تونیم این تابع را حتی در یک سطر خلاصه کنیم

In [19]:

```
1 def square (num): return num**2
```

In [20]:

```
1 square(7)
```

Out[20] :

49

در واقع هدف عبارات لامبدا ساخت توابعی مشابه بالا میباشد تابع بالا را میتوانیم با استفاده از لامبدا بصورت زیر بازنویسی کنیم.

In [۲۱]:

```
1 lambda num: num**2
```

Out[۲۱] :

<function main.<lambda> (num)>

In [22]:

```
1 # You wouldn't usually assign a name to a Lambda expression, this is just for demons
2 lbd_square= lambda num: num ** 2
```

In [23]:

```
1 lbd_square (8)
```

Out[23] :

64

سوالی که شاید پیش بیاید این است که چرا باید از چنین تابعی استفاده نماییم در بسیاری از توابع مانند نگاشت و فیلتر ما از تابع ورودی که غالبا ساختار ساده ای دارند یکبار استفاده میکنیم از این رو معمولا از عبارات لامبدا در اینگونه سناریوها استفاده می نماییم.

In [۲۴]:

```
1 [map(lambda num: num ** 2, my_nums) ]
```

Out[۲۴] :

[1, 4, 9, 16, 25]

In [۲۵]:

```
1 [* filter(lambda n:n % 2== 0, nums)]
```

Out[۲۵] :

[0, 2, 4, 6, 8, 10]

در ادامه به بررسی چند مثال می پردازیم. توجه به این نکته ضروری است که هرچه یک تابع پیچیده تر باشد که تبدیل آن به یک عبارت لامبدا سخت تر خواهد بود و گاهی اوقات تنها راه استفاده از یک تابع معمولی می باشد.

مثال ۱: یک عبارت لامبدا برای استخراج اولین کاراکتر یک رشته:

In [۲۶]:

```
1 first_letter = lambda s: s[0]
```

In [۲۷]:

```
1 first_letter ('Hello')
```

Out[27]:

'H'

مثال ۲: یک عبارت لامبدا برای معکوس کردن یک رشته:

In [ ]:

```
1 lambda s: s[: :-1]
```

همچنین ما میتوانیم تعداد پارامتر ورودی بیشتری را برای لامبدا تعریف کنیم:

In [28]:

```
1 Add = lambda x,y: x + y
```

In [29]:

```
1 add(5, 6)
```

Out[29]:

11

عبارات لامبدا در بسیاری از کتابخانه مورد استفاده میشود برای مثلا در کتابخانه Pandas از این عبارات بسیار استفاده می شود. این کتابخانه در بحث یادگیری ماشین داده کاوی و هوش مصنوعی بشدت کاربردی می باشد.

دستورات تو در تو و محدودهNested Statements and Scope

حالا که ما در مورد نوشتن توابع خودمان صحبت کردیم مهم است که بدانیم پایتون چگونه با نامهای متغیری که شما تعیین می کنید برخورد میکند. زمانی که شما یک نام متغیر در پایتون ایجاد میکنید این نام در یک فضای نام ذخیره میشود. نامهای متغیر همچنین یک محدوده یا scope دارند این محدوده قابلیت دیدن آن نام متغیر را برای بخشهای دیگر کد شما تعیین میکند.

بیایید با یک تجربه فکری سریع شروع کنیم؛ تصور کنید کد زیر را:

In [2]:

```
1  x = 25
2
3  def printer():
4      x = 50
5      return x
6
7  printer()
8  # print(x)
9  # print(printer())
```

Out[2]:

50

بنظر شما خروجی تابع printer() چه خواهد بود؟ ۲۵ یا ۵۰؟ خروجی چاپ x در سطر ۷ چطور؟ ۲۵ یا ۵۰؟

In [3]:

```
1  print(x)
```

In [4]:

```
1  print (printer())
```

50

جالب است اما پایتون چگونه میداند که شما در کد خود به کدام اشاره میکنید؟ در اینجا ایده محدوده وارد میشود. پایتون یک مجموعه قوانین دارد که برای تصمیم گیری در مورد اینکه شما به چه متغیرهایی (مانند x در این مورد) در کد خود اشاره میکنید آنها را دنبال می.کند. بیایید قوانین را تجزیه و تحلیل کنیم:

این ایده محدوده در کد شما بسیار مهم است تا بتوانید به درستی نام های متغیر را تعیین و فراخوانی کنید. به زبان ساده ایده محدوده را میتوان با ۳ قانون عمومی توصیف کرد:

*

1. تخصیص نام به طور پیش فرض نامهای محلی را ایجاد یا تغییر میدهد .۲ جستجوی نام (حداکثر) در چهار محدوده. اینها عبارتند از * -Built * Global (module * Local Enclosing function locals in نام های اعلام شده در جملات جهانی و نام های غیر محلی تخصیص داده شده را به محدوده های ماژول و تابع محصور نگاشت میکنند.

جمله در ۲ بالا را میتوان با قانون LEGB تعریف کرد.

قانون LEGB

L: Local - نامهای تخصیص داده شده به هر نحو در داخل یک تابع de) یا (lambda که در سطح جهانی اعلام نشده است.

E: Enclosing function locals - نامهایی در محدوده محلی هر و همه توابع محصور کننده der) یا لامیدا)، از درونی بهبیرونی...

G: Global (module) - نامهایی که در سطح بالای یک فایل ماژول اختصاص داده میشوند یا در یک تعریف جهانی در قابل تعریف میشوند.

B: Built-in (Python)- نامهای از پیش تعیین شده در ماژول نامهای داخلی open، range، SyntaxError....

مثالهای ساده ازLEGB

محلی

In [5]:

```
# x is Local here:
f = lambda x: x ** 2
```

Enclosing function locals

این حالت زمانی رخ میدهد که ما یک تابع درون یک تابع دیگر داشته باشیم.

In [8]:

```
name = "This is a global name"

def greet():
    # Enclosing function
    name = 'Sammy'

    def hello():
        # name = 'John ==> Local
        print('Hello ' + name)
    hello()

greet()
```

Hello John

توجه کنید که چگونه تابع hello() به مقدار متغیر name دسترسی پیدا کرد! زیرا تابع hello() در داخل تابع greet() محصور شده است.

Global

در داخل جوپیتر براحتی میتوان جهانی بودن یک متغیر را چک کرد اگر یک متغیر در سلول دیگری قابل دسترسی باشد پس آن متغیر بصورت جهانی تعریف شده است:

In [9]:

```
name = "This is a global name"

def greet():
    # Enclosing function
    #name = 'Sammy'

    def hello():
        #name = 'John'==> Local
        print('Hello+ name)
    hello()

greet()
```

Hello This is a global name

In [10]:

```
print(name)
```

This is a global name

Built-in

برای مثال تابع len یا id جزو توابع پیش فرض پایتون هستند.

In [11]:

```
print
```

Out[11]:

<function print>

In [12]:

```
len
```

Out[12]:

<function len(obj, /)>

In [13]:

```
1 id
```

Out[13]:

<function id(obj, /)>

In [15]:

```
# Global scope
# name = "This is a global name"

def greet():
    # Enclosing function scope
    # name = 'Sammy'

  def hello():
      #name = 'John' ==> Local scope
      print('Hello+ firstname)
 hello()

greet()
```

NameError

Traceback (most recent call last)

Cell In[15], line 13

10 print('Hello+ firstname)

11 hello()

---->13 greet()

Cell In[15], line 11, in greet()

```
      8   def hello():
      9   # name = 'John' ==> Local scope
     10    print('Hello' + firstname)
----> 11   hello()

Cell In[15], line 10 , in greet.<locals>.hello()
      8   def hello():
      9  # name = 'John' ==> Local scope
----> 10    print('Hello+ firstname)
NameError: name 'firstname' is not defined
```

متغیرهای محلی

وقتی متغیرها را در داخل تعریف تابع اعلام میکنید به هیچ وجه به متغیرهای دیگر با همین نام که خارج از تابع استفاده می شوند. ارتباطی ندارند به عبارت دیگر نام متغیرها برای تابع محلی است. این مسئله را دامنه متغیر (Variable scope) نامیده اند. همه متغیرها دامنه بلوکی را که در آن تعریف شده اند از نقطه تعریف نام خود شروع کرده و دارای دامنه بلوک هستند.

مثال:

In [15]:

```
1  x = 50
2
3  def func(x):
4      print('x is', x)
5      X = 2
6      print('Changed local x to', x)
7
8  func(x)
9  print('x is still', x)
```

```
x is 50

Changed local x to 2

x is still 50
```

اولین باری که مقدار نام را با خط اول در بدنه تابع چاپ میکنیم پایتون از مقدار پارامتری استفاده میکند که در بلوک اصلی بالای تعریف تابع تعریف شده است.

سپس مقدار ۲ را به x اختصاص میدهیم نام محلی برای تابع ،ماست ،بنابراین هنگامی که مقدار x را در تابع تغییر می دهیم، در بلوک اصلی تحت تأثیر قرار نمی گیرد

با آخرین دستور ،چاپ مقدار x را به عنوان در بلوک اصلی تعریف شده نمایش میدهیم و به همین ترتیب تأیید می کنیم که در واقع تحت تأثیر قرار نگرفته است.

دستور global

اگر میخواهید به یک نام در سطح بالای برنامه (بدون هرگونه دامنه از جمله توابع یا کلاس ها مقدار اختصاص دهید، باید به پایتون بگوئید که نام مورد نظر شما محلی نبوده و بلکه جهانی است. این کار را با استفاده از دستور global انجام می دهیم. اختصاص دادن مقدار به یک متغیر خارج از یک تابع بدون استفاده از دستور global غیرممکن است.

شما می توانید از اینگونه متغیرهای خارج از تابع ب) فرض عدم وجود هرگونه متغیر با همین نام در داخل تابع استفاده کنید با این حال این کار پاسخ دادن به خواننده برنامه را سخت و غیر واضح می.کند استفاده از دستور global باعث وضوح شود که متغیر در یک بلوک بالاترین سطح تعریف شده است.

مثال:

In [20]:

```
1  x = 50
2
3  def func():
4          global x
5          print('This function is now using the global x!')
6         print('Because of global x is:, x, id(x))
7        x = 2
8       print('Ran func(), changed global x to', x, id(x))
9
10 print('Before calling func(), x is: ', x, id(x))
11 func()
12 print('Value of x (outside of func()) is:
```

Before calling func(), x is: 50 3032342136592

This function is now using the global x!

Because of global x is:50 3032342136592

Ran func(), changed global x to 2 3032342135056

Value of x (outside of func()) is:2 3032342135056

دستور global برای اعلام این مورد استفاده میشود که یک متغیر جهانی است ،بنابراین هنگامی که مقدار x را درون تابع اختصاص میدهیم آن تغییر در هنگام استفاده از مقدار x در بلوک ،اصلی نمایش داده می شود.

شما میتوانید با استفاده از یک دستور global بیشتر از یک متغیر جهانی را مشخص کنید به عنوان مثال global x,z, y,Z

نتیجه گیری

شما باید در حال حاضر درک خوبی از Scope داشته باشید ممکن است قبلاً به طور ذاتی درست درباره Scope حس کرده باشید که عالی (است چیز دیگری که باید به خاطر داشته باشید این است که همه چیز در پایتون یک شیء است! من می توانم متغیرها را به توابع نسبت دهم مانند کاری که با اعداد انجام میدادیم ما در بخش decorator دوره به این موضوع دوباره خواهیم پرداخت!

سری دوم تمرینات توابع

به سوالات زیر پاسخ دهید

تابعی بنویسید تا حجم کره را با دریافت شعاع آن محاسبه نماید.

حجم کره براساس رابطه زیر محاسبه می شود

$$\frac{4}{3}\pi r^3$$

In [6]:

```
1  import math
2
3
4  def vol(rad):
5      return (4 / 3) * (math.pi)*(rad ** 3)
```

In [7]:

```
1  # Check
2  vol(2)
```

out[7]:

33.510321638291124

In [8]:

```
1  math.pi
```

out[8]:

3.141592653589793

تابعی بنویسید که بررسی کند یک عدد در بازه بسته داده شده قرار دارد یا نه و برای هرکدام از حالات یک جمله چاپ کند

تابع سه ورودی خواهد داشت مقدار عددی که باید بررسی شود ابتدای بازه و انتهای بازه

In [۱۰]:

```
1  def ran check (num, low, high):
۲      if num in range (low, high + 1):
۳          print(f' {num} is in the range between {low} and {high}')
۴      else:
۵          print('The number is outside of the range')
```

In [۱۳]:

```
1 # Check
2 ran_check (5,2,7)
```

5 is in the range between 2 and 7

در صورتیکه بخواهیم برای گزارش نتیجه یک مقدار بولین برگرداند چطور؟

In [۱۴]:

```
1 def ran_bool (num, low, high):
2   return num in range(low, high + 1)
```

In [۱۶]:

```
1 ran_bool(15,1,10)
```

Out[16] :

False

تابع پایتونی بنویسید که یک رشته را میپذیرد و تعداد حروف بزرگ و کوچک را محاسبه می کند.

رشته نمونه *Hello Rogers how are you this fine Tuesday* خروجی مورد انتظار تعداد حروف بزرگ ۴ تعداد کاراکترهای کوچک: ۳۳

راهنمایی دو متد رشته ای که ممکن است مفید باشد *.isupper*() و *.islower*()

اگر احساس کردید بهتر است ماژول *Collections* را برای حل این مسئله بررسی کنید!

In [۱۷]:

```
def up_low(text):
        d = {'upper' : 0, 'lower' : 0}

      for char in text:
           if char.isupper():
               d['upper'] += 1
          elif char.islower():
               d['lower'] += 1
           else:
              pass
    print(f'Original String{text}')
   print (f"No. of Upper case characters: {d[ 'upper']}")
    print (f"No. of Lower case characters:{d['lower']}")
```

In [۱۸]:

```
s = 'Hello Mr. Rogers, how are you this fine Tuesday?'
up low(s)
```

Original String Hello Mr. Rogers, how are you this fine Tuesday?

*No. of Upper case characters:*۴

No. of Lower case characters: 33

تابع پایتونی بنویسید که یک لیست را میگیرد و لیست جدیدی با عناصر منحصر به فرد لیست اول برمی گرداند.

Sample List: [1,1,1,1,2,2,3,3,3,3,4,5]

Unique List [1, 2, 3, 4, 5]

In [۱9]:

```
1  def unique_list_set(1st):
2      return list(set(1st))
```

In [20]:

```
1  unique_list_set([1,1,1,1,2,2,3,3,3,3,4,5])
```

Out[20]:

[1, 2, 3, 4, 5]

In [21]:

```
1  def unique_list_for_loop (1st):
2
3      x = [ ]
4
5    for a in 1st:
6        if a not in x:
7            x.append(a)
8  return x
9
10
```

In [22]:

```
1  unique_list_for_loop([1,1,1,1,2,2,3,3,3,3,4,5])
```

Out[22]:

[1, 2, 3, 4, 5]

تابع پایتونی بنویسید که تمام اعداد یک لیست را در هم ضرب کند

Sample List: [1, 2, 3, 4]

Expected Output: -24

In [23]:

```
1  def multiply (numbers):
2      total = 1
3
4      for x in numbers:
5  total *= x # total = total *X
6  return total
```

In [24]:

```
1  multiply([1,2,3,-4])
```

Out[24]:

-24

تابع پایتونی بنویسید که چک کند که یک کلمه یا عبارت پالیندروم است یا خیر.

توجه پالیندروم کلمه عبارت یا دنباله ای است که از هر دو طرف مانند هم خوانده میشود به عنوان مثال، *madam.nurses run", kayak. racecar"*

راهنمایی :شما ممکن است بخواهید به روش .*replace*() در رشته برای کمک به مقابله با فضاها نگاه کنید. همچنین جستجو در گوگل برای روشهای باور نکردنی برای برعکس کردن یک رشته در پایتون با استفاده از نوتاسیون اسلایسینگ.

In [27]:

```
1  def palindrome(s):
2      s = s.replace('', '')
3
4      return s == s[::-1]
```

In [31]:

```
1  palindrome ('raccar')
```

Out[31]:

True

تمرین سخت:

تابع پایتونی بنویسید که بررسی کند که یک رشته پانگرام است یا خیر فرض کنید رشته وارد شده هیچ علامت نگارشی ندارد)

توجه: پانگرام ها کلمات یا جملاتی هستند که حداقل یک بار هر حرف الفبای انگلیسی را دارا میباشند به عنوان مثال: *Thequick brown fox jumps over the lazy dog*

راهنمایی :شما ممکن است بخواهید از روش ***replace*** برای رهایی از فضاها استفاده کنید.

In [32]:

```
import string

def ispangram (str1, alphabet-string.ascii_lowercase):

     alphabet set (alphabet)

    # Remove spaces from str1
   str1=str1.replace(' ',' ')

    # Lowercase all strings in the passed string
  str1 = str1.lower()

   # Grab all unique Letters in the string as a set
  str1=set(str1)

  # Now check that the alpahbet set is same as string set

  return str1 == alphabet

```

In [33]:

```
ispangram ("The quick brown fox jumps over the lazy dog")
```

Out[33]:

True

In [34]:

```
1 string.ascii_lowercase
```

Out[34]:

'abcdefghijklmnopqrstuvwxyz'

دست گرمی برای پروژه عملی ۱

زمان آن رسیده است که شما تمام مهارتهای خود را برای شروع ساخت پروژههای قابل استفاده به کار ببندید قبل از اینکه به پروژه عملی خود بپردازید ما از طریق تمرینات دست ،گرمی شما را با چند ایده کلیدی که در پروژه عملی و پروژه های بزرگتر استفاده میکنیم آشنا می کنیم به ویژه:

- دریافت ورودی کاربر
- ایجاد توابع که متغیرها را بر اساس ورودی کاربر ویرایش میکنند
- تولید خروجی
- پیوستن ورودیهای کاربر و جریان منطق

تابع نمایش اطلاعات

ایجاد یک تابع که لیست را برای کاربر نشان میدهد

In [1]:

```
1 def display_list (mylist):
2     print(mylist)
```

In [2]:

```
1 st [0,1,2,3,4,5,6,7,8,9,10]
2 display_list(lst)
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

پذیرش ورودی کاربر

ایجاد تابعی که ورودی را از کاربر دریافت کرده و نتیجه را با نوع داده اعداد صحیح برمی گرداند
هنگام استفاده از تابع ***input()*** مواظب باشید اجرای دوباره سلول بدون ارائه مقدار ورودی باعث میشود پایتون در حالت انتظار برای مقدار اولیه در اجرای اول گیر کند. اگر سلول گیر کردید با توجه به ***in []*** کنار سلول هستید در این صورت به راحتی ***kernel*** را راه اندازی مجدد کنید و هر سلول لازم را دوباره اجرا کنید

In [3]:

```
1 input('Please enter a value: ')
```

Please enter a value: 2

Out[3]:

'۲'

In [4]:

```
1 input('Please enter a value: ')
```

Please enter a value: 2

In [5]:

```
1 result
```

Out[5]:

'2'

In [6]:

```
1 type(result)
```

Out[6]:

str

In [7]:

```
1 int(result)
```

Out[7]:

2

In [8]:

```
1 input('Please enter a value: ')
```

Please enter a value: 7

In [9]:

```
1 type(result)
```

Out[9]:

Int

In [10]:

```
1 # Example of an error!
2 result = int(input('Please enter a value: '))
```

Please enter a value: two

ValueError

Traceback (most recent call last)

Cell In[10], line 2

1 # Example of an error!

----->2 result = int(input('Please enter a value: '))

ValueError: invalid literal for int() with base 10: 'two'

تابع نهایی برای اینکار:

In [11]:

```
def user choice():
    ...
    User inputs a number (0-10) and we return this in integer form. No
    parameter is passed when calling this function.
    ...
    choice input('Please input a number (0-10): ')

    return int(choice)
```

In [12]:

```
user choice()
```

Please input a number (0-10): 5

Out [12]:

5

In [13]:

```
result = user_choice()
```

Please input a number (0-10): 7

In [14]:

```
type(result)
```

Out [14]:

Int

In [15]:

```
result
```

Out [15]:

7

تأیید ورودی کاربر

قبل از تلاش برای تبدیل بررسی کنید که ورودی معتبر است.

ما از یک روش ساده استفاده خواهیم کرد.

هنگامی که پیشرفت میکنید میتوانید شروع به نگاه کردن به روش های دیگر برای انجام این کار کنید این روش ها در درس های بعدی ،دوره بیشتر معنا خواهند داشت بنابراین در حال حاضر نگران آنها نباشید

پلاگهای متعدد:

(*https://www.google.com/search?q=python check+if+input+is+number*)

پست ۱ در استک اور فلو:

(*https://stackoverflow.com/questions/5424716/how-to-check-if-string input-is-a-number*)

پست ۲ در استک اورفلو:

(*https://stackoverflow.com/questions/1265665/how-can-i-check-if-a-*)

In [۲۰]:

```
1 some_input = '10'
```

In [۲۱]:

```
1 some_input = '10'
```

Out [۲۱]:

True

حال تابع را براساس مطالب فوق اصلاح نمایید تا ورودی مناسب را دریافت و پردازش کند

In [۲2]:

```
def user_choice():

    # This original choice value can be anything that isn't an integer
    choice= 'wrong'

    # While the choice is not a digit, keep asking for input
    while choice.isdigit() == False:

    # we shouldn't convert here, otherwise we get an error on a wrong input
    choice = input("Choose a number: ")

    # We can convert once the while loop above has confirmed we have a digit.
    return int(choice)
```

In [۲3]:

```
user choice()
```

Choose a number: hello

Choose a number: two

Choose a number: 5

Out[23]:

5

In [۲4]:

```
def user choice():

    # This original choice value can be anything that isn't an integer
    choice = 'wrong'

    # While the choice is not a digit, keep asking for input.
    while choice.isdigit() == False:

    # we shouldn't convert here, otherwise we get an error on a wrong input
    choice = input("Choose a number: ")

    # Error Message check:
    if choice.isdigit() == False:
        print('Sorry, but you did not enter an integer number. Please try again!

    # We can convert once the while Loop above has confirmed we have a digit.
    return int(choice)
```

In [۲6]:

```
user_choice()
```

Choose a number: asd

Sorry, but you did not enter an integer number. Please try again! Choose a number: asda

Sorry, but you did not enter an integer number. Please try again! Choose a number: asdsa

Sorry, but you did not enter an integer number. Please try again! Choose a number: ads

Sorry, but you did not enter an integer number. Please try again! Choose a number: asdafsadsf

Sorry, but you did not enter an integer number. Please try again! Choose a number: 5

Out[26]:

5

حالا بیایید بررسی کنیم چگونه خروجی را پاک کنیم به این ترتیب ما تاریخچه اظهارات انتخاب یک شماره را نمی بینیم. :توجه کاربران *Jupyter Notebook* از روش *IPython* نشان داده شده در اینجا استفاده خواهند کرد. سایر کاربران *ID* (*PyCharm. VS*)

In [۲۷]:

```
1 from IPython.display import clear_output
۲ clear_output()
```

In [۲۸]:

```
1  def user choice():
۲
۳      # This original choice value can be anything that isn't an integer
۴     choice = 'wrong'
۵
۶    #While the choice is not a digit, keep asking for input.
۷    while choice.isdigit() == False:
۸
۹       # we shouldn't convert here, otherwise we get an error on a wrong input
۱۰     choice input("Choose a number: ")
۱۱
۱۲      # Error Message check:
۱۳      if choice.isdigit() == False:
۱۴
۱۵         clear_output()
۱۶
۱۷         print('Sorry, but you did not enter an integer number. Please try again!
۱۸
۱۹  # We can convert once the while Loop above has confirmed we have a digit.
۲۰ return int(choice)
```

In [۲۹]:

```
1 user_choice()
```

Sorry, but you did not enter an integer number. Please try again! Choose a number: 5

Out[29]:

5

بررسی صحت ورودی برای اطمینان از انتخاب یکی از گزینه های قابل قبول

In [30]:

```
1 result = 'wrong value'
2 acceptable values = ['0', '1', '2']
```

In [31]:

```
1 result in acceptable_values
```

Out[31]:

False

In [32]:

```
1 result not in acceptable_values
```

Out[32]:

True

In [33]:

```
1 from IPython.display import clear_output
2 clear_output()
```

In [34]:

```
def user_choice():

    # This original choice value can be anything that isn't an integer
    choice = 'wrong'

    #While the choice is not a digit, keep asking for input.
    while choice not in ['0', '1', '2']:

        # we shouldn't convert here, otherwise we get an error on a wrong input
        choice = input("Choose one of these numbers (0, 1, 2): ")

        # Error Message check:
        if choice not in ['0', '1', '2']:

            clear_output()

            print('Sorry, but you did not enter an integer number. Please try again!

    # We can convert once the while loop above has confirmed we have a digit.
    return int(choice)
```

In [35]:

```
user_choice()
```

Sorry, but you did not enter an integer number. Please try again!

Choose one of these numbers (0, 1, 2): 2

Out [35]:

۲

مثال انعطاف پذیرتر

In [36]:

```
def user choice():

    choice 'wrong'

    within_range= False

    while choice.isdigit() == False or within_range == False:
        choice = input("Choose a number (0-10): ")

        if choice.isdigit() == False:
            print('Sorry that is not a digit!')

        if choice.isdigit() == True:
            if int(choice) in range(0, 11):
                within_range= True
            else:
                within_range= False

return int(choice)
```

In [37]:

```
user_choice()
```

Choose a number (0-10): asd

Sorry that is not a digit!

Choose a number (0-10): 15

Choose a number (0-10): 5

Out [37]:

5

تعامل ساده کاربر

در نهایت بیایید تمام این ایده ها را ترکیب کنیم تا یک بازی کوچک ایجاد کنیم که در آن کاربر میتواند یک موقعیت را در یک لیست موجود انتخاب کرده و آن را با یک مقدار از انتخاب خود جایگزین کند.

In [38]:

```
1 game_list = [0,1,2]
```

In [39]:

```
1 def display_game (game_list):
2     print("Here is the current list")
3     print (game_list)
```

In [40]:

```
1 display_game (game_list)
```

```
Here is the current list
[0, 1, 2]
```

In [41]:

```
def position_choice():

    # This original choice value can be anything that isn't an integer
    choice= 'wrong'

    #While the choice is not a digit, keep asking for input.
    while choice not in ['0', '1', '2']:

    # we shouldn't convert here, otherwise we get an error on a wrong input
    choice = input("Pick a position to replace (0,1,2): ")

    if choice not in ['0', '1', '2']:
        # THIS CLEARS THE CURRENT OUTPUT BELOW THE CELL
        clear_output()

        print("Sorry, but you did not choose a valid position (0,1,2)")

    # Optionally you can clear everything after running the function
    # clear_output()

    # We can convert once the while Loop above has confirmed we have a digit.

    return int(choice)
```

In [42]:

```
def replacement_choice (game_list, position):
    user_placement = input("Type a string to palce at the position: ")

    game_list [position] = user_placement

    return game_list
```

In [43]:

```
def gameon_choice():
    # This original choice value can be anything that isn't a Y or N
    choice = 'wrong'

    # While the choice is not a digit, keep asking for input.
    while choice not in ['Y', 'N']:

        # we shouldn't convert here, otherwise we get an error on a wrong input
        choice = input("Would you like to keep playing? Y or N ")


        if choice not in ['Y', 'N']:
            # THIS CLEARS THE CURRENT OUTPUT BELOW THE CELL
            clear_output()

            print("Sorry, I didn't understand. Please make sure to choose Y or N.")



    # Optionally you can clear everything after running the function
    # clear_output()

    if choice == "Y":
        # Game is still on
        return True
    else:
        # Game is over
        return False
```

کنار هم قرار دادن منطق بازی

In [43]:

```
1
2  game_on =True
3
4  game_list =[0, 1, 2]
5
6  while game_on:
7
8        clear_output()
9        display_game (game_list)
10
11       position position_choice()
12
13      game_list =replacement_choice (game_list, position)
14
15      clear_output()
16
17     display_game (game_list)
18
19
20     game_on gameon_choice()
21
22
٢٣ print('Goodbye')
```

Here is the current list

['hi', 'hello', 2]

Would you like to keep playing? Y or NN

Goodbye

عالی کار بزرگی انجام دادید شما در حال حاضر درکی از ترکیب توابع و منطق حلقه برای ساخت یک بازی ساده دارید این در پروژه مرحله بعدی گسترش خواهد یافت!

پروژه عملی ۱

تبریک بابت رسیدن به این مرحله شما!

شما قبلاً بسیاری از مهارتهای خود را یاد گرفته اید و آماده کار با یک پروژه واقعی هستید. وظیفه شما ایجاد یک بازی *Tic Tac Toe* شما آزاد هستید از هر *IDE* ای که دوست دارید استفاده کنید اینجا نیازمندی های آن را مشاهده میکنید

- بازیکن باید قادر باشند بازی را انجام دهند(هر دو در کامپیوتر یکسان نشسته اند).
- باید هر بار که بازیکن حرکت میکند تخته چاپ شود
- شما باید قادر باشید ورودی موقعیت بازیکن را پذیرفته و سپس نماد را در تخته قرار دهید

با استفاده از *Google* به راحتی میتوانید هر چیزی را کشف کنید اما فقط *Tic Tac Toe in Python* را جستجو نکنید در غیر این صورت چیز جدیدی یاد نخواهید گرفت به خاطر داشته باشید که این پروژه ممکن است بین چند ساعت تا چند روز طول بکشد.

۳دفترچه *Jupyter* مربوط به این وظیفه وجود دارد:

- این دفترچه وظایف
- دفترچه کاربردی *W*
- *alkthrough Steps* دفترچه راهنمای گام به گام برای حل *Walkthrough help*
- دفترچه حل *Walkthrough Solution*

من شما را تشویق میکنم فقط سعی کنید پروژه را به تنهایی شروع کنید بدون مرجع گیری به هر یک از دفاتر.

بخش های این پروژه سخت خواهد بود .... و این خوب است! من به شما اعتقاد دارم که اگر تا الان با من همراه بوده اید، می توانید از پس انجام پروژه بربیایید توجه کنید که پروژه کاملا جزوه باز ،هست پس زمان بگذارید و مطالعه و فکر بیشتر سعی نمایید پاسخ خود را ارائه کنید

پروژه عملی ۱ :بازی دوز - راهنمای گام به گام

در این بخش راهنمای گام به گام بازی دوز یا *Tic Tac Toe* را در اختیار شما قرار می دهیم.

مرور برخی از ابزارهایی که در بخش دست گرمی استفاده کردیم:

دریافت ورودی کاربر:

```
player1 =input("Please pick a marker 'X' or '0'")
```

توجه نمایید که تابع *input* ورودی کاربر را در قالب رشته برمیگرداند به همین دلیل برای تبدیل به عدد صحیح از روش زیر برای *cast* استفاده نمایید:

```
position = int(input('Please enter a number'))
```

برای پاک کردن صفحه نمایش میان تعویض نوبت بازی بازیکنان از روش زیر استفاده نمایید:

```
from IPython.display import clear_output

clear_output()
```

توجه نمایید که دستور *clear_output ()* تنها در محیط *pyter Notebook* قابل اجرا میباشد و برای *IDE* های دیگر از روش زیر استفاده نمایید

```
print('\n' *100)
```

دستور بالا خروجی کنسول قبلی را از محدوده نمایش خارج می نماید. حالا بریم سراغ کد:

مرحله ۱ :تابعی بنویسید که بتواند یک تخته را چاپ کند تخته خود را به عنوان یک لیست تنظیم کنید به طوری که هر شاخص *1-9*با یک شماره در پد شماره مطابقت داشته باشد بنابراین نمایش ۳ در ۳ تخته را دریافت میکنید

X	O	X
O	X	O
X	O	X

In [ ]:

```
1  from IPython.display import clear_output
2
3  def display_board (board) :
4
5          pass
```

تست مرحله ۱ :تابع خود را با ورودی آزمایش زیر از نقشه بورد مورد آزمایش قرار دهید و در صورت نیاز فاصله ها را تنظیم نمایید.

In [ ]:

```
1  test_board = [ '#', 'X', 'O', 'X', 'O', 'X', 'O', 'X','O','X']
۲  display_board(test_board)
```

مرحله ۲ :تابع بنویسید تا ورودی کاربران در قالب یک عدد که مکان علامت یا را بیان میکند دریافت نماید. می توانید از یک حلقه *while* برای انجام پرسش تا زمان دریافت ورودی صحیح استفاده نمایید.

In [ ]:

```
1  def player_input():
۲
3      pass
```

تست گام دوم :تابع را اجرا نمایید تا از دریافت خروجی مورد نظر اطمینان حاصل نمایید.

In [ ]:

```
1  player_input()
```

گام سوم :تابعی بنویسید که لیست علامتهای روی بورد بازی *X* یا *O* و مکان علامت گذاری در بازه صفر تا ۹ را دریافت کرده و علامت مورد نظر را در مکان مناسب انتخاب شده قرار دهد.

In [ ]:

```
1  ef place_narker(board, marker, position):
2
3      pass
```

تست مرحله ۳ :تابع فوق را با ورودی بورد و مکان انتخاب شده آزمایش نمایید

In [ ]:

```
1  place_marker(test_board, '$',8)
2  display_board(test_board)
```

مرحله ۴ :تابعی بنویسید تا نشانه *X* یا *O* را بعنوان ورودی دریافت کرده تشخیص دهد که آیا کاربر برنده شده است یا نه

In [ ]:

```
1  def win_check (board, mark):
2
3      pass
```

تست مرحله ۴ :تابع *win_check* را برروی بورد آزمایش خودمان تست کنید باید *True* را برگرداند.

In [ ]:

```
1  win_check(test_board, 'X' )
```

مرحله ۵ :تابعی بنویسید تا با استفاده از ماژول *random* یک بازیکن را بعنوان نفر اول انتخاب کرده و یک رشته برای این کار چاپ .کند میتوانید از تابع *random.randint*() استفاده نمایید.

In [ ]:

```
1  import random
2
3  def choose_first():
4      pass
```

مرحله ۶: تابعی بنویسید که چک کند آیا فضای خالی در بورد بازی در موقعیت مشخص شده وجود دارد یا نه

In [ ]:

```
1  def space_check(board, position):
2
3      pass
```

مرحله ۷ :تابعی بنویسید تا بررسی کند یا بورد بازی توسط علامتهای بازیکنها پر شده است یا نه؟ *True* برای حالت پر و *False* برای حالت دیگر

In [ ]:

```
1  def full_board_check(board):
2
3      pass
```

مرحله ۸ :تابعی بنویسید تا از کاربر موقعیت حرکت بعدی بازیکن را بپرسد (۱-۹) و سپس از تابع مرحله ۶ برای بررسی خالی بودن موقعیت انتخاب شده استفاده نماید و در صورتیکه موقعیت خالی باشد مقدار موقعیت انتخاب شده را برای ادامه بازی برگرداند.

In [ ]:

```
1  def player_choice(board) :
2
3      pass
```

مرحله ۹ : تابعی بنویسید تا از بازیکن بپرسد که تمایل به انجام بازی مجدد دارد یا نه

In [ ]:

```
1  def replay():
2
3      pass
```

مرحله ۱۰ : حالا به مرحله سخت رسیدیم، سعی کنین تمام منطق بازی را در یک حلقه ***while*** با شرط همواره درست قرار دهید

In [ ]:

```
1  print('Welcome to Tic Tac Toe!')
2
3  #while True:
۴      # Set the game up here
۵      #pass
۶
۷      #while game_on:
۸          #PLayer 1 Turn
۹
۱۰
۱۱         # Player2's turn.
۱۲
۱۳             #pass
۱۴
۱۵
۱۶     #if not replay():
          #break
```

دست مریزاد!

پروژه عملی ۱ :بازی دوز - راهنمای گام به گام

در این بخش راهنمای گام به گام بازی دوز یا *Tic Tac Toe* را در اختیار شما قرار می دهیم.

مرور برخی از ابزارهایی که در بخش دست گرمی استفاده کردیم:

دریافت ورودی کاربر:

```
player1 =input("Please pick a marker 'X' or '0'")
```

توجه نمایید که تابع *input* ورودی کاربر را در قالب رشته برمیگرداند به همین دلیل برای تبدیل به عدد صحیح از روش زیر برای *cast* استفاده نمایید:

```
position = int(input('Please enter a number'))
```

برای پاک کردن صفحه نمایش میان تعویض نوبت بازی بازیکنان از روش زیر استفاده نمایید:

```
from IPython.display import clear_output

clear_output()
```

توجه نمایید که دستور *clear_output ()* تنها در محیط *pyter Notebook* قابل اجرا میباشد و برای *IDE* های دیگر از روش زیر استفاده نمایید

```
print('\n' *100)
```

دستور بالا خروجی کنسول قبلی را از محدوده نمایش خارج می نماید. حالا بریم سراغ کد:

مرحله ۱ :تابعی بنویسید که بتواند یک تخته را چاپ کند تخته خود را به عنوان یک لیست تنظیم کنید به طوری که هر شاخص *1-9*با یک شماره در پد شماره مطابقت داشته باشد بنابراین نمایش ۳ در ۳ تخته را دریافت میکنید

X	O	X
O	X	O
X	O	X

In [۱]:

```
from IPython.display import clear_output

def display_board(board):

    clear_output()

    print('     |    |    ')
    print(f" (board[7])  |  {board [8]} (board[9]}")
    print('     |    |    ')

    print('------------')

    print('     |    |    ')
    print(f" (board[۴])  |  {board [۵]} (board[۶]}")
    print('     |    |    ')

    print('------------')

    print('     |    |    ')
    print(f" (board[۱])  |  {board [۲]} (board[۳]}")
    print('     |    |    ')
```

تست مرحله ۱ :تابع خود را با ورودی آزمایش زیر از نقشه بورد مورد آزمایش قرار دهید و در صورت نیاز فاصله ها را تنظیم نمایید.

In [ ]:

```
test_board = [ '#', 'X', 'O', 'X', 'O', 'X', 'O', 'X','O','X']
display_board(test_board)
```

مرحله ۲ :تابع بنویسید تا ورودی کاربران در قالب یک عدد که مکان علامت یا را بیان میکند دریافت نماید. می توانید از یک حلقه ***while*** برای انجام پرسش تا زمان دریافت ورودی صحیح استفاده نمایید.

In [2]:

```
def player_input():

    marker = ' '

    while not (marker == 'X' or marker == '0'):
        marker input('Player 1: Do you want to be X or O? ').upper()

    if marker == 'X':
        return ('X', '0')
    else:
        return ('0', 'X')
```

تست گام دوم :تابع را اجرا نمایید تا از دریافت خروجی مورد نظر اطمینان حاصل نمایید.

In [ ]:

```
player_input()
```

گام سوم :تابعی بنویسید که لیست علامتهای روی بورد بازی *X* یا *O* و مکان علامت گذاری در بازه صفر تا ۹ را دریافت کرده و علامت مورد نظر را در مکان مناسب انتخاب شده قرار دهد.

In [3]:

```
def place_marker (board, marker, position):

    board [position] = marker
```

تست مرحله ۳ :تابع فوق را با ورودی بورد و مکان انتخاب شده آزمایش نمایید

In [ ]:

```
place_marker(test_board, '$',8)
display_board(test_board)
```

مرحله ۴ :تابعی بنویسید تا نشانه *X* یا *O* را بعنوان ورودی دریافت کرده تشخیص دهد که آیا کاربر برنده شده است یا نه

In [4]:

```
def win_check(board, mark):

return ((board [7] == mark and board [8] == mark and board [9] == mark) or
(board [4] == mark and board [5] == mark and board [6] == mark) or
 (board[1] == mark and board [2] == mark and board [3] == mark) or
 (board [7] == mark and  board [4] == mark and board [1] == mark) or
(board [8] == mark and  board [5] == mark and board [2] == mark) or
(board [9]  == mark and board [6] == mark and board[3] ==mark) or
(board [7] == mark and board [5] == mark and board [3] == mark) or
(board [9] == mark and board [5] == mark and board[1]  ==mark))
```

تست مرحله ۴ :تابع *win_check* را برروی بورد آزمایش خودمان تست کنید باید *True* را برگرداند.

In [ ]:

```
win_check(test_board, 'X' )
```

مرحله ۵ :تابعی بنویسید تا با استفاده از ماژول *random* یک بازیکن را بعنوان نفر اول انتخاب کرده و یک رشته برای این کار چاپ .کند میتوانید از تابع *random.randint*() استفاده نمایید.

In [5]:

```
import random

def choose first():

    if random.randint(0, 1) == 0:
        return 'Player 2'
    else:
        return 'Player 1'
```

In [ ]:

```
1 choose first()
```

مرحله ۶: تابعی بنویسید که چک کند آیا فضای خالی در بورد بازی در موقعیت مشخص شده وجود دارد یا نه

In [6]:

```
1 def space_check (board, position):
2
3     return board [position] == ' '
```

مرحله ۷ :تابعی بنویسید تا بررسی کند یا بورد بازی توسط علامتهای بازیکنها پر شده است یا نه؟ *True* برای حالت پر و *False* برای حالت دیگر

In [7]:

```
1 def full board_check(board):
2
3     for i in range(1, 10):
4         if space check (board, i):
5             return False
6     return True
```

مرحله ۸ :تابعی بنویسید تا از کاربر موقعیت حرکت بعدی بازیکن را بپرسد (۱-۹) و سپس از تابع مرحله ۶ برای بررسی خالی بودن موقعیت انتخاب شده استفاده نماید و در صورتیکه موقعیت خالی باشد مقدار موقعیت انتخاب شده را برای ادامه بازی برگرداند.

In [8]:

```
1 def player_choice(board):
2
3     position = '0'
4
5     while position not in ['1', '2', '3', '4', '5', '6', '7', '8', '9'] or not space_check (b
6         position input('Choose your next position: (1-9)')
7
8     return int(position)
```

مرحله ۹ : *تابعی بنویسید تا از بازیکن بپرسد که تمایل به انجام بازی مجدد دارد یا نه*

In [9]:

```
Hef replay():

return input('Do you want to play again? Enter Yes or No: ').lower().startswith(
```

مرحله ۱۰ : حالا به مرحله سخت رسیدیم، سعی کنین تمام منطق بازی را در یک حلقه ***while*** با شرط همواره درست قرار دهید

In [10]:

```
print('Welcome to Tic Tac Toe!')

while True:
      theBoard=[] * 10 # [' ', ' ', ' ',...]

     player1_marker, player2_marker =player_input()

     turn = choose_first()

    print(f"{turn) will go first.")


    play_game = input ('Are you ready to play? Enter Yes or No: ')

   if play game.lower()[0]=='y":
       game_on = True
  else:
      game_on = False

  while game_on:
     if turn == 'Player 1':

         display_board(theBoard)
        position player_choice (theBoard)
```

```
        place marker (theBoard, player1_marker, position)

       if win_check (theBoard, player1_marker):
            display_board(theBoard)
             print('Congratulations! You have won the GAME!')
            game_on False
      else:
           if full_board_check (theBoard):
                display board (theBoard)
                print('The Game is a DRAW!')
               break
          else:
               turn 'Player 2'

    else:
       # player2's turn
      display board(theBoard)
     position player_choice (theBoard)
     place_marker (theBoard, player2_marker, position)

      if win check(theBoard, player2_marker):
           display board(theBoard)
          print("Player 2 has WON!')
         game_on = False
      else:
         if full board check (theBoard):
             display board (theBoard)
             print('The game is a DRAW!')
             break
        else:
            turn 'Player 1'
if not replay():
     break
```

دست مریزاد!

برنامه نویسی شی گرا

برنامه نویسی شی گرا (*OOP*) یکی از موانع اصلی برای مبتدیان در زمان شروع یادگیری پایتون است. در این درس با ساختار *OOP* در پایتون با استفاده از مباحث زیر آشنا خواهید شد:

- اشیاء
- استفاده از کلمه کلیدی *class*
- ساخت و تعریف ویژگی های کلاس
- ساخت متدها در یک کلاس
- آموزش در مورد *Inheritance*
- آموزش در مورد *Polymorphism*
- آموزش در مورد *Special Methods* برای کلاس ها
- و بسیاری دیگر .....

با یادآوری درباره اشیاء پایه پایتون شروع به یادگیری *OOP* خواهید کرد. به عنوان مثال:

In [4]:

```
1 1ST = [1, 2, 3]
```

بخاطر بیاورید چگونه متدهای یک لیست را میتوانیم فراخوانی کنیم

In [5]:

```
1 type(1st)
```

Out[5]:

List

In [6]:

```
1 1st.count (2)
```

Out[6]:

1

ساخت اشیاء در پایتون

در این درس ما به بررسی این میپردازیم که چگونه میتوانیم یک نوع شیء مانند لیست ایجاد کنیم. قبلاً درباره ساخت توابع یاد گرفته بودیم بنابراین بیایید به طور کلی درباره اشیاء صحبت کنیم

اشیاء

در پایتون همه چیز شی است. به خاطر درسهای ،قبل ما میتوانیم از تابع *type* برای بررسی نوع شئ استفاده کنیم:

In [7]:

```
1 print(type(1))
2 print(type(''))
3 print(type([]))
4 print (type(()))
5 print(type({}))
```

Out[7]:

<class 'int'>

<class 'str'>

<class 'list'>

<class 'tuple'>

<class 'dict'>

پس ما می دانیم که همه این چیزها شیء ،هستند پس چگونه میتوانیم نوع شی خود را ایجاد کنیم؟
اینجاست که کلید واژه *class* وارد میشود.

شروع کار با کلاس های پایتون

پایتون یک زبان برنامه نویسی چند الگویی است که با کلاسهایی که میتوانید با کلید واژه *class* تعریف کنید، برنامه نویسی شی گرا (*OOP*) را پشتیبانی می.کند میتوانید یک کلاس را به عنوان یک قطعه کد

در نظر بگیرید که داده ها و رفتار را مشخص میکند و یک نوع خاص از شئ را نمایش میدهد و مدل می کند.

کلاس در پایتون چیست؟ یک شباهت رایج این است که یک کلاس مانند طرح بلوپرینت خانه است شما میتوانید از طرح بلوپرینت برای ایجاد چندین خانه و حتی یک محله کامل استفاده کنید. هر خانه بتنی یک شئ یا نمونه است که از طرح بلوپرینت به دست آمده است.

هر نمونه ممکن است دارای ویژگی های خود باشد مانند ،رنگ ،صاحب و طراحی داخلی این ویژگی ها حالت شی را به خود مشخص میکنند نمونه ها همچنین ممکن است رفتارهای مختلف داشته باشند مانند قفل کردن درب ها و پنجره ها باز کردن درب ،گاراژ روشن و خاموش کردن ،نور آب دادن به باغچه و غیره.

در *OOP*، به طور عمومی از عبارات ویژگی ها(*attributes*) برای اشاره به خصوصیات یا داده های مرتبط با يك شئ خاص از يك كلاس داده شده استفاده می شود در پایتون ویژگی ها به عنوان متغیرهای تعریف شده در داخل يك كلاس با هدف ذخیره کلیه داده های لازم برای کار کلاس تعریف شده از کلاس کاربردي استفاده مي شود.

بطور مشابه به متدها برای اشاره به رفتارهای مختلف که شیئ ها نشان خواهند داد استفاده خواهیم کرد متد ها تابع هستند که در یک کلاس تعریف میشود این توابع به طور عمده بروی یک نمونه یا کلاس پشتیبانی شده یک سطح عمل کرده و یا با ویژگی های نمونه یا کلاس پایه کار می.کنند ویژگی ها و متدها به طور کلی به عنوان اعضای یک کلاس یا شئ ارجاع داده میشوند.

با استفاده از کلاس ها میتوانید جهان واقعی را مدل کنید این کلاس ها به شما در سازماندهی بهتر کد و حل مسائل برنامه نویسی پیچیده کمک خواهند کرد.

به عنوان مثال شما میتوانید از کلاس ها برای ایجاد شئ هایی استفاده کنید که ،انسان ،حیوانات وسایل نقلیه کتاب ها ساختمان ها خودروها یا دیگر اشیاء را شبیه سازی میکنند همچنین میتوانید اشیاء مجازی را نمایش دهید. مانند یک سرور وب درخت دایرکتوری چت ،بات، مدیر فایل و غیره

تعریف کلاس با پایتون

برای تعریف یک کلاس در پایتون باید از کلمه کلیدی *class* و نام کلاس و دو نقطه استفاده کنید
همانطور که برای دیگر عبارات ترکیبی در پایتون انجام میدهید سپس باید بدنه کلاس را تعریف کنید که در سطح بعدی تورفتگی شروع خواهد شد:

In [۹]:

```
# Create a new object type called Sample
class Sample:
    pass

# Instance of sample
x = Sample()

print(type(x))
```

<class '_main_.Sample>

در بدنه یک ،کلاس میتوانید ویژگی ها و متدها را به عنوان نیاز تعریف کنید. همانطور که قبلاً یاد گرفتید، ویژگی ها متغیرهایی هستند که داده های کلاس را نگه میدارند در حالی که متدها توابعی هستند که رفتار را فراهم میکنند و به طور عمده بر روی داده های کلاس عمل می کنند.

توجه: در پایتون بدنه یک کلاس به عنوان یک فضای نام عمل میکند که در آن ویژگی ها و متدها زندگی می کنند. شما فقط میتوانید به این ویژگی ها و متدها از طریق کلاس یا شی های آن دسترسی داشته باشید.

به عنوان مثال برای تعریف ویژگی ها و متدها فرض کنید نیاز به یک کلاس دایره برای شبیه سازی دایره های مختلف در یک برنامه نقاشی دارید در ابتدا کلاس شما یک ویژگی تک رادیوس برای نگه داری خواهد داشت. همچنین یک روش برای محاسبه مساحت دایره خود خواهید داشت:

In [10]:

```
import math

class Circle:
    def init(self, radius):
        self.radius = radius

    def calculate_area(self):
        return round (math.pi * self.radius ** 2, 2)
```

در این قطعه کد شما با استفاده از کلمه کلیدی *class*، کلاس دایره را تعریف می.کنید در داخل کلاس دو متد را می نویسید. متد ()init _ در کلاس های پایتون معنای خاصی دارد این متد به عنوان مبدأ شيء شناخته شود زیرا مقادیر اولیه و ویژگیهای شما را تعریف و تنظیم می.کند در بخش ویژگیهای نمونه بیشتر در مورد این متد یاد خواهید گرفت.

متد دوم دایره به نام *calculate_area* است و با استفاده از شعاع خود مساحت یک دایره خاص را محاسبه خواهد کرد. برای توصیف عملکرد متد، معمول است که نام های متد حاوی فعل باشند، مانند *calculate*. در این مثال، شما از ماژول *math* برای دسترسی به ثابت *pi* استفاده کرده اید زیرا در آن ماژول تعریف شده است.

توجه :در پایتون اولین آرگومان بسیاری از متدها *self* است. این آرگومان یک ارجاع به شی فعلی را نگه می دارد تا بتوانید آن را در داخل کلاس استفاده کنید در بخش متدهای نمونه با *self* بیشتر درباره این آرگومان یاد خواهید گرفت. خب شما اولین کلاس خود را نوشتید حالا چگونه میتوانید از این کلاس در کد خود استفاده کنید تا چندین دایره واقعی را نشان دهید؟ خب باید کلاس خود را نمونه سازی کنید تا شیهای دایره خاص را از آن بسازید.

ایجاد شی های جدید از یک کلاس در پایتون

عمل ساخت شی های واقعی از یک کلاس موجود به عنوان نمونه سازی شناخته میشود با هر نمونه سازی یک شی جدید از کلاس هدف ایجاد میشود. برای آغاز با اجرای کد زیر در جلسات *REPL* پایتون چندین نمونه *Circle* بسازید:

In [12]:

```
1 circle_1 = Circle(42)
2 circle_2 = Circle(7)
```

برای ایجاد یک شیء از یک کلاس پایتون مانند دایره باید با جفت پرانتز و مجموعه مناسبی از آرگومان ها سازنده کلاس *Circle*() را فراخوانی کنید چه آرگومان هایی؟ در پایتون، سازنده کلاس همان آرگومان های*init*_() ___.را قبول می کند. در این مثال کلاس دایره آرگومان شعاع را انتظار دارد.

فراخوانی سازنده کلاس با مقادیر آرگومان مختلف به شما اجازه میدهد تا اشیاء یا نمونه های مختلفی از کلاس هدف ایجاد کنید. در مثال بالا ۱ *circle* و ۲ *circle* نمونه های جداگانه از *Circle* هستند. به عبارت دیگر آنها دو دایره متفاوت و ملموس هستند همانطور که میتوانید از خروجی که استنباط کنید.

عالی شما قبلاً یاد گرفتید چگونه با فراخوانی سازنده کلاس مورد نظر با آرگومان های لازم اشیاء یک کلاس موجود را ایجاد کنید. حال چگونه میتوانید به ویژگیها و متدهای یک کلاس داده شده دسترسی پیدا کنید؟ این چیزی است که در بخش بعدی خواهید آموخت.

دسترسی به ویژگی ها و متدها

در پایتون شما میتوانید با استفاده از نشانه نقطه با عامل نقطه به ویژگی ها و متدهای یک شیء دسترسی پیدا کنید. قطعه کد زیر نحو لازم را نشان میدهد

- *ahi attributa namn*

توجه کنید که نقطه (.) در این نحو به طور کلی به معنای دسترسی به ویژگی یا متد زیر این شی است. خط اول مقدار ذخیره شده در ویژگی هدف را برمی گرداند در حالی که خط دوم به روش هدف دسترسی پیدا می کند تا بتوانید آن را فراخوانی کنید.

توجه :به یاد داشته باشید که برای فراخوانی یک تابع یا متد باید از جفت پرانتز و سری آرگومان ها استفاده کنید. اگر قابل اجرا باشد. حال به شی های دایره خود بازگردید و کد زیر را اجرا کنید:

In [۱۳]:

```
circle_1.radius
```

Out[۱۳]:

۴۲

In [۱۴]:

```
circle_ ۲.radius
```

Out[۱۴]:

۷

In [۱۵]:

```
circle_1.calculate_area()
```

Out[۱۵]:

۵۵۴۱٫۷۷

In [۱۶]:

```
۱ circle_۲.calculate_area()
```

Out[۱۶]:

۱۵۳٫۹۴

In [ ]:

```
۱
```

در چند خط اول پس از نمونه ،سازی شما به ویژگی *radius* در شیء ۱ *circle* خود دسترسی پیدا می کنید. سپس با فراخوانی متد () *calculate_area* مساحت دایره را محاسبه می.کنید در جفت دوم از بیانیه ها شما همان کار را انجام می دهید اما در شیء ۲ *circle_*

شما همچنین میتوانید از نقطه نوتاسیون و یک بیانیه اختصاص به تغییر مقدار فعلی یک ویژگی استفاده کنید:

In [۱۷]:

```
۱ circle_1.radius = 100
2 circle_1.radius
```

Out[۱7]:

100

In [۱8]:

```
1 circle_1.calculate_area()
```

Out[۱8]:

31415.93

حال شعاع۱-*circle* به طور کامل متفاوت است هنگام فراخوانی () *calculate_area* ، نتیجه به طور فوری این تغییر را نشان میدهد شما وضعیت داخلی یا داده های شیء را تغییر داده اید که به طور معمول بر رفتارهای یا متدهای آن تاثیر میگذارد.

مثال دیگر:

In [۱۹]:

```
class Dog:
    def init_(self, breed):
        self.breed = breed

sam = Dog breed = 'Lab')
frank =Dog (breed = 'Huski')
```

In [۲۰]:

```
sam.breed
```

Out[۲۰]:

'*Lab*'

In [۲۱]:

```
frank.breed
```

Out[۲۱]:

'Huski'

توجه کنید که ما پس از نام نژاد هیچ پرانتزی نداریم این به این دلیل است که یک ویژگی است و هیچ آرگومانی را نمی گیرد. در پایتون همچنین ویژگی های شی کلاس وجود دارد این ویژگی های شی کلاس برای هر نمونه از کلاس یکسان است. به عنوان مثال ما میتوانیم ویژگی گونه را برای کلاس سگ ایجاد کنیم سگها بدون توجه به نژاد نام یا ویژگی های دیگر خود همیشه پستانداران خواهند بود ما این منطق را به صورت زیر اعمال می کنیم

In [۳۵]:

```
class Dogs:
    species = 'mammal
   def _init_(self, breed, name):
        self.breed = breed
        self.name = name
        #print(Dog.species)
        print(type(self).species)

sam Dog (breed = 'Lab', name = 'sam')
frank = Dog breed = 'Huski', name = 'frank' )
```

mammal

mammal

In [۲۳]:

```
sam.name
```

Out[۲۳]:

'sam'

In [۲۴]:

```
sam.breed
```

Out[۲۴]:

'Lab'

توجه کنید که ویژگی شی کلاس خارج از هر متد در کلاس تعریف میشود. همچنین به طور معمول آنها را قبل از *init* قرارمیدهیم.

In [۲۶]:

```
۱ frank.species
```

Out[۲۶]:

'mammal'

In [۲7]:

```
۱ Dog.species
```

Out[۲7]:

'mammal'

In [۲8]:

```
۱ Dog.breed
```

AttributeError

Traceback (most recent call last)

Cell In[28], line 1

-----> 1 Dog.breed

AttributeError: type object 'Dog' has no attribute 'breed'

In [38]:

```
class Dog:
    species = 'mammal'
    def init(self, breed, name):
        self.breed breed
        self.name = name
        #print (Dog. species)
        print (type(self). species)
    def bark(self):
        print('Wooof!')

sam = Dog (breed = 'Lab', name = 'sam')
frank = Dog (breed = 'Huski', name = 'frank')
```

'mammal'

'mammal'

In [39]:

```
sam.bark()
```

Wooof!

وراثت

ارث بری یک روش برای ایجاد کلاس های جدید با استفاده از کلاس هایی است که قبلاً تعریف شده اند. کلاس های تازه شکل داده شده کلاس های مشتق شده نامیده میشوند و کلاس هایی که از آنها به دست میآوریم کلاس های پایه نامیده می شوند. مزایای مهم ارث بری استفاده مجدد از کد و کاهش پیچیدگی برنامه است. کلاس های مشتق شده (فرزندان) عملکرد کلاس های پایه پدران را بازنویسی یا گسترش می دهند.

بیایید با یک مثال به کلاس سگ بپردازیم:

In [55]:

```
class Animal:
    def _init_(self):
        print('Animal Created')

    def whoAmI (self):
        print('Animal')

    def eat(self):
        print('Eating')

class Dog (Animal):
    def init(self):
        Animal.__init_(self)
        print("Dog Created")
    def whoAmI (self):
        print("Dog")

    def bark(self):
        print("Woof!')

class Lion (Animal):
    def init(self):
        Animal. Init_(self)
        print("Lion Created")
```

In [56]:

```
d = Dog()
```

Animal Created

Dog Created

In [57]:

```
d.whoAmI()
```

Dog

In [58]:

```
d.eat()
```

Eating

In [59]:

```
d.bark()
```

Woof!

In [60]:

```
a = Animal()
```

Animal Created

In [46]:

```
a.bark()
```

AttributeError

Traceback (most recent call last)

Cell In[46], line 1

-----> 1 a.bark()

AttributeError: 'Animal' object has no attribute 'bark'

In [54]:

```
l= Lion()

l.WhoAmI()
```

Animal Created

Lion Created

Animal

در این مثال، دو کلاس داریم :*Animal* و *DogAnimal* کلاس پایه است، *Dog* کلاس مشتق شده است.

کلاس مشتق شده متدهای کلاس پایه را به ارث می برد.

- این با روش() *eat* نشان داده شده است

کلاس مشتق شده رفتار موجود کلاس پایه را تغییر میدهد.

- با روش( )*whoAmI* نشان داده شده است.

در نهایت کلاس مشتق شده عملکرد کلاس پایه را گسترش میدهد با تعریف یک روش جدید ()*bark*

چندریختی *Polymorphism*

ما یاد گرفتیم که در حالی که توابع میتوانند با آرگومان های مختلف فراخوانی ،شوند متدها متعلق به شی هایی هستند که بر روی آنها عمل میکنند در پایتون *polymorphism* به روشی اشاره دارد که کلاس های شی مختلف می توانند نام یکسان را به اشتراک بگذارند و این روش ها میتوانند از یک مکان فراخوانی شوند حتی اگر اشیاء مختلفی از طریق آنها منتقل شوند. بهترین راه برای توضیح این مسئله با یک مثال است:

In [۶۲]:

```
class Dog:
    def init(self, name):
        self.name = name

    def speak(self):
         return self.name + says WOOOF!

class Cat:
    def _init_(self, name):
        self.name = name

    def speak (self):
        return self.name + ' says Meow
niko=Dog('Niko')
felix =Cat('Felix')

print (niko.speak())
print (felix.speak())
```

Niko says WOOOF!

Felix says Meow

در اینجا یک کلاس سگ و یک کلاس گربه داریم و هر کدام دارای یک روش ()*speak* .هستند هنگام فراخوانی، روش ()*speak* هر شیء نتیجه ای منحصر به فرد برای شیء خود باز می گرداند.

چندین روش مختلف برای نمایش *polymorphism* وجود دارد اول با یک حلقه *for*:

In [۶۳]:

```
for pet in [niko, felix]:
    print(pet.speak())
```

Niko says WOOOF!

Felix says Meow

یا از طریق یک تابع این کا را انجام دهیم :

In [۶۳]:

```
def pet_speak (pet) :
    print(pet.speak())

pet speak (niko)
pet_speak(felix)
```

Niko says WOOOF!

Felix says Meow

در هر دو مورد ما توانستیم انواع شی مختلف را منتقل کنیم و نتایج منحصر به شیء را از همان مکان بدست آوریم. یک شیوه کاربردی تر استفاده از کلاس های انتزاعی ***abstract class*** و وراثت است. یک کلاس انتزاعی یک کلاس است که هرگز انتظار ندارد نمونه سازی شود. به عنوان مثال هرگز شی ***Animal*** را نخواهیم داشت فقط شی های ***Dog*** و ***Cat*** را خواهیم داشت اگرچه سگ ها و گربه ها از حیوانات به دست می آیند:

In [۶۶]:

```
class Animal:
     def _init_(self, name):
         self.name

    def speak (self):
        raise NotImplementedError('Subclass must implement speak() method!')
name
class Dog (Animal):
    def speak (self): #override
         return self.name + says WOOOF!'

class Cat (Animal):
   def speak (self):
         return self.name + 'says Meow'

fido=Dog('Fido')
isis = Cat('Isis')

print (fido.speak())
print(isis.speak())
```

Fido says WOOOF!

Isis says Meow

مثالهای واقعی از *polymorphism* شامل:

- باز کردن انواع مختلف فایل - ابزارهای مختلف برای نمایش فایل های *Word pdf* و *Excel* لازم است
- اضافه کردن اشیاء مختلف - عملگر + عملیات حسابی و ادغام را انجام می دهد

متدهای ویژه

در نهایت بیایید به متدهای ویژه بپردازیم کلاس ها در *Python* میتوانند برخی از عملیات ها را با نام های متدهای ویژه پیاده سازی کنند این متدها در واقع به صورت مستقیم فراخوانی نمیشوند بلکه با دستور زبان خاص *Python* فراخوانی میشوند به عنوان مثال بیایید یک کلاس *Book* ایجاد کنیم:

In [88]:

```
class Book:
    def init_(self, title, author, pages):
        print('A book is created!')
        self.title = title
        self.author = author
        self.pages = pages

    def str_(self):
        return f"Title: (self.title}, author: (self.author), pages: (self.pages)"

    def len(self):
        return self.pages
    def del_(self):
        print("A book is destroyed")
```

In [82]:

```
book Book ('Python Rocks!', 'J P', 209)
```

A book is created!

In [83]:

```
1 print(book)
```

Title: Python Rocks!, author: J P, pages: 209

In [84]:

```
1 len(book)
```

Out[84]:

209

In [85]:

```
1 del book
```

A book is destroyed

متدهای ویژه init_ _str len و del

این متدهای ویژه با استفاده از زیرخط تعریف شده اند آنها به ما اجازه میدهند تا از توابع خاص *Python* بر روی اشیاء ایجاد شده از طریق کلاس خود استفاده کنیم.

عالی پس از این سخنرانی شما باید درک پایه ای از نحوه ایجاد شیهای خود با کلاس در *Python* داشته باشید. در پروژه مرحله بعدی خود به طور فراوان از آن استفاده خواهید کرد!

برنامه نویسی شی گرا

تمرینات

مسئله ۱

متدهای کلاس خط را برای پذیرفتن مختصات به عنوان جفتی از تایلها و بازگشت شیب و فاصله خط پر کنید.

In [1]:

```
class Line:

      def _init_(self, coor1, coor2):
          self.coor1 = coor1
          self.coor2 = coor2

     def distance(self):
          x1, y1= self.coor1
          x2, y2 = self.coor2
         return ((x2 - x1) ** 2 + (y2 y1) ** 2) ** 0.5

   def slope (self):
          self.coor1 = coor1
          self.coor2 = coor2

return (y2- y1) / (x2-x1)
```

In [2]:

```
# EXAMPLE OUTPUT

coordinate1 = (3,2)
coordinate2 = (8,10)

li= Line(coordinate1, coordinate2)
```

In [3]:

```
li.distance()
```

Out[3]:

9.433981132056603

In [4]:

```
1 li.slope()
```

Out[4]:

1.6

مسئله ۲

کلاس زیر را تکمیل نمایید

In [5]:

```
1  class Cylinder:
2
3      def init(self, height=1, radius=1):
4          self.height=height
5          self.radius = radius
6
7      def volume(self):
8          return self.height * (3.14 * self.radius ** 2)
9
10     def surface area(self):
11         top =3.14 * (self.radius) ** 2
12         return (2* top) + (2 * 3.14 * self.radius self.height)
```

In [6]:

```
1 # EXAMPLE OUTPUT
2 C = Cylinder(2,3)
```

In [7]:

```
1 c.volume()
```

Out[7]:

56.52

In [8]:

```
1 c.surface_area()
```

Out[8]:

94.2

In [9]:

```
1 c2 = Cylinder()
```

In [10]:

```
1 c2.radius
```

Out[10]:

1

In [11]:

```
1 c2.height
```

Out[11]:

1

In [12]:

```
1 c2.volume()
```

Out[12]:

3.14

چالش بخش برنامه نویسی شی گرا

برای این چالش یک کلاس حساب بانکی ایجاد کنید که دو ویژگی داشته باشد

- صاحب

- موجودی

و دو متد:

- سپرده
- برداشت

به عنوان یک الزام اضافی، برداشت ها نباید از موجودی موجود بیشتر شود.

کلاس خود را نمونه سازی کنید، چندین سپرده و برداشت انجام دهید و تست کنید تا اطمینان حاصل شود که حساب بانکی نمی تواند بیش از حد برداشت شود.

In [1]:

```
class Account:
    def _init_(self, owner, balance = 0):
        self.owner = owner
        self.balance = balance

    def _str_(self):
        return f"Account Owner: {self. owner} \nAccount Balance: {self.balance}"

    def deposit (self, dep_amt):
        self.balance += dep_amt
        print('Deposit Accepted')

    def withdraw(self, wd_amt):
        if self.balance >= wd_amt:
            self.balance -= wd_amt
            print('Withdrawal Accepted!')
        else:
            print('Funds Unavailable!!!')
```

In [۲]:

```
# 1 .Instantiate the class
acct1 = Account ( 'Jose',100)
```

In [۳]:

```
# 2. Print the object
print (acct1)
```

Account Owner: Jose

Account Balance: 100

In [۴]:

```
# 3. Show the account owner attribute
acct1.owner
```

Out[4]:

'Jose'

In [5]:

```
# 4. Show the account balance attribute
acct1.balance
```

Out[5]:

100

In [6]:

```
# 5. Make a series of deposits and withdrawals
acct1.deposit (50)
```

Deposit Accepted

In [7]:

```
print(acct1)
```

Account Owner: Jose

Account Balance: 150

In [8]:

```
1 acct1.withdraw(75)
```

Withdrawal Accepted!

In [9]:

```
١ # 6. Make a withdrawal that exceeds the available balance
٢ acct1.withdraw(500)
```

Funds Unavailable!!!

عالی!

مباحث پیشرفته در برنامه نویسی شی گرا

نامگذاری کلاس ها در پایتون

قبل از ادامه دادن به کلاس ها باید با برخی از قواعد نامگذاری مهم آشنا شوید که پایتون در زمینه کلاس ها استفاده می کند. پایتون یک زبان انعطاف پذیر است که آزادی را دوست دارد و دوست ندارد محدودیت های صریح داشته باشد. به همین دلیل این زبان و جامعه برنامه نویسان آن بر روی روش های مرسوم نام گذاری تکیه میکنند تا به جای محدودیت ها از قراردادها استفاده کنند.

توجه بیشتر برنامه نویسان پایتون از قانون *Snake_case* استفاده میکنند که شامل استفاده از خط تیره (-) برای جداسازی چندین کلمه است. با این حال قانون نامگذاری توصیه شده برای کلاسهای پایتون *PascalCase* است به طوری که هر کلمه با حروف بزرگ شروع شود.

در دو بخش زیره در مورد دو قانون نامگذاری مهم صفات کلاس آموزش خواهید دید.

Public vs Non-Public Members

اولین قانون نامگذاری که باید در مورد آن اطلاعاتی داشته باشید مربوط به این است که پایتون مانند جاوا و زبان های دیگر بین ویژگی های خصوصی محافظت شده و عمومی تفاوت قائل نیست در پایتون تمام ویژگی ها به یک شکل یا دیگر قابل دسترسی هستند با این حال پایتون یک قانون نامگذاری خوب دارد

که باید از آن استفاده کنید تا اعلام کنید که یک ویژگی یا روش برای استفاده از خارج از کلاس یا شیء حاوی آن مناسب نیست.

قانون نامگذاری شامل اضافه کردن خط تیره به عنوان پیشوند به نام عضو است ،بنابراین در یک کلاس پایتون شما قانون زیر را خواهید داشت

عضویت	نام گذاری	مثال
Public	**از الگوی معمولی نام گذاری استفاده نمایید**	*calculate_area(), radius*
Non-public	از- بیش از نام ها استفاده نمایید	*calculate_area() _و _radius*

عضو های عمومی بخش رسمی رابط یا *API* کلاس های شما هستند در حالی که اعضای غیر عمومی قرار نبوده بخشی از آن *API* بخش باشد. این بدان معناست که شما نباید از اعضای غیر عمومی خارج از کلاس تعریف کنید.

لطفا توجه داشته باشید که قانون نامگذاری دوم فقط نشان می دهد که ویژگی به صورت مستقیم از خارج از کلاس حاوی استفاده نمیشود با این حال دسترسی مستقیم را جلو نمیگیرد به عنوان مثال میتوانید *obj name* را اجرا کنید و به محتوای *name* - . دسترسی پیدا کنید با این حال این شیوه بد است و باید از آن خودداری کنید.

عضوهای غیر عمومی فقط برای پشتیبانی از پیاده سازی داخلی یک کلاس خاص وجود دارند و ممکن است در هر زمان حذف شوند بنابراین نباید به آنها اعتماد کنید. وجود این اعضا به نحوی است که به پیاده سازی کلاس بستگی دارد. بنابراین شما نباید در کد مشتری به صورت مستقیم از آنها استفاده کنید در صورت استفاده کد شما ممکن است در هر لحظه خراب شود.

In [۱]:

```
class Account:
      def _init_(self, owner, balance=0):
           self.owner = owner
          self._balance = balance

    def str_(self):
          return f'Account owner:

    def get_balance(self):
          return self._balance

   def deposit(self, dep_amt):
          self. balance += dep_ant
          print('Deposit Accepted')

   def withdraw(self, wd_amt):
          if self._balance >= wd_amt:
                self._balance -= wd_amt
               print('Withdrawal Accepted')
        else:
              print('Funds Unavailable!')
```

In [۲]:

```
acct1 = Account('Jose',100)
```

In [۳]:

```
acct1.get_balance()
```

Out[3]:

100

In [4]:

```
١ acct1._balance
```

Out[4]:

100

Name Mangling

نام *Mangling* یکی دیگر از قواعد نامگذاری که میتوانید در کلاس های پایتون ببینید و استفاده کنید، اضافه کردن دو زیر خط به نام و متدها است. این قانون نامگذاری باعث فعال شدن چیزی به نام *name mangling* می شود.

Name mangling یک تبدیل خودکار نام است که نام کلاس را به نام عضو اضافه میکند مانند *_ClassName_attribute* یا *ClassName_method*. این باعث مخفی شدن نام ها میشود به عبارت دیگر نام های *mangling* برای دسترسی مستقیم در دسترس نیستند آنها بخشی از *API* عمومی یک کلاس نیستند.

برای مثال به کلاس نمونه زیر توجه کنید:

In [5]:

```
١ class Sample:
٢     def _init_(self):
٣         self._x = 1
٤         self.y = 2
٥
٦   def foo(self):
٧       print("This is a private method.")
٨
٩   def bar(self) :
١٠      print("This is a public method.") self._foo()
١١     self.__foo()
١٢
١٣ obj1= Sample()
```

In [6]:

۱	*obj1.y*

Out[6]:

2

In [7]:

۱	*obj1.x*

AttributeError

Traceback (most recent call last)

Cell In[7], line 1

-----> 1 obj1._x

AttributeError: 'Sample' object has no attribute '__x'

در این کلاس دو زیرخط ،دارند بنابراین نام های آنها به ( )*Sample_foo* تغییر می،کند همانطور که در خطوط برجسته مشاهده میکنید پایتون به صورت خودکار پیشوند *Sample* را به هر دو نام اضافه کرده است به دلیل این تغییر نام داخلی نمی توانید با استفاده از نامهای اصلی خود به ویژگی ها از خارج از کلاس دسترسی پیدا کنید. اگر سعی کنید این کار را انجام دهید خطای *AttributeError* دریافت می

کنید.

توجه: در مثال بالا شما از تابع *vars* () داخلی استفاده میکنید که یک دیکشنری از تمام عضوهای مرتبط با شیء داده شده را برمی گرداند این دیکشنری نقش مهمی در کلاسهای پایتون بازی می.کند در بخش ویژگی *dict* بیشتر درباره آن یاد خواهید گرفت.

این رفتار داخلی نام ها را پنهان میکند و حس خصوصی بودن ویژگی با روش را ایجاد میکند با این حال، آنها به صورت سختگیرانه خصوصی نیستند. شما میتوانید به آنها از طریق نامهای *mangling* دسترسی پیدا کنید:

In [8]:

```
۱ vars(obj1)
```

Out[8]:

```
{'_Sample_x': 1, 'y': 2}
```

In [9]:

```
۱ obj1._Sample_x
```

Out[9]:

```
1
```

هنوز هم میتوانید به ویژگی ها یا روش های *named-mangled* با استفاده از نام های *mangling* دسترسی پیدا کنید. اگرچه این یک شیوه بد است و باید از آن در کد خود دوری کنید اگر نامی را که از این شیوه در کد شخص دیگری مشاهده می کنید، ببینید، آنگاه سعی نکنید کد را به استفاده از نام خارج از کلاس حاوی آن وادار کنید.

Name mangling به خصوص زمانی مفید است که میخواهید اطمینان حاصل کنید یک ویژگی یا روش داده شده به طور تصادفی بازنویسی نمیشود این یک راه برای جلوگیری از تداخل نام ها بین کلاس ها یا زیر کلاس ها است. همچنین برای جلوگیری از زیرکلاسهای بازنویسی روش های بهینه سازی شده برای عملکرد بهتر مفید است.

واو*!* تا اینجای کار شما مبانی کلاس های پایتون و چند تا بیشتر را فرا گرفتید. در لحظات آینده به عمق کارکرد کلاس های پایتون خواهید پرداخت اما قبل از آن زمان آن رسیده است تا به چند دلیل بپردازید که باید درباره کلاس ها چیزهای بیشتر یاد بگیرید و در پروژه های پایتون خود از آنها استفاده کنید.

مزایای استفاده از کلاس ها در پایتون

آیا استفاده از کلاس ها در پایتون ارزش دارد؟ قطعاً کلاس ها بلوک های ساختمانی برنامه نویسی شی گرا در پایتون هستند. آنها به شما اجازه میدهند تا از قدرت پایتون در حین نوشتن و سازماندهی کد خود بهره ببرید با یادگیری درباره کلاس ها شما قادر خواهید بود از تمام مزایایی که آنها فراهم میکنند استفاده کنید با کلاس ها شما می توانید:

- مدل و حل مسائل پیچیده واقعی :در بسیاری از موارد شی های موجود در کد شما به شی های واقعی جهان نقش میبینند این میتواند به شما در فکر کردن درباره مسائل پیچیده کمک کند که نتیجه آن بهبود روش حل مسائل برنامه نویسی شما خواهد بود.
- باز استفاده از کد و جلوگیری از تکرار: شما میتوانید سلسله مراتبی از کلاس های مرتبط را تعریف کنید. کلاس های پایین سلسله مراتب عملکردهای مشترک را فراهم میکنند که بعدها در زیر کلاس های سلسله مراتب استفاده مجدد خواهید کرد. این به شما اجازه میدهد تکرار کد را کم کنید و باز استفاده از کد را ترویج دهید.
- پوشش دادن دادههای و رفتارهای مرتبط در یک جانب :شما میتوانید از کلاسهای پایتون برای گروه بندی وابستگیهای و روشهای مرتبط در یک جانب چگال شده استفاده کنید این به شما در سازماندهی بهتر کدهای خود با استفاده از جانبهای قابل استفاده و خودکار استفاده از آنچه حتی در پروژه های چندگانه قابل استفاده است به شما کمک خواهد کرد.
- پوشش جزئیات پیاده سازی مفصل و اشیاء: شما میتوانید از کلاس ها برای پوشش جزئیات پیاده سازی مفصل و اشیاء اصلی استفاده کنید. این به شما در فراهم کردن رابط های*(API)*ها قابل فهم برای کاربران خود برای پردازش داده های و رفتارهای پیچیده کمک خواهد کرد.
- با رابطهای مشترک چند ریختی را باز کنید :شما میتوانید یک رابط خاص را در چند کلاس کمی متفاوت پیاده سازی کنید و آنها را به صورت قابل تعویض در کد خود استفاده کنید. این باعث میشود کد شما انعطاف پذیرتر و قابل تطبیق تر شود.

به طور خلاصه کلاس های پایتون میتوانند به شما در نوشتن کدهای سازماندهی شده ساختار یافته قابل نگهداری، قابل استفاده مجدد انعطاف پذیر و دوستانه به کاربران شما کمک کنند آنها یک ابزار عالی برای دسترسی به آنچه در دستان

Class attributes vs Instance attributes

همانطور که یاد گرفتید کلاس ها زمانی عالی هستند که باید داده ها و رفتار را در یک موجودیت واحد بسته بندی کنید. داده ها به صورت ویژگیها و رفتار به عنوان متدها ارائه خواهد شد. شما در حال حاضر ایده ای از ویژگی دارید. حالا وقت آن است که عمیق تر به آن بپردازید که چگونه میتوانید ویژگی ها را در کلاس های سفارشی خود اضافه دسترسی و تغییر دهید.

ابتدا باید بدانید که کلاس های شما میتوانند دو نوع ویژگی در پایتون داشته باشند:

- ویژگی های کلاس یک ویژگی کلاس متغیری است که به صورت مستقیم در بدنه کلاس تعریف می شود. ویژگی های کلاس متعلق به کلاس حاوی آن است. داده های آن مشترک بین کلاس و تمام نمونه های آن است.

- ویژگی های نمونه یک نمونه متغیری است که درون یک متد تعریف می.شود. ویژگی های نمونه به یک نمونه خاص از یک کلاس خاص تعلق دارد. داده های آن فقط برای آن نمونه در دسترس است و حالت آن را تعریف می کند.

هر دو نوع ویژگی مورد استفاده خود را دارند ویژگي هاي نمونه به طور قطع پرکاربردترین نوع ویژگی است که در برنامه نویسي روزانه خود استفاده خواهید کرد اما ویژگی های کلاس نیز مفید خواهد بود.

Class attributes

ویژگی های کلاس متغیرهایی هستند که به صورت مستقیم در بدنه کلاس تعریف میشوند اما خارج از هر متدی هستند. این ویژگی ها به کلاس خود و نه به شیء خاصی از آن کلاس مربوط هستند.

تمام شیء هایی که از یک کلاس خاص ایجاد میکنید ویژگی های کلاس را با مقادیر اصلی یکسان به اشتراک می گذارند. به دلیل این اگر یک ویژگی کلاس را تغییر دهید آن تغییر بر تمام شی های مشتق شده تأثیر خواهد گذاشت.

به عنوان مثال، فرض کنید می خواهید یک کلاس ایجاد کنید که شمارش داخلی نمونه های ایجاد شده را نگه دارد. در آن صورت میتوانید از ویژگی های کلاس استفاده کنید:

In [۱۰]:

```
۱  class ObjectCounter:
۲      num_instances = 0
۳      def _init_(self):
۴          #ObjectCounter.num_instances += 1
۵          type(self). num_instances += 1
```

In [۱۱]:

```
۱  X1 =ObjectCounter()
۲  x2 = ObjectCounter()
۳  x3= ObjectCounter()
۴  x4 = ObjectCounter()
```

In [۱۲]:

```
1 ObjectCounter.num_instances
```

Out[12]:

4

In [۱۳]:

```
1 counter = ObjectCounter()
2 counter.num_instances
```

Out[13]:

5

In [۱۴]:

```
1 ObjectCounter.num_instances
```

Out[14]:

5

عنوان یک شمارنده از نمونه ها عمل می کند. هنگامی که پایتون این کلاس را تجزیه و تحلیل میکند شمارنده را به صفر مقداردهی اولیه میکند و آن را تنها می گذارد.ایجاد نمونه های این کلاس به معنای خودکار فراخوانی روش *init* () - . و افزایش *num_instances* به ۱ است.

مهم است که بدانید میتوانید از ویژگی های کلاس با استفاده از کلاس یا یکی از نمونه های آن دسترسی پیدا کنید به همین دلیل میتوانید از شیء شمارنده برای بازیابی مقدار *num_instances*. استفاده کنید با این حال اگر نیاز به تغییر ویژگی کلاس دارید باید از خود کلاس به جای یکی از نمونه های آن استفاده کنید.

به عنوان مثال اگر از *self* برای تغییر *num_instances* استفاده کنید با ساخت یک ویژگی نمونه جدید ویژگی کلاس اصلی را بازنویسی خواهید کرد:

In [۱5]:

```
1  class ObjectCounter:
2      num_instances = 0
3     def _init_(self):
4         self.num_instances += 1
```

In [۱7]:

```
1  x1 = ObjectCounter()
2  x2 = ObjectCounter()
3  x3 = ObjectCounter()
4  x4 = ObjectCounter()
5
6  ObjectCounter.num_instance
```

Out[17]:

0

In [۱8]:

```
1  x1.num_instances
```

Out[18]:

1

In [۱9]:

```
1  X2.num_instances
```

Out[19]:

1

شما نمی توانید از طریق نمونه های کلاس ویژگی های کلاس را تغییر دهید. این کار باعث ایجاد ویژگی های نمونه جدید با همان نام ویژگی های کلاس اصلی میشود. به همین دلیل
ObjectCounter.num_instances در این مثال ۰ برمی گرداند. شما ویژگی کلاس را در خط برجسته شده بازنویسی کرده اید.

بطور کلی باید از ویژگی های کلاس برای به اشتراک گذاشتن دادهها بین نمونه های یک کلاس استفاده کنید. هر تغییر در یک ویژگی کلاس به تمام نمونه های آن کلاس قابل مشاهده است.

Instance Attributes

ویژگی های نمونه متغیرهایی هستند که به یک شیء خاص از یک کلاس داده شده اند. مقدار ویژگی نمونه به خود شیء متصل است. بنابراین مقدار ویژگی به نمونه خود اختصاص دارد.

پایتون به شما اجازه میدهد تا ویژگی های جدید را به شی های موجود که قبلاً ایجاد کرده اید اضافه کنید. با این حال، شما بیشتر ویژگی های نمونه را درون متدهای نمونه تعریف میکنید که متدهایی هستند که *self* را به عنوان آرگومان اول خود دریافت می کنند.

برای مثال کلاس *car* زیر را در نظر بگیرید که تعدادی ویژگی نمونه دارد

In [20]:

```
1  class Car:
2          def _init_(self, make, model, year, color):
3              self.make = make
4              self.model = model
5              self.year = year
6              self.color = color
7              self.started = False
8              self.speed = 0
9              self.max_speed = 200
```

در این کلاس شما هفت ویژگی نمونه را درون () *init* . تعریف میکنید ویژگیهای *make model year color* مقادیر را از آرگومان های *() init* _ . دریافت می،کنند که آرگومان هایی هستند که باید برای سازنده کلاس، *()Car* ، به منظور ایجاد شیء های محسوس گذاشته شوند.

سپس ،شما ویژگی های *started. speed* و *max_speed* را با مقادیر منطقی که از کاربر نمی آیند، به صورت صریح مقداردهی اولیه میکنید.

In [21]:

```
1 toyota_camry = Car("Toyota", "Camry", 2022 "Red")
2 ford_mustang = Car("Ford", "Mustang", 2022, "Black") 2 ford_mustang)
```

In [22]:

```
1 print(toyota_camry.make, toyota_camry.model, toyota_camry.year,
  toyota_ camry.max_spe
```

Toyota Camry 2022 200

In [23]:

```
1 print(ford_mustang.make, ford_mustang.model, ford_mustang.year,
  ford_mustang.max_spe
```

Ford Mustang 2022 200

In [24]:

```
1 Car.make
```

AttributeError

Traceback (most recent call last)

Cell In[24], line 1

----> 1 Car.make

AttributeError: type object 'Car' has no attribute 'make'

The dict. Attribute

در پایتون هر دو کلاس ها و نمونه ها دارای یک ویژگی خاص به نام *dict* هستند این ویژگی یک دیکشنری را که حاوی اعضای قابل نوشتن کلاس یا نمونه زیرین است نگه میدارد به یاد داشته باشید این اعضا می توانند ویژگی ها یا متد ها باشند. هر کلید در *dict*_ یک نام ویژگی را نشان میدهد. مقدار مرتبط با یک کلید خاص، مقدار ویژگی مربوطه را نشان میدهد

.

در یک کلاس .*dict* شامل ویژگی های کلاس و متدها خواهد بود. در یک نمونه *dict*_ ویژگی های نمونه را نگه می دارد.

زمانی که شما به عضو کلاس از طریق شیء کلاس دسترسی پیدا میکنید پایتون به طور خودکار برای نام عضو در کلاس جستجو میکند اگر نام در آنجا وجود نداشته باشد آنگاه شما یک *AttributeError* دریافت خواهید کرد

بطور مشابه زمانی که شما به عضو نمونه از طریق یک شئ محسوس از یک کلاس دسترسی پیدا میکنید پایتون برای نام عضو در *dict*_. نمونه جستجو میکند اگر نام در آنجا ظاهر نشود پایتون در *dict* کلاس جستجو می اگر نام پیدا نشود آنگاه شما یک *NameError* در خودتان خواهید دید.

اینجا چندین کلاس برای آشنایی بیشتر در نظر گرفته ایم تا نحوه عملکرد این مکانیزم را بهتر درک نمایید.

In [۲۵]:

```
1  class SampleClass:
۲      class_attr = 100
۳
۴      def init(self, instance_attr):
۵          self.instance_attr = instance_attr
۶
۷      def method(self):
۸          print("Class attribute: {self.class_attr}")
۹          print (f"Instance attribute: {self.instance_attr}")
```

در این کلاس شما یک ویژگی کلاس با مقدار ۱۰۰ تعریف می.کنید در متد()__ *init* __ ، شما یک ویژگی نمونه را تعریف میکنید که مقدار آن از ورودی کاربر است در نهایت شما یک روش برای چاپ هر دو ویژگی تعریف می کنید.

حالا وقت آن رسیده است تا محتوای .*dict* را در شی کلاس بررسی کنید به جلو بروید و کد زیر را اجرا کنید:

In [۲۶]:

```
۱  SampleClass.class_attr
```

Out[26]:

100

In [27]:

```
۱ SampleClass._dict____
```

Out[27]:

```
mappingproxy({'_module _' : '_main_' ,
              'class_attr': 100,
               init': <function main. SampleClass.init(self, in
stance_attr)>,
             'method'<function _main_.SampleClass.method (self)>
             _dict_: <attribute '_dict_' of 'SampleClass' objects>,
             _weakref_': <attribute weakref_' of 'SampleClass' obj
ects>,
            '_doc_': None})
```

In [28]:

```
۱ SampleClass._dict_["class_attr"]
```

Out[28]:

```
100
```

خطوط مشخص شده نشان میدهند که هر دو ویژگی کلاس و متد در دیکشنری dict *کلاس هستند توجه کنید که چگونه می توانید از* dict *برای دسترسی به مقدار ویژگی های کلاس با مشخص کردن نام ویژگی در پرانتز مربعی استفاده کنید، به عنوان معمول در یک دیکشنری به کلیدها دسترسی پیدا می کنید.*

در نمونه ها دیکشنری dict. *فقط شامل ویژگیهای نمونه خواهد بود:*

In [29]:

```
۱ instance SampleClass("Hello!")
```

In [30]:

```
۱ instance. instance_attr
```

Out[30]:

'Hello!'

In [31]:

```
1 instance.method()
```

Class attribute: 100

Instance attribute: Hello!

In [32]:

```
1 instance._dict_
```

Out[32]:

{"instance_attr': 'Hello!')

In [33]:

```
1 vars(instance)
```

Out[33]:

{"instance_attr': 'Hello!')

In [34]:

```
1 instance._dict__["instance_attr"]
```

Out[34]:

'Hello!'

In [36]:

```
1 SampleClass._dict_["class_attr"]
```

In [37]:

```
۱ instance. instance_attr
```

Out[37]:

'Goodbye'

در این مثال دیکشنری *dict.* نمونه شامل *instance_attr* و مقدار خاص آن برای شیء در دست است باز هم شما می توانید با استفاده از *dict.* و نام ویژگی در پرانتز مربعی به هر ویژگی نمونه موجود دسترسی پیدا کنید.

شما میتوانید بصورت پویا دیکشنری *dict.* نمونه را تغییر دهید این بدان معناست که شما میتوانید ارزش ویژگی های نمونه موجود را از طریق *dict.* تغییر دهید همانطور که در مثال نهایی بالا انجام دادید شما حتی می توانید با استفاده از دیکشنری *dict* خود ،نمونه ویژگیهای جدید را به یک نمونه اضافه کنید.

استفاده از *dict* برای تغییر مقادیر ویژگی های نمونه به شما اجازه می دهد که در هنگام سیم بندي ویژگي هاي در پایتون خطاء *RecursionFrror* را که به علت بازگشت تابع بیش از حد به محمد می آید، جلوگری کنید.

ویژگی های کلاسی و نمونه ای پویا

در پایتون شما میتوانید ویژگیهای جدید را به کلاس ها و نمونه های خود به صورت دینامیک اضافه کنید. این امکان به شما اجازه میدهد تا در پاسخ به نیازها یا زمینه های تغییر کننده داده ها و رفتار جدید را به کلاس ها و اشیاء خود اضافه کنید. همچنین به شما اجازه میدهد تا کلاس های موجود را برای نیازهای خاص و پویا سفارشی کنید.

به عنوان مثال شما میتوانید از این ویژگی پایتون استفاده کنید زمانی که در زمان تعریف خود ،کلاس ویژگی های مورد نیاز یک کلاس مشخص نمی باشد.

کلاس زیر را در نظر بگیرید که قصد دارد یک ردیف داده از جدول پایگاه داده یا یک فایل *CSV* را ذخیره کند:

In [۳۸]:

```
۱ class Record:
۲     """Hold a record of data """
```

ر این کلاس هیچ ویژگی یا متد تعریف نکرده اید زیرا نمی دانید کلاس چه داده هایی را ذخیره می.کند. خوشبختانه شما می توانید ویژگی ها و حتی متدها را به این کلاس به صورت دینامیک اضافه کنید.

به عنوان مثال فرض کنید یک ردیف داده را از یک فایل *employees.cs* با استفاده از *csv* *DictReader* خوانده اید. این کلاس داده ها را میخواند و آنها را در یک شئ شبیه به دیکشنری برمیگرداند. حال فرض کنید که شما دارای دیکشنری داده های زیر هستید:

In [۴۰]:

```
john = {
    "name": "John Doe",
    "position": "Python Developer",
    "department": "Engineering",
    "salary": 80000,
    "hire_date": "2020-01-01",
    "is_manager": False,
}
```

سپس شما میخواهید این داده ها را به یک نمونه از کلاس *Record* خود اضافه کنید و باید هر فیلد داده را به عنوان ویژگی نمونه نشان دهید. به این صورت می توانید این کار را انجام دهید:

In [۴۱]:

```
john_record = Record()

for field, value in john.items():
    setattr(john_record, field, value)
```

In [۴۲]:

```
john_record.name
```

Out[42]:

'John Doe'

In [۴3]:

```
john_record.department
```

Out[43]:

'Engineering'

In [۴4]:

```
۱ john_record._dict_
```

Out[44]:

{'name': 'John Doe',

'position': 'Python Developer',

'department': 'Engineering', 'salary: 80000,

'hire_date': '2020-01-01',

'is_manager': False}

هم چمنین می توانیم با استفاده از نوتشین نقطه ای متدها و ویژگی های جدیدی را اضافه نماییم. در واقع می توانیم یک شی را با هر ویژگی یا متدی از صفر بسازیم

In [۴5]:

```
۱ class User:
2     pass
3
4 jane User()
5
6 jane.name = "Jane Doe"
7 jane.job= "Data Engineer"
8 jane. _dict_
```

Out[45]:

{'name': 'Jane Doe', 'job': 'Data Engineer'}

In [۴6]:

```
# Add methods dynamically
def _init_(self, name, job):
    self.name = name
    self.job = job

  User._init_ = _init_

User._dict_
```

Out[46]:

mappingproxy({' module ':main_ ',

dict_': <attributedictof 'User' objects>,

doc ': None,

init': <function _maininit(self, name, job)>})

In [۴7]:

```
linda User ("Linda Smith", "Team Lead")
linda._dict_
```

Out[47]:

{'name': 'Linda Smith', 'job' : 'Team Lead' }

@Class Methods With classmethod

شما همچنین میتوانید متدهای کلاس را به کلاس های پایتون سفارشی خود اضافه کنید یک متد کلاس یک متد است که شیء کلاس را به عنوان اولین آرگومان خود به جای *self* دریافت میکند در این صورت آرگومان باید با *cls* نامیده شود که همچنین یک قرارداد قوی در پایتون است بنابراین شما باید به آن چسبیده باشید.

شما می توانید با استفاده از دکوراتور *@classmethod* متدهای کلاس را ایجاد کنید فراهم کردن سازنده های چندگانه برای کلاس های شما یکی از معمول ترین موارد استفاده از متدهای کلاس در پایتون است.

به عنوان مثال فرض کنید میخواهید یک سازنده جایگزین به *ThreePoint* خود اضافه کنید تا بتوانید به سرعت نقاط را از *tuples* یا لیست های مختصات ایجاد کنید:

In [۴8]:

```
class ThreeDPoint:
    def _init_(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

    def _iter_(self):
        yield from (self.x, self.y, self.z)

    @classmethod
    def from sequence (cls, sequence):
        return cls(*sequence)

    def __repr_(self):
        return f"{type(self).__name_}({self.x), (self.y), (self.z))"
```

در متد کلاس*from sequence ()* یک دنباله از مختصات را به عنوان آرگومان دریافت میکنید یک شی *ThreeDPoint* از آن ایجاد میکنید و شی را به فراخواننده باز میگردانید برای ایجاد شی ،جدید از آرگومان *cls* استفاده می کنید که حاوی یک مرجع ضمنی به کلاس فعلی است که پایتون به صورت خودکار در متد شما درج می کند.

اینجاست که چگونه این متد کلاس کار می کند:

In [51]:

```
yy = ThreeDPoint (9, -6, 3)
```

In [52]:

```
yy
```

Out[52]:

ThreeDPoint (9, -6, 3)

In [53]:

```
x = ThreeDPoint.from_sequence ((4, 8, 16))
```

In [54]:

```
1 x
```

Out[54]:

ThreeDPoint(4, 8, 16)

In [55]:

```
1 ThreeDPoint. From_sequence((4, 8, 16))
```

Out[55]:

ThreeDPoint (4, 8, 16)

In [56]:

```
1 point=ThreeDPoint (7, 14, 21)
2 point.from_sequence( 3, 6, 9)
```

Out[56]:

ThreeDPoint(3, 6, 9)

In [57]:

```
1 point
```

Out[57]:

ThreeDPoint (7, 14, 21)

در این مثال کلاس *ThreeDPoint* را به طور مستقیم برای دسترسی به متد کلاس *from sequence*() استفاده می کنید. توجه داشته باشید که همچنین میتوانید با استفاده از یک نمونه محکم مانند *point* در مثال به متد دسترسی پیدا کنید. در هر یک از تماسهای *from_sequence*() شما یک نمونه کاملاً جدید از *ThreePoint* دریافت خواهید کرد. با این حال برای بهترین وضوح و جلوگیری از ابهام باید به متدهای کلاس از طریق نام کلاس مربوطه دسترسی پیدا کنید.

static Methods With @staticmethod

کلاس های پایتون شما همچنین میتوانند دارای متدهای استاتیک .باشند این متدها نمونه یا کلاس را به عنوان آرگومان دریافت نمی کنند ،بنابراین آنها توابع معمولی هستند که در یک کلاس تعریف شده اند شما همچنین میتوانید آنها را به عنوان تابع مستقل خارج از کلاس تعریف کرده باشید.

به طور معمول به جای یک تابع معمولی خارج از کلاس یک متد استاتیک را تعریف خواهید کرد زمانی که آن تابع به صورت نزدیک با کلاس شما مرتبط است و میخواهید آن را برای راحتی یا سازگاری با *API* کد خود بسته بندی کنید به خاطر داشته باشید که فراخوانی یک تابع از فراخوانی یک متد کمی متفاوت است برای فراخوانی یک متد شما باید یک کلاس یا شیء را مشخص کنید که این متد را فراهم میکند

اگر می خواهید یک متد استاتیک در یکی از کلاسهای سفارشی خود بنویسید آنگاه باید از دکوراتور *@staticmethod* استفاده کنید به روش *show_intro_message ()* در زیر نگاه کنید:

In [58]:

```
class ThreeDPoint:
        def _init_(self, x, y, z):
                self.x = x
                self.y = y
                self.z = z

        def _iter (self):
                yield from (self.x, self.y, self.z)

        @classmethod
        def fron_sequence(cls, sequence):
                return cls(*sequence)

        @staticmethod
        def show_intro_message(name) :
                print (f"Hey {name}! This is your 3D Point!")

    def _repr_(self):
                return f" {type(self).name}({self.x}, {self.y}, {self.z})"
```

متد استاتیک *show_intro_message*() یک نام به عنوان آرگومان دریافت میکند و پیامی را روی صفحه نمایش چاپ می کند. توجه داشته باشید که این فقط یک مثال بازیگوشانه از نحوه نوشتن متدهای استاتیک در کلاس های شما است. متدهای استاتیک مانند *show_intro_message*() بر روی نمونه فعلی، *self* یا کلاس فعلی *cls* عمل نمی کنند. آنها به عنوان توابع مستقل در یک کلاس قرار دارند به طور معمول آنها را درون یک کلاس قرار میدهید زمانی که به صورت نزدیک با آن کلاس مرتبط هستند اما لزوماً بر کلاس یا نمونه های آن تأثیر ندارند.

اینجاست که چگونگی عملکرد این متد است:

In [59]:

```
1 ThreeDPoint. show_intro_message("Pythonista")
```

Hey Pythonista! This is your 3D Point!

In [60]:

```
1 point ThreeDPoint(2, 4, 6)
```

In [61]:

```
1 point. show_intro_nessage("Python developer )
```

Hey Python developer! This is your 3D Point

Getter and Setter Methods vs Properties

زبان های برنامه نویسی مانند جاوا و سی پلاس پلاس ویژگی های خود را به عنوان بخشی از رابط برنامه نویسی عمومی کلاس های خود نشان نمیدهند به جای آن این زبان های برنامه نویسی از روش های *getter* و *setters* برای دسترسی به ویژگی ها استفاده فراوان میکنند.

استفاده از روش های دسترسی به و به روزرسانی ویژگی *encapsulation* را ترویج میدهد *encapsulation* یک اصل *OOP* بنیادین است که توصیه میکند حالت یا داده یک شیء را از دنیای خارج جلوگیری از دسترسی مستقیم حفاظت کند. حالت شئ فقط از طریق یک رابط عمومی که شامل متدهای *getter* و *setter* است قابل دسترس است.

به عنوان مثال فرض کنید ،حا جوب *Person* با ویژگ *name* دارید. شما من توانید *name* را به عنوان یک ویژگی

In [68]:

```
class Person:
    def _init_(self, name):
        self.set_name (nane)

    def_get_name(self):
        return self._name

    def set_name(self, value):
        self._name = value
```

در این مثال ()*get_name* روش *getter* است و به شما اجازه میدهد به ویژگی پایه دسترسی پیدا کنید. به طور مشابه ()*set_name* روش *setter* است و به شما اجازه میدهد تا مقدار فعلی *name* را تغییر دهید ویژگی *name* غیر عمومی است و جایی است که داده های واقعی ذخیره می شوند. اینجاست که چگونه میتوانید از کلاس *Person* خود استفاده کنید:

In [69]:

```
Jane = Person("Jane")
jane.get_name()
```

Out[69]:

'Jane'

In [70]:

```
jane.set_name("Jane Doe")
jane.get_name()
```

Out[70]:

'Jane Doe'

In [71]:

```
1 vars(Jane)
```

Out[71]:

{'_name': 'Jane Doe'}

In [72]:

```
1 jane._name
```

Out[72]:

'Jane Doe'

در اینجا شما یک نمونه از *Person* با استفاده از سازنده کلاس و *"Jane"* به عنوان نام مورد نیاز ایجاد می کنید. این بدان معناست که میتوانید از روش *get_name*() برای دسترسی به نام *Jane* و روش *set_name*() برای به روزرسانی آن استفاده کنید.

الگوی *getter* و *setter* در زبان هایی مانند جاوا و *C++* رایج است. علاوه بر ترویج تعلقات و *API*های متمرکز بر فراخوانی روش این الگو به شما امکان میدهد تا به سرعت رفتار شبیه به تابع را به ویژگی های خود اضافه کنید بدون اینکه تغییرات شکست آور در *API*های خود را به دنبال داشته باشید.

با این حال این الگو در جامعه پایتون کمتر محبوب است در پایتون کاملاً عادی است که ویژگی ها را به عنوان بخشی از *API* عمومی یک شئ فراهم کنید اگر همیشه نیاز به اضافه کردن رفتار شبیه به تابع به بالای یک ویژگی عمومی دارید آنگاه می توانید آن را به جای آنکه با یک متد *API* جایگزین شود به یک خصوصیت تبدیل کنید.

در اینجاست که بسیاری از توسعه دهندگان پایتون کلاس *Person* را خود را مینویسند:

In [65]:

```
1 class Person:
2     def_init_(self, name):
3         self.name = name
```

این کلاس روش getter و setter برای ویژگی name. ندارد . به جای ،آن ویژگی را به عنوان بخشی از API خود فراهم می کند بنابراین می توانید آن را به صورت مستقیم استفاده کنید

In [66]:

```
1 jane Person ("Jane")
2 jane.name
```

Out[66]:

'Jane'

In [67]:

```
1 jane.name = "Jane Doe"
2 jane.name
```

Out[67]:

'Jane Doe'

در این مثال به جای استفاده از یک متد *setter* برای تغییر مقدار *name* ویژگی را به صورت مستقیم در یک عبارت اختصاص استفاده میکنید این عمل در کد پایتون رایج است. اگر کلاس *Person* شما به یک نقطه ای برسد که نیاز به اضافه کردن رفتار شبیه به تابع به بالای *name*. دارید آنگاه میتوانید ویژگی را به یک خصوصیت تبدیل کنید.

برای مثال فرض کنید نیاز دارید ویژگی را با حروف بزرگ ذخیره کنید سپس میتوانید شبیه به زیر عمل کنید:

In [73]:

```
1  class Person:
2      def _init_(self, name):
3          self.name = name
4          #self.set_name (name)
5
6      @property
7      def name(self):
8          return self._name
9
10     @name.setter
11     def name(self, value):
12         self._name = value.upper()
```

این کلاس *name* را به عنوان یک خصوصیت با روش *getter* و *setter* مناسب تعریف میکند هنگامی که شما به مقدار ویژگی دسترسی یا آن را به روز میکنید پایتون به ترتیب این متدها را به صورت خودکار فراخوانی می کند. متد *setter* از بالا بردن مقدار ورودی قبل از اختصاص دادن آن به *name* مراقبت میکند:

In [74]:

```
1 jane = Person ("Jane")
2 jane.name
```

Out[74]:

'JANE'

In [75]:

```
1 vars (jane)
```

Out[75]:

{'_name': 'JANE ' }

In [76]:

```
1 jane.name = "Jane Doe"
2 jane.name
```

Out[76]:

'JANE DOE'

خصوصیات پایتون به شما اجازه میدهند تا رفتار شبیه به تابع را به ویژگی های خود اضافه کنید در حالی که همچنان از آن ها به عنوان ویژگی های عادی به جای روش ها استفاده میکنید توجه کنید که هنوز میتوانید با استفاده از یک عبارت اختصاص به *name.*مقادیر جدید را اختصاص دهید به جای فراخوانی متد. اجرای این عبارت روش *setter* را فعال میکند که مقدار ورودی را بالا می برد.

بررسی کلاس های ویژه از کتابخانه استاندارد پایتون

در کتابخانه استاندارد پایتون ،شما ابزارهای زیادی را که چالش های مختلف را حل میکنند پیدا خواهید کرد. در میان تمام این ابزارها، شما چندین ابزار خواهید دید که شما را در زمان نوشتن کلاس های سفارشی به صورت مؤثرتری کمک می کنند.

به عنوان مثال اگر شما به دنبال یک ابزار هستید که شما را از نوشتن بسیاری از کدهای *boilerplate* مربوط به کلاس نجات دهد، آنگاه شما میتوانید از *data classes* و ماژول *dataclasses* استفاده کنید.

به طور مشابه اگر به دنبال یک ابزار هستید که به شما اجازه میدهد به سرعت تعریف شمارش های بر پایه کلاس از ثابت ها را داشته باشید آنگاه میتوانید چشم خود را به ماژول *enum* و نوع های مختلف کلاس شمارش خود بپردازید.

در بخش های زیر شما با اصول استفاده از *data classes* و *enumerations* برای نوشتن ،بهینه قابل اطمینان و کلاس های ویژه در پایتون آشنا خواهید شد.

Data Classes

پایتون در کلاس های دادهای خود به ذخیره داده ها میپردازد با این حال آنها همچنین تولید کننده کد هستند که برای شما در پشت صحنه کد *boilerplate* بسیاری از کلاس های مربوط به شما را تولید میکنند.

به عنوان مثال اگر از زیرساخت کلاس داده استفاده کنید تا یک کلاس سفارشی بنویسید آنگاه نیازی به پیاده سازی متدهای ویژه مانند __*init*__() و __*repr*__() و __*eq*__() و __*hash*__(). نخواهید داشت کلاس داده آنها را برای شما خواهد نوشت علاوه بر این کلاس داده این متدها را با رعایت بهترین شیوه های کاربردی و جلوگیری از خطاهای احتمالی نوشته است.

همانطور که میدانید متدهای ویژه قابلیت های مهمی را در کلاس های پایتون پشتیبانی میکنند در مورد کلاس های داده شما یک نمایش رشته ،دقیق قابلیت های ،مقایسه قابلیت هش کردن و غیره خواهید داشت.

اگرچه نام کلاس داده ممکن است این نوع کلاس را به محدود بودن در بردارنده داده ها محدود ،کند اما همچنین متدهای دیگری را ارائه میدهد بنابراین کلاس های داده شبیه به کلاس های عادی هستند اما با قابلیت های فوق العاده

برای ایجاد یک کلاس داده به سادگی *@dataclass decorator*را از ماژول *dataclasses* وارد کنید شما از این *decorator* در تعریف کلاس خود استفاده خواهید کرد در این حالت شما نیاز به نوشتن یک متد __*init*__().نخواهید داشت. شما فقط با استفاده از *type hints* فیلدهای داده را به عنوان ویژگی های کلاس تعریف خواهید کرد.

بطور مثال، به این صورت میتوانید کلاس *ThreeDPoint* را به عنوان یک کلاس داده بنویسید:

In [77]:

```
from dataclasses import dataclass

@dataclass
class ThreeDPoint:
      x: int | float
     y: int | float
     z: int | float

     @classmethod
     def from_sequence (cls, sequence):
          return cls (*sequence)

    @staticmethod
   def show_intro_message(name) :
          print("Hey (name)! This is your 3D Point!")
```

این پیاده سازی جدید *ThreePoint* از *@dataclass decorator*پایتون استفاده میکند تا کلاس عادی را به کلاس داده تبدیل کند به جای تعریف یک متد *()__init__*، شما ویژگی های نمونه را با نوع مربوطه آنها لیست میکنید. کلاس داده به شما کمک خواهد کرد تا یک مقداردهی مناسب برای شما بنویسد توجه داشته باشید که شما *()iter* یا *()repr* را نیز تعریف نمیکنید

بعد از تعریف فیلدهای داده یا ویژگی ها میتوانید شروع به اضافه کردن روش های مورد نیاز خود کنید. در این مثال شما روش کلاس *()from_sequence* و روش استاتیک *()show_intro_message* را حفظ میکنید،برای بررسی قابلیت های اضافی که *@dataclass*به این نسخه از *ThreeDPoint* اضافه کرده است به جلو بروید و کد زیر را اجرا کنید.

In [78]:

```
point_1 = ThreeDPoint (1.0, 2.0, 3.0)

point 2 = ThreeDPoint (2, 3, 4)

point_1 == point_2
```

Out[78]:

False

In [79]:

```
1 point_3 = ThreeDPoint(1, 2, 3)
2
3 point_1 == point_3
```

Out[79]:

True

In [80]:

```
1 point_1
```

Out[80]:

ThreeDPoint (x=1.0, y=2.0, z=3.0)

In [81]:

```
1 point_3
```

Out[81]:

ThreeDPoint (x=1, y=2, z=3)

In [82]:

```
1 from dataclasses import astuple
2
3 astuple (point_1)
```

Out[82]:

(1.0, 2.0, 3.0)

Enumerations

شما در چندین زبان برنامه نویسی نوع دادهای *enum* یا *enumeration* را خواهید یافت *Enums* به شما اجازه میدهند تا مجموعه ای از ثابت های نامگذاری شده را ایجاد کنید که به عنوان اعضا شناخته میشوند و میتوانند از طریق خود *enum* دسترسی پیدا کنند.

پایتون نوع دادهای *enum* داخلی ندارد. خوشبختانه پایتون ۳٫۴ ماژول *enum* را معرفی کرده است تا کلاس *Enum* را برای پشتیبانی از *enum*های عمومی فراهم کند.

روزهای هفته ماه ها و فصول سال کدهای وضعیت *HTTP*، رنگ های چراغ ترافیک و برنامه های قیمت گذاری یک سرویس وب، همگی نمونه های خوبی از ثابت هایی هستند که میتوانید در *enum* گروه بندی کنید به طور خلاصه، شما میتوانید از *enums* برای نشان دادن متغیرهایی استفاده کنید که مقدار گیری آنها تحت یک مجموعه محدود از مقادیر ممکن است.

کلاس *Enum*، در میان کلاس های مشابه دیگر در ماژول *enum* به شما اجازه میدهد تا به سرعت و با کارآمدی *enum* های سفارشی یا گروه های ثابت مشابه با ویژگی های خوب و جذاب از خود استفاده کنید علاوه بر ثابت ،عضو *enums* همچنین ممکن است دارای روش های عمل با آن ثابت باشد.

برای تعریف یک *enum* ،سفارشی میتوانید از کلاس *Enum* به عنوان زیر کلاس استفاده کنید در ادامه مثالی از یک *enum* را مشاهده میکنید که روزهای هفته را گروه بندی میکند

In [83]:

```
1  from enum import Enum
2
3  class WeekDay (Enum):
4      MONDAY = 1
5      TUESDAY = 2
6      WEDNESDAY = 3
7      THURSDAY = 4
8      FRIDAY = 5
9      SATURDAY = 6
10     SUNDAY = 7
```

اگر سعی کنید مقدار یک عضو *enum* را تغییر دهید خطای *AttributeError* دریافت خواهید کرد ،بنابراین اعضای *enum* به صورت دقیق ثابت هستند شما میتوانید بر روی اعضای *enum* به صورت مستقیم حلقه بزنید زیرا *enum* ها به طور پیشفرض حلقه را پشتیبانی میکنند.

In [84]:

```
1 WeekDay. MONDAY = 0
```

AttributeError

Traceback (most recent call last)

Cell In[84], line 1

---> 1 WeekDay. MONDAY = 0

File C:\ProgramData\Anaconda3\lib\enum.py: 480, in EnumMeta._setattr_(cls, name, value)

478 member_map = cls._dict_.get('_member_map_', {})

479 if name in member_map:

--> 480 raise AttributeError('Cannot reassign members.')

481 super().setattr(name, value)

AttributeError: Cannot reassign members.

In [85]:

```
1 list(WeekDay)
```

Out [85]:

[<WeekDay.MONDAY: 1>,

<WeekDay. TUESDAY: 2>,

<WeekDay.WEDNESDAY: 3>,

<WeekDay. THURSDAY: 4>,

<WeekDay. FRIDAY: 5>,

<WeekDay.SATURDAY: 6>,

<WeekDay. SUNDAY: 7>]

شما میتوانید با استفاده از نحو مختلف به صورت مستقیم به اعضای آنها دسترسی پیدا کنید:

In [86]:

```
1 WeekDay (1)
```

Out [86]:

In [89]:

```
1 <WeekDay.MONDAY: 1>
```

Out [89]:

<WeekDay.MONDAY: 1>

In [87]:

```
1 WeekDay (2)
```

Out [87]:

<WeekDay. TUESDAY: 2>

In [90]:

```
1 WeekDay["WEDNESDAY"]
```

Out [90]:

<WeekDay. TUESDAY: 3>

در مثال اول شما با استفاده از نمایش نقطه ای به یک عضو ***enum*** دسترسی پیدا میکنید که بسیار شفاف و خوانا است. در مثال دوم شما با فراخوانی ***enum*** با مقدار آن عضو به عنوان یک ،آرگومان به یک عضو دسترسی پیدا میکنید در نهایت شما از نحو شبیه به دیکشنری برای دسترسی به یک عضو دیگر با نام استفاده میکنید.اگر میخواهید به اجزای یک عضو دسترسی دقیق داشته باشید میتوانید از ویژگی های ***name*** و ***value.***استفاده کنید که در زمینه حلقه بسیار مفید هستند:

In [91]:

```
1 WeekDay. THURSDAY.name
```

Out [91]:

'THURSDAY'

In [92]:

```
1 WeekDay. THURSDAY.value
```

Out [92]:

4

In [93]:

```
1 for day in WeekDay:
2     print (day.name, "->", day.value)
```

Out [93]:

MONDAY -> 1

TUESDAY -> 2

WEDNESDAY -> 3

THURSDAY -> 4

FRIDAY -> 5

SATURDAY -> 6

SUNDAY -> 7

در نهایت ما میتوانیم متدهای دلخواه خودمان را نیز اضافه نماییم:

In [94]:

```
1  from enum import Enum
2
3  class WeekDay (Enum):
4          MONDAY = 1
5          TUESDAY = 2
6          WEDNESDAY = 3
7          THURSDAY 4
8          FRIDAY = 5
9          SATURDAY = 6
10         SUNDAY = 7
11
12     @classmethod
13     def favorite_day (cls):
14             return cls. FRIDAY
15
16     def _str_(self):
17             return f"Current day: {self.name}"
```

In [95]:

```
1  WeekDay. favorite_day()
```

Out [95]:

<WeekDay. FRIDAY: 5>

In [96]:

```
1  print(WeekDay.FRIDAY)
```

Current day: FRIDAY

سلسه مراتب کلاس ها

استفاده از *inheritance* شما میتوانید سلسله مراتب کلاس ها را طراحی و ساختاردهی کنید سلسله مراتب کلاس یک مجموعه از کلاس های نزدیک به هم هستند که از طریق *inheritance* به یکدیگر متصل شده اند و در یک ساختار شبیه به درخت قرار دارند.

کلاس یا کلاس های بالای سلسله مراتب کلاس های پایه هستند در حالی که کلاس های پایین تر کلاس های مشتق یا زیر کلاس هستند سلسله مراتب مبتنی بر *inheritance* رابطه *Mis-a-type-of-* بین زیر کلاس ها و کلاس پایه خود را بیان می کنند.

در هر سطح از سلسله مراتب ویژگی ها و رفتار از سطوح بالاتر به ارث میبرند بنابراین کلاس های بالای سلسله مراتب کلاس های جنرال با عملکرد مشترک هستند در حالی که کلاس های پایین تر در سلسله مراتب تخصص بیشتری دارند. آنها ویژگی ها و رفتار از *superclass* خود را به ارث میبرند و همچنین ویژگیها و رفتار خود را نیز اضافه می کنند طبقه بندی تاکسونومی حیوانات یک مثال رایج است برای توضیح سلسله مراتب کلاس در این سلسله مراتب شما یک کلاس *Animal* جنرال دارید در زیر این ،کلاس شما ممکن است زیر کلاس های *Mammal Bird Fish*، و غیره داشته باشید. این زیرکلاس ها کلاسهای خاص تر از *Animal* هستند و ویژگی ها و روش های آن را به ارث می.برند. آنها نیز ممکن است ویژگي هاي خود را داشته باشند.

برای ادامه با سلسله مراتب شما ممکن است زیرکلاس *Mammal Bird* و *Fish* را داشته باشید و زیرکلاس های مشتق شده وا با خصوصیات حتي خصوصیات خصوصی تر را ایجاد کنید در اینجا یک نمونه بازیچه کوتاه است

In [97]:

```
class Animal:
    def _init_(self, name, sex, habitat):
        self.name = name
        self.sex = sex
        self.habitat = habitat

class Mammal (Animal):
        unique_feature = "Mammary glands"

 class Bird (Animal):
      unique_feature = "Feathers"

class Fish (Animal):
    unique_feature = "Gills"

class Dog (Mammal):
    def walk(self):
        print("The dog is walking")

class Cat (Mammal):
```

```
21      def walk(self):
22          print("The cat is walking")
23
24  class Eagle(Bird) :
25       def fly(self):
26          print("The eagle is flying")
27
28  class Penguin(Bird):
29      def swin(self):
30          print("The penguin is swimming")
31
32  class Salmon (Fish):
33       def swin(self):
34          print("The salmon is swimming")
35
36  class Shark (Fish):
37      def swin(self):
38          print("The shark is swimming")
```

در بالای سلسله مراتب شما کلاس *Animal* را دارید این کلاس پایه سلسله مراتب شما است این ویژگی های *name* و *sex* و *habitat* را دارد که به ترتیب شیء های رشت هایی هستند این ویژگی ها برای تمام حیوانات مشترک هستند. شما با به ارث بردن از *Animal* کلاس های *Mammal Bird* و *Fish* را تعریف میکنید این کلاس ها یک ویژگی سپس کلاس .*unique_feature* دارند که ویژگی تمایز دهنده هر گروه حیوانات را نگه می دارد.

Extended vs Overridden Methods

سپس شما موجودات خاص مانند *Dog* و *Cat* را ایجاد میکنید این کلاس ها دارای روش های خاصی هستند که به تمام سگ ها و گربه ها مشترک است به طور مشابه شما دو کلاس را تعریف میکنید که به ترتیب از *Bird* و دو کلاس دیگر به ارث برده شده است.

زمانی که از ارث بری استفاده میکنید ممکن است با یک مسئله جالب و چالش برانگیز روبرو شوید در برخی از موارد یک کلاس پدر تنها در سطح پایه یک قابلیت خاص را فراهم میکند و شما ممکن است بخواهید آن قابلیت را در زیر کلاس های خود گسترش دهید در دیگر موارد ویژگی در کلاس پدر برای زیر کلاس مناسب نیست.

در این موارد میتوانید از یکی از راهبردهای زیر استفاده کنید به تفکیک مورد خاص شما

extend یک متد به ارث برده شده در یک زیر کلاس به این معنی است که شما قابلیت فراهم شده توسط کلاس پدر را دوباره استفاده کرده و قابلیت جدید را بر روی آن اضافه خواهید کرد ***override*** یک متد به ارث برده شده در یک زیر کلاس به این معنی است که شما به طور کامل قابلیت از کلاس پدر را دور خواهید انداخت و قابلیت جدید را در زیر کلاس فرا خواهید داد اینجا یک مثال از سلسله مراتب کلاس کوچک است که روش اول را برای فراهم کردن قابلیت های گسترده بر اساس قابلیت های به ارث برده شده به کار می برد

In [98]:

```
1  Class Aircraft:
2      def_init_(self, thrust, lift, max_speed) :
3              self.thrust = thrust
4              self.lift = lift
5              self.max_speed = max_speed
6
7     def show_technical_specs(self) :
8              print(f"Thrust: (self.thrust) km")
9              print("Lift: (self.lift) kg")
10             print (f"Max speed: {self.max_speed} km/h")
```

In [99]:

```
1  class Helicopter (Aircraft):
2        def_init_(self, thrust, lift, max_speed, num_rotors):
3              super().init_(thrust, lift, max_speed)
4              self.num_rotors = num_rotors
5
6       def show_technical_specs(self):
7             super().show_technical_specs()
8            print("Number of rotors: (self.num_rotors}")
```

In [100]:

```
1  h1 = Helicopter(25,300,150, 2)
```

In [101]:

```
1 h1
```

Out[101]:

<main.Helicopter at 0x1a1bdd16e90>

در این مثال شما کلاس هواپیما را به عنوان کلاس پایه تعریف میکنید در (() *_int_*.، چند ویژگی نمونه ایجاد میکنید. سپس متد *show_technical_specs*() را تعریف میکنید که اطلاعاتی در مورد مشخصات فنی هواپیما چاپ می کند.

بعد، شما هلیکوپتر را تعریف میکنید و از هواپیما به عنوان پدر بهره می برید. متد *_init_*() هلیکوپتر، رومتدش مربوطه هواپیما را با فراخوانی *super*() برای مقداردهی اولیه ویژگیهای *thrust* و *lift* و *max_speed.*گسترش می دهد. شما قبلاً چنین چیزی را در بخش قبل دیده بودید.

هلیکوپتر همچنین قابلیت متد *show_technical_specs*() را گسترش میدهد. در این حالت شما ابتدا با استفاده از *super show_technical_specs*() را از هواپیما فراخوانی می.کنید. سپس یک فراخوانی جدید به *print*() می کنید که اطلاعات جدید را به توصیف فنی هلیکوپتر در دست اضافه می کند.

اینجا نحوه عملکرد نمونه های هلیکوپتر است:

In [102]:

```
1 sikorsky_UH60 Helicopter (1490, 9979, 278, 2)
```

In [103]:

```
1 sikorsky_UH68. show_technical_specs()
```

Thrust: 1490 kW

Lift: 9979 kg

Max speed: 278 km/h

Number of rotors: 2

زمانی که روی یک نمونه هلیکوپتر *show_technical_specs()* را فراخوانی میکنید اطلاعات ارائه شده توسط کلاس پایه هواپیما و همچنین اطلاعات خاص اضافه شده توسط خود هلیکوپتر را دریافت میکنید شما قابلیت های هواپیما را در زیر کلاس خود ،هلیکوپتر گسترش داده اید.

حال زمان آن رسیده است که نگاهی به این بیندازید که چگونه میتوانید یک متد را در یک زیر کلاس بازنویسی کنید. به عنوان مثال بگویید که یک کلاس پایه به نام *Worker* دارید که چندین ویژگی و متد مانند مثال زیر را تعریف می کند:

In [104]:

```
class Worker:
      def init(self, name, address, hourly_salary):
            self.name = name
           self.address = address
           self.hourly_salary « hourly_salary

     def show_profile(self):
           print("Worker profile")
          print(f"Name: (self.name}")
         print (f"Address: {self.address}")
        print(f"Hourly salary: {self.hourly_salary}")

def calculate payroll (self, hours=48):
       return self.hourly_salary * hours
```

در این کلاس شما چند ویژگی نمونه را برای ذخیره داده های مهم در مورد کارگر فعلی تعریف می کنید. همچنین متد *show_profile*() را برای نمایش اطلاعات مربوط به کارگر ارائه می دهید. در نهایت شما یک متد کلی *calculate_payroll*() را برای محاسبه حقوق کارگران از حقوق ساعتی و تعداد ساعات کارکرد شان ارائه می دهید.

در دوره توسعه ،بعدی برخی از الزامات تغییر میکنند حالا متوجه میشوید که مدیران حقوق خود را به روش دیگری محاسبه می کنند. آنها یک پاداش ساعتی دارند که شما باید قبل از محاسبه مبلغ نهایی به حقوق ساعتی عادی آن اضافه کنید.

پس از فکر کردن به مسئله تصمیم می گیرید که *Manager* باید *calculate_payro11*() را به طور کامل بازنویسی کند. در اینجا پیاده سازی است که شما با آن آمده اید:

In [ ]:

```
class Manager (Worker):
        def init(self, name, address, hourly_salary, hourly_bonus):
              super().__init_(name, address, hourly salary)
             self.hourly_bonus = hourly_bonus

    def calculate_payroll (self, hours=40):
              return (self.hourly_salary + self.hourly_bonus) * hours
```

در مقدمه مدیر شما پاداش ساعتی را به عنوان یک آرگومان در نظر میگیرید سپس به طور معمول روش ()*init* والدین را فراخوانی کرده و ویژگی نمونه *hourly_bonus* را تعریف می.کنید. در نهایت شما ()*calculate_payroll* را با یک پیاده سازی کاملاً متفاوت که از قابلیت های به ارث برده شده استفاده نمی کند بازنویسی می کنید.

وراثت چندگانه

در پایتون شما میتوانید از چندین وراثت استفاده کنید این نوع وراثت به شما اجازه میدهد کلاسی را ایجاد کنید که از چندین والدین به ارث برده شده است زیر کلاس دسترسی به ویژگی ها و روشهای همه والدین خود دارد. وراثت چندگانه به شما اجازه میدهد کد را از چندین کلاس موجود استفاده مجدد کنید با این حال شما باید با مراقبتی از پیچیدگی وراثت چندگانه برخورد کنید در غیر این صورت مشکلاتی مانند مشکل الماس را ممکن است مواجه شوید. در بخش ترتیب روش *(MRO)*بیشتر درباره این موضوع بیاموزید.

اینجا یک مثال کوچک از وراثت چندگانه در پایتون است:

In [ ]:

```
class Vehicle:
      def _init_(self, make, model, color):
          self.make = make
          self.model = model
         self.color = color

     def start(self):
         print("Starting the engine...")

    def stop(self):
        print("Stopping the engine...")

    def show_technical_specs(self):
        print (f"Make: {self.make}")
       print("Model: (self.model)")
       print("Color: (self.color}")

class Car (Vehicle):
   def drive(self):
      print("Driving on the road...")

class Aircraft (Vehicle):
   def fly(self) :
       print("Flying in the sky...")

class FlyingCar (Car, Aircraft):
    pass
```

در این مثال شما یک کلاس *Vehicle* با ویژگی های *make* و *model* و *color* مینویسید این کلاس همچنین متدهای *start*() و *stop()*و *show_technical_specs*() را دارد. سپس شما یک کلاس *Car* ایجاد میکنید که از *Vehicle* به ارث برده شده است و با یک متد جدید به نام() *drive* توسعه می یابد همچنین شما یک کلاس *Aircraft* ایجاد میکنید که از *Vehicle* به ارث برده شده است و یک متد *Fly*() را اضافه می کند.

در نهایت شما یک *Flying class* تعریف میکنید تا یک ماشین را نشان دهد که میتوانید آن را در جاده رانده یا در آسمان پرواز کنید آیا این جالب نیست؟ توجه داشته باشید که این کلاس هر دو *Car* و *Aircraft* را در لیست کلاس های والد خود شامل میشود بنابراین عملکرد خود را از هر دو *superclass* به ارث خواهد برد.

اینجا نحوه استفاده از *Flyingcar class* را مشاهده می کنید:

In [106]:

```
1 space_flyer FlyingCar ("Space", "Flyer", "Black")
```

In [107]:

```
1 space_flyer. show_technical_specs()
```

Make: Space

Model: Flyer

Color: Black

In [108]:

```
1 space flyer.start()
```

Starting the engine....

In [109]:

```
1 space_flyer.drive()
```

Driving on the road...

In [110]:

```
1 space_flyer.fly()
```

Flying in the sky...

در این قطعه کد شما ابتدا یک نمونه از *Flyingcar* ایجاد میکنید سپس تمام متدهای آن را فراخوانی می کنید، از جمله متدهای به ارث برده شده همانطور که میبینید وراثت چندگانه باعث افزایش استفاده مجدد کد میشود و به شما اجازه می دهد تا به طور همزمان از عملکرد چندین کلاس پایه استفاده کنید به هر حال اگر این *Flyingcar* را واقعاً پرواز کنید مطمئن شوید که در حال پرواز موتور را نمی خاموشید!

نحوه تمایز متدها در فراخوانی (*Method Resolution Order (MR*

زمانی که از وراثت چندگانه استفاده میکنید ممکن است با مواردی روبرو شوید که یک کلاس از دو یا چند کلاس به ارث برده شده است که پایه یکسانی دارند این به عنوان مشکل الماس شناخته میشود. مسئله واقعی زمانی پدیدار می شود که والدین چندین نسخه از همان متد را فراهم می.کنند در این صورت سخت است تعیین کردن اینکه کدام نسخه از آن روش زیر کلاس در نهایت استفاده خواهد کرد.

پایتون با استفاده از ترتیب روش خاص (*MRO*)با این مشکل برخورد می.کند پس *MRO* در پایتون چیست؟ این الگوریتمی است که به پایتون میگوید چگونه در یک زمینه وراثت چندگانه به دنبال روش های به ارث برده شده بگردید. *MRO* پایتون تعیین میکند کدام پیاده سازی روش یا ویژگی را باید استفاده کنید زمانی که چندین نسخه از آن در سلسله مراتب کلاس وجود دارد.

MRO پایتون بر اساس ترتیب کلاس های والد در تعریف زیر کلاس است. به عنوان مثال *Car* قبل از *Aircraft* در *Flying Car class* در بخش قبل قرار دارد. *MRO* همچنین رابطه وراثت بین کلا سها را در نظر میگیرد به طور کلی پایتون به دنبال متدها و ویژگی ها به ترتیب زیر می گرد

- کلاس فعلی
- سوپر کلاسهای سمت چپ
- .سوپر کلاس بعدی که از چپ به راست تا آخرین سوپر کلاس لیست شده است
- سوپر کلاس های کلاس های به ارث برده شده
- کلاس شیء

مهم است به یاد داشته باشید که زیر کلاس ها در جستجو اول قرار میگیرند علاوه بر این اگر شما چندین والدین داشته باشید که هر کدام نسخه متفاوتی از یک متد یا ویژگی خاص را پیاده سازی کنند پایتون آنها را با ترتیبی که در تعریف کلاس لیست شده اند جستجو می کند.

برای توضیح*MRO* ، سلسله مراتب کلاس نمونه زیر را در نظر بگیرید:

In [112]:

```
class A:
    def method(self) :
        print("A.method")

class B(A):
    def method(self):
        print("B.method")

class c(A):
    def method(self):
        print("C.method")

class D(B, C):
    pass
```

در این مثال *D* از *B* و *C* به ارث برده شده است که از *A* به ارث برده شده است تمام سوپر کلاس ها در سلسله مراتب نسخه متفاوتی از() *method* را تعریف میکنند کدام یک از این نسخه ها در نهایت توسط *D* فراخوانی خواهد شد؟ برای پاسخ به این سوال به آرامی() *method* را روی یک نمونه *D* فراخوانی کنید:

In [113]:

```
D(). method()
```

B.method

وقتی شما *method( )* را روی یک نمونه *D* فراخوانی میکنید *B.method* را در صفحه خود مشاهده میکنید. این بدان معناست که پایتون ابتدا *method()*. را در کلاس *B* پیدا کرده است این نسخه از *method()*است که شما فراخوانی می کنید شما نسخه های *C* و *A* را نادیده می گیرید.

شما می توانید *MRO* فعلی یک کلاس داده شده را با استفاده از ویژگی خاص *_mro_*.بررسی کنید:

In [114]:

```
D._mro_
```

Out[114] :

(__main_.D, _main_.B, _main_.C, _main_.A, object)

در خروجی میتوانید ببینید که پایتون با عبور از *D* خود، سپس *B*سپس *c*، سپس *A* و در نهایت *object* کلاس پایه تمام کلاس های پایتون به دنبال متدها و ویژگی ها در *D* جستجو میکند.

ویژگی *_mro_*. میتواند به شما در تغییر کلاس های خود و تعریف *MRO* خاص کلاس شما کمک کند. روش تغییر این

Mixin Classes

یک کلاس *mixin* متدهایی را فراهم میکند که میتوانید در بسیاری از کلاس های دیگر استفاده کنید. کلاس های *mixin* نوع جدیدی را تعریف نمیکنند بنابراین قرار نیست به صورت نمونه سازی شوند شما از قابلیت آنها برای اضافه کردن ویژگی های اضافی به سرعت به کلاس های دیگر استفاده می کنید.

شما میتوانید به قابلیت یک کلاس *mixin* به متدهای مختلف دسترسی پیدا کنید یکی از این روش ها *inheritance* است. با این حال به ارث بردن از کلاس های *mixin* به معنای یک رابطه *isa* نیست زیرا این کلاس ها نوع های قابل تعریف را بیان نمیکنند آنها فقط وظایف خاص خود را بسته بندی میکنند که قرار است در کلاس های دیگر استفاده مجدد شود.

برای توضیح نحوه استفاده از کلاسهای *mixin* فرض کنید در حال ساخت یک سلسله مراتب کلاس با یک کلاس *Person* در بالاترین سطح هستید از این کلاس شما به کلاس های مشتق شده مانند *Employee Student Professor* و چندین دستگاه دیگر مشتق خواهید شد. سپس متوجه میشوید که تمام زیرکلاس های *Person* نیاز به متدهای *serialize* داده خود در فرمتهای مختلف از جمله *JSON* و *pickle* دارند.

با توجه به این شما به نوشتن یک *SerializerMixin class* فکر میکنید که این وظایف را بر عهده دارد. در زیر آنچه را پیدا میکنید:

In [115]:

```
import json
import pickle

class Person:
      def _init_(self, name, age):
            self.name = name
            self.age = age

class SerializerMixin:
      def to_json(self) :
            return json.dumps (self._dict_)

      def to_pickle(self):
            return pickle.dumps (self._dict_)

class Employee (SerializerMixin, Person):
      def _init_(self, name, age, salary):
            super().__init_(name, age)
            self.salary = salary
```

در این مثال *Person* کلاس والدین است و *SerializerMixin* یک کلاس *mixin* است که قابلیت سریال سازی را فراهم می کند. کلاس *Employee* از هر دو *SerializerMixin* و *Person* به ارث میبرد بنابراین، این متدهای *to_json* () و *topickle*() را به ارث خواهد برد که میتوانید از آنها برای سریال سازی نمونه های *Employee* در کد خود استفاده کنید. در این مثال *Employee* یک *Person*. است با این حال یک *SerializerMixin* نیست زیرا این کلاس نوع شیء را تعریف نمی کند فقط یک کلاس *mixin* است که قابلیتهای سریال سازی را بسته بندی میکند.

نحوه عملکرد *Employee* بصورت زیر خواهد بود:

In [116]:

```
john = Employee("John Doe", 30, 50000)
John.to_json()
```

Out[116] :

{"name": "John Doe", "age": 30, "salary": 50000}"

In [117]:

```
1 john.to_pickle()
```

Out[117] :

b'\x80\x04\x95+\x00\x00\x00\x00\x00\x00\x00}\x94(\x8c\x04name\x94\x8c\x08 john Doe\x94\x8c\x03age\x94K\x1e\x8c\x06salary\x94MP \xc3u.

اکنون کلاس *Employee* شما قادر به سریال سازی داده های خود با استفاده از فرمتهای *JSON* و *pickle* است. عالی است آیا می توانید به هر کلاس *mixin* مفید دیگری فکر کنید؟

تا این لحظه شما بسیاری در مورد *inheritance* ساده و چندگانه در *Python* یاد گرفته اید در بخش بعدی شما به برخی از مزایای استفاده از *inheritance* در هنگام نوشتن و سازماندهی که خود می پردازید.

مزایای استفاده از وراثت

- قابلیت استفاده مجدد() *Reusability*:شما میتوانید به سرعت کد کاری را از یک یا چند کلاس والدین به عنوان زیر کلاس های خود به تعداد دلخواه استفاده کنید.
- ماژولاریتی *Modularity()*:شما میتوانید از *inheritance* برای سازماندهی که خود در سلسله مراتب کلاس های مرتبط استفاده کنید.
- قابلیت نگهداری *Maintainability*:شما میتوانید به سرعت مشکلات را در یک کلاس والدین رفع کنید یا ویژگی های جدید را اضافه کنید این تغییرات به صورت خودکار در تمام زیرکلاس های آن در دسترس خواهند بود. *Inheritance* همچنین تکرار کد را کاهش میدهد
- *Polymorphism* :شما میتوانید زیرکلاس هایی را ایجاد کنید که میتوانند جایگزین کلاس والدین خود شوند و قابلیت های مشابه یا معادل را فراهم کنند.
- قابلیت گسترش *Extensibility()*:شما میتوانید به سرعت یک کلاس قبلی را با اضافه کردن داده های جدید و رفتار به زیر کلاس های خود گسترش دهید.

راهکارهای جایگزین وراثت در پایتون

Inheritance به خصوص *inheritance*، چندگانه میتواند یک موضوع پیچیده و سخت برای درک باشد. خوشبختانه *inheritance* تنها تکنیکی نیست که به شما اجازه میدهد تا قابلیت استفاده مجدد را در برنامه نویسی شی گرا داشته باشید. شما همچنین دارای *composition* هستید که یک رابطه *hasa* بین کلاس ها را نشان می دهد.

Composition به شما اجازه میدهد یک شیء را از اجزای آن .بسازید شیء *composite* دسترسی مستقیم به رابط هر اجزای خود ندارد. با این حال می تواند از پیاده سازی هر اجزای استفاده کند.

Delegation یک تکنیک دیگر است که میتوانید برای ارتقاء قابلیت استفاده مجدد کد در برنامه های *OOP* خود استفاده کنید. با *delegation* میتوانید رابطه های *can-do* را نشان دهید جایی که یک شی بر روی یک شیء دیگر برای انجام یک وظیفه خاص تکیه می کند

در بخش های بعد شما بیشتر در مورد این تکنولوژی ها و نحوه استفاده از آنها در کدهای شی گرا خود برای ساخت کدهای قابل اطمینان و قابل انعطاف خواهید آموخت.

Composition

همانطور که قبلاً گفته شد میتوانید از *composition* برای مدل کردن یک رابطه *hasa* بین اشیاء استفاده کنید. به عبارت دیگر از طریق *composition* میتوانید با ترکیب اشیاء که به عنوان اجزا عمل می،کنند اشیاء پیچیده ای را ایجاد کنید. توجه داشته باشید که این اجزا به عنوان کلاس های مستقل معنایی نخواهند داشت.

ترجیح *composition* بر *inheritance* به طراحی کلاس های قابل انعطاف تر منجر میشود بر خلاف *inheritance composition* در زمان اجرا تعریف میشود به این معنی که میتوانید به صورت پویا یک جزء فعلی را با یک جزء دیگر از همان نوع جایگزین کنید این ویژگی باعث میشود تا رفتار *composite* در زمان اجرا قابل تغییر باشد.

در مثال زیر شما از *composition* برای ساخت یک کلاس *Industrial Robot* از *Body* و *Arm* *components* استفاده میکنید:

In [119]:

```
class Industrial Robot:
    def _init_(self):
        self.body = Body()
        self.arm = Arm()

    def rotate_body_left(self, degrees=10):
        self.body.rotate_left(degrees)

    def rotate_body_right (self, degrees=10):
        self.body.rotate_right (degrees)

    def move_arm up (self, distance=1):
```

```
13          self.arm.move_up(distance)
14
15      def move_arm_down (self, distance=1):
16          self.arm.nove_down (distance)
17
18      def weld(self):
19          self.arm.weld()
20
21  class Body:
22      def _init_(self):
23          self.rotation = 0
24
25      def rotate_left(self, degrees=10):
26          self.rotation -= degrees
27          print("Rotating body (degrees) degrees to the left...")
28
29      def rotate_right (self, degrees=10):
30          self.rotation += degrees
31          print (f"Rotating body {degrees} degrees to the right...")
32
33  class Arm:
34      def _init_(self):
35          self.position= 0
36
37      def move_up(self, distance=1):
38          self.position += distance
39          print("Moving arm (distance) cm up...")
40
41      def move down(self, distance=1):
42          self.position -= distance
43          print (f"Moving arm {distance} cm down...")
44
45      def weld(self):
46          print("Welding...")
```

در این مثال شما یک کلاس *IndustrialRobot* را از اجزای آن *Body* و *Arm* ساخته اید کلاس *Body* حرکات افقی را فراهم می کند در حالی که کلاس *Arm* نماینده بازوی ربات است و حرکت عمودی و قابلیت جوشکاری را فراهم می کند.

اینجاست که چگونه میتوانید از *IndustrialRobot* در کد خود استفاده کنید:

In [120]:

```
1 robot Industrial Robot ()
```

In [122]:

```
1 robot
```

Out[122]:

<_main_. Industrial Robot at 0x1a1bddb7e20>

In [123]:

```
1 robot.rotate_body_left()
```

Rotating body 10 degrees to the left...

In [124]:

```
1 robot.move_arm_up (15)
```

Moving arm 15 cm up...

In [125]:

```
1 robot.move_arm_up(15)
```

Moving arm 15 cm up...

In [126]:

```
1 robot.rotate_body_right (20)
```

Rotating body 20 degrees to the right....

In [127]:

```
1 robot.move_arm_down(5)
```

Moving arm 5 cm down...

In [128]:

```
1 robot.weld()
```

Welding...

In [129]:

```
1 vars (robot)
```

Out[129]:

{'body' <main. Body at 0x1a1bddb5540>, 'arm': <_main_.Arm at 0x1a1bddb58d0>}

In [130]:

```
1 robot.body
```

Out[130]:

<main. Body at 0x1a1bddb5540>

In [131]:

```
1 robot.body.rotation
```

Out[131]:

10

In [132]:

```
1 robot.arm.position
```

Out[132]:

عالی ربات شما به عنوان انتظار کار میکند به شما اجازه میدهد بدن و بازوی خود را بر اساس نیازهای حرکتی خود حرکت دهید همچنین به شما اجازه میدهد تا قطعات مکانیکی مختلف را با هم جوش بدهید.

یک ایده برای ساخت ربات حتی پیشرفته تر پیاده سازی چندین نوع بازو با فناوری های جوشکاری مختلف است. سپس می توانید با اجرای *robot.arm = Newarm* بازو را تغییر دهید حتی میتوانید یک متد *change_arm*() به کلاس ربات خود اضافه کنید چطور به عنوان یک تمرین یادگیری به نظر میرسد؟

بر خلاف *inheritance composition* کل رابط اجزای را نمایش نمی دهد بنابراین *encapsulation* را حفظ می کند. در عوض اشیاء *composite* فقط به عملکردهای لازم خود از اجزای خود دسترسی و استفاده میکنند. این ویژگی باعث شود طراحی کلاس شما قابل اطمینان و قابل اعتماد تر شود زیرا عضوهای نامبرده را نشان نخواهد داد.

با توجه به مثال ربات فرض کنید چندین ربات مختلف در یک کارخانه دارید هر ربات میتواند قابلیت های مختلف مانند جوشکاری ،برش، شکل دادن پولیش و غیره داشته باشد همچنین چندین بازو مستقل دارید بعضی از آنها می توانند همه آن عملکردها را انجام دهند. بعضی دیگر فقط یك زیر مجموعه از عملکردها را انجام می دهند.

حال بگویید که یک ربات خاص فقط قادر به جوشکاری است. با این حال این ربات میتواند از بازوهای مختلف با فن آوری های جوشکاری مختلف استفاده کند در صورت استفاده از *inheritance* پس از آن که ربات به عملیات دیگری مثل برش و شکل دادن دسترسی پیدا کرده است میتواند حادثه یا خراب شدگی ایجاد کند.

در صورت استفاده از *composition* ربات جوشکار فقط به قابلیت جوشکاری بازو دسترسی خواهد داشت. با این حال، *composition* می تواند به شما در محافظت از کلاس های خود در برابر استفاده ناخواسته کمك کند.

Delegation

Delegation یک تکنیک دیگری است که میتوانید به عنوان جایگزین *inheritance* استفاده کنید با *delegation* می توانید رابطه های *cando* را مدل کنید جایی که یک شیء یک کار را به شی دیگری منتقل میکند که از اجرای کار مراقبت میکند. توجه داشته باشید که شیء منتقل شده مستقل از *delegator* وجود دارد.

می توانید از *delegation* برای دستیابی به *Code reuse*، *separation of concerns* و *modularity* استفاده کنید. به عنوان مثال فرض کنید میخواهید یک ساختار داده *stack* ایجاد کنید.

می توانید از *delegation* برای دستیابی به *Code reuse*، *separation of concerns* و *modularity* استفاده کنید.

In [133]:

```
class Stack:
        def _init_(self, items=None):
            if items is None:
                self._items = []
            else:
                self_items = list(items)

        def push(self, item):
            self._items.append(item)

        def pop(self) :
            return self._items.pop()

        def _repr_(self) -> str:
            return f"{type(self)._name_}({self. itens))"
```

در () *_init_*.، شما یک شیء لیست به نام *items* تعریف میکنید که میتواند داده های اولیه خود را از آرگومان های *items* بگیرد شما از این لیست برای ذخیره داده ها در *Stack* حاوی استفاده خواهید کرد، بنابراین تمام عملیات مربوط به ذخیره اضافه کردن و حذف داده ها را به این شیء لیست منتقل میکنید سپس عملیات های معمول *Stack* و ()*push*. و() *pop*. را پیاده سازی می کنید.

توجه کنید که این عملیات ها به راحتی مسئولیتهای خود را در () *items.append_*. و () *items.pop_*. به ترتیب منتقل می کنند کلاس *Stack* شما عملیات خود را به شیء لیست منتقل کرده است که در حال حاضر می داند چگونه آنها را انجام دهد. است مهم به یاد داشته باشید که این کلاس بسیار قابل انعطاف است شما میتوانید شیء لیست در *items_*.خود را با هر شیء دیگری جایگزین کنید، تا زمانی که ()*pop*. و () *append*. روش های آن را پیاده سازی کند. به عنوان مثال می توانید از یک شئ *deque* از ماژول *collections* استفاده کنید.

بدلیل استفاده از *delegation* برای نوشتن کلاس خود پیاده سازي داخلي لیست در *Stack* قابل مشاهده یا به صورت مستقیم در دسترس نیست که *encapsulation* را حفظ میکند

In [134]:

```
stack = Stack ([1, 2, 3])
```

In [135]:

```
1 stack
```

Out[135]:

Stack ([1, 2, 3])

In [136]:

```
1 stack.push(4)
```

In [137]:

```
1 stack
```

Out[137]:

Stack([1, 2, 3, 4])

In [138]:

```
1 stack.pop()
```

Out[138]:

4

In [139]:

```
1 stack.pop()
```

Out[139]:

3

In [140]:

```
1 stack
```

Out[140]:

Stack([1, 2])

In [141]:

```
1 dir(stack)
```

Out[141]:

['_class_

delattr

dict

dir

doc

eq

format_ ge'

getattribute',

_gt_ hash

init

init_subclass_' ,

le

lt

module

ne

new

reduce

reduce_ex

repr

setattr

sizeof

str',

_subclasshook__'.

weakref_',

'items',

'pop', 'push']

رابط عمومی کلاس *Stack* شما فقط شامل متدهای مربوط به *Stack* و () *push*.و()*pop*. است همانطور که در خروجی تابع ()*dir* مشاهده میکنید این باعث میشود که کاربران کلاس شما از استفاده از متدهای خاص لیست که با ساختار داده کلاسیک *stack* سازگار نیستند جلوگیری شود.

در صورت استفاده از *inheritance* کلاس فرزند شما *Stack* تمام قابلیت هاي پدر خود لیست را به ارث خواهد برد:

In [142]:

```
1  class Stack(list):
2          def push(self, item):
3                  self.append(item)
4          def pop(self):
5                  return super().pop()
6          def repr_(self) -> str:
7                  return f" {type(self)._name_}({ super()._repr_()})"
8
9  stack Stack()
10 dir(stack)
```

Out[142]:

['_add_', _class_',

_class_getiten_",

contains

delattr

delitem

dict',

dir

doc_ eq

format', _ge_',

getattribute_',

getitem,

_gt_',

hash

iadd_

imul

init

_init_subclass_',

iter',

_le_',

len _',

lt

_module___', mul

ne

new

_reduce___',

_reduce_ex__',

repr',

reversed',

rmul

setattr

setitem

sizeof

str',

subclasshook,

_weakref _',

'append',

'clear',

'copy',

'count',

'extend',

'index',

'insert',

'pop',

'push',

'remove', 'reverse',

'sort']

در نهایت در پایتون روش سریعتر برای *Delegate* کردن وجود دارد. این متد در پایتون به نام *Betattr*() وجود دارد. هر زمان که به یک ویژگی یا متد نمونه دسترسی پیدا کنید پایتون به صورت خودکار این متد را فراخوانی می کند. شما می توانید از این متد برای هدایت درخواست به یک شی دیگر که ممکن است متد یا ویژگی مناسب را فراهم کند، استفاده کنید.

برای توضیح این تکنیک به مثال *mixin* برگردید که از یک کلاس *mixin* برای فراهم کردن قابلیت های سریال سازی به

In [ ]:

```
import json
import pickle

class Person:
        def _init_(self, name, age):
                self.name = name
                self.age = age

class Serializer:
        def init(self, instance):
                self.instance = instance

        def to_json(self) :
                return json.dumps (self.instance._dict_)

       def to_pickle(self):
                return pickle.dumps (self.instance._dict__)

class Employee(Person):
      def init(self, name, age, salary):
              super().__init_(name, age)
              self.salary = salary

      def getattr_(self, attr):
               return getattr(Serializer (self), attr)
```

در این پیاده سازی جدید کلاس سریال ساز نمونه ای که داده ها را فراهم میکند را به عنوان یک آرگومان دریافت می کند. *Employee* یک متد *Betattr*() را تعریف میکند که از تابع *getattr*() داخلی برای دسترسی به متدهای کلاس *Serializer* استفاده میکند.

به عنوان مثال اگر شما *to_json*() را روی یک نمونه از *Employee* فراخوانی کنید آن تماس به صورت خودکار به فراخوانی *to_son*() در نمونه *Serializer* هدایت خواهد شد. این را امتحان کنید این یک ویژگی خوب پایتون است. شما در یک مثال سریع از *delegation* در پایتون سعی کرده اید تا چگونگی هدایت بخشی از مسئولیت های یک کلاس به یک کلاس دیگر را برای دسترسی مجدد به کد و جداسازی مسئولیت ها یاد بگیرید باز هم باید توجه داشته باشید که این تکنیک به طور غیر مستقیم تمام ویژگی های و متدهای منتقل شده را نشان میدهد .بنابراین با دقت از آن استفاده کنید.

Dependency Injection

تزریق وابستگی الگوی طراحی است که میتوانید از آن برای دستیابی به جداسازی کم از یک کلاس و اجزای آن استفاده کنید. با استفاده از این تکنیک میتوانید وابستگی های یک شیء را از خارج فراهم کنید به جای ارث بری یا پیاده سازی آن ها در خود شیء با این روش میتوانید کلاس های انعطاف پذیری را ایجاد کنید که قادر به تغییر رفتار خود به صورت پویا هستند بسته به قابلیتهای وارد شده

در مثال ربات شما میتوانید از تزریق وابستگی برای جداسازی کلاس های *Arm* و *Body* از *IndustrialRobot* استفاده کنید، که باعث میشود که شما انعطاف پذیرتر و چند منظوره تر شود.

مثال به روز شده در زیر آمده است:

In [ ]:

```
1  class Industrial Robot:
2      def_init_(self, body, arm):
3          self.body = body
4          self.arm = arm
5          # rest of the class methods ...
```

در این نسخه جدید از *IndustrialRobot* فقط دو تغییر کوچک در *init_()*.انجام دادید اکنون این روش *body* و *arm* را به عنوان آرگومان ها می پذیرد و مقادیر آنها را به ویژگی های نمونه مربوطه *body* و *arm*، اختصاص می دهد. این به شما اجازه میدهد تا شی های *body* و *arm* مناسب را در کلاس تزریق کنید تا بتواند کار خود را انجام دهد. اینجاست که چگونه میتوانید از *IndustrialRobot* با این پیاده سازی جدید استفاده کنید . و

In [ ]:

```
1  robot = IndustrialRobot(Body(), Arm())
```

در کل عملکرد کلاس همانند نسخه اول شما باقی میماند تنها تفاوت این است که اکنون باید شیهای *body* و *arm* را به سازنده کلاس منتقل کنید این مرحله یک روش رایج برای پیاده سازی تزریق وابستگی است.

Interfaces و Abstract Base Classes (ABCs)

گاهی اوقات میخواهید سلسله مراتب کلاس را ایجاد کنید که تمام کلاس ها یک رابط یا *API* پیش تعریف شده را پیاده سازی کنند به عبارت دیگر میخواهید مجموعه خاصی از روش ها و ویژگی های عمومی را تعریف کنید که تمام کلاس های سلسله مراتب باید آنها را پیاده سازی کنند در ،پایتون میتوانید این کار را با استفاده از چیزی به نام یک کلاس پایه انتزاعی *ABC()*انجام دهید.

ماژول *abc* در کتابخانه استاندارد چندین *ABC* و ابزار مرتبط دیگر را صادر میکند که میتوانید از آنها برای تعریف کلاس های پایه سفارشی استفاده کنید که تمام زیر کلاس های خود را برای پیاده سازی رابط های خاص الزام می دارند.

شما نمی توانید به طور مستقیم *ABCS* را نمونه سازی کنید شما باید آنها از طریق زیرکلاس تعریف و استفاده نمایید. به نوعی *ABCS* به عنوان الگوهای برای دیگر کلاس ها برای به ارث بردن عمل میکنند.

برای نشان دادن چگونگی استفاده از *ABCS* در پایتون فرض کنید میخواهید یک سلسله مراتب کلاس برای نمایش شکل های مختلف مانند دایره، مربع و غیره ایجاد کنید شما تصمیم میگیرید که تمام کلاس ها باید *get_area*() و *get_perimeter*() را دارا باشند. در این شرایط شما میتوانید با چنین کلاس پایه اولیه زیر شروع کنید:

In [143]:

```
1  from abc import ABC, abstractmethod
2
3  class Shape (ABC):
4      @abstractmethod
5      def Bet_area (self) :
6          pass
7
8      @abstractmethod
9      def Bet_perimeter(self):
10         pass
```

کلاس *Shape* از *abc.ABC* به ارث می برد به این معنی که یک کلاس پایه انتزاعی است. سپس شما با استفاده از تزئین کننده *@abstractmethod* روش های *get area()* و *get_perimeter()* را تعریف میکنید. با استفاده از تزئین کننده *abstract method* شما اعلام میکنید که این دو روش رابط عمومی هستند که تمام زیرکلاس های *Shape* باید پیاده سازی کنند.

حال میتوانید کلاس *Circle* را ایجاد کنید در اینجا رویکرد اول به این کلاس وجود دارد:

In [144]:

```
from abc import ABC, abstractmethod
from math import pi

 # ...

class Circle (Shape):
        def _init_(self, radius):
                self.radius = radius

        def get_area(self):
                return pi* self.radius ** 2
```

در این قطعه کد شما با به ارث بردن از *Shape* کلاس *Circle* را تعریف میکنید در این نقطه فقط روش *get area* () را اضافه کرده اید. حالا به جلو بروید و کد زیر را اجرا کنید:

In [145]:

```
circle = Circle(100)
```

TypeError

Traceback (most recent call last)

----> 1 circle = Circle(100)

TypeError: Can't instantiate abstract class Circle with abstract method ge
t_perimeter

In [146]:

```
from abc import ABC, abstractmethod
from math import pi

# ...

class Circle (Shape):
        def _init_(self, radius):
                self.radius = radius

    def get_area(self):
                return pi * self.radius ** 2

    def get_perimeter(self):
                return 2 * pi * self.radius
```

این بار کلاس *Circle* شما تمام متدهای مورد نیاز را پیاده سازی میکند این متد ها برای تمام کلاس های سلسله مراتب شکل شما مشترک هستند پس از تعریف پیاده سازی های سفارشی مناسب برای تمام متدهای انتزاعی، می توانید به ایجاد نمونه *Circle* بپردازید همانطور که در مثال زیر استفاده شده است:

In [147]:

```
circle = Circle(100)
```

In [148]:

```
circle.radius
```

Out[148]:

100

In [149]:

```
circle.get_area()
```

Out[149]:

31415.926535897932

In [150]:

```
1 circle.get_perimeter()
```

Out[150]:

628.3185307179587

پس از پیاده سازی متد های سفارشی برای جایگزین کردن پیاده سازی های انتزاعی *get_area* () *get_perimeter*() ، میتوانید *Circle* را در کد خود نمونه سازی و استفاده کنید.

حال برای تمرین میتوانید یک کلاس مربع با استفاده رابط انتزاعی *shape* تعریف کنید.

In [151]:

```
1 class Square (Shape):
2         def_init_(self, side):
3                 self.side=side
4
5         def get_area(self):
6                 return self.side ** 2
7
8         def get_perimeter (self):
9                 return 4 * self.side
```

خدا قوت عالی بود تمام مباحث مربوط به شی گرایی در پایتون از صفر تا صد رو یاد گرفتیم

ماژول ها و پکیج ها

برنامه نویسی ماژولار به فرآیند تقسیم یک وظیفه برنامه نویسی بزرگ و سخت به زیر وظایف یا ماژول های کوچکتر، قابل مدیریت تر و جداگانه اشاره دارد سپس ماژول های جداگانه میتوانند مانند بلوک های ساختمانی با هم ترکیب شوند تا یک برنامه بزرگتر ایجاد شود.

چندین مزیت برای ماژولار کردن کد در یک برنامه بزرگ وجود دارد:

سادگی: به جای تمرکز بر روی کل مسئله یک ماژول به طور معمول بر روی یک قسمت نسبتاً کوچکتر از مسئله تمرکز دارد اگر شما در حال کار روی یک ماژول واحد هستید دامنه مسئله کوچکتر است. این باعث میشود توسعه آسانتر و کم خطای تر باشد.

- قابلیت :نگهداری ماژول ها به طور معمول طوری طراحی شدهاند که محدوده منطقی بین دامنه های مختلف مسئله را اعمال کنند اگر ماژول ها به گونه ای نوشته شده باشند که حداقل وابستگی را داشته باشند، احتمال کمتری وجود دارد که تغییرات در یک ماژول تأثیرات آن را بر سایر قسمت های برنامه داشته باشد. (حتی شاید بتوانید تغییرات را در یک ماژول اعمال کنید بدون این که هیچ دانش خارج از آن ماژول را داشته باشید. این باعث میشود توانایی همکاری تعداد زیادی برنامه نویس در یک برنامه بزرگ را فراهم کند.
- قابل استفاده مجدد: قابل استفاده بودن عملکردهای تعریف شده در یک ماژول به راحتی با استفاده از گسترده مناسب توسط سایر قسمتهای برنامه استفاده شده است. این باعث حذف نیاز به تکثیر کدهای تکثیر شده خواهد شد.
- *Scoping*: معمولا ماژول ها فضای نام جدید را تعریف میکنند که به جلوگیری از تصادف نام هویت ها در مناطق مختلف برنامه کمک میکند(یکی از اصول در *Zen of Python* فضای نامها یک ایده بزرگ و عالی هستند - بیایید بیشتر از آنها استفاده کنیم!)

توابع ماژول ها و پکیج ها همه ساختارهایی در پایتون هستند که به مدولارسازی کد کمک میکنند.

ماژول های پایتون مروری در واقع سه روش مختلف برای تعریف یک ماژول در پایتون وجود دارد

۱. یک ماژول میتواند به زبان پایتون نوشته شود.
۲. یک ماژول میتواند به زبان *C* نوشته شده و در زمان اجرا به صورت پویا بارگذاری شود مانند ماژول *re*(عبارات با قاعده).
۳. یک ماژول داخلی در خود ترجمه گر وجود دارد مانند ماژول *itertools*

محتویات یک ماژول در همه سه حالت به همان شکل با استفاده از دستور *import* قابل دسترسی هستند.

در اینجا تمرکز بیشتر بر روی ماژول های نوشته شده به زبان پایتون خواهد بود چیز جالب دربارهٔ ماژول های نوشته شده به زبان پایتون این است که بسیار ساده هستند. تنها کافی است یک فایل ایجاد کنید که حاوی کد پایتون قابل قبول باشد سپس به فایل یک نام با پسوند *py* بدهید همین است نیاز به دستورات خاص یا جادو ندارید.

برای مثال فرض کنید یک فایل با نام *mod.py* ایجاد کرده اید که شامل مطالب زیر است:

In [ ]:

```
## wrtie this in separate file named mod.py
s = "If Comrade Napoleon says it, it must be right."
a = [100, 200, 300]

def foo(arg):
    print(f'arg {arg}')

class Foo:
    pass
```

چندین شی در *mod. py* تعریف شده اند:

- *s* (یک رشته)
- *a* (یک لیست)
- *Foo()*یک (تابع)
- *Foo*(یک کلاس)

با فرض اینکه *mod.py* در محل مناسبی قرار دارد که به زودی بیشتر درباره آن یاد خواهید ،گرفت این شی ها با وارد کردن ماژول به عنوان زیرمجموعه به صورت زیر قابل دسترسی هستند:

In [1]:

```
import mod
```

In [2]:

```
dir()
```

In [3]:

```
print(mod)
```

<module 'mod' from 'C:\\Users\\babak\\Python For Everyone\\07-Modules and Packages\\nod.py">

In [4]:

```
print(mod.a)
```

[100, 200, 300]

In [5]:

```
1 print(mod.s)
```

If Comrade Napoleon says it, it must be right.

In [6]:

```
1 mod.foo(['quux', 'corge', 'grault'])
```

Arg = ['quux', 'corge', 'grault']

In [8]:

```
1 x = mod.Foo()
2 x
```

Out [8]:

<mod. Foo at 0x159d0648e50>

دستور *import*

محتویات ماژول با دستور *import* به فراخوانی کننده *(caller)*ارائه میشود دستور *import* در چندین فرم مختلف وجود دارد که در زیر نشان داده شده اند.

import ساده ترین فرم همان فرمی است که در بالا نشان داده شده است:

In [ ]:

```
1 import <module_name>
```

توجه کنید که این کار محتویات ماژول را به صورت مستقیم برای *caller* قابل دسترسی نمیکند. هر ماژول دارای جدول نمادهای خصوصی خود است که به عنوان جدول نمادهای سراسری برای تمام شی های تعریف شده در ماژول عمل میکند بنابراین یک ماژول فضای نام جداگانه ای ایجاد میکند همانطور که در قبل گفته شد.

با دستور *import* ، فقط در جدول نماد *caller* قرار میگیرد. شی هایی که در ماژول تعریف شده اند در جدول نماد خصوصی ماژول باقی می مانند.

شی های موجود در ماژول فقط زمانی با استفاده از نام و با استفاده از نشان دهندهٔ نقطه به صورت قابل دسترسی هستند. بعد از دستور *import* ،زیر *mod* در جدول نماد محلی قرار میگیرد بنابراین *mod* در زمینه محلی *caller* معنا دارد:

In [12]:

```
1 import mod
2
3 mod
```

Out[12]:

<module 'mod from 'C:\\Users\\babak\\Python For Everyone\\07-Modules and Packages\\mod. py'

In [13]:

```
1 s
```

NameError

Traceback (most recent call last)

Cell In[13], line 1

----> 1 s

NameError: name 's' is not defined.

In [14]:

```
1 foo('quux")
```

NameError

Traceback (most recent call last)

Cell In[14], line 1

----> 1 foo('quux")

NameError: name 's' is not defined.

اما شما میتوانید با استفاده از نام ماژول و علامت به فضای خصوصی ماژول دسترسی پیدا نمایید

In [15]:

```
1 mod.s
```

Out[15]:

'If Comrade Napoleon says it, it must be right.'

In [16]:

```
1 mod. foo('Quuux')
```

arg= Quuux

همچنین در صورت نیاز میتوانین چندین ماژول رو بصورت همزمان وارد نمایید:

In [ ]:

```
1 from <module_name> import <name(s) >
```

در صورتیکه به بخشی از یک ماژول نیاز دارید میتوانید به نحو دیگری عملیات ***import*** را انجام دهید:

In [17]:

```
1 from mod imports, foo
```

In [18]:

```
1 s
```

Out [18]:

'If Comrade Napoleon says it, it must be right.'

In [19]:

```
1 foo('Quux')
```

arg=Quux

حتی در صورت نیاز میتوانید تمام فضای خصوصی ماژول را بصورت کامل و عمومی در کد خود وارد کنید اما در برنامه های بزرگ به هیچ عنوان توصیه نمیشود

In [ ]:

```
1 from <module_name> import *
```

In [1]:

```
1 from mod import *
```

In [2]:

```
1 a
```

Out [2]:

[100, 200, 300]

In [3]:

```
1 s
```

Out[3]:

'If Comrade Napoleon says it, it must be right.'

In [4]:

```
1 x = Foo()
2
3 x
```

Out [4]:

<mod. Foo at 0x1b16963d660>

در صورت نیاز میتوانید نام جایگزین نیز برای ماژول یا بخشی از ماژول وارد شده تعریف نمایید:

In [ ]:

```
1 from <module_name> import <name> as <alt_name> [, <name> as <alt_name> ...]
```

In [1]:

```
1 from mod import s as string, a as alist
```

In [2]:

```
1 s
```

NameError

Traceback (most recent call last)

Cell In[2], line 1

----> 1 s

NameError: name 's' is not defined

In [3]:

```
1 string
```

Out [3]:

'If Comrade Napoleon says it, it must be right.'

In [4]:

```
1 a
```

NameError

Traceback (most recent call last)

Cell In[4], line 1

----> 1 a

NameError: name 'a' is not defined

In [5]:

```
1 alist
```

out[5] :

[100, 200, 300]

In [ ]:

```
1 import <module_name> as <alt_name>
```

In [1]:

```
1 import mod as my_module
```

In [2]:

```
1 my_module.a
```

Out[2]:

[100, 200, 300]

اجرای یک ماژول بعنوان یک اسکریپت مستقل

اجرای یک ماژول به عنوان یک اسکریپت هر فایل *py* که شامل یک ماژول است در واقع همچنین یک اسکریبت پایتون است و هیچ دلیلی برای این نیست که مانند یک اسکریپت اجرا نشود. دوباره *mod.py* را همانطور که در بالا تعریف شده بود، در زیر نشان میدهیم:

mod.py

In [ ]:

```
1  s = "If Comrade Napoleon says it, it must be right."
2  a = [100, 200, 300]
3
4  def foo (arg):
5      print (f'arg= {arg}')
6
7  class Foo:
8      pass
```

In [ ]:

```
1  C:\Users\john\Documents>python mod.py
```

هیچ خطایی وجود ندارد بنابراین به نظر میرسد کار کرده است. البته خیلی جالب نیست همانطور که نوشته شده است. فقط شی ها را تعریف میکند هیچ کاری با آنها انجام نمیدهد و هیچ خروجی ای تولید نمیکند. بیایید ماژول پایتون بالا را به گونه ای تغییر دهیم که هنگام اجرا به عنوان یک اسکریپت خروجی تولید کند:

In [ ]:

```
1  s = "If Comrade Napoleon says it, it must be right."
2  a = [100, 200, 300]
3
4  def foo(arg):
5      print(f'arg = (arg')
6
7  class Foo:
8      pass
9
10 print(s)
11 print(a)
12 foo('quux')
13 x = Foo()
14 print(x)
```

حالا در *cmd* خود میتوانید فایل مربوطه را اجرا نمایید:

In [ ]:

```
1 C:\Users\john\Documents>python mod.py
```

با این وجود تغییر اخیر باعث خواهد شد تا در زمان *import* هم دستورات داخل ماژول اجرا شوند.

In [1]:

```
1 import mod
```

If Comrade Napoleon says it, it must be right.

[100, 200, 300]

arg = quux

<mod. Foo object at 0x00000197302ACA30>

این احتمالاً آنچه میخواهید نیست معمول نیست که یک ماژول هنگام وارد شدن خروجی تولید کند.

آیا خوب نیست که بتوانید بین زمان بارگیری فایل به عنوان یک ماژول و زمان اجرای آن به عنوان یک اسکریپت مستقل تفاوت قائل شوید؟

بپرسید و دریافت کنید.

زمانی که یک فایل *py*. به عنوان یک ماژول وارد میشود پایتون متغیر خاص داندر *name*_ را به نام ماژول تنظیم میکند. با این حال اگر یک فایل به عنوان یک اسکریبت مستقل اجرا شود *nane*_ به صورت خلاقانه به رشته *main*_ تنظیم میشود با استفاده از این واقعیت میتوانید در زمان اجرا تشخیص دهید کدام حالت صادق است و رفتار را به منظور تغییرات مناسب تغییر دهید:

In [1]:

```
# mod.py

s = "If Comrade Napoleon says it, it must be right."
a = [100, 200, 300]

def foo(arg):
        print (f'arg = {arg} ' )

class Foo:
       pass

 if (_name_== '_main_'):
       print('Executing as standalone script')
      print(s)
     print(a)
     foo("quux')
     x = Foo( )
    print(x)
```

حالا در *cmd* خود میتوانید فایل مربوطه را اجرا نمایید:

In [ ]:

```
C:\Users\john\Documents>python mod.py
```

اما با این تغییر دیگر دستورات داخل if در هنگام import اجرا نخواهند شد

In [1]:

```
import mod
```

In [2]:

```
mod. foo('grault')
```

arg = grault

برای بارگذاری مجدد یک پکیج کافی هست آن را دوباره *import* نمایید.

پکیج های پایتون

فرض کنید یک برنامه بسیار بزرگ توسعه داده اید که شامل بسیاری از ماژول ها است با افزایش تعداد ماژول ها، اگر آنها در یک مکان قرار داده شوند سخت است که همه را پیگیری کرد این به خصوص در صورتی است که نام ها یا عملکردهای مشابه داشته باشند. شما ممکن است به دنبال یک روش برای گروهبندی و سازماندهی آنها باشید.

پکیج ها به شما اجازه میدهند تا با استفاده از نمایش نقطه ای فضای نام ماژول را به صورت سلسله مراتبی سازماندهی کنید. به همان شکلی که ماژول ها به جلوگیری از تداخل نام متغیر جهانی کمک میکنند بسته ها به جلوگیری از تداخل نام ماژول کمک میکنند

ایجاد یک پکیج بسیار ساده است زیرا از ساختار فایل سلسله مراتبی پایین سیستم عامل استفاده میکند به عنوان یک نمونه ترتیب زیر را در نظر بگیرید:



در اینجا یک پوشه به نام *pkg* وجود دارد که دو ماژول *mod1.py* و *mod2.py* را شامل میشود.
محتوای ماژولها به صورت زیر است:

In [ ]:

```
1  #modi.py
2
3  def foo():
4      print('[mod1] foo()')
5
6  class Foo:
7      pass
```

In [ ]:

```
1  #mod2.py
2
3  def bar():
4      print('[mod2] bar()')
5
6  class Bar:
7      pass
```

با توجه به این ساختار اگر پوشه ***pkg*** در مکانی قرار داده شود که بتوان آن را پیدا کرد(در یکی از پوشه های موجود در ***sys.path)*** میتوانید با نمایش نقطه ای به دو ماژول با نام ***pkg.mod1*** و ***pkg.mod2*** ارجاع دهید و آنها را با دستوری که قبلاً با آن آشنا شده اید وارد کنید:

In [ ]:

```
1  import <module_name>[, <module_name> ...]
```

In [3]:

```
1  import pkg.mod1, pkg.mod2
```

In [4]:

```
1  pkg.mod1.foo()
```

[mod1] foo()

In [5]:

```
1  x= pkg.nod2. Bar()
```

In [6]:

```
1  x
```

Out[6] :

<pkg.mod2.Bar at 0x16209f2ff40>

In [7]:

```
1 from pkg.modi import foo
```

In [8]:

```
1 foo()
```

```
[mod1] foo()
```

In [9]:

```
1 from pkg.mod2 import Bar as Qux
```

In [10]:

```
1 x = Qux()
```

In [11]:

```
1 x
```

Out[11] :

<pkg.mod2.Bar at 0x16209f2ebf0>

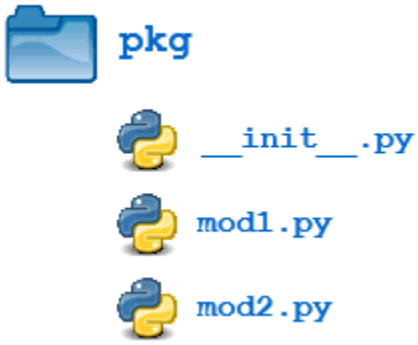
Package Initialization

اگر یک فایل با نام *init_py* در یک پوشه پکیج وجود داشته باشد هنگام وارد کردن بسته یا یک ماژول در پکیج فراخوانی میشود. این میتواند برای اجرای کد اولیه پکیج مانند اولویت دادن به داده های سطح پکیج استفاده شود. به عنوان مثال، فایل *init_py* زیر را در نظر بگیرید:

In [ ]:

```
1 # __init_.py
2
3 print (f'Invoking_init_.py for _name_}') ['quux', 'corge', 'grault']
4 A = ['quux', 'corge', 'grault']
```

بیایید این فایل را به پوشه *pkg* از مثال بالا اضافه کنیم:



حالا هنگام وارد کردن ،پکیج ،لیست سراسری *A* مقداردهی اولیه میشود:

In [1]:

```
1 import pkg
```

Invoking_init__.py for pkg

In [2]:

```
1 pkg.A
```

out[2]:

['quux', 'corge', 'grault']

یک ماژول در پکیج میتواند با وارد کردن آن به ترتیب به متغیر سراسری دسترسی پیدا کند:

همچنین میتواند برای ایجاد وارد کردن خودکار ماژول ها از یک پکیج استفاده شود. به عنوان مثال، قبلاً دیدید که دستور *import pkg* فقط نام *pkg* را در جدول نمادین محلی *caller* قرار میدهد و هیچ ماژولی را وارد نمی کند. اما اگر *init_.py* در پوشه *pkg* شامل موارد زیر باشد:

In [2]:

```
1 ## init__.py
2
3 print(f'Invoking_init__.py for {name}")
4 import pkg.mod1, pkg.mod2
```

در این صورت هنگام اجرای *import.pkg* ماژولهای *mod1* و *mod2* به طور خودکار وارد میشوند:

In [1]:

```
1 import pkg
```

Invoking_init__.py for pkg

In [2]:

```
1 pkg.mod1. foo()
```

[mod1] foo()

In [3]:

```
1 pkg.mod2,bar( )
```

[mod2] bar()

*Importing * From a Package*

برای اهداف بحث زیر پکیج تعریف شده قبلی به منظور شامل بودن برخی ماژول های اضافی گسترش می یابد:



در حال حاضر چهار ماژول در پوشه *pkg* تعریف شدهاند. محتوای آنها به شکل زیر است:

In [ ]:

```
1 ## mod1.py
2 def foo():
3     print('[mod1] foo()')
4
5 class Foo:
6     pass
```

In [ ]:

```
## mod2.py

def bar():
    print('[mod2] bar()')

class Bar:
    pass
```

In [ ]:

```
## mod3.py

def car():
    print('[mod2] bar()")

class Car:
    pass
```

In [ ]:

```
## mod4.py

def door():
    print('[mod2] bar()")

class Door:
    pass
```

شما قبلاً دیده اید که هنگام استفاده از *import** برای یک ماژول تمام شی های ماژول وارد جدول نمادین محلی میشوند. به جز آنهایی که با یک زیرخط شروع میشوند، همانطور که همیشه:

In [1]:

```
dir()
```

In [2]:

```
1 from pkg.mod3 import *
```

In [ ]:

```
1 dir()
```

In [4]:

```
1 from pkg import"
2 pkg.mod1.foo()
```

NameError

Traceback (most recent call last)

Cell In[4], line 2

1 from pkg import

---->2 pkg.mod1.foo()

NameError: name 'pkg' is not defined

چگونه در *import* * ماژول های قابل ورود را محدود نمایید؟

برای اینکار میتوانیم از یک لیست با اسم خاص در داخل فایل *_init_* استفاده نماییم نام این لیست *_a11_* می باشد و مشخص می نماید که چه ماژول های اجازه دارند توسط در دستور *import* * به *caller* مستقیما وارد شوند.

In [ ]:

```
1 # phg/_init_.py
2
3 _all_ = [
4       'mod1'
5       'mod2'
6      'mod3'
7       'mod4'
8 ]
```

In [ ]:

```
1 dir()
```

In [2]:

```
1 from pkg import*
```

In [ ]:

```
1 dir()
```

علاوه بر این، شما میتوانید داخل هر یک از ماژول های موجود نیز با استفاده از لیست ***all*** توابع یا کلاس ها یا متغییر های قابل ***import*** را مشخص نمایید

In [ ]:

```
1
2
3
4
5
6
7
8
9
```

In [ ]:

```
1 dir()
```

In [2]:

```
1 from pkg.mod1 import*
```

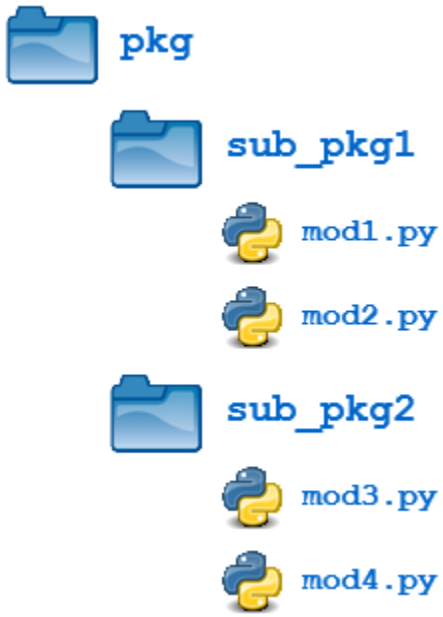
In [ ]:

```
1 dir()
```

Subpackages

بسته ها میتوانند شامل زیرپکیج های تو در تو با عمق دلخواه باشند به عنوان مثال بیایید یک تغییر دیگر به پوشه بسته مثال اعمال کنیم:

چهار ماژول (*mod1.py, mod2.py. mod3.py* و *mod4.py*)به شکل قبلی تعریف شده اند اما حالا به جای این که در پوشه *pkg* قرار داشته باشند، آنها به دو پوشه زیرپکیج *sub_pkg1* و *sub_pkg2* تقسیم شده اند.



وارد کردن همچنان همانطور کار میکند که قبلاً نشان داده شده است. نحو شبیه است اما نحو نقطه اضافی برای جداسازی نام بسته از نام زیر پکیج استفاده میشود:

In [1]:

```
1 import pkg1.sub_pkg1.mod1
```

In [2]:

```
1 pkg1.sub_pkg1.mod1.foo()
```

[mod1] foo()

In [3]:

```
1 from pkg1.sub_pkg1 import mod2
```

In [4]:

```
1 mod2.bar()
```

[mod2] bar()

In [6]:

```
1 from pkg1.sub_pkg2.mod3 import car
```

In [7]:

```
1 car()
```

[mod2] bar()

In [1]:

```
1 from pkg1.sub_pkg2. mod4 import door as grault
```

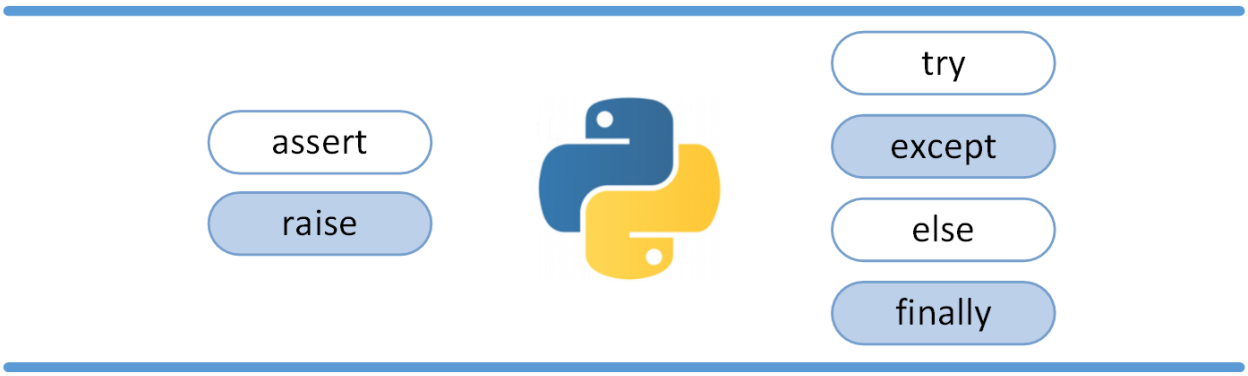
In [2]:

```
1 grault()
```

[mod2] door ()

Python Exceptions

یک برنامه پایتون به محض برخورد با خطا متوقف می شود. در پایتون خطا می تواند یک خطای نحوی یا یک استثناء باشد. در این مقاله شما خواهید دید که استثناء چیست و چگونه از خطای نحوی متفاوت است. پس از آن در مورد برانگیختن استثناءها و ادعاها صحبت خواهید کرد. سپس با نمایش بلوک *try* و *except* به پایان خواهید رسید.



Exception* ها در مقابل خطای *syntax

خطاهای *syntax* هنگامی رخ می دهند که پارسر یک بیانیه نادرست را تشخیص میدهد. به عنوان مثال زیر را مشاهده کنید:

In [23]:

```
1 print(0/0))
```

Cell In[23], line 1

print(0/0))

SyntaxError: unmatched')'

نشانگر نشان میدهد کجا پارسر به خطای نحوی برخورد کرده است. در این مثال یک پرانتز بیش از حد وجود دارد. آن را حذف کنید و کد خود را دوباره اجرا کنید:

In [24]:

```
1 print(0/0))
```

ZeroDivisionError

Traceback (most recent call last)

Cell In[24], line 1

---->1 print(0 / 0)

ZeroDivisionError: division by zero

این بار شما با خطای استثناء مواجه شدید. این نوع خطا هرگاه که پایتون نحوی صحیح باعث بروز خطا شود رخ می دهد. آخرین خط پیام نشان میدهد چه نوع خطای استثناء با آن روبرو شده اید.

بجای نمایش پیام خطای استثناء پایتون جزئیات نوع خطای استثناء را که با آن روبرو شده است توضیح می دهد. در این مورد *ZeroDivisionError* بود. پایتون با استثناء های داخلی مختلف همراه است و همچنین امکان ایجاد استثناء های تعریف شده توسط کاربر وجود دارد.

ایجاد یک*Exception*

می توانیم از *raise* برای پرتاب یک استثناء در صورت بروز یک شرط استفاده کنیم. این عبارت میتواند با یک استثناء سفارشی تکمیل شود.

Use raise to force an exception:



اگر می خواهید هنگام بروز یک شرط خاص با استفاده از *raise* خطا را پرتاب کنید میتوانید به این صورت عمل کنید:

In [25]:

```
1  x = 10
2
3  if x> 5:
4      raise Exception(f'X should not exceed 5. The value of x was: (x)')
```

Exception

Traceback (most recent call last)

Cell In[25], line 4

1x = 10

3 if x> 5:

4----> raise Exception(f'X should not exceed 5. The value of x was: (x)')

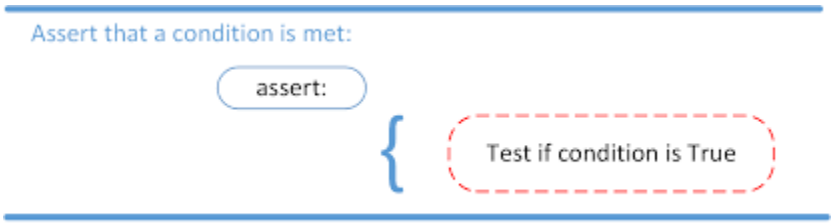
Exception: X should not exceed 5. The value of x was: 101

وقتی این کد را اجرا میکنید خروجی به شکل زیر خواهد بود:

برنامه به پایان می رسد و استثناء ما را به صفحه نمایش میکشاند و درباره آنچه اشتباه رفت راهنمایی هایی ارائه می دهد.

AssertionError Exception

به جای اینکه منتظر بمانید تا برنامه در نیمه راه خراب شود میتوانید با یک ادعا کار را در پایتون شروع کنید. ما ادعا می کنیم که یک شرط خاص برآورده شده است. اگر این شرط به درستی ثابت شود آنگاه عالی است برنامه می تواند ادامه پیدا کند. اگر شرط ثابت شود که نادرست است میتوانید برنامه را به یک استثناء *Assertion Error* پرتاب کنید.



به مثال زیر نگاه کنید جایی که ادعا شده است که بر روی یک سیستم لینوکس اجرا خواهد شد.:

In [26]:

```
1 import sys
2
3 assert ('Linux' in sys.platform), "This code runs on Linux only."
```

AssertionError

Traceback (most recent call last)

Cell In[26], line 3

```
    1 import sys
----> 3 assert ('linux' in sys.platform), "This code runs on Linux only."
AssertionError: This code runs on Linux only.
```

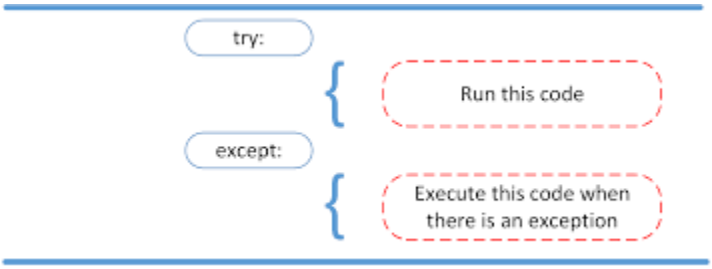
اگر این کد را روی یک سیستم لینوکس اجرا کنید ادعا عبور میکند. اگر شما این کد را روی یک سیستم ویندوز اجرا کنید.

نتیجه ادعا *False* خواهد بود و نتیجه به شرح زیر خواهد بود:

در این مثال پرتاب یک استثناء *AssertionError* آخرین چیزی است که برنامه انجام خواهد داد. برنامه به سرعت متوقف می شود و ادامه نمی دهد. اگر همین چیزی نباشد که می خواهید چه؟

مدیریت *Exception* ها از طریق بلوکهای *try & except*

بلوک *try* و *except* در پایتون برای گرفتن و کنترل استثناءها استفاده میشود. پایتون کد را که پس از عبارت *try* قرار دارد. به عنوان بخشی "عادی" از برنامه اجرا میکند کدی که پس از عبارت *except* قرار دارد پاسخ برنامه به هر استثناء در بخش *try* قبلی است.



همانطور که قبلاً دیدید هنگامی که که نحوی به خطا برخورد میکند پایتون یک خطای استثناء را پرتاب می کند. این خطای استثناء در صورت عدم برخورد با آن برنامه را خراب می کند. شرط *excent* نحوه پاسخ برنامه شما به به استثناء ها را

In [27]:

```
1 def linux interaction():
2         assert ('linux' in sys.platform), "This code runs on Linux only."
3         print("Doing Something')
```

تابع *linux interaction* فقط در سیستم عامل لینوکس قابل اجرا است. اگر شما این تابع را در یک سیستم عامل غیر از لینوکس فراخوانی کنید *assert* در این تابع یک استثناء *AssertionError* را پرتاب می کند.

شما می توانید با استفاده از کد زیر تابع را امتحان کنید:

In [28]:

```
1  try:
2      linux interaction()
3  except:
4      pass
```

روشی که شما در اینجا خطای خود را دست کاری کرده اید به صورت دادن یک *pass* است. اگر شما این کد را در یک ماشین ویندوز اجرا کنید خروجی خالی دریافت خواهید کرد.

شما هیچ چیز نگرفتید. نکته مثبت در اینجا این است که برنامه خراب نشده است. اما خوب بود مشخص شود هرگز هنگام اجرای که شما چه نوع استثناءی رخ داده است. به این منظور می توانید *pass* را به چیزی تغییر دهید که پیام اطلاعاتی تولید کند، مانند زیر:

In [30]:

```
1  try:
2      linux interaction()
3  except:
4      print("Linux function was not executed')
```

Linux function was not executed

هنگامی که یک استثناء در یک برنامه در حال اجرا این تابع رخ می دهد برنامه شما را در مورد اینکه فراخوانی تابع موفق نبود، مطلع می کند.

آنچه که شما نتوانستید ،ببینید نوع خطایی بود که به دلیل فراخوانی تابع پرتاب شده بود برای دیدن دقیق آنچه که اشتباه رفت شما باید خطایی را که تابع پرتاب کرده است. گرفتار کنید.

In [31]:

```
1  try:
2      Linux_interaction()
3  except AssertionError as error:
4      print(error)
5      print('Linux function was not executed')
```

This code runs on Linux only.

Linux function was not executed

پیام اول *AssertionError* است که شما را مطلع میکند تابع فقط در یک ماشین لینوکس قابل اجرا است.
پیام دوم به شما می گوید کدام تابع اجرا نشده است.

در مثال قبل شما یک تابع را که خودتان نوشته بودید فراخوانی کردید. هنگامی که تابع را اجرا کردید.
استثناء *AssertionError* را گرفتید و آن را به صفحه نمایش چاپ کردید.

اینجا مثال دیگری وجود دارد که در آن یک فایل را باز میکنید و از استثناء داخلی استفاده می کنید:

In [32]:

```
1  try:
2  with open(file.log') as file:
3      read_data = file.read()
4  except:
5      print('Could not open file.log')
```

Could not open file.log

این یک پیام اطلاعاتی است و برنامه ما همچنان ادامه خواهد داد. در مستندات پایتون

https://docs.python.org/3/library/exceptions.html.

https://docs. python.org/3/library/exceptions.html%D8%8C

می توانید ببینید که تعداد زیادی استثناء داخلی وجود دارد که میتوانید در اینجا استفاده کنید. یک استثناء که در آن صفحه توصیف شده است، به شرح زیر است:

Exception FileNotFoundError

وقتی درخواست فایل با پوشه ای که وجود ندارد ارسال میشود این استثناء برای شما پرتاب می شود. این استثناء با *errno ENOENT* مطابقت دارد.

برای گرفتن این نوع استثناء و چاپ آن روی صفحه می توانید از کد زیر استفاده کنید:

In [36]:

```
try:
      with open(file.log') as file:
           read_data = file.read()
except FileNotFoundError as fnf_error:
    print (fnf_error)
   print('Could not open file.log')

print(5** 5)
```

[Errno 2] No such file or directory: 'file.log'

Could not open file.log

3125

شما میتوانید بیش از یک فراخوانی تابع در شرط *try* خود داشته باشید و پیش بینی کنید که چندین استثناء را میگیرید. چیزی که در اینجا باید توجه داشت این است که کد در شرط *try* به محض بروز استثناء متوقف میشود.

توجه :پرتاب استثناء *Exception* تمام خطاها را مخفی میکند ... حتی آنهایی که کاملاً غیر منتظره هستند. به همین دلیل باید از بلوک های *except* بدون پارامتر در برنامه های پایتون خودداری کنید به جای آن شما می خواهید به کلاس های استثناء خاصی که میخواهید آنها را پرتاب و پردازش کنید ارجاع دهید شما میتوانید در این آموزش بیشتر درباره اینکه چرا این یک ایده خوب است، بیاموزید.

به کد زیر نگاه کنید. در اینجا شما ابتدا تابع *Olinux interaction* را فراخوانی میکنید و سپس سعی می کنید یک فایل بازکنید:

In [38]:

```
try:
   linux_interaction()
  with open('file.log') as file:
      read_data = file.read()
except FileNotFoundError as fnf_error:
   print(fnf_error)
except AssertionError as error:
  print(error)
   print('Linux linux_interaction() function was not executed)
```

[Errno 2] No such file or directory: 'file.log'

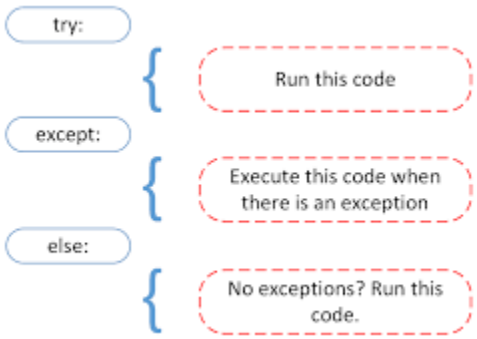
در داخل بلوک *try* با اولین *exception* اجرای دستورات متوقف خواهد شد.

نکات کلیدی که به خاطر داشتن آنها مفید است:

- یک بلوک *try* تا زمانی که اولین استثنا رخ دهد، اجرا میشود.
- در داخل بخش *except* یا *exception handler* شما مشخص میکنید که برنامه به استثنا چگونه پاسخ می دهد.
- شما میتوانید چندین استثنا را پیش بینی کرده و تفاوت رفتار برنامه در برابر آنها را تعیین کنید.
- بهتر است از *bare except clauses* خودداری کنید.

عبارت *else*

در پایتون با استفاده از دستور *else* میتوانید به برنامه دستوری را بدهید تا فقط در صورت عدم وجود استثنا، یک بلوک خاص از کد را اجرا کند.



به مثال زیر نگاه کنید:

In [39]:

```
1  def win32_interaction():
2      assert ('win32' in sys.platform), "This code runs on Windows only."
3      print('Doing Something')
```

In [40]:

```
1  try:
2      win32_interaction()
3  except AssertionError as error:
4      print(error)
5  else:
6      print('Executing the else Clause.").
```

Doing Something

Executing the else clause.

بلوک *else* اجرا شد زیرا برنامه با هیچ استثنایی مواجه نشد.

همچنین میتوانید کد را درون بلوک *else* اجرا کرده و در آنجا نیز استثناهای ممکن را گرفته و پردازش کنید

In [41]:

```
1  try:
2      win32_interaction()
3  except AssertionError as error:
4      print(error)
5  else:
6       try:
7           with open('file.log') as file:
8                   read_data = file.read()
9      except FileNotFoundError as fnf_error:
10          print(fnf_error)
```

Doing Something

[Errno 2] No such file or directory: 'file.log"

از خروجی میتوانید ببینید که تابع *Win32_interaction*() اجرا شده است زیرا هیچ استثنایی وجود نداشت تلاش برای باز کردن فایل *file. log* انجام شد. این فایل وجود نداشت و به جای باز کردن فایل استثنای *File NotFoundError* را گرفتید.

عبارت*finally*

تصور کنید همیشه باید عملیاتی را برای جمع بندی نهایی پس از اجرای کد خود پیاده کنید با استفاده از بخش *finally* در پایتون میتوانید این کار را انجام دهید.



به مثال زیر نگاهی بیندازید:

In [43]:

```
1  try:
2      win32_interaction()
3  except AssertionError as error:
4      print(error)
5  else:
6      try:
7          with open('file.log') as file:
8              read_data file.read()
9      except FileNotFoundError as fnf_error:
10         print(fnf_error)
11 finally:
12     print('Cleaning up, irrespective of any exceptions.')
```

Doing Something

[Errno 2] No such file or directory: 'file.log"

Cleaning up, irrespective of any exceptions.

مثال

ما می توانیم این را به همراه *except* استفاده کنیم بیایید یک مثال جدید را ببینیم که با ورود اشتباه کاربر مواجه میشود:

In [44]:

```
1  def askint():
2      try:
3          val = int(input('Please enter an integer: '))
4      except:
5          print('Looks like you did not enter an integer!')
6      finally:
7          print('Finally, I executed!!')
8
9  print(val)
```

In [45]:

```
1  askint()
```

Please enter an integer: 5

Finally, I executed!!

5

In [46]:

```
1  askint()
```

Please enter an integer: five\

Looks like you did not enter an integer!

Finally, I executed!!

UnboundLocalError

Traceback (most recent call last)

Cell In[46], line 1

----> 1 askint()

Cell In[44], line 9, in askint()

6 finally:

7 print("Finally, I executed!!)

----> 9 print(val)

UnboundLocalError: local variable 'val' referenced before assignment

توجه کنید که وقتی سعی کردیم *val* را چاپ کنیم (زیرا به درستی اختصاص داده نشده بود)خطایی دریافت کردیم. بیایید با پرسیدن کاربر و بررسی اینکه نوع ورودی یک عدد صحیح است این مشکل را برطرف کنیم:

In [47]:

```
1  def askint():
2      try:
3          val int(input('Please enter an integer: '))
4      except:
5          print("Looks like you did not enter an integer!')
6          val =int(input('Please agian-enter an integer: '))
7      finally
8          print("Finally, I executed!!')
9
10     print(val)
```

In [48]:

```
1  askint()
```

Please enter an integer: five

Looks like you did not enter an integer!

Please agian-enter an integer: five

Finally, I executed!!

ValueError

Traceback (most recent call last)

```
Cell In[47], line 3, in askint()
      2 try:
---> 3       val int(input('Please enter an integer: '))
      4 except:
ValueError: invalid literal for int() with base 18: 'five'

During handling of the above exception, another exception occurred:

ValueError
Traceback (most recent call last)
Cell In[48], line 1
---> 1 askint()
Cell In[47], line 6, in askint()
      4 except:
      5 print("Looks like you did not enter an integer!')
----> 6 val int(input('Please agian-enter an integer: '))
      7 finally:
      8 print('Finally, I executed!!")
ValueError: invalid literal for int() with base 10: 'five"
```

هممم... فقط یک بار چک کردیم چطور میتوانیم به طور مداوم بررسی کنیم؟ ما میتوانیم از یک حلقه while استفاده کنیم!

In [49]:

```
1  def askint():
2      while True:
3          try:
4              val = int(input('Please enter an integer: '))
5          except:
6              print('Looks like you did not enter an integer!')
7              continue
8          else:
9              print("Yep that's an integer")
10             break
11         finally:
12             print("Finally, I executed!!')
13
14 print(val)
```

In [50]:

```
1  askint()
```

Please enter an integer: five

Looks like you did not enter an integer!

Finally, I executed!!

Please enter an integer: 5

Yep that's an integer

Finally, I executed!!

پس چرا تابع ما "بالاخره من اجرا شدم" را پس از هر دور چاپ کرد اما هیچگاه val را چاپ نکرد؟ این به این دلیل است که با یک بلوک try except finally هر دستور continue یا break تا پس از اتمام بلوک try رزرو شده اند. این بدان معنی است که با وجود اینکه ورودی موفقیت آمیزی به ما آورد ۳ وارد بلوک else: شدیم و یک دستور break پرتاب شد بلوک try تا قبل از خروج از حلقه while ادامه یافت و از آنجا که ((printval خارج از بلوک try بود، دستور break از اجرای آن جلوگیری کرد.

بیایید یک تنظیم نهایی انجام دهیم:

بیایید یک تنظیم نهایی انجام دهیم:

In [51]:

```
def askint():
    while True:
            try:
                val = int(input('Please enter an integer: '))
    except:
            print('Looks like you did not enter an integer!')
            continue
     else:

            print("Yep that's an integer")

             break

finally:

print('Finally, I executed!!')

print(val)
```

In [52]:

```
askint()
```

Please enter an integer: ddsf

Looks like you did not enter an integer!

Finally, I executed!!

Please enter an integer: sdfsf

Looks like you did not enter an integer!

Finally, I executed!!

Please enter an integer: sasda

Looks like you did not enter an integer!

Finally, I executed!!

Please enter an integer: 77

Yep that's an integer

Finally, I executed!!

77

خطاها و استثناءها را finally و try except else عالی اکنون شما میدانید چگونه با استفاده از نمادهای در پایتون کنترل کنید

تمرینات خطاها و Exception ها

مسئله ۱

استثنائات پرتاب شده توسط کد زیر را با استفاده از بلوک های try و except کنترل کنید

In [2]:

```
1  try:
2      for i in ['a', 'b','']:
3          print(1**2)
4  except:
5      print('Can not calculate the power of string types")
```

Can not calculate the power of string types

مسئله ۲

برای برخورد با استثناء پرتاب شده توسط کد زیر از بلوک های try و except استفاده کنید. سپس از یک بلوک finally استفاده کنید تا عبارت All Done را چاپ کنید.

In [5]:

```
x=5
y = 10

try:
      z = x/y
except ZeroDivisionError:
      print("Can't divide by Zero!!!")
else:
      print('The calculation was successful') 10 Finally:

print('All Done!')
```

The calculation was successful

All Done!

مسئله ۳

یک تابع بنویسید که یک عدد صحیح را درخواست کند و مربع آن را چاپ کند. از یک حلقه while با یک بلوک ، try except else استفاده کنید تا به ورودی های نادرست پاسخ دهید.

In [6]:

```
def ask():
      while True:
         try:
            n = int(input('Please enter an integer: ))
         except:
            print('An error occurred! Please try again!')
            continue
         else:
            break
      print('Thank you, your number squared is:, n** 2)
```

In [7]:

```
1 ask()
```

```
Please enter an integer: five

An error occurred! Please try again!

Please enter an integer: null

An error occurred! Please try again!

Please enter an integer: 7

Thank you, your number squared is: 49
```

دست مریزاد!

Unit Testing

همان قدر که نوشتن که خوب مهم است نوشتن تست های خوب نیز مهم است بهتر است خودتان باگ ها را پیدا کنید تا اینکه کاربران نهایی آنها را به شما گزارش دهند!

در این بخش با فایل ها خارج از نوت بوک کار خواهیم کرد کد خود را در یک فایل Py. ذخیره خواهیم کرد و سپس اسکریپت تست خود را در یک فایل py. دیگر ذخیره خواهیم کرد به طور معمول ما این فایل ها را با استفاده از یک ویرایشگر متن مانند Brackets یا Atom یا درون یک محیط توسعه یکپارچه مانند Spyder با Pycharm کد مینویسیم. اما از آنجا که در اینجا هستیم بیایید از Jupyter استفاده کنیم!

با استفاده از یکی از ویژگی های IPython میتوانیم محتوای یک سلول را با استفاده از writefile در یک قابل ذخیره کنیم. چیزی که تاکنون ندیده ایم میتوانید دستورات ترمینال را از طریق یک سلول Jupyter با استفاده از ! اجرا کنید.

Testing tools

ده ها کتابخانه تست خوب وجود دارند بیشتر آنها پکیج های شخص ثالثی هستند که نیاز به نصب دارند، مانند:

- gylint (https://www.gylint.org/)
- Qyflakes (https://pypi.python.org/pypi/pyflakes/)
- pep8 (https://pypi.python.org/pypi/pep8)

این ابزارها ابزارهای ساده ای هستند که فقط به کد شما نگاه میکنند و اگر مشکلاتی مانند مسائل استایل یا مشکلات ساده ای مانند فراخوانی نام متغیرها قبل از اختصاص به آنها وجود داشته باشد به شما اطلاع میدهند.

روشی بسیار بهتر برای تست کردن که شما نوشتن تسته ایی است که داده های نمونه را به برنامه شما ارسال کنند و مقایسه کنند که چه چیزی برگردانده میشود و چه نتیجه مورد انتظار است.

دو ابزار از کتابخانه استاندارد در دسترس هستند:

- unittest (https://docs.python.org/3/library/unittest.html)
- doctest (https://docs.python.org/3/library/doctest.html)

بیایید ابتدا به pylint نگاهی بیندازیم سپس با استفاده از unittest برخی تست های سنگین تر انجام دهیم.

pylint

Pylint برای استایل و همچنین برخی از منطق بسیار ابتدایی برنامه تست انجام میدهد.

ابتدا ،اگر هنوز ندارید (و احتمالاً دارید ،زیرا قسمتی از توزیع آناکوندا است) باید pylint را نصب کنید. بعد از انجام این کار میتوانید سلول را کامنت کنید دیگر به آن نیاز نخواهید داشت.

In [3]:

```
1 pip install pylint
```

Defaulting to user installation because normal site-packages is not writea ble

Requirement already satisfied: pylint in c:\programdata\anaconda\lib\site - packages (2.16.2)

Requirement already satisfied: astroid <=2.16.8-dev8,>=2.14.2 in c:\program data\anaconda3\lib\site-packages (from py1int) (2.14.2)

Requirement already satisfied: tomli>-1.1.0 in c:\programdata\anaconda}\li b\site-packages (from pylint) (2.0.1)

Requirement already satisfied: platformdirs-2.2.0 in c:\programdata\anaco nda3\lib\site-packages (from pylint) (2.5.2)

Requirement already satisfied: dill-8.2 in c:\programdata\anaconda3\lib\s. ite-packages (from pylint) (0.3.6)

Requirement already satisfied: tomlkit>-0.10.1 in c:\programdata\anaconda3 \lib\site-packages (from pylint) (0.11.1)

Requirement already satisfied: isort<6,>-4.2.5 in c:\prograndata\anaconda3 \lib\site-packages (from pylint) (5.9.3)

Requirement already satisfied: mccabece.8,>=0.6 in c:\programdata\anaconda 3\lib\site-packag

es (from pylint) (0.7.0)

Requirement already satisfied: colorama>-0.4.5 in c:\programdata\anaconda3 \lib\site-packages (from pylint) (0.4.6).

Requirement already satisfied: lazy-object-proxy>-1.4.0 in c:\programdata \anaconda3\lib\site-packages (from astroid<=2.16.0-deve,>=2.14.2->pylint)

(1.6.0)

Requirement already satisfied: wrapt<2,>=1.11 in c:\programdata\anaconda3
\lib\site-packages (from astroid<=2.16.0-dev0, >=2.14.2->pylint) (1.14.1)
Requirement already satisfied: typing-extensions >=4.8.8 in c:\programdata
\anaconda3\lib\site-packages (from astroid<-2.16.0-deve,>=2.14.2->pylint)

(4.5.0)

بیایید یک اسکریپت بسیار ساده را ذخیره کنیم:

In [4]:

```
1 %%writefile simple1.py
2 a=1
3 b=2
4 print(a)
5 print(b)
6
```

writing simple1.py

حالا بیایید آن را با استفاده از *pylint* بررسی کنیم

In [5]:

```
1 pylint simple1.py
```

****** Module simple1*

simple1.py:1:0: C0114: Missing module docstring (missing-module-docstring)
simple1.py:1:0: C0103: Constant name "a" doesn't conform to UPPER CASE nam
ing style (invalid-name)

simple1.py:2:0: C0183: Constant name "b" doesn't conform to UPPER CASE nam
ing style (invalid-name)

Your code has been rated at 2.50/10 (previous run: 10.00/10, -7.50)

ابتدا، *Pylint* برخی از مشکلات استایلی را لیست میکند - میخواهد در انتها یک خط جدید اضافه شود، ماژول ها و تعاریف تابع باید از *docstring* توصیفی برخوردار باشند و استفاده از کاراکترهای تکی برای نامگذاری متغیرها انتخاب ضعیفی است.

اما مهمتر از همه *pylint* یک خطا در برنامه شناسایی کرده است - استفاده از متغیر قبل از اختصاص به آن این نیاز به اصلاح دارد.

توجه کنید که *Pylint* برنامه ما را با امتیاز منفی ۱۲٫۵ از ۱۰ امتیاز گرفته است. بیایید سعی کنیم این امتیاز را بهبود ببخشیم!

In [12]:

```
%%writefile simple1.py
"""
A very simple script.
"""

def myfunc():
      '''
       An extremely simple function
      '''
     first = 1
    second = 2
    print(first)
    print(second)

myfunc()
```

Overwriting simple1.py

In [13]:

```
!pylint simple1.py
```

Your code has been rated at 10.00/10 (previous run: 8.33/10, +1.67)

بسیار بهتر شد امتیاز ما به ۱۰/۰۰ از ۱۰ ارتقا یافت. متأسفانه خط جدید نهایی با نحوه نوشتن *Jupyter* در یک فایل مربوط است و در اینجا چیز زیادی که بتوانیم درباره آن انجام دهیم وجود ندارد با این حال *Pylint* به ما در رفع برخی از مشکلاتمان کمک کرد اما اگر مشکل پیچیده تر بود چه اتفاقی می افتاد؟

In [14]:

```
%%writefile simple2.py
"""
A very simple script.
"""

def myfunc():
    '''
    An extremely simple function
    '''
    first = 1
    second = 2
    print(first)
    print('second').

myfunc()
```

Writing simple2.py

In [15]:

```
!pylint simple2.py
```

**** Module simple2

simple2.py:18:4: W8612: Unused variable 'second' (unused-variable).

Your code has been rated at 8.33/10 (previous run: 8.33/10, +0.00)

pylint به ما میگوید که در خط ۱۰ یک متغیر بی استفاده وجود دارد اما این نمیداند که ممکن است خروجی غیرمنتظره ای از خط ۱۳ دریافت کنیم برای این کار نیاز به مجموعه ای از ابزارهای قدرتمندتر است. به همین دلیل *unittest* وارد صحنه می شود.

unittest

unittest به شما امکان میدهد برنامه های تست خود را بنویسید. هدف این است که یک مجموعه خاصی از داده ها را به برنامه ی خود ارسال کنید و نتایج بازگشتی را با یک نتیجه مورد انتظار مقایسه کنید.
بیایید یک اسکریپت ساده بسازیم که کلمات در یک رشته داده شده را بزرگ کند اسم آن را *cap.py* خواهیم گذاشت.

In [16]:

```
1 XXwritefile cap.py
2 def cap_text(text):
3         return text.capitalize()
```

Writing cap.py

In [17]:

```
1 def cap_text(text):
2         return text.capitalize()
3
4 cap_text("monty python")
```

Out[17] :

'Monty python'

حالا یک اسکریپت تست خواهیم نوشت میتوانیم آن را هر چیزی که میخواهیم صدا بزنیم اما *test_cap.py* به نظر گزینه ای منطقی می رسد.

در هنگام نوشتن توابع تست بهتر است از ساده به پیچیده رفت، زیرا هر تابع به ترتیب اجرا خواهد شد.
در اینجا ابتدا رشته های ساده و یک کلمه ای را تست میکنیم سپس تستی برای رشته های چند کلمه ای انجام میدهیم.

In [2]:

```
writefile test_cap.py
import unittest
import cap

class TestCap(unittest.TestCase):

        def test_one_word(self):
             text = python"
            result = cap.cap_text(text)

            self.assertEqual(result, 'Python')

      def test_multiple_words (self):
            text = "monty python"
            result = cap.cap_text(text)

            self.assertEqual(result, 'Monty Python')

if(_name__main_');
unittest.main()
```

Overwriting test_cap.py

In [2]:

```
python test_cap.py
```

Ran 2 tests in 0.000s

OK

چه اتفاقی افتاد؟ به نظر میرسد که متد *capitalize*() تنها حرف اول کلمه اول در یک رشته را بزرگ میکند. با انجام یک تحقیق کوچک درباره متدهای رشته متوجه میشویم که ممکن است متد *title*(). ما را به هدفمان برساند.

In [3]:

```
1 %%writefile cap.py
2 def cap_text(text):
3     return text.title()
```

Overwriting cap.py

In [3]:

```
1 python test_cap.py
```

Ran 2 tests in 0.000s

OK

عالی هر دو تست ما پاس شدن اما آیا همه موارد را تست کردیم؟ بیایید یک تست دیگر به *test_cap.py* اضافه کنیم تا ببینیم آیا با کلماتی که حاوی واژه های ایتالیک هستند مانند "*don't*"به درستی برخورد میکند.

در یک ویرایشگر متنی این کار آسان است اما در *Jupyter* ما باید از ابتدا شروع کنیم.

In [4]:

```
%%writefile test_cap.py
import unittest
import cap.

class TestCap (unittest. TestCase):
        def test_one_word(self):
                text = 'python'
                result = cap.cap_text(text)

                self.assertEqual(result, 'Python')

        def test_multiple_words (self):
                text = "monty python"
                result=cap.cap_text(text)

                self.assertEqual(result, 'Monty Python')

        def test_with_apostrophes (self):
                text="monty python's flying circus"
                result= cap.cap_text(text)

                self.assertEqual(result, "Monty Python's Flying Circus")

if( _name_ == '_main_'):

        unittest.main()
```

Overwriting test_cap.py

In [5]:

```
python test_cap.py
```

..F

==

FAIL: test_with_apostrophes (main. TestCap)

```
----------------------------------------------------------------------------------------------------

Traceback (most recent call last):

File "C:\Users\babak\Python For Everyone \08-Errors and Exception Handlin
g\test_cap.py", line 22, in test_with_apostrophes

self.assertEqual(result, "Monty Python's Flying Circus")

AssertionError: "Monty Python's Flying Circus" != "Monty Python's Flying C

ircus"

- Monty Python'S Flying Circus

?

+ Monty Python's Flying Circus

?

----------------------------------------------------------------------------------------------------

Ran 3 tests in 0.001s

FAILED (failures=1)
```

الان باید یک راه حل پیدا کنیم که اپستروف ها را مدیریت کند! یک راه وجود دارد جستجوی *capwords* از ماژول *(string)*اما ما آن را به عنوان یک تمرین برای خواننده واگذار می کنیم.

عالی حالا باید یک درک پایه ای از تست واحد داشته باشید!

پروژه دست گرمی

بازی جنگ ساده

قبل از شروع به پروژه مرحله ۲ *OOP* بیایید با هم نحوه استفاده از *OOP* برای یک برنامه قدرتمند و پیچیده مانند یک بازی را مرور کنیم ما از *OOP* پایتون برای شبیه سازی نسخه ساده ای از بازی جنگ استفاده خواهیم کرد. دو بازیکن هر کدام با نیمی از *deck* شروع میکنند سپس هر کدام یک کارت بر می دارند کارت ها را با یکدیگر مقایسه میکنند و بازیکنی که کارت با بالاترین ارزش را دارد هر دو کارت را برای خود بر می دارد. در صورت برابری ارزش کارت ها یک تساوی اتفاق می افتد.

کلاس کارت تکی

ایجاد یک کلاس کارت با استفاده از متغیرهای خارجی

در اینجا ما از برخی متغیرهای خارجی استفاده خواهیم کرد که بدون توجه به شرایط موجود تغییر نمی کنند، مانند یک دک کارت بدون توجه به اینکه در چه دورهای مسابقه ای یا بازی ای هستیم همچنان به یک دک کارت نیاز خواهیم داشت.

In [1]:

```
# We'LL use this later
import random
```

In [2]:

```
suits = ('Hearts', 'Diamonds', "Spades', 'Clubs)
ranks = ('Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine', 'Ten',
'la
values = {'Two: 2, 'Three':3, 'Four' :4, 'Five':5, 'Six':6, Seven' :7,
'Eight':8, 'Nine':9, 'Ten':10, 'lack':11, 'Queen':12, 'King" :13, Ace': 14
```

In [3]:

```
class Card:

def_init_(self, suit, rank):
        self.suit = suit
        self.rank = rank
       self.value = values[rank]

def_str_(self):
    return self.rank + ' of ' + self.suit
```

ساخت یک نمونه از کارت ها

In [ ]:

```
suits[0]
```

In [ ]:

```
1 ranks[0]
```

In [ ]:

```
1 two_hearts = Card(sults[@],ranks[e])
```

In [ ]:

```
1 two_hearts
```

In [ ]:

```
1 print(two_hearts)
```

In [ ]:

```
1 two_hearts: rank
```

In [ ]:

```
1 two_hearts.value
```

In [ ]:

```
1 values[two_hearts.rank]
```

کلاس دسته کارت ها

استفاده از یک کلاس درون یک کلاس دیگر

تا به حال ما فقط یک کارت تکی ایجاد کرده ایم اما چگونه میتوانیم یک دسته کامل از کارت ها ایجاد کنیم؟ بیایید با استفاده از یک کلاس که از کلاس کارت استفاده میکند این کار را بررسی کنیم.

یک ***deck*** از چندین کارت تشکیل میشود این به این معناست که در ***init*** کلاس ***deck*** به واقع از کلاس کارت استفاده خواهیم کرد.

In [4]:

```
class Deck:

  def_ init_(self):
      #Note this only happens once upon creation of a new Deck
      self.all_cards = []
      for suit in suits:
          for rank in ranks:
              # This assumes the Card class has already been defined!
              self.all_cards.append(Card(suit,rank))

  def shuffle(self):
     #Note this doesn't return anything
     randon.shuffle(self.all_cards)

  def deal_one(self):
     # Note we remove one card from the List of all cards
     return self.all_cards.pop()
```

ایجاد یک deck

In [ ]:

```
mydeck = Deck()
```

In [ ]:

```
len(mydeck.all_cards)
```

In [ ]:

```
mydeck.all_cards[0]
```

In [ ]:

```
print(mydeck.ali_cards[0])
```

In [ ]:

```
1 mydeck.shuffle()
```

In [ ]:

```
1 print(mydeek.all_cards[0])
```

In [ ]:

```
1 my_card = mydeck.deal_one()
```

In [ ]:

```
1 print(my_card)
```

کلاس بازیکن

بیایید یک کلاس بازیکن ایجاد کنیم یک بازیکن باید قادر باشد نمونه هایی از کارت ها را در اختیار داشته باشد و همچنین بتواند آنها را از دست خود حذف یا اضافه کند ما میخواهیم که کلاس بازیکن به اندازه کافی انعطاف پذیر باشد تا بتواند یک کارت یا چند کارت را به آن اضافه کند بنابراین از یک بررسی ساده استفاده میکنیم تا همه این کارها در یک متد یکسان قرار گیرد.

همه این موارد را در نظر خواهیم داشت هنگامی که متدهای کلاس بازیکن را ایجاد میکنیم.

کلاس بازیکن

In [5]:

```
class Player:

    def_init_(self,name):
            self.name = name
            #A new player has no cards
           self.all cards = []

   def remove_one(self):
           # Note we remove one card from the list of all cards
          #We state 8 to remove from the "top" of the deck
         #We'LL imagine index 1 as the bottom of the deck.
        return self.all_cards.pop(a)

  def add cards (self,new_cards):
        if type(new_cards) == type([]):
              self.all_cards.extend(new_cards)
       else:
             self.all cards.append(new_cards)


def _str_(self):
      return 'Player (self.name} has (len(self.all_cards)} cards."
```

In [ ]:

```
jose Player("Jose")
```

In [ ]:

```
Jose
```

In [ ]:

```
print(Jose)
```

In [ ]:

```
1 two hearts
```

In [ ]:

```
1 jose.add_cards (two_hearts)
```

In [ ]:

```
1 print(Jose)
```

In [ ]:

```
1 jose.add_cards ([two_hearts, two_hearts, two_hearts])
```

In [ ]:

```
1 print(Jose)
```

In [ ]:

```
1 jose.remove_one()
```

In [ ]:

```
1 print(jose)
```

منطق بازی جنگ

In [6]:

```
1 player one Player("One")
```

In [7]:

```
1 player two Player("Two")
```

راه اندازی تنظیمات بازی

In [8]:

```
1 new deck Deck()
```

In [9]:

```
1 new deck.shuffle()
```

تقسیم Deck میان دو بازیکن

In [ ]:

```
1 len(new_deck.all_cards)/2
```

In [10]:

```
1 for x in range(26):
2     player_one.add_cards (new_deck.deal_one())
3     player_two.add_cards (new_deck.deal_one())
```

In [ ]:

```
1 len(new deck.all_cards)
```

In [ ]:

```
1 len(player_one.all_cards)
```

In [ ]:

```
1 len(player_two.all_cards)
```

اجرای بازی

In [27]:

```
import pdb.
```

In [11]:

```
game on = True
```

In [13]:

```
round_num = 0

while game_on:

    round_num+= 1
    print("Round (round_num)")

   if len(player one.all_cards) = 8:
      print("Player One out of cards! Game Over")
     print("Player Two Wins!")
     game on = False
      break
  if len(player two.all_cards) == 0:
    print("Player Two out of cards! Game Over")
    print("Player One Wins!")
    game on = False
    break

 player_one_cards = []
player_one_cards.append(player_one.remove_one())

player_two_cards=[]
player_two_cards.append(player_two.remove_one())

 at_war = True

 while at war:
```

```
28
29          if player_one_cards[-1].value> player_two_cards[-1].value:
30                player_one.add_cards (player_one_cards)
31               player_one.add_cards (player_two_cards)
32
33              at_war = False
34
35       elif player_one_cards[-1]. value < player_two_cards [-1].value:
36
37             # Player Two gets the cords
38            player_two.add_cards (player_one_cards)
39            player two.add_cards (player_two_cards)
40
41           #No Longer at "war", time for next round at war False
42           at_war = False
43
44       else:
45
46           print("WAR!")
47
48          if len(player_one.all_cards) < 5:
49                print("Player One unable to play war! Game Over at War")
50                print("Player Two Wins! Player One Loses!")
51               game on False
52               break
53       elif len(player_two.all_cards) < 5:
54               print("Player Two unable to play war! Game Over at War")
55               print("Player One Wins! Player One Loses!")
56               game on False
57               break
58       else:
59              for num in range(5):
60                   player_one_cards.append(player_one.remove_one())
61                   player_two_cards.append(player_two.remove_one())
62
```

Round 561

Round 562

Round 563

Round 564

Round 565

Round 566

Round 567

Round 568

Round 569

Round 570

Round 571

Round 572

Round $73.

Round 574

Round 575

Round 576

WAR!

Player One unable to play war! Game Over at War Player Two Wins! Player One Loses!

گرداوری منطق بازی در یک سلول

In [14]:

```
player_one player_two = Player ("One")
Player_ ("Two") = Player ("Two")

new_deck = Deck()
new_deck.shuffle()

for x in range(26):
        player_one.add_cards (new_deck.deal_one())
        player_two.add_cards (new_deck.deal_one())

game_on = True
```

In [15]:

```
round_num=0
while game on:

    round_num 1
     print("Round (round_num}")

    # Check to see if a player is out of cards:
    if len(player_one.all_cards) == 0:
         print("Player One out of cards! Game Over")
          print("Player Two Wins!")
         game on= False
        break

    if len(player_two.all_cards) == 0:
          print("Player Two out of cards! Game Over")
          print("Player One Wins!")
         game on = False
        break

   # Otherwise, the game is still on!

```

```
22    #Start a new round and reset current cards "on the table"
23    player_one_cards = []
24    player_one_cards.append(player_one.remove_one())
25
26   player_two_cards = []
27   player_two_cards.append(player_two.remove_one())
28
29   at war True
30
31  while at war:
32
33
34        if player_one_cards [-1].value> player_two_cards [-1].value:
35
36            # Player One gets the cards
37            player_one.add_cards (player_one_cards)
38           player_one.add_cards (player_two_cards)
39
40
41          # No Longer at "war", time for next round
42         at war= False
43
44     # Player Two Has higher Card
45    elif player_one_cards[-1].value < player_two_cards[-1].value:
46
47       #PLayer Two gets the cards
48      player two.add_cards (player_one_cards)
49      player_two.add_cards (player_two_cards)
50
51     #No Longer at "war" time for next round
52     at war False
53
54 else:
55     print('WAR!')
56    # This occurs when the cards are equal.
57   # We'll grab another card each and continue the current war.
58
```

```
59   # First check to see if player has enough cards.
60
61   # Check to see if a player is out of cards:
62   if len(player_one.all_cards) < 5:
63       print("Player One unable to play war! Game Over at War")
64       print("Player Two Wins! Player One Loses!")
65       game_on = False
66       break
67
68   elif len(player_two.all_cards) < 5:
69           print("Player Two unable to play war! Game Over at War")
70           print("Player One Wins! Player One Loses!")
71           game on = False
72           break
73   # Otherwise, we're still at war, so we'll add the next cards
74   else:
75         for num in range(5):
76             player_one_cards.append(player_one.remove_one())
77             player_two_cards.append(player_two.remove_one())
78
```

Round 1

Round 2

Round 3

Round 4

Round 5

Round 6

Round 7

Round 8

Round 9

Round 10

Round 11

Round 12

Round 13.

Round 14

Round 15

Round 16

Round 17

Round 18

Round 19

WAR!

Round 20

Round 21

Round 22

Round 23

Round 24

Round 25

Round 26

Round 27

Round 28

Round 29

Round 30

Round 31

Round 32

WAR!

Round 33

Round 34

Round 35

Round 36

Round 37

Round 38

Round 39

Round 40

Round 41

Round 42

Round 43

Round 44

Round 45

Round 46

Round 47

Round 48

Round 49

Round 50

Round 51

Round 52

Round 53

Round 54

Round 55

Round 56

Round 57

Round 58

Round 59

Round 60

Round 61

Round 62

Round 63

Round 64

Round 65

Round 66

Round 67

Round 68

Round 69

WAR!

Round 70

Round 71

Round 72

Round 73

Round 74

WAR!

Player One unable to play war! Game Over at War Player Two Wins! Player One Loses!

In [16]:

```
1 len(player_one.all_cards)
```

Out [16]:

3

In [17]:

```
1 len(player two.all cards)
```

Out[17]:

47

In [18]:

```
1 print(player_one_cards[-1])
```

Jack of Hearts.

In [19]:

```
1 print(player_two_cards[-1])
```

Jack of Clubs

عالی

لینک های که به درک بهتر شما می توانند کمک کنند.

- https://www.reddit.com/r/learnpython/comments/7ay83p/war.card.game/

https://www.reddit.com/r/leamovthon/comments/7av830/war.card.name/)

- https://codereview.stackexchange.com/pumations/121174/wat-card-same-sibo.classes

(https://codereview.stackexchange.com/questions/131174/war-card-game-using-classes)

- https://gist.github.com/damianesteban/6896120

https://cist.github.com/damianesteban/6896120)*)*

- https://lethein.com/war-card.game-in-python/(htts://lethai.com/war-cant-game-in-python/
- https://hectorpefa, github.io/2017-09-13-Cand- Wars/Chttps://hectorpefo.github.io/2017-09-13- Card-Wars
- https://www.wimuvprogrammer.com/the-statistics of war the card garme
- https-//www.winwwwnrammer.com/the-stathetics-of-wse-the-cant-name)

پروژه عملی ۲ بازی بلک جک

در این پروژه مرحله ای شما باید یک بازی کامل بلک جک با استفاده از پایتون ایجاد کنید

در زیر شرایط مورد نیاز آمده است:

- شما باید یک بازی متنی ساده بلک جک بلک جک (https://en.wikipedia.org/wiki/Blackjack)را ایجاد کنید.

- بازی باید بین یک بازیکن و یک دلال خودکار انجام شود.
- بازیکن میتواند وضعیت *stand* یا *hit* بگیرد.
- بازیکن باید قادر باشد مقدار سرمایه گذاری خود را انتخاب کند.
- شما باید مجموع پول بازیکن را در نظر بگیرید
- شما باید بازیکن را در مورد برد باخت و پیروزی ها و... آگاه سازید.

و بیشتر از همه :

- **شما باید در بخشی از بازی خود از *OOP* و کلاس ها استفاده کنید نمیتوانید فقط از توابع در بازی خود استفاده کنید از کلاس ها برای تعریف دسته کارت ها و دست بازیکن استفاده کنید متدهای درست بسیاری برای انجام این کار وجود دارد پس به خوبی آن را بررسی کنید!**

احساس راحتی کنید که این بازی را گسترش دهید. سعی کنید چندین بازیکن را اضافه کنید تلاش کنید تا قابلیت های همچون *Double-Down* و تقسیم کارت ها را به بازی اضافه کنید به یاد داشته باشید که میتوانید از هر منبعی که میخواهید استفاده کنید و همواره:

سرگرم شوید

پروژه مرحله ۲ کتاب کار گام به گام

در زیر مجموعه ای از مراحل برای شما آورده شده است تا سعی کنید بازی پروژه مرحله ۲ بلک جک را ایجاد کنید!

بازی

برای بازی یک دست بلک جک باید مراحل زیر را دنبال کنید:

۱. یک دسته کارت ۵۲ تایی ایجاد کنید.
۲. دسته کارت را مخلوط کنید.
۳. از بازیکن شرط خود را بپرسید.
۴. اطمینان حاصل کنید که شرط بازیکن از تعداد تراشه های در دسترسش بیشتر نشود.
۵. دو کارت به دلال و دو کارت به بازیکن بدهید.
۶. فقط یکی از کارت های دلال را نشان دهید و دیگری مخفی بماند.
۷. هر دو کارت بازیکن را نشان دهید.
۸. از بازیکن بپرسید که آیا میخواهند کارت بگیرند یا خیر و کارت دیگری بگیرند.
۹. اگر دست بازیکن زیادی بشود بیش از (۲۱) بپرسید که آیا میخواهند دوباره کارت بگیرند یا خیر
۱۰. اگر بازیکن ایستاد دست دلال را بازی کنید دلال همیشه کارت بگیرد تا مقدار دلال برابر یا بیشتر از ۱۷ شود.
۱۱. برنده را تعیین کرده و تعدیل *chips* بازیکن را مطابق با آن انجام دهید.
۱۲. از بازیکن بپرسید که آیا میخواهند دوباره بازی کنند یا خیر.

کارت های بازی

یک دسته استاندارد کارت بازی شامل چهار نوع(،قلب ،دیموند، پیک و تپه) و سیزده رتبه (از ۲ تا ۱۰ و سپس کارت های صورتی ،جک کوین و کینگ و آس است که در مجموع ۵۲ کارت دارد. جک ها، کوین ها و کینگ ها همه رتبه ۱۰ دارند آس ها رتبه ۱۱ یا ۱ دارند تا به ۲۱ برسند بدون اینکه بیشتر از این مقدار شوند به عنوان نقطه شروع برنامه میتوانید متغیرهایی را اختصاص دهید تا یک لیست از نوع ها رتبه ها را ذخیره کنید و سپس از یک دیکشینری برای نگاشت رتبه ها به ارزش ها استفاده کنید.

بازی

Imports and Global Variables

گام ۱: ماژول *random* را وارد کنید این ماژول برای مخلوط کردن دسته کارت قبل از توزیع استفاده خواهد شد. سپس متغیرهایی را برای ذخیره کردن انواع کارت ها رتبه ها و ارزش ها تعریف کنید شما میتوانید سیستم خودتان را توسعه دهید یا میتوانید سیستم ما را کپی کنید که در زیر آمده است در نهایت یک مقدار بولین تعریف کنید که برای کنترل حلقه های *while* استفاده میشود. این یک شیوه رایج است که برای کنترل جریان بازی استفاده میشود.

```
suits =  ('Hearts', 'Diamonds', 'Spades', 'Clubs')

ranks = ('Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine', 'Te
```

In [ ]:

```
import random

suits = pass
ranks = pass
values = pass

playing = True
```

تعاریف کلاس

در نظر بگیرید یک کلاس *Card* ایجاد کنید که هر شی *Card* دارای نوع و رتبه است سپس یک کلاس *Deck* برای نگهداری تمامی ۵۲ شی *Card* و مخلوط کردن آنها و در نهایت یک کلاس *Hand* که شامل کارتهایی است که از دسته به هر بازیکن داده شده است.

گام ۲ :ایجاد کلاس *Card*

یک شیء *Card* واقعاً تنها به دو ویژگی نیاز دارد نوع و رتبه شما میتوانید یک ویژگی برای ارزش اضافه کنید - ما تصمیم گرفتیم ارزش را در زمان توسعه کلاس *Hand* مدیریت کنیم. علاوه بر روش *init* کارت در نظر بگیرید یک روش *str* اضافه کنید که هنگام درخواست چاپ یک کارت یک رشته به شکل *Two of Hearts* را برمی گرداند.

In [ ]:

```
import random

suits = pass
ranks = pass
values = pass

playing = True
```

گام۳ : ایجاد کلاس *Deck*

در این قسمت میتوانیم ۵۲ شی کارت را در یک لیست ذخیره کنیم که در آینده بتوانیم آنها را مخلوط کنیم. اما در ابتدا، باید تمامی ۵۲ شیء کارت یکتایی را ایجاد کرده و آنها را به لیستمان اضافه کنیم تا زمانی که تعریف کلاس *Card* در کد ما وجود دارد ما میتوانیم شی کارت را درون متد *init* کلاس *Deck* بسازیم. در نظر بگیرید که با استفاده از حلقه ها بر روی دنباله های انواع و رتبه ها هر کارت را بسازیم این ممکن است درون متد *init* کلاس *Deck* ظاهر شود:

```
for suit in suits:

    for rank in ranks:
```

علاوه بر متد _init_ ما میخواهیم متدهایی را اضافه کنیم تا دسته کارت هایمان را مخلوط کنیم و کارت ها را در طول بازی توزیع کنیم. اختیاری ممکن است هرگز نیازی به چاپ محتویات دسته کارت ها در طول بازی نداشته باشیم اما داشتن امکان مشاهده کارت ها درون آن میتواند به رفع مشکلاتی که در طول توسعه به وجود میاید کمک کند. با این در نظر گرفته شود، در نظر بگیرید یک متد *str* را به تعریف کلاس اضافه کنید.

In [ ]:

```
class Deck:

    def init (self):
        self.deck = [] # start with an empty list
        for suit in suits:
            for rank in ranks:
                pass

    def str_(self):
        pass

    def shuffle(self):
        random, shuffle(self.deck)

    def deal(self):
        pass
```

آزمایش فقط برای دیدن اینکه تا الان همه چیز کار میکند چه شکلی دسته کارت هایمان است. بیایید ببینیم!

In [ ]:

```
1  test_deck = Deck()
2  print(test_deck)
```

عالی حالا بیایید به کلاس *Hand* برویم.

گام ۴ :ایجاد کلاس *Hand*

علاوه بر نگهداری از شیرهای کارتی که از دسته داده میشوند کلاس *Hand* میتواند برای محاسبه ارزش این کارت ها با استفاده از فهرست ارزش هایی که در بالا تعریف شده اند استفاده شود. همچنین در صورت لزوم باید برای ارزش آس ها تعدیلی انجام دهد

In [ ]:

```
1  class Hand:
2      def _init_(self):
3          self.cards = []  # start with an empty List as we did in the Deck class
4          self.value=0    # start with zero value
5          self.aces = 0   # add an attribute to keep track of oces
6
7      def add card(self, card) :
8          pass
9
10     def adjust_for_ace(self):
11         pass
```

گام ۵ :ایجاد کلاس*Chips*

علاوه بر دسته های کارت و دست ها ما باید موجودی اولیه بازیکن شرط ها و برنده شدن های مداوم یک بازیکن را نیز نگهداری کنیم. این کار میتواند با استفاده از متغیرهای عمومی انجام شود اما با توجه به روح برنامه نویسی شی گرا، به جای آن یک کلاس *Chips* بسازیم

In [ ]:

```
1  class Chips:
2
3      def init_(self):
```

```
4           self.total = 100 # This can be set to a default value or supplied by a user
5           self.bet = 0
6
7       def win_bet(self):
8           pass
9
10      def lose_bet(self):
11          pass
```

تعریف توابع

بسیاری از مراحل تکراری خواهند بود اینجاست که توابع به کار میآیند مراحل زیر راهنماییهایی هستند - توابع را بر اساس نیاز خود در برنامه خود اضافه یا حذف کنید.

گام ۶ :نوشتن تابع برای شرط ها

از آنجا که از کاربر مقدار عددی میخواهیم اینجا مکان خوبی است برای استفاده از try/exceptاست به خاطر داشته باشید که چک کنید که شرط بازیکن توسط تعداد تراشه های موجود تامین شود.

In [ ]:

```
1   def take_bet():
2
3       pass
```

گام ۷: نوشتن تابع برای گرفتن کارت

هر یک از بازیکنان میتوانند تا زمانی که تابع ***bust*** را بدهند کارت بگیرند این تابع هنگام بازی فراخوانی میشود هر زمانی که یک بازیکن درخواست کارت بدهد یا دست یک دیلر کمتر از ۱۷ .باشد باید شیهای دسته و دست را به عنوان آرگومان دریافت کند و یک کارت از دسته را برداشت کند و به دست اضافه کند ممکن است بخواهید در صورتی که دست یک بازیکن بیش از ۲۱ باشد برای مشخص کردن آس ها بررسی کنید.

In [ ]:

```
1   def hit(deck, hand):
2
```

```
3    pass
```

گام ۸: نوشتن تابع برای درخواست بازیکن برای گرفتن کارت یا ایستادن

این تابع باید دسته و دست بازیکن را به عنوان آرگومان دریافت کند و بازی را به عنوان یک متغیر عمومی تعیین کند. اگر بازیکن کارت میگیرد .(*Hit*)از تابع() *hit* استفاده کنید اگر بازیکن ایستاد . *(Stand)*متغیر *playing* را به *False* تنظیم کنید - این متغیر عملکرد یک حلقه *while* در بخشی دیگر از کد را کنترل خواهد کرد.

In [ ]:

```
1  def hit_or_stand(deck,hand) :
2      global playing # to control an upcoming while Loop.
3
4      pass
```

گام ۹: نوشتن توابع برای نمایش کارت ها

هنگام شروع بازی و پس از هر باری که بازیکن یک کارت میگیرد کارت اول دیلر مخفی است و تمام کارتهای بازیکن قابل مشاهده است. در پایان دست تمام کارتها نمایش داده میشوند و ممکن است بخواهید مجموع ارزش هر دست را نشان دهید. برای هر یک از این ،حالات، یک تابع بنویسید.

In [ ]:

```
1  def show some (player, dealer):
2
3      pass
4
5  def show_all(player, dealer):
6
7      pass
```

گام ۱۰ :نوشتن توابع برای مدیریت حالات پایانی بازی

به یاد داشته باشید که در صورت نیاز دست بازیکن دست دیلر و تراشه ها را به عنوان آرگومان ها منتقل کنید.

In [ ]:

```
def player_busts():
    pass

def player_wins():
    pass

def dealer_busts():
    pass

def dealer_wins():
    pass

def push():
    pass
```

و حالا به بازی میپردازیم!!

In [ ]:

```
while True:
    # Print an opening statement


    # Create & shuffle the deck, deal two cards to each player

```

```
7
8
9       # Set up the Player's chips
10
11
12     #Prompt the Player for their bet
13
14
15     #Show cards (but keep one dealer card hidden)
16
17
18      while playing: # recall this variable from our hit_or_stand function
19
20      # Prompt for Player to Hit or Stand
21
22
23     #Show cards (but keep one dealer card hidden)
24
25
26    # If player's hand exceeds 21, run player_busts() and break out of
27 Loop
28
29        break
30
31     # If Player hasn't busted, play Dealer's hand until Dealer reaches 17
32
33
34    # Show all cards
35
36    # Run different winning scenarios
37
38
39    #Inform Player of their chips total
40
41    #Ask to play again
42
43       break
```

تمام شد به یاد داشته باشید که این مراحل ممکن است با راه حل خودتان به طور قابل توجهی متفاوت باشد. این مهم نیست تا زمانی که نتیجه مورد نظر را بدست نیاورید روی بخش های مختلف برنامه ای خودکار کنید. این مسئله نیازمند زمان و صبر زیادی است همیشه از پرسش ها و نظرات خود در انجمن های پرسش و پاسخ استفاده کنید.

خدا قوت

پروژه مرحله ۲ کتاب کار گام به گام

در زیر مجموعه ای از مراحل برای شما آورده شده است تا سعی کنید بازی پروژه مرحله ۲ بلک جک را ایجاد کنید!

بازی

برای بازی یک دست بلک جک باید مراحل زیر را دنبال کنید:

۱. یک دسته کارت ۵۲ تایی ایجاد کنید.
۲. دسته کارت را مخلوط کنید.
۳. از بازیکن شرط خود را بپرسید.
۴. اطمینان حاصل کنید که شرط بازیکن از تعداد تراشه های در دسترسش بیشتر نشود.
۵. دو کارت به دلال و دو کارت به بازیکن بدهید.
۶. فقط یکی از کارت های دلال را نشان دهید و دیگری مخفی بماند.
۷. هر دو کارت بازیکن را نشان دهید.
۸. از بازیکن بپرسید که آیا میخواهند کارت بگیرند یا خیر و کارت دیگری بگیرند.
۹. اگر دست بازیکن زیادی بشود بیش از (۲۱) بپرسید که آیا میخواهند دوباره کارت بگیرند یا خیر
۱۰. اگر بازیکن ایستاد دست دلال را بازی کنید دلال همیشه کارت بگیرد تا مقدار دلال برابر یا بیشتر از ۱۷ شود.
۱۱. برنده را تعیین کرده و تعدیل *chips* بازیکن را مطابق با آن انجام دهید.
۱۲. از بازیکن بپرسید که آیا میخواهند دوباره بازی کنند یا خیر.

کارت های بازی

یک دسته استاندارد کارت بازی شامل چهار نوع(،قلب ،دیموند، پیک و تپه) و سیزده رتبه (از ۲ تا ۱۰ و سپس کارت های صورتی ،جک کوین و کینگ و آس است که در مجموع ۵۲ کارت دارد. جک ها، کوین ها و کینگ ها همه رتبه ۱۰ دارند آس ها رتبه ۱۱ یا ۱ دارند تا به ۲۱ برسند بدون اینکه بیشتر از این مقدار شوند به عنوان نقطه شروع برنامه میتوانید متغیرهایی را اختصاص دهید تا یک لیست از نوع ها رتبه ها را ذخیره کنید و سپس از یک دیکشینری برای نگاشت رتبه ها به ارزش ها استفاده کنید.

بازی

Imports and Global Variables

گام ۱: ماژول *random* را وارد کنید این ماژول برای مخلوط کردن دسته کارت قبل از توزیع استفاده خواهد شد. سپس متغیرهایی را برای ذخیره کردن انواع کارت ها رتبه ها و ارزش ها تعریف کنید شما میتوانید سیستم خودتان را توسعه دهید یا میتوانید سیستم ما را کپی کنید که در زیر آمده است در نهایت یک مقدار بولین تعریف کنید که برای کنترل حلقه های *while* استفاده میشود. این یک شیوه رایج است که برای کنترل جریان بازی استفاده میشود.

suits = ('Hearts', 'Diamonds', 'Spades', 'Clubs')

ranks = ('Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine', 'Te

In [4]:

```
1  import random
2
3  suits = ('Hearts', 'Diamonds', 'Spades', 'Clubs')
4  ranks = ('Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine', 'Ten', 'Jac)
5  values = {'Two':2, 'Three':3, 'Four':4, 'Five':5, 'Six':6, 'Seven':7,
6  'Eight':8, 'Nir 'Queen':10, 'King':10, 'Ace':11}
7
8  playing = True
```

تعاریف کلاس

در نظر بگیرید یک کلاس *Card* ایجاد کنید که هر شی *Card* دارای نوع و رتبه است سپس یک کلاس *Deck* برای نگهداری تمامی ۵۲ شی *Card* و مخلوط کردن آنها و در نهایت یک کلاس *Hand* که شامل کارتهایی است که از دسته به هر بازیکن داده شده است.

گام ۲ :ایجاد کلاس *Card*

یک شیء *Card* واقعاً تنها به دو ویژگی نیاز دارد نوع و رتبه شما میتوانید یک ویژگی برای ارزش اضافه کنید - ما تصمیم گرفتیم ارزش را در زمان توسعه کلاس *Hand* مدیریت کنیم. علاوه بر روش *init* کارت در نظر بگیرید یک روش *str* اضافه کنید که هنگام درخواست چاپ یک کارت یک رشته به شکل *Two of Hearts* را برمی گرداند.

In [1]:

```
class Card:

    def _init_(self, suit, rank):
        self.suit suit
       self.rank rank

    def _str_(self):
        return self.rank + 'of '+ self.suit
```

گام۳ : ایجاد کلاس *Deck*

در این قسمت میتوانیم ۵۲ شی کارت را در یک لیست ذخیره کنیم که در آینده بتوانیم آنها را مخلوط کنیم. اما در ابتدا، باید تمامی ۵۲ شیء کارت یکتایی را ایجاد کرده و آنها را به لیستمان اضافه کنیم تا زمانی که تعریف کلاس *Card* در کد ما وجود دارد ما میتوانیم شی کارت را درون متد *init* کلاس *Deck* بسازیم. در نظر بگیرید که با استفاده از حلقه ها بر روی دنباله های انواع و رتبه ها هر کارت را بسازیم این ممکن است درون متد *init* کلاس *Deck* ظاهر شود:

```
for suit in suits:

    for rank in ranks:
```

علاوه بر متد _init_ ما میخواهیم متدهایی را اضافه کنیم تا دسته کارت هایمان را مخلوط کنیم و کارت ها را در طول بازی توزیع کنیم. اختیاری ممکن است هرگز نیازی به چاپ محتویات دسته کارت ها در طول بازی نداشته باشیم اما داشتن امکان مشاهده کارت ها درون آن میتواند به رفع مشکلاتی که در طول

توسعه به وجود میاید کمک کند. با این در نظر گرفته شود، در نظر بگیرید یک متد *str* را به تعریف کلاس اضافه کنید.

In [2]:

```
class Deck:

    def _init_(self):
         self.deck [] # start with an empty list
        for suit in suits:
              for rank in ranks:
                    self.deck.append(Card(suit, rank))

def_str_(self):
    deck_comp=
    for card in self.deck:
       deck_comp+= '\n' + card._str_()

     return 'The deck has: + deck_comp

 def shuffle(self):
    random.shuffle(self.deck)

 def deal(self):
       signle_card self.deck.pop()
       return signle_card
```

آزمایش فقط برای دیدن اینکه تا الان همه چیز کار میکند چه شکلی دسته کارت هایمان است. بیایید ببینیم!

In [ ]:

```
test_deck = Deck()
print(test_deck)
```

The deck has:

Two of Hearts

Three of Hearts

Four of Hearts

Five of Hearts

Six of Hearts

Seven of Hearts

Eight of Hearts

Nine of Hearts

Ten of Hearts

Jack of Hearts

Queen of Hearts

King of Hearts

Ace of Hearts

Two of Diamonds

Three of Diamonds

Four of Diamonds

Five of Diamonds

Six of Diamonds

Seven of Diamonds

Eight of Diamonds

Nine of Diamonds

Ten of Diamonds

Jack of Diamonds

Queen of Diamonds

King of Diamonds

Ace of Diamonds

Two of Spades

Three of Spades

Four of Spades

Five of Spades

Six of Spades

Seven of Spades

Eight of Spades.

Nine of Spades

Ten of Spades

Jack of Spades

Queen of Spades

King of Spades

Ace of Spades

Two of Clubs.

Three of Clubs

Four of Clubs.

Five of Clubs

Six of Clubs

Seven of Clubs

Eight of Clubs

Nine of Clubs.

Ten of Clubs

Jack of Clubs

Queen of Clubs

King of Clubs

Ace of Clubs

عالی حالا بیایید به کلاس *Hand* برویم.

گام ۴ :ایجاد کلاس *Hand*

علاوه بر نگهداری از شیرهای کارتی که از دسته داده میشوند کلاس *Hand* میتواند برای محاسبه ارزش این کارت ها با استفاده از فهرست ارزش هایی که در بالا تعریف شده اند استفاده شود. همچنین در صورت لزوم باید برای ارزش آس ها تعدیلی انجام دهد

In [6]:

```
class Hand:
    def _init_(self):
        self.cards = []  # start with an empty List as we did in the Deck class
        self.value=0    # start with zero value
        self.aces = 0   # add an attribute to keep track of oces

    def add card(self, card) :
        self.cards.append(card)
        self.value = values[card.rank]

    def adjust_for_ace(self):
        pass
```

In [7]:

```
test_deck = Deck()
test_deck.shuffle()

test_player = Hand()
test_player.add_card (test_deck.deal())
test_player.add_card (test_deck.deal())

test_player.value
```

Out [7]:

18

In [8]:

```
for card in test_player.cards:
    print(card)
```

Ten of Clubs

Eight of Diamonds

In [9]:

```
def take bet(chips):

 while True:
     try:
        chips.bet = int(input("How many chips would like to bet? "))
    except ValueError:
        print("Sorry, a bet must be an integer!")
   else:
      if chips.bet > chips.total:
          print("Sorry, your bet can't exceed", chips.total)
     else:
          break
```

گام ۵ :ایجاد کلاس*Chips*

علاوه بر دسته های کارت و دست ها ما باید موجودی اولیه بازیکن شرط ها و برنده شدن های مداوم یک بازیکن را نیز نگهداری کنیم. این کار میتواند با استفاده از متغیرهای عمومی انجام شود اما با توجه به روح برنامه نویسی شی گرا، به جای آن یک کلاس *Chips* بسازیم

In [10]:

```
class Chips:

    def init (self, total 108):
        self.total total # This can be set to a default value or supplied by a usi
        self.bet = 0

    def win bet(self):
        self.total = self.bet

    def lose bet(self):
        self.total self.bet
```

تعریف توابع

بسیاری از مراحل تکراری خواهند بود اینجاست که توابع به کار میآیند مراحل زیر راهنماییهایی هستند - توابع را بر اساس نیاز خود در برنامه خود اضافه یا حذف کنید.

گام ۶ :نوشتن تابع برای شرط ها

از آنجا که از کاربر مقدار عددی میخواهیم اینجا مکان خوبی است برای استفاده از try/exceptاست به خاطر داشته باشید که چک کنید که شرط بازیکن توسط تعداد تراشه های موجود تامین شود.

In [11]:

```
def take bet(chips):

  while True:
      try:
         chips.bet = int(input("How many chips would like to bet? "))
      except ValueError:
```

```
            print("Sorry, a bet must be an integer!")
        else:
            if chips.bet > chips.total:
                print("Sorry, your bet can't exceed", chips.total)
            else:
                break
```

گام ۷: نوشتن تابع برای گرفتن کارت

هر یک از بازیکنان میتوانند تا زمانی که تابع ***bust*** را بدهند کارت بگیرند این تابع هنگام بازی فراخوانی میشود هر زمانی که یک بازیکن درخواست کارت بدهد یا دست یک دیلر کمتر از ۱۷ .باشد باید شیهای دسته و دست را به عنوان آرگومان دریافت کند و یک کارت از دسته را برداشت کند و به دست اضافه کند ممکن است بخواهید در صورتی که دست یک بازیکن بیش از ۲۱ باشد برای مشخص کردن آس ها بررسی کنید.

In [12]:

```
def hit(deck, hand):

    hand.add_card (deck.deal())
    hand.adjust_for_ace()
```

گام ۸: نوشتن تابع برای درخواست بازیکن برای گرفتن کارت یا ایستادن

این تابع باید دسته و دست بازیکن را به عنوان آرگومان دریافت کند و بازی را به عنوان یک متغیر عمومی تعیین کند. اگر بازیکن کارت میگیرد .(***Hit***)از تابع() ***hit*** استفاده کنید اگر بازیکن ایستاد . ***(Stand)***متغیر ***playing*** را به ***False*** تنظیم کنید - این متغیر عملکرد یک حلقه ***while*** در بخشی دیگر از کد را کنترل خواهد کرد.

In [13]:

```
def hit or stand (deck, hand):
    global playing # to control an upcoming while Loop
```

```
3
4   while True:
5         x = input("Would you like to Hit or Stand? enter 'h' or 's'")
6
7         if x[0].lower()=='h':
8               hit(deck, hand)
9         elif x[0].lower() == 's':
10              print("Player stands. Dealer is playing.")
11              playing= False
12
13        else:
14              print("Sorry, Please try again!")
15              continue
16        break
```

گام ۹: نوشتن توابع برای نمایش کارت ها

هنگام شروع بازی و پس از هر باری که بازیکن یک کارت میگیرد کارت اول دیلر مخفی است و تمام کارتهای بازیکن قابل مشاهده است. در پایان دست تمام کارتها نمایش داده میشوند و ممکن است بخواهید مجموع ارزش هر دست را نشان دهید. برای هر یک از این ،حالات، یک تابع بنویسید.

In [14]:

```
1  def show some(player, dealer):
2         print("\nDealer's Hand:")
3         print(" <card hidden>")
4         print(dealer.cards[1])
5         print("\nPlayer's Hand:", "player.cards, sep='\n')
6
7  def show_all(player, dealer):
8         print("\nDealer's Hand: ", "dealer. cards, sep='\n')
9         print("Dealer's Hand ", dealer.value)
10        print("\nPlayer's Hand:", "player.cards, sep='\n')
11        print("Player's Hand ",player.value)
```

گام ۱۰ :نوشتن توابع برای مدیریت حالات پایانی بازی

به یاد داشته باشید که در صورت نیاز دست بازیکن دست دیلر و تراشه ها را به عنوان آرگومان ها منتقل کنید.

In [21]:

```
def player busts (player, dealer, chips):
      print("Player busts!")
      chips.lose_bet()

def player_wins (player, dealer, chips):
      print("Player wins!")
      chips, win bet()

def dealer busts (player, dealer, chips):
      print("Dealer busts!")
      chips.win_bet()

def dealer wins (player, dealer, chips):
      print("Dealer wins!")
      chips.lose_bet()

def push(player, dealer):
      print("Dealer and Player tie! It's a push.")
```

و حالا به بازی میپردازیم!!

In [ ]:

```
while True:
  # Print an opening statement
```

```
print('Welcome to BlackJack! Get as close to 21 as you can without
going over!\n' Dealer hits until she reaches 17. Aces count as 1 or 11.)

 # Create & shuffle the deck, deal two cards to each player
 Deck=Deck()
 deck.shuffle()

 player hand Hand()
 player hand.add_card (deck.deal())
 player_hand.add_card (deck.deal())

dealer hand =Hand())
dealer hand.add_card (deck.deal())
dealer hand.add_card (deck.deal())

 # Set up the Player's chips
 player_chips =Chips()

 # Prompt the Player for their bet
take_bet(player_chips)

#Show cards (but keep one dealer card hidden)
show some(player hand, dealer_hand)

while playing: # recall this variable from our hit or stand function

#Prompt for Player to Hit or Stand
 hit_or_stand (deck, player_hand)

#Show cards (but keep one dealer card hidden)
show some(player_hand, dealer_hand)

# If player's hand exceeds 21, run player busts() and break out of   Loop.
 if player_hand.value> 21:
     player_busts(player_hand, dealer_hand, player_chips)
    break

```

```
40
41  # If Player hasn't busted, play Dealer's hand until Dealer reaches 17
42  if player_hand.value <= 21:
43
44          while dealer hand. value < 17:
45              hit(deck, dealer_hand)
46
47          # Show all cards
48          show_all(player_hand, dealer_hand)
49
50          # Run different winning scenarios
51          if dealer hand.value > 21:
52              dealer busts (player_hand, dealer_hand, player_chips)
53
54          elif dealer_hand.value> player_hand.value:
55              dealer wins (player hand, dealer_hand, player_chips)
56
57          elif dealer_hand.value < player_hand.value:
58              player wins (player_hand, dealer_hand, player_chips)
59
60          else:
61              push(player_hand, dealer_hand)
62
63      # Inform Player of their chips total
64      print("\nPlayer's winnings stand at", player_chips.total)
65
66      #Ask to play again
67      new game = input("Would you like to play another hand? Enter 'y'
68  or 'n")
69          if new game[e].lower()=='y':
70              playing =True
71              continue
72          else:
73              print("Thanks for playing!")
74              break
```

Welcome to BlackJack! Get as close to 21 as you can without going over!

Dealer hits until she reaches 17. Aces count as 1 or 11.

How many chips would like to bet? 42

Dealer's Hand:

<card hidden> Five of Clubs

Player's Hand:

King of Clubs. Ace of Hearts

Dealer's Hand:

Ten of Hearts

Five of Clubs Queen of Diamonds

Dealer's Hand 25

Player's Hand: King of Clubs

Ace of Hearts

Player's Hand 21 Dealer busts!

Player's winnings stand at 142

Would you like to play another hand? Enter 'y' or 'n' y Welcome to BlackJack! Get as close to 21 as you can

without going over! Dealer hits until she reaches 17. Aces count as 1 or 11.

How many chips would like to bet? 40

Dealer's Hand: <card hidden>

Three of Hearts

Player's Hand:

Jack of Spades King of Spades

Would you like to Hit or Stand? enter 'h' or 's's Player stands. Dealer is playing.

Dealer's Hand:

<card hidden>

Three of Hearts

Player's Hand: Jack of Spades

King of Spades

Dealer's Hand:

Queen of Hearts

Three of Hearts

Jack of Diamonds Dealer's Hand 23

Player's Hand:

Jack of Spades King of Spades

Player's Hand 28

Dealer busts!

Player's winnings stand at 148.

Would you like to play another hand? Enter 'y' or 'n' n Thanks for playing!

تمام شد به یاد داشته باشید که این مراحل ممکن است با راه حل خودتان به طور قابل توجهی متفاوت باشد. این مهم نیست تا زمانی که نتیجه مورد نظر را بدست نیاورید روی بخش های مختلف برنامه ای خودکار کنید. این مسئله نیازمند زمان و صبر زیادی است همیشه از پرسش ها و نظرات خود در انجمن های پرسش و پاسخ استفاده کنید.

خدا قوت

ecorators

دکوراتورها میتوانند به عنوان توابعی در نظر گرفته شوند که قابلیت تغییر عملکرد تابع دیگری را دارند. آنها به کوتاه تر و بیشتر پایتونی شدن کد شما کمک میکنند.

برای توضیح مناسب دکوراتورها، ما از توابع شروع کرده و به آرامی آنها را گسترش خواهیم داد اطمینان حاصل کنید که تمام سلول های این نوت بوک را اجرا کنید تا این درس در رایانه شما به همان شکل نشان داده شود.

توابع

قبل از اینکه بتوانید دکورانورها را درک کنید باید ابتدا نحوه عملکرد توابع را درک کنید برای مقاصد ما، یک تابع بر اساس آرگومان های داده شده یک مقدار را برمی گرداند. در ادامه یک مثال بسیار ساده وجود دارد:

In [1]:

```
1  def add_one(number) :
2      return number + 1
3
4  add_one(5)
```

out[1] :

6

در کل توابع در پایتون ممکن است تأثیرات جانبی داشته باشند به جای اینکه فقط یک ورودی را به خروجی تبدیل کنند. تابع ()*print* مثالی ابتدایی در این مورد است آن مقداری را برگردانده و در عین حال تأثیر جانبی خروجی دادن چیزی به کنسول دارد. با این حال برای درک ،دکوراتورها کافی است به توابع به عنوان چیزی که آرگومان های داده شده را به یک مقدار تبدیل میکنند، فکر کنید.

First-Class Objects

در پایتون توابع یک قسمت از اولین کلاس ها هستند. این به این معنی است که توابع میتوانند به عنوان آرگومان به تابع دیگری منتقل شوند و مانند هر شی دیگری ،رشته عدد صحیح عدد اعشاری لیست و غیره استفاده شوند. به مثال زیر توجه کنید:

In [2]:

```
1  def say_hello(name) :
2      return "Hello {name}"
3
4  def be awesome (name):
5      return "Yo {name), together we are the awesomest!"
6
7  def greet_bob(greeter_func)
8      return greeter_func("Bob")
```

In [3]:

```
1  type(say_hello)
```

Out[3] :

Function

In [5]:

```
1  ss = say_hello
```

In [7]:

```
1  say hello('John')
```

out[7] :

'Hello John'

In [8]:

```
1  ss('John')
```

Out [8]:

'Hello John'In

[ ]:

```
1 add lambda x, y: x+y
```

در اینجا توابع *say_hello*() و *be_awesome* () توابع معمولی هستند که یک نام به عنوان ورودی دریافت میکنند. اما تابع *greet_bob*() توقع دارد یک تابع را به عنوان آرگومان خود دریافت کند مثلاً میتوانیم آن را به عنوان آرگومان تابع *say_hello*() یا *be_awesome*() منتقل کنیم

In [9]:

```
1 greet_bob(say_hello)
```

Out [9]:

'Hello Bob'

In [10]:

```
1 greet_bobbe_awesome )
```

Out [10]:

'Yo Bob, together we are the awesomest!'

توجه کنید که *greet_bob(say_hello*) به دو تابع ارجاع دارد اما به روشهای متفاوتی تابع *greet_bob*() و *say_hello* تابع *say_hello* بدون پرانتز نامگذاری شده است. این بدان معنی است که فقط یک ارجاع به تابع منتقل میشود و تابع اجرا نمیشود از طرف دیگر تابع *greet_bob*() با پرانتز نوشته شده است بنابراین به طور معمول فراخوانی خواهد شد.

توابع داخلی

ممکن است توابع را در داخل توابع دیگر تعریف کنیم اینگونه توابع را توابع داخلی مینامیم در ادامه مثالی از یک تابع با دو تابع داخلی آورده شده است.

In [11]:

```
1  def parent():
2      print("Printing from the parent() function")
3
4      def first_child():
5          print("Printing from the first_child() function")
6
7      def second_child() :
8          print("Printing from the second child() function")
9
10     second_child()
11     first_child()
```

چه اتفاقی می افتد وقتی تابع ()*parent* را فراخوانی میکنید؟ برای یک دقیقه به این موضوع فکر کنید. خروجی به شرح زیر خواهد بود:

In [12]:

```
1  parent()
```

Printing from the parent() function

Printing from the second child() function

Printing from the first_child() function

توجه کنید که ترتیب تعریف توابع داخلی اهمیتی ندارد مانند هر تابع دیگری چاپ فقط در زمانی انجام میشود که توابع داخلی اجرا می شوند.

علاوه بر این توابع داخلی تا زمانی که تابع والد(*parent( )*)فراخوانی نشده باشد فرزندان تعریف نمیشوند. آنها به عنوان متغیرهای محلی در ()*parent* وجود دارند تلاش کنید تابع ()*first_child* را فراخوانی کنید. باید خطایی دریافت کنید:

In [13]:

```
1  first_child()
```

NameError

Traceback (most recent call last)

Cell In[13], line 1

----> 1 first_child()

NameError: name first_child' is not defined

هربار که تابع *parent*() را فراخوانی میکنید توابع داخلی *first_child*() و *second_child*() نیز فراخوانی میشوند. اما به دلیل اینکه دارای محدوده محلی هستند،بیرون تابع از تابع *parent()* در دسترس نیستند.

برگرداندن توابع از داخل توابع

پایتون به شما اجازه میدهد تا از توابع به عنوان مقادیر برگشتی استفاده کنید مثال زیر یکی از توابع داخلی را از تابع *parent*() برمی گرداند.

In [14]:

```
def parent (num):
    def first_child() :
        return "Hi, I an Emma"

    def second_child():
        return "Call ne Liam"

    if num mm 1:
        return first_child
    else:
        return second child
```

توجه کنید که تابع *first_child* را بدون پرانتز برمیگردانید به یاد داشته باشید که این بدان معناست که شما یک ارجاع به تابع *First_child* را برمی گردانید در مقابل *first_child*() با استفاده از پرانتز به نتیجه ارزیابی تابع اشاره میکند. این میتواند در مثال زیر مشاهده شود:

In [15]:

```
first = parent(1)
```

In [16]:

```
1 second = parent(2)
```

In [18]:

```
1 parent
```

out[18] :

<function__main__.parent(num) >

In [17]:

```
1 first
```

out[17] :

<function _main_.parent.<locals>.first_child()>

In [19]:

```
1 second
```

Out[19] :

<function_main_parent, clocals).second_child())

خروجی نسبتاً رمزآمیز به این معناست که متغیر اول به تابع محلی *first_child* داخل *parent* اشاره دارد، در حالی که متغیر دوم به تابع *second_child* اشاره دارد.

اکنون میتوانید از *second, first* به به عنوان توابع عادی استفاده کنید اگرچه توابعی که به آنها اشاره دارند نمیتوانند صورت مستقیم دسترسی داشته باشند:

In [20]:

```
1 first()
```

Out[20] :

'Hi, I am Enma'

In [21]:

```
1 second()
```

Out[21] :

'Call me Liam'

در نهایت توجه کنید که در مثال قبل توابع داخلی را در داخل تابع والد (*parent*) اجرا کرده بودید، به عنوان مثال () *first_child*.با این حال در این مثال ،آخر شما پرانتز را به توابع داخلی (*first_child*)

در هنگام برگشت برنامه اضافه نکرده اید به این ترتیب شما یک ارجاع به هر تابع بدست آورده اید که در آینده میتوانید آن را فراخوانی کنید. منطقی است؟

دکوراتورهای ساده

اکنون که دیدید توابع همانند هر شی دیگری در پایتون هستند آماده اید به سمت جادوگری بروید که دکوراتور پایتون است. همین الان با یک مثال شروع کنیم:

In [29]:

```
1  def my decorator (func) :
2      def wrapper():
3          print("Something is happening before the function is called.")
4          func()
5          print("Something is happening after the function is called.")
6      return wrapper
7
8  def say whee():
9      print("Whee!")
10 def be awesome():
11     print("Yo, together we are the awesomest!")
12
13
14 be_awesome = my_decorator (be_awesome)
15 say_whee = my_decorator(say_whee)
16
```

میتوانید حدس بزنید که چه اتفاقی می افتد وقتی تابع *say whee* را صدا میزنید؟ امتحان کنید:

In [30]:

```
1 say_whee()
```

Something is happening before the function is called.

Whee!

Something is happening after the function is called.

برای درک آنچه در اینجا اتفاق میافتد به مثال های قبلی نگاهی بیندازید. به طور کامل همان چیزی که تاکنون یاد گرفته اید را به کار میگیریم. دکوراسیون مورد نظر در خط زیر رخ میدهد:

In [ ]:

```
1 say_whee() = my_decorator(say_whee)
```

در واقع نام *say_whee* اکنون به تابع داخلی *wrapper* اشاره میکند. به یاد داشته باشید که وقتی *my_decorator(say_whee*) را صدا میزنید تابع *wrapper* را به عنوان یک تابع برمی گردانید.

In [31]:

```
1 say_whee()
```

Out[31]:

<function _main_.my_decorator.clocals.wrapper()>

با این حال تابع *wrapper* یک ارجاع به تابع اصلی *say whee* را به عنوان *func* دارد و این تابع را بین دو فراخوانی *print* فرا میخواند.

به طور ساده میتوان گفت دکوراتورها تابعی را دربر میگیرند و رفتار آن را تغییر میدهند.

قبل از ادامه نگاهی به مثال دوم بیندازیم به این دلیل که تابع *wrapper* یک تابع معمولی در پایتون است. شیوه ای که دکوراتور رفتار یک تابع را تغییر میدهد به طور پویا میتواند تغییر کند به منظور از همسایگانتان خیلی آزرده نشوید، در مثال زیر تنها در طول روز کد دکوره شده اجرا میشود:

In [32]:

```
from datetime import datetime

def not_during_the_night (func) :
    def wrapper():
        if 7 <= datetime.now().hours 22:
            func()
        else:
            pass #Hush, the neighbors are asleep
    return wrapper

def say whee():
    print("Wheel")

say_whee = not_during_the_night(say_whee)
```

اگر بعد از خوابیدن همسایه ها تابع *say_whee* را فراخوانی کنید هیچ اتفاقی نخواهد افتاد:

In [33]:

```
say whee()
```

whee!

***Syntactic Sugar*!**

توابع را میتوان با استفاده از سینتکس شکرآمیزتری دکوره کرد. ابتدا باید به یاد داشته باشید که در این حالت نام تابع را سه بار تایپ میکنید به علاوه دکوراسیون یکم در زیر تعریف تابع پنهان میشود.

در عوض پایتون به شما اجازه میدهد تا از دکوراتورها با استفاده از نماد @ که گاهی به آن نماد "پای" میگویند، به ساده ترین شکل ممکن استفاده کنید مثال زیر دقیقاً همان کاری را انجام میدهد که در مثال اول دکوراتور صورت گرفت:

In [ ]:

```
def my decorator (func):
     def wrapper():
         print("Something is happening before the function is called.")
         func()
         print("Something is happening after the function is called.")
    return wrapper


# say whee my decorator (say_whee)
@my_decorator
def say_whee()
    print("Whee!")
```

بنابراین *Emy_decorator* تنها یک راه آسان برای گفتن

say_whee = my_decorator(say_whee) است. این روشی است که یک دکوراتور را روی یک تابع اعمال میکنید.

استفاده مجدد از دکوراتورها

بیاد آورید که یک دکوراتور فقط یک تابع معمولی در پایتون است. تمام ابزارهای معمول برای قابلیت استفاده مجدد آسان در دسترس هستند. بیایید دکوراتور را به یک ماژول جداگانه منتقل کنیم که در بسیاری از توابع دیگر قابل استفاده باشد. یک فایل با نام *decorators.py* ایجاد کنید و محتوای زیر را در آن قرار دهید:

In [ ]:

```
# decorators.py


def do twice(func):
    def wrapper_do_twice():
        func()
       func()
   return wrapper_do_twic
```

توجه میتوانید نام تابع داخلی را هر چیزی که میخواهید بگذارید و نام عمومی مانند *wrapper*() به طور معمول قابل قبول است. شما در این مقاله بسیاری از دکوراتورها را خواهید دید برای تمیز نگه داشتن آنها ما نام تابع داخلی را با همان نام دکوراتور اما با پیشوند *wrapper* نامگذاری میکنیم.

حالا میتوانید از این دکوراتور جدید در فایل های دیگر با استفاده از *import* عادی استفاده کنید:

In [34]:

```
1  from decorators import do_twice
2
3  @do_twice
4  def say whee():
5      print("Whee!")
```

هنگام اجرای این مثال باید ببینید که تابع اصلی *say whee*() دو بار اجرا میشود.

In [35]:

```
1  say whee()
```

Whee!

Wheel

Decorating Functions With Arguments

فرض کنید یک تابع دارید که چندین آرگومان را میپذیرد آیا هنوز میتوانید آن را دکور کنید؟ بیایید امتحان کنیم:

In [36]:

```
1  from decorators import do_twice
2
3  Edo_twice
4  def greet (name):
5      print("Hello {name}")
```

متأسفانه اجرای این کد خطا را برمی گرداند:

In [37]:

```
1 greet("World")
```

TypeError

Traceback (most recent call last)

Cell In[37], line 1

---> 1 greet("world")

TypeError: do_twice.<locals>.wrapper_do_twice() takes @ positional argumen ts but 1 was given

مشکل این است که تابع داخلی *wrapper_do_twice*() هیچ آرگومانی را نمی پذیرد اما *name="world"*به آن ارسال شده است. میتوانید این مشکل را با اینکه به *wrapper_do_twice*() اجازه دهید یک آرگومان را بپذیرد، رفع کنید، اما در این صورت برای تابع *say whee*() که قبلاً ایجاد کرده بودید کار نخواهد کرد.

راه حل این استفاده از* *args* و ***kwargs* در تابع داخلی *wrapper* است. در این صورت این تابع تعداد خاصی از آرگومان های موقعیتی و کلمه ای را پذیرفته و آنها را به تابعی که دکور میکند منتقل میکند به عنوان مثال *decorators.py* را به صورت زیر بازنویسی کنید:

In [ ]:

```
1 def do_twice(func):
2     def wrapper_do_twice(*args, **kwargs):
3         func(*args, **kwargs)
4         func(*args, **kwargs)
5     return wrapper_do_twice
```

تابع داخلی *wrapper_do_twice()* در حال حاضر تعداد دلخواهی از آرگومان ها را پذیرفته و آنها را به تابعی که دکور میکند منتقل میکند حالا هر دو مثال *greet() say nhee*() کار می کنند.

In [1]:

```
from decorators import do_twice

@do_twice
def say whee():
    print("Whee!")

@do_twice
def greet (name):

    print(f"Hello fname}")
```

In [2]:

```
say whee()
```

Whee!

Whee!

In [3]:

```
greet("World")
```

Hello World

Hello World

In [4]:

```
greet
```

Out[4]:

*<function decorators.do_twice.<locals>.wrapper_do_twice(*args, **kwargs)>*

بازگرداندن مقادیر از توابع دکوراتور شده

مقدار بازگشتی از توابع تدکوراتور شده چه اتفاقی می افتد؟ خب این بستگی به دکوراتور دارد. فرض کنید یک تابع ساده را به شکل زیر دکوراتور کنید:

In [1]:

```
1  from decorators import do_twice
2
3  @do_twice
4  def return_greeting(nane):
5      print("Creating greeting")
6      return "Hi (name)"
```

In [2]:

```
1  hi_adam = return_greeting( "Adan")
```

```
Creating greeting

Creating greeting
```

In [3]:

```
1  print(hi_adan)
```

```
None
```

اوه دکوراتور شما مقدار بازگشتی تابع را خورده است.

زیرا تابع *do_twice_wrapper()* بخیه صورت صریح یک مقدار را بازنمی گرداند. فراخوانی *return greeting("Adam")* بازگشتی از *None* داشت.

برای رفع این مشکل باید مطمئن شوید که تابع *wrapper* مقدار بازگشتی تابع دکوراتور شده را برمی گرداند. فایل *decorators.py* را تغییر دهید:

In [ ]:

```
def do_twice(func):
    def wrapper_do_twice(*args, **kwargs):
        func(*args, **kwargs)
        return func(*args, **kwargs)
    return wrapper_do_twice
```

In [1]:

```
from decorators import do twice

@do_twice
def return greeting (name):
    print("Creating greeting")
    return "Hi (name)"
```

مقدار بازگشتی از آخرین اجرای تابع برگردانده می شود.

In [2]:

```
return greeting("Adam")
```

Creating greeting

Creating greeting

Out[2]:

"Hi Adam"

مشخص کردن منشا یک تابع

یکی از مزایای بزرگ کار با زبان پایتون به ویژه در محیط تعاملی آن قابلیت تفحص قدرتمند آن است. تفحص قابلیت یک شی است که به شیء مربوطه اجازه میدهد تا در زمان اجرا درباره ویژگی های خود اطلاعاتی داشته باشد. به عنوان مثال، یک تابع نام و مستندات خود را میشناسد:

In [3]:

```
print
```

Out[3]:

<function print>

In [4]:

```
1 print._name_
```

Out[4]:

'print'

In [5]:

```
1 help(print)
```

Help on built-in function print in module builtins:

print(...).

t.

print(value,..., sep=', end='\n', file-sys.stdout, flush=False)

Prints the values to a stream, or to sys.stdout by default. Optional keyword arguments:

file: a file-like object (stream); defaults to the current sys.stdou

sep: string inserted between values, default a space.

end:

string appended after the last value, default a newline. flush: whether to forcibly flush the stream.

قابلیت تفحص برای توابعی که خودتان تعریف میکنید نیز کار میکند:

In [6]:

```
1 @do_twice
2 def say whee():
3     print("Whee!")
4
5 say whee
```

Out[6]:

*<function decorators.do_twice.<locals>.wrapper_do_twice(*args, **kwargs)>*

In [7]:

```
1 say whee._name,
```

Out[7]:

'wrapper_do_twice'

In [8]:

```
1 help(say_whee)
```

Help on function wrapper_do_twice in module decorators:

wrapper_do_twice(*args, **kwargs)

با این حال پس از دکوراتور تابع *say whee*() بسیار گیج شده است درباره هویت خود اکنون به عنوان تابع داخلی *wrapper_do_twice*() درون دکوراتور *do_twice*() اعلام میشود اگرچه این اصولاً درست است، اما اطلاعات خیلی مفیدی نیست. برای رفع این مشکل دکوراتور کننده ها باید از دکوراتور کننده *functools wrapse* استفاده کنند که اطلاعات درباره تابع اصلی را حفظ میکند دوباره فایل *decorators.py* را به روز کنید

In [9]:

```
1  import functools
2
3  def do_twice(func):
4      @functools.wraps(func)
5      def wrapper_do_twice (*args, **kwargs):
6          func(*args, **kwargs)
7          return func(*args, **kwargs)
8      return wrapper_do_twice
```

شما نیازی به تغییر هیچ چیز در تابع دکوراتور شده say_whee() ندارید:

In [1]:

```
1  from decorators import do_twice
2
3  @do_twice
4  def say whee():
5      print("Whee!")
6
7  say_whee
```

Out[1] :

<function_main_say_whee())

خیلی بهتر! حالا say_whee() پس از تزئین همچنان خودش است.

In [2]:

```
1  say_whee._name
```

Out[2] :

'say_whee'

In [3]:

```
1  help(say_whee)
```

Help on function say whee in module_main_ :

say_whee()

چند مثال واقعی

بیایید به چند مثال مفید دیگر از دکوراتورها نگاهی بیندازیم خواهید دید که آنها اغلب الگویی مشابه با آنچه تاکنون یاد گرفته اید را دنبال میکنند

این الگوی قالب بسیار خوبی برای ساخت دکوراتورهای پیچیده تر است.

توجه در مثال های بعدی فرض میکنیم این دکوراتورها را در فایل *decorators.py* خود ذخیره کرده اید. به یاد داشته باشید که میتوانید تمام مثال ها در این آموزش را دانلود کنید.

تایمینگ توابع

بیایید با ایجاد یک دکوراتور با نام *@timer*شروع کنیم این دکوراتور زمانی که یک تابع را اجرا میکند زمان اجرای آن را اندازه گیری کرده و مدت زمان را در کنسول چاپ میکند. کد زیر را ببینید:

In [5]:

```
1  import functools, time
2
3  def timer (func):
4     """print the runtime of the decorated function"""
5     @functools.wraps(func)
6     def wrapper_timer (*arg, **kwargs):
7        start_time = time.perf_counter()
8        value= func(*arg, **kwargs)
9        end_time = time.perf_counter()
10        run_time = end_time - start_time
11        print("Finished [func._name_Ir) in (run time:.4f) secs")
12        return value
13     return wrapper_timer
14
15
16
17 @timer
18 def waste_some_time(num_times):
19      for in range(num_times):
20           Sun([1 ** 2 for 1 in range(10000)])
```

این دکوراتور با ذخیره زمان در لحظه شروع اجرای تابع(در خط مشخص شده با # *1)* و زمان پایان آن (در خط مشخص شده با #۲) کار میکند مدت زمانی که تابع طول میکشد تفاوت بین دو زمان است(در خط مشخص شده با # ۳). ما از تابع *time.perf_counter()* استفاده میکنیم که عملکرد خوبی در اندازه گیری فواصل زمانی دارد. در زیر چند نمونه از زمان ها را مشاهده میکنید:

In [6]:

```
1 waste_some_time(1)
```

Finished waste_some_time in 0.0028 secs

In [7]:

```
1 waste_some_time(999)
```

Finished waste_some_time in 2.7858 secs

In [8]:

```
1 waste_some_tine
```

Out[8] :

<function main.waste_some_time(num_times)

با اجرای آن به صورت خودکار کد را خط به خط اجرا کنید. اطمینان حاصل کنید که نحوه عملکرد آن را متوجه شده اید. اگر نتوانستید آن را درک کنید نگران نباشید دکوراتورها موجودات پیشرفته ای هستند. سعی کنید روی نحوه عملکرد آنها بیشتر تمرکز کنید یا یک *flowchart* برنامه را بکشید.

توجه دکوراتور *timer* عالی است اگر شما فقط میخواهید یک ایده از زمان اجرای توابع خود داشته باشید. اگر می خواهید اندازه گیری های دقیق تری از کد خود انجام دهید به جای آن باید ماژول *timeit* را در کتابخانه استاندارد در نظر بگیرید این ماژول گردآوری زباله را به طور موقت غیر فعال میکند و چندین آزمایش اجرا میکند تا نویز از *call*های سریع تابع را حذف کند.

اشکال زدایی کد

دکوراتور ***debug*** زیر هر بار که تابعی فراخوانی میشود آرگومان هایی که با آن فراخوانی شده است و همچنین مقدار بازگشتی آن را چاپ میکند:

In [9]:

```
1 x = [1,2,3]
2  repr(x)
```

Out[9] :

' [1, 2, 3]'

In [11]:

```
1 y = {'age : 100, 'name' : 'John'}
2 repr(y)
```

Out[11] :

"{'age': 100, 'name': 'John'}"

In [12]:

```
#Calling make_greeting(Richard, age=112)
# make greeting' returned 'Whoa Richard! 112 already, you are growing up!'
import functools

def debug(func):
      """print the function signature and return value """
     @functools.wraps(func)
     def wrapper_debug(*args, **kwargs):
         args_repr = [repr(a) for a in args] # ['Richard, "handsome " ]
          kwargs repr= [f"{k}-{v!r}" for k,v in kwargs.items()] # ["age-112", "scors-
     arguments = args_repr + kwargs_repr # ['Richard", "handsome,  age-112","se
      signature = ", ".join(arguments) d Richard, handsome, age=112, scors=[12,29

      print("Calling (func._name_}({signature})")
     value = func(*args, **kwargs)
     print(f" (func. _name_Ir) returned {value!r}")

      return value
   return wrapper_debug
```

امضا توسط اتصال نمایش رشته های مربوط به همه آرگومان ها ایجاد میشود اعداد در لیست زیر با نظرات شماره گذاری شده در که مطابقت دارند

- یک لیست از آرگومان های موقعیتی ایجاد میشود از تابع ()*repr* برای به دست آوردن یک رشته مناسب برای نمایش هر آرگومان استفاده میشود.
- یک لیست از آرگومان های کلیدواژه ای ایجاد میشود عبارت *f-string* هر آرگومان را به صورت کلید مقدار فرمت می کند جایی که مشخص کننده ۱۳ به این معنی است که از تابع ()*repr* برای نمایش مقدار استفاده شده است.
- لیست های آرگومان های موقعیتی و کلیدواژه ای به صورت یک رشته امضا با هر آرگومان که توسط یک کاما جدا شده اند ترکیب میشوند.
- مقدار بازگشتی پس از اجرای تابع چاپ میشود.

بیایید با استفاده از این دکوراتور به یک تابع ساده با یک آرگومان موقعیتی و یک آرگومان کلیدواژه ای اعمال کنیم:

In [13]:

```
1 @debug
2 def make_greeting(nane, age=None):
3     if age is None:
4         return f"Howdy {name}!"
5     else:
6         return "Whoa {name}! {age} already, you are growing up!"
```

توجه کنید که دکوراتور *debug* امضا و مقدار بازگشتی تابع *make_greeting*() را چاپ میکند.

In [14]:

```
make greeting("Benjamin")
```

Calling make_greeting('Benjamin')

'make greeting' returned 'Howdy Benjamin!'

out[14] :

'Howdy Benjamin!'

In [15]:

```
make_greeting("Richard", agen112)
```

Calling make_greeting('Richard', age=112)

'make greeting' returned 'Whoa Richard! 112 already, you are growing up!'
Out[15]:

'Whoa Richard! 112 already, you are growing up!!

In [16]:

```
1 make_greeting (name="Dorrisile", age=116)
```

Calling make_greeting (name="Dorrisile', age-116)

'make greeting' returned 'Whoa Dorrisile! 116 already, you are growing u p!'

out[16] :

'Whoa Dorrisile! 116 already, you are growing up!!

این مثال در ابتدا به ظاهر مفید نمی آید زیرا دکوراتور *@debug*فقط تکرار میکند آنچه که به تازگی نوشته اید. اما وقتی به توابع کمکی کوچکی اعمال شود که به طور مستقیم خودتان فراخوانی نمی کنید، قدرتمندتر است.

مثال زیر یک تقریب از ثابت ریاضی را محاسبه میکند:

In [18]:

```
1  import math
2  #from decorators import debug
3
4  # Apply a decorator to a standard Library function
5  math.factorial = debug(math.factorial)
6
7  def approximate_e(terns=18) :
8     return sun(1/math. factorial (n) for n in range(terms))
```

این مثال نشان میدهد چگونه میتوانید یک دکوراتور را به یک تابعی که قبلاً تعریف شده است اعمال کنید. تقریب و براساس گسترش سری زیر است:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \cdots = \frac{1}{1} + \frac{1}{1} + \frac{1}{1 \cdot 2} + \cdots$$

هنگام فراخوانی تابع *approximate_e*() ، میتوانید دکوراتور *debug* در حال کار را ببینید.

In [19]:

```
1  approximate_e(10)
```

Calling factorial(0)

'factorial returned 1

Calling factorial(1)

'factorial' returned 1

Calling factorial(2)

'factorial returned 2

Calling factorial(3)

'factorial returned 6

Calling factorial(4)

'factorial' returned 24

Calling factorial(5)

'factorial returned 120

Calling factorial(6)

'factorial' returned 720

Calling factorial(7)

'factorial returned 5040

Calling factorial (8)

'factorial returned 40320

Calling factorial(9)

'factorial' returned 362880

Out[19] :

2.7182815255731922

در این مثال، تقریب مناسبی از مقدار واقعی ۲٫۷۱۸۲۸۱۸۲۸ = *e* حاصل میشود با اضافه کردن تنها ۵ عبارت.

In [22]:

```
1  import functools, time
2
3  def slow down (func):
4      """sleep 1 second before calling the function"""
5      @functools.wraps (func)
6       def wrapper_slow_down (*args, **kwargs):
7            time.sleep(1)
8           return func(*args, **kwargs)
9       return wrapper_slow_down
10
11 @slow down
12 def countdown (from_number):
13     if from number < 1:
14          print('Liftoff')
15     else:
16         print (from_number)
17         countdown (from_number - 1)
```

برای دیدن تأثیر دکوراتور *slow_down* واقعاً باید مثال را خودتان اجرا کنید:

In [24]:

```
1  countdown(10)
```

10

9

8

7

6

5

4

3

2

1

Liftoff

دکوراتور *Aslow_down* همیشه به مدت یک ثانیه تاخیر میکند بعدها خواهید دید که چگونه با ارسال یک آرگومان به دکوراتور میتوانید نرخ را کنترل کنید.

Fancy Decorators

تا اینجا شما چگونگی ایجاد دکوراتورهای ساده را دیده اید شما در حال حاضر درک خوبی از آنچه دکوراتورها هستند و چگونه کار میکنند دارید به آرامی از این مقاله استفاده کرده و تمرین کردن همه چیزی که یاد گرفته اید را آزادانه انجام دهید. در بخش دوم این آموزش ویژگی های پیشرفته تری را بررسی خواهیم کرد از جمله استفاده از موارد زیر:

- دکوراتورها در کلاس ها
- چندین دکوراتور بر روی یک تابع یا دکوراتورهای تودرتو
- دکوراتورها با آرگومان ها
- دکوراتورهایی با وضعیت کلاس ها به عنوان دکوراتورها
- دکوراتور کلاس ها

دو روش متفاوت برای استفاده از دکوراتورها در کلاس ها وجود دارد اولین روش بسیار شبیه به آنچه که قبلاً با توابع انجام داده اید است شما میتوانید متدهای یک کلاس را تزئین کنید این یکی از دلایل معرفی دکوراتورها در گذشته بود. تعدادی از دکوراتورهای معمول استفاده شده که در واقع در پایتون داخلی هستند عبارتند از *@classmethod*یا *@staticnethod*و . *@property*دکوراتورهای *classmethod* و *static method* برای تعریف متدها در فضای نام یک کلاس استفاده میشوند که به هیچ نمونه خاصی از آن کلاس متصل نیستند دکوراتور *gproperty* برای سفارشی سازی *Better* و *setter* برای ویژگی های کلاس استفاده میشود به مثال زیر برای استفاده از این دکوراتورها مراجعه کنید.

بیایید یک کلاس را تعریف کنیم که برخی از متدهای آن را با استفاده از دکوراتورهای *debug* و *timer* از قبل تعریف شده دکور کنیم

In [1]:

```
from decorators import debug, timer

class Timewaster:

    @debug
    def init_(self, max_num):
          self.nax_num = max_num

    @timer
    def waste_time(self, num_times):
        for in range(num_times):
              Sum([1 ** 2 for 1 in range(num_times)])
```

با استفاده از این کلاس تأثیر دکوراتورها را مشاهده میکنید:

In [2]:

```
tw = TimeWaster(1000)
```

Calling init__init__(_main_.TimeWaster object at 8x8000025C1972FFD0), 1000) returned None

In [3]:

```
tw.waste time (999)
```

Finished 'waste time' in 0.2529 secs

روش دیگری برای استفاده از دکوراتورها در کلاسها دکوراتور کل کلاس است. این کار به عنوان مثال در ماژول*dataclasses* جدید در پایتون ۳٫۷ انجام میشود:

In [ ]:

```
from dataclasses import dataclass

# Playing Card = dataclass (PlayingCard)
@dataclass
class PlayingCard:
   rank: str
    suit: str
```

معنای دستور نحوی مشابه دکوراتورهای تابع است در مثال بالا، شما میتوانستید با نوشتن

PlayingCard =(dataclass(PlayingCard ، تزئین کنید. استفاده رایجی از دکوراتورهای کلاس جایگزینی ساده تری برای برخی موارد استفاده از متاکلاس ها است. در هر دو مورد، شما در حال تغییر تعریف یک کلاس به صورت پویا هستید.

نوشتن یک دکوراتور کلاس بسیار شبیه به نوشتن یک دکوراتور تابع است. تنها تفاوت این است که دکوراتور یک کلاس را به عنوان آرگومان دریافت میکند و نه یک تابع در واقع همه دکوراتورهایی که در بالا ،دیدید میتوانند به عنوان دکوراتورهای کلاس عمل کنند. هنگام استفاده از آنها بر روی یک کلاس به جای یک تابع تأثیر آنها ممکن است همانطور که میخواهید نباشد. در مثال زیر دکوراتور *timer* بر روی یک کلاس اعمال میشود:

In [4]:

```
1  from decorators import timer
2
3  timer
4  class TimeWaster:
5
6      def_init_(self, max_num):
7          self.nax_num = max_num
8
9      def waste_time(self, num_times):
10         for.in range(num_times):
11             sum([12 for i in range(num_times)])
```

تزئین یک کلاس متدهای آن را تزئین نمیکند. به یاد داشته باشید که *@timer*فقط اختصاری برای

Tinemaster = timer(TimeWaster) است.

در اینجا *@timer*فقط زمانی را که برای نمونه سازی کلاس لازم است، اندازه گیری میکند.

In [5]:

```
1  Tw = Timewaster (1000)
```

In [6]:

```
1  tw.waste_time(999)
```

بعدها یک مثال از تعریف یک دکوراتور کلاس مناسب به نام *singleton* را خواهید دید که اطمینان میدهد تنها یک نمونه از یک کلاس وجود داشته باشد.

دکوراتورهای تودرتو

In [11]:

```
1  from decorators import debut, do_twice
2
3  @debug
4  @do twice
5  def greet (name):
6      print(f"Hello {name}")
7
8  # greet do_twice(greet)
9  #greet debug(do_twice (greet))
```

لطفاً در نظر بگیرید که دکوراتورها به ترتیبی که در لیست آنها قرار دارند اجرا میشوند به عبارت دیگر، *adebug* تماس می گیرد با *Bdo_twice* که تماس میگیرد با *greet*() ، به عبارت دیگر *debug(do_twice(greet :*()))

In [12]:

```
1  greet("Eva")
```

Calling greet('Eva')

Hello Eva

Hello Eva

'greet returned None

تفاوت را مشاهده کنید اگر ترتیب *debug* و *@do_twice*را تغییر دهیم:

In [13]:

```
from decorators import debug, do_twice

@do twice
@debug
def greet (name):
    print("Hello (nane)")
```

در این حالت @*do_twice*همچنین بر روی *debug* اعمال خواهد شد.

In [14]:

```
greet("Eva")
```

Calling greet('Eva')

Hello Eva

'greet' returned None Calling greet('Eva') Hello Eva

'greet' returned None.

ارسال آرگومان به دکوراتورها

گاهی اوقات ارسال آرگومانها به دکوراتورهای شما مفید است. به عنوان مثال *Bdo_twice* میتواند به یک دکوراتور (*repeat(num_times* گسترش یابد تعداد بار اجرای تابع تزئین شده را میتوان به عنوان آرگومان ارائه کرد.

این به شما امکان میدهد که مانند زیر عمل کنید:

In [ ]:

```
@repeat(num_times=4)
def greet (name):
    print(f"Hello nane}"
```

In [ ]:

```
greet("World")
```

In [15]:

```
1  import functools
2
3  def repeat (num_times):
4
5      @functools.wraps(func)
6      def wrapper_repeat (*arg, **kwargs):
7          for _ in range(num_times):
8              value func(*args, **kwargs)
9          return value
10     return wrapper_repeat
11
```

In [16]:

```
1  grepeat(num_tines=4)
2  def greet (name):
3      print("Hello {name}")
```

NameError

Traceback (most recent call last)

Cell In[16], line 1

----> 1 repeat(nun_times-4)

2 def greet (name):

3 print("Hello (name)")

Cell In[15], line 5, in repeat (num_times)

3 def repeat(num_tines):

```
---> 5 @functools.wraps (func)
      6  def wrapper_repeat (*arg, kwargs):
      7        for in range(num_times):
      8              value func(*args, kwargs)
NameError: name 'func' is not defined
```

درباره اینکه چگونه این کار را انجام دهید، فکر کنید.

تا به حال نام نوشته شده پس از ۴ به یک شیء تابع اشاره کرده است که میتواند با یک تابع دیگر فراخوانی شود. برای انطباق، نیاز دارید (۴-*repeat(num_times* یک شی تابع را برگرداند که به عنوان یک دکوراتور عمل کند. خوشبختانه، شما در حال حاضر میدانید چگونه توابع را برگردانید به طور کلی شما چیزی شبیه به زیر را میخواهید:

In [ ]:

```
def repeat (num_times):
    def decorator_repeat (func) :
      … # Create and return a wrapper function
    return decorator_repeat
```

معمولا دکوراتور یک تابع پوششی را ایجاد و بر می گرداند بنابراین نوشتن مثال را به طور کامل درون یک *def* اضافی قرار میدهیم که آرگومان های دکوراتور را رسیدگی میکند با یک تابع درونی بیشتر شروع کنیم:

In [17]:

```
def repeat (num_times):
    def decorator_repeat (func):
        @functools.wraps(func)
        def wrapper_repeat (*args, **kwargs):
            for _ in range(nun_times):
                value= func(*args, **kwargs)
            return value
        return wrapper_repeat
    return decorator_repeat
```

این کمی پریشان کننده به نظر میرسد اما ما تنها الگوی دکوراتور مشابهی که تاکنون چندین بار دیده اید را درون یک *def* اضافی قرار داده ایم که آرگومان های دکوراتور را رسیدگی میکند بیایید با تابع درونی شروع کنیم:

In [ ]:

```
1  def wrapper_repeat (*args, **kwargs):
2      for in range(num_times):
3          value= func(*args, **kwargs)
4    return value
```

این تابع *wrapper_repeat*() آرگومان های دلخواهی را میپذیرد و مقدار تابع تزئین شده یعنی *func*() را برمی گرداند. این تابع پوششی همچنین حلقه ای را شامل میشود که تابع تزئین شده را *num_times* بار فراخوانی میکند. این با تفاوتی نسبت به تابع های پوششی قبلی که دیده اید نیست به جز استفاده از پارامتر *num_times* که باید از بیرون ارسال شود.

یک قدم بیرون تر، تابع دکوراتور است:

In [ ]:

```
1  def decorator_repeat (func):
2      @functools.wraps(func)
3      def wrapper_repeat (*args, **kwargs):
4          …
5      return wrapper_repeat
```

دوباره *decorator_repeat*() دقیقاً شبیه به توابع دکوراتوری است که قبلاً نوشته اید به جز اینکه با یک نام متفاوت است. این بدلیل این است که نام پایه را *repeat()* برای تابع بیرونی که کاربر فراخوانی خواهد کرد، رزرو کرده ایم. همانطور که قبلاً دیده اید تابع بیرونی یک ارجاع به تابع دکوراتور را برمی گرداند:

In [ ]:

```
1  def repeat(nun_times):
2      def decorator_repeat(func):
3          …
4    return decorator_repeat
```

در تابع *repeat*() چند نکته ی پنهانی وجود دارد:

- تعریف *decorator_repeat*() به عنوان یک تابع درونی به این معناست که *repeat*() به یک شیء تابع ارجاع خواهد داد - *decorator repeat.* قبلاً از *repeat* بدون پرانتز برای ارجاع به شیء تابع استفاده کردیم. پرانتزهای اضافی هنگام تعریف دکوراتورهایی که آرگومان می پذیرند، ضروری است.
- آرگومان *num_times* به ظاهر در *repeat*() استفاده نمی شود. اما با ارسال *num_times* ، یک بسته بندی ایجاد میشود که مقدار *num_times* تا زمان استفاده بعدی توسط *wrapper_repeat*() ذخیره میشود.

با همه چیز آماده شده بیایید ببینیم آیا نتایج مورد انتظار هستند:

In [18]:

```
1 @repeat (num_times=4)  # 1) > decorator repeat (func)
2                        #2) @decorator_reapte
3                        #3) wrapper_repeat ("arg, **kwargs)
4 def greet (name):
5     print("Hello {name}")
```

In [19]:

```
1 greet("world")
```

Hello World

Hello World

Hello World

Hello World

دکوراتورهای دارای وضعیت *(Stateful Decorators)*

گاهی اوقات داشتن یک دکوراتور که قادر به پیگیری وضعیت باشد مفید است. به عنوان مثال ساده، ما یک دکوراتور ایجاد میکنیم که تعداد بارهایی که یک تابع فراخوانی میشود را شمارش میکند.

توجه در ابتدای این راهنما درباره توابع خالص که بر اساس آرگومان های داده شده مقداری را برمی گردانند صحبت کردیم. دکوراتورهای دارای وضعیت کاملاً متضاد هستند که در آن مقدار بازگشتی به وضعیت کنونی و همچنین آرگومان های داده شده بستگی دارد.

In [20]:

```
1  import functools
2
3  def count_calls(func) :
4      @functools.wraps(func)
5      def wrapper_count_calls (*args, **kwargs):
6          wrapper_count_calls.num_calls + 1
7          print(f"Call (wrapper_count_calls.num_calis} of (func._name_Ir}")
8          return func(*args, **kwargs)
9      wrapper_count_calls.num_calls = 0
10     return wrapper_count_calls
11
12 @count_calls
13 def say whee():
14     print("Whee!")
```

وضعیت - تعداد فراخوانی های تابع در ویژگی *num_calls* روی تابع پوشش داده می.شود اینجا تأثیر استفاده از آن را میبینید:

In [21]:

```
1  say_whee()
```

Call 1 of 'say_whee Whee!

In [22]:

```
1  say_whee()
```

Call 2 of 'say_whee Whee!

In [23]:

```
1  say_whee.num_calls
```

Out[23]:

2

استفاده از کلاس ها به عنوان دکوراتورها

راه معمول برای نگهداری وضعیت استفاده از کلاس ها است. در این بخش خواهید دید که چگونه مثال *@count_calls* را از بخش قبل با استفاده از یک کلاس به عنوان تدکوراتور، بازنویسی کنید.

به یاد داشته باشید که سینتکس دکوراتور *erry_decorator* تنها یک روش آسان تر برای

گفتن *my decorator(func) =func* است. بنابراین اگر *my_decorator* یک کلاس باشد باید *func* را به عنوان آرگومان در متد *init_()* آن دریافت کند علاوه بر این نمونه کلاس باید قابل فراخوانی باشد تا بتواند به جای تابع تزئین شده قرار بگیرد.

برای اینکه یک نمونه کلاس قابل فراخوانی باشد شما باید متد ویژه *call_()* _.را پیاده سازی کنید:

In [24]:

```
1  class Counter:
2      def_init_(self, start = 0):
3          self.count = start
4
5      def_call_(self):
6          self.count + 1
7          print("Current count is (self.count}")
```

متد *call_* _()هر بار که تلاش میکنید یک نمونه از کلاس را صدا بزنید اجرا میشود:

In [25]:

```
1  counter = Counter()
```

In [26]:

```
1  counter()
```

Current count is 1

In [27]:

```
1 counter()
```

Current count is 21

In [28]:

```
1 counter.count
```

out[28] :

2

بنابراین، پیاده سازی معمول برای یک کلاس دکوراتور نیازمند پیاده سازی متد *int*() و *call_*. __() است:

In [31]:

```
1  import functools
2
3  class CountCalls:
4        def_init_(self, func):
5              functools.update_wrapper(self, func)
6             self.num_calls = 6
7             self.func = func
8
9      def _call_(self, *args, **kwargs):
10            self.num_calls += 1
11            print(f"Call (self.num_calls) of (self.func._name_Ir}")
12            return self.func(*args, **kwargs)
13
14 @CountCalls
15 def say_whee():
16     print("Whee!")
```

متد *init* _().باید یک مرجع به تابع را ذخیره کند و میتواند هرگونه مقداردهی اولیه مورد نیاز را انجام دهد. متد *call* _().به جای تابع تزئین شده فراخوانی میشود این تقریباً همان کاری را انجام میدهد که

تابع *wrapper*() در مثال های قبلی ما انجام می دهد. توجه کنید که باید از تابع *functools.update_wrapper*() به جای *functools.wraps@* استفاده کنید.

دکوراتور *countCalls@* همانطور که در بخش قبل استفاده شده است. کار می کند.

In [32]:

```
1 say_whee()
```

Call 1 of 'say_whee Whee!

In [33]:

```
1 say_whee()
```

Call 2 of 'say_whee Whee!

In [34]:

```
1 say_whee.num_calls
```

Out[34] :

2

مثال های دیگر در کاربرد واقعی

تا اینجا راه زیادی را طی کرده ایم و توانسته ایم ببینیم که چگونه میتوانیم انواع دکوراتورها را ایجاد کنیم بیایید آن را به پایان برسانیم و با استفاده از دانش جدید خود چندین مثال دیگری را ایجاد کنیم که ممکن است در دنیای واقعی مفید باشند.

کند کردن کد، دوباره بررسی شود

همانطور که قبلاً توضیح داده شد نسخه قبلی ما از *slow_down* همیشه به مدت یک ثانیه منتظر میماند. اکنون میدانید چگونه پارامترها را به دکوراتورها اضافه کنید بنابراین بیایید *slow_down* را با استفاده از یک آرگومان اختیاری بنویسیم که مدت زمان خواب را کنترل میکند:

In [35]:

```
import functools, time

def slow_down (func = None, *, rate = 1):
    """Sleep given amount of seconds before calling the function"""
    def decorator_slow_down (func):
        @functools.wraps(func)
        def wrapper_slow_down (*args, **kwargs):
            time.sleep(rate)
            return func(*args, **kwargs)
        return wrapper_slow_down
    if _func is None:
        return decorator_slow_down
    else:
        return decorator_slow_down (_func)
```

در اینجا از کد پایه که در بخش قبل معرفی شد استفاده میکنیم تا *slow_down* هم با آرگومان ها و هم بدون آرگومان قابل فراخوانی باشد. تابع بازگشتی *countdown*() مشابه نسخه قبلی است اما حالا دو ثانیه در هر شمارش خوابیده میشوند

In [38]:

```
@slow_down (rate=2)
def countdown (from_number) :
    if from_number < 1:
        print("Liftoff!")
    else:
        print (from_number)
        countdown (from_number -1 )
```

همانند قبل باید نمونه را خودتان اجرا کنید تا تأثیر دکوراتور را ببینید.

In [39]:

```
countdown(3)
```

3

2

1

Liftoff!

ایجاد *Singletons*

یک *singleton* یک کلاس با تنها یک نمونه است در پایتون چندین *singleton* وجود دارد که به طور متداول از آنها استفاده میکنید، از جمله *None True* و *False* این واقعیت که *None* یک *singleton* است به شما اجازه میدهد تا با استفاده از عبارت *is* برای بررسی مساوی بودن با *None* استفاده کنید.

In [ ]:

```
1  if _func is None:
2      return decorator_name
3  else:
4      return decorator_name(_func)
```

استفاده از *is* فقط برای اشیاءی که دقیقاً همان نمونه هستند *True* را برمی گرداند دکوراتور *singleton* زیر را با ذخیره کردن نمونه اول کلاس به عنوان یک ویژگی یک کلاس را به یک *singleton* تبدیل میکند تلاش های بعدی برای ایجاد نمونه به سادگی نمونه ذخیره شده را برمی گردانند:

In [42]:

```
1  import functools
2
3  def singleton (cls):
4      """Make a class a Singleton class (only one instance)"""
5      @functools.wraps (cls)
6
7       def wrapper_singleton (*args, **kwargs):
8              if not wrapper_singleton.instance:
9                     wrapper_singleton. instance cls(*args, **kwargs)
10            return wrapper_singleton. instance
11     wrapper_singleton.instance = None
12      return wrapper_singleton
13
14 @singleton
15 class TheOne:
16      pass
```

همانطور که میبینید این دکوراتور ،کلاس الگویی مشابه دکوراتورهای تابع ما دنبال میکند تنها تفاوت این است که از *cls* به جای *func* به عنوان نام پارامتر استفاده میکنیم تا نشان دهیم که به عنوان یک دکوراتور کلاس قرار داده شده است.

In [43]:

```
1 first_one = Theone()
```

In [44]:

```
1 another_one = Theone()
```

In [45]:

```
1 id(first_one)
```

Out [45]:

2594632824960

In [46]:

```
1 id(another_one)
```

Out [46]:

2594632824960

In [47]:

```
1 first_one is another_one
```

Out [47]:

True

به نظر میرسد که *firstone* واقعاً همان نمونه دیگر است.

توجه: کلاس های *Singleton* در پایتون به طور معمول به اندازه سایر زبان ها استفاده نمیشوند تأثیر یک *singleton* معمولاً بهتر در یک متغیر عمومی در یک ماژول پیاده سازی میشود.

کش کردن مقادیر بازگشتی

دکوراتورها میتوانند یک مکانیزم خوب برای کش کردن *(Caching)* و ذخیره سازی حافظه باشند به عنوان مثال بیایید به تعریف بازگشتی دنباله فیبوناچی نگاهی بیندازیم

۱’۱’۲’۳’۵’۸’۱۳’۲۱

In [51]:

```
count calls
def fibonacci(num):
    if num <2:
        return num
    return fibonacci(num 1) + fibonacci(num 2)
```

اگرچه پیاده سازی ساده است اما عملکرد زمان اجرای آن وحشتناک است.

In [52]:

```
fibonacci(10)
```

```
Call 154 of fibonacci'

Call 155 of fibonacci'

Call 156 of fibonacci'

Call 157 of 'fibonacci'

Call 158 of 'fibonacci'

Call 159 of 'fibonacci'

Call 160 of fibonacci'

Call 161 of fibonacci'

Call 162 of fibonacci'
```

Call 163 of fibonacci'

Call 164 of 'fibonacci'

Call 165 of 'fibonacci'

Call 166 of 'fibonacci'

Call 167 of fibonacci'

In [53]:

```
1 fibonacci.num_calls
```

Out[53]:

177

برای محاسبه عدد دهم فیبوناچی در واقع فقط باید عددهای فیبوناچی قبلی را محاسبه کنید اما این پیاده سازی به یک راه حل عدد ۱۷۷ نیاز دارد. وضعیت بسیار بدتر میشود: ۲۱۸۹۱ محاسبه برای (۲۰) *Fibonacci* و تقریباً ۲.۷ میلیون محاسبه برای عدد سی ام لازم است این به دلیل این است که کد عددهای فیبوناچی را که قبلا محاسبه شده اند، دوباره محاسبه می کند. راه حل معمول این است که از حلقه *for* و یک جدول جستجو برای پیاده سازی عددهای فیبوناچی استفاده کنید. با این حال کش ساده کردن محاسبات نیز میتواند کار را انجام دهد

In [54]:

```
import functools

def cache(func):
    """Keep a cache of previous function calls"""
    @functools.wraps (func)
    def wrapper_cache(*args, **kwargs):
        cache_key= args + tuple (kwargs.items())
        if cache key not in wrapper_cache.cache
            wrapper_cache.cache[cache_key] = func(*args, **kwargs)
        return wrapper_cache.cache[cache_key]
    wrapper_cache.cache dict()
    return wrapper_cache

@cache
@count calls
def fibonacci(num):
    if num < 2:
        return num
    return fibonacci(num 1) fibonacci(num 2)
```

کش به عنوان یک جدول جستجو عمل میکند بنابراین اکنون *Fibonacci* () فقط یکبار محاسبات لازم را انجام میدهد:

In [55]:

```
Fibonacci(8)
```

Call 1 of fibonacci'

Call 2 of fibonacci'

Call 3 of

fibonacci

fibonacci'

Call 4 of

Call 5 of fibonacci'

Call 6 of fibonacci'

Call 7 of

fibonacci'

Call 8 of

fibonacci'

Call 9 of fibonacci'

out[55] :

21

In [56]:

1	*Fibonacci(10)*

Call 10 of fibonacci'

Call 11 of fibonacci'

Out[56]:

55

In [58]:

1	*fibonacci(30)*

Call 22 of fibonacci'

Call 23 of fibonacci'

Call 24 of 'fibonacci'

Call 25 of fibonacci'

Call 26 of 'fibonacci'

Call 27 of fibonacci'

Call 28 of fibonacci'

Call 29 of fibonacci'

Call 30 of fibonacci'

Call 31 of fibonacci

Out [58]:

832040

توجه کنید که در *call* نهایی با *Fibonacci*(*8*) ، محاسبات جدیدی لازم ،نبودند زیرا عدد هشتم فیبوناچی قبلاً برای *Fibonacci*(۱۰) محاسبه شده بود.

در کتابخانه استاندارد یک کش *Least Recently Used (LRU)* با نام *functools.iru_cache* در دسترس است. این دکوراتور دارای ویژگی های بیشتری نسبت به دکوراتور بالا است. بجای نوشتن دکوراتور کش خودتان باید از *functools.iru_cache* استفاده کنید:

In [59]:

```
1  import functools
2
3  functools.lru_cache(maxsize=4)
4  def fibonacci(num):
5      print("Calculating fibonacci((num))")
6      if num < 2:
7          return num
8      return fibonacci(num 1) fibonacci(num 2)
```

پارامتر *maxsize* تعداد *call*های اخیری که کش میشود را مشخص میکند مقدار پیش فرض ۱۲۸ است، اما میتوانید *maxsize None* را مشخص کنید تا تمام *call* های تابع را کش کنید با این حال توجه کنید که اگر تعداد زیادی اشیاء بزرگ را کش کنید این ممکن است باعث مشکلات حافظه شود.

می توانید از روش *cache_info()* برای مشاهده عملکرد کش استفاده کنید و در صورت نیاز آن را تنظیم کنید. در مثال ما از یک *maxsize* مصنوعی کوچک استفاده کردیم تا تأثیر حذف عناصر از کش را مشاهده کنیم.

In [60]:

```
1 fibonacci(10)
```

```
Calculating fibonacci(10)
Calculating fibonacci(9)
Calculating fibonacci(8)
Calculating fibonacci(7)
Calculating fibonacci(6)
Calculating fibonacci(5)
Calculating fibonacci(4)
Calculating fibonacci(3)
Calculating fibonacci(2)
Calculating fibonacci(1)
Calculating fibonacci(0)
```

Out[60]:

```
55
```

In [61]:

```
1 fibonacci(8)
```

Out [61]:

```
21
```

In [62]:

```
1 fibonacci(5)
```

```
Calculating fibonacci(5)

Calculating fibonacci(4)

Calculating fibonacci(3)

Calculating fibonacci(2)

Calculating fibonacci(1)

Calculating fibonacci(0)
```

Out[62]:

5

In [63]:

```
1 fibonacci(8)
```

```
Calculating fibonacci(8)

Calculating fibonacci(7)

Calculating fibonacci(6)
```

Out[63]:

21

In [64]:

```
1 fibonacci(5)
```

Out[64]:

5

خیلی عالی الان شما تقریبا بر تمام مباحث دکوراتورهای پایتون مسلط شدین و به راحتی میتوانین از کتابخانه ها فریمورک های پایتون از جمله *Flask* یا *Django* و بسیاری دیگر استفاده نمایید.

Iterators, Generators

در این بخش از دوره تفاوت بین تکرار و تولید در پایتون را خواهیم آموخت و یاد خواهیم گرفت چگونه تولید کننده های خود را با استفاده از دستور . *yield*بسازیم مولدها به ما امکان میدهند تا در طول اجراء مقادیر را تولید کنیم به جای نگهداری همه چیز در حافظه

قبلاً در بحثی درباره برخی از توابع پیش فرض پایتون مانند ()*range*و()*map* و() *filter* به این موضوع اشاره کرده ایم. بیایید کمی عمیق تر بررسی کنیم یاد گرفتیم که چگونه تابعی با استفاده از دستور *def* و دستور *return* بنویسیم توابع مولد به ما امکان میدهند تا یک مقدار را ارسال کنیم و سپس بعداً به آنجا برگردیم و از جایی که متوقف شده بود ادامه دهیم. این نوع تابع در پایتون یک تولید کننده است که به ما امکان میدهد تا به مرور زمان یک دنباله از مقادیر را تولید کنیم تفاوت اصلی در نحو ساختاری استفاده از دستور *yield* خواهد بود.

از نظر بیشتر جنبه ها، تابع مولد بسیار شبیه یک تابع عادی به نظر خواهد رسید. تفاوت اصلی این است که وقتی یک تابع تولید کننده کامپایل میشود. آنها به شیدی تبدیل میشوند که پروتکل تکرار را پشتیبانی میکند این بدان معناست که وقتی آنها در که شما فراخوانی میشوند در واقع یک مقدار برنمیگردانند و سپس خارج نمیشوند به جای آن، توابع مولد به طور خودکار اجرای خود را متوقف و از نقطه آخر تولید مقدار ادامه میدهند. مزیت اصلی در اینجا این است که به جای محاسبه یک دنباله کامل از مقادیر به صورت پیش فرض مولد یک مقدار را محاسبه کرده و سپس فعالیت خود را متوقف می کند و منتظر دستور بعدی میماند این ویژگی به عنوان متوقف کردن وضعیت شناخته میشود.

برای بهتر درک کردن مولدها بیایید به ساخت آنها بپردازیم.

In [1]:

```
1  def cubes(n):
2      c=[ ]
3      for num in range(n):
4          c.append (ncam * 3)
5      return c
6
7  cubes (20)
```

out[1]:

[0, 1, 8, 27, 64, 125, 216, 343, 512, 729]

In [2]:

```
1  # Generator function for the cube of numbers (power of 3)
2  def gencubes(n):
3      for num in range(n):
4          yield nun **
5          #print(num)
```

In [3]:

```
1  g = gencubes (5)
```

In [4]:

```
1  g
```

out[4] :

<generator object gencubes at exe0e001FB44288190>

In [5]:

```
1  next(g)
```

out[5]:

0

In [6]:

```
1  next(g)
```

out[6]:

1

In [7]:

```
1 next(g)
```

out[7]:

8

In [8]:

```
1 for x in gencubes (30) :
2     print(x)
```

0

1

8

27

64

125

216

343

512

729

خوب حالا که یک تابع مولد داریم نیازی نیست که از هر مکعبی که ایجاد کرده ایم، پیگیری کنیم. مولدها برای محاسبه مجموعه های بزرگی از نتایج به ویژه در محاسباتی که شامل حلقه های خودشان هستند در مواردی که نخواهیم حافظه را برای همه نتایج به صورت همزمان تخصیص دهیم بهترین راه حل هستند.

بیایید یک مثال دیگر از یک تولید کننده بسازیم که اعداد فیبوناچی

(*https://en.wikipedia.org/wiki/Fibonacci number*) را محاسبه میکند.

In [9]:

```
def genfibon(n):
 """
  Generate a fibonnaci sequence up to n
 """
 a = 1
 b = 1
 for i in range(n):
        yield a
        a,b=b,a+b
```

In [10]:

```
for num in genfibon(18):
    print(num)
```

```
1

3

5

8

13

34

55
```

اگر این یک تابع عادی بود چگونه به نظر میرسید؟

In [11]:

```
1  def fibon(n) :
2    a = 1
3    b = 1
4    output = []
5
6    for i in range(n):
7        output.append(a)
8        a,b = bratb
9
10    return output
```

In [12]:

```
1  fibon(10)
```

Out[12]:

[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]

توجه کنید که اگر مقدار بسیار بزرگی از " را مانند ۱۰۰۰۰۰ صدا بزنیم تابع دوم باید از هر نتیجه ای پیگیری کند. در حالی که در مورد ما تنها اهمیت داریم که نتیجه قبلی را برای تولید نتیجه بعدی داشته باشیم

درک بهتر مولدها

تا اینجا درباره دو روش اصلی ایجاد مولدها آموخته اید استفاده از توابع تولید کننده و عبارات تولید کننده. شاید حتی درکی شهودی از نحوه کار مولدها داشته باشید. بیایید یک لحظه وقت بگذاریم تا این دانش را یکمی صریح تر کنیم.

توابع مولد به نظر میرسند و همانند توابع عادی عمل میکنند با این تفاوت که یک ویژگی تمایز دهنده دارند. توابع مولد از کلیدواژه *yield Python* به جای *return* استفاده میکنند به خاطر بیاورید تابع مولدی که قبلا نوشته بودید.

In [ ]:

```
1  def infinite_sequence():
2      num=0
3      while True:
4          yield nun
5          num+= 1
```

In [ ]:

```
1  for num in infinite_sequence() :
2      print(num)
```

این به نظر میرسد مانند یک تعریف تابع معمولی است به استثنای عبارت *yield Python* و کدی که بعد از آن قرار دارد. *yield* نشان میدهد که کدام مقدار به *call* ارسال میشود اما بر خلاف *return* شما پس از آن از تابع خارج نمی شوید.

به جای آن حالت تابع به یاد آورده میشود. در این صورت وقتی ()*next* بر روی یک شیء مولد(به صورت صریح یا ضمنی در یک حلقه *for*) فراخوانی میشود متغیر *num* که قبلاً *yield* شده بود افزایش مییابد و دوباره *yield* میشود. از آنجا که توابع تولید کننده مانند سایر توابع به نظر میرسند و به شدت شبیه به آنها عمل میکنند میتوانید فرض کنید که عبارات تولید کننده بسیار شبیه به سایر تواناییهای موجود در پایتون هستند.

ساخت مولد با استفاده از *Generator Expressions*

مانند عبارت های *comprehensions* عبارت های مولد به شما اجازه میدهند تا در چند خط کد، به سرعت یک شیء تولید کننده بسازید. آنها همچنین در همان مواردی که عبارات *comprehensions* استفاده میشوند، مفید هستند یا این تفاوت که شما میتوانید آنها را بدون ساخت و نگهداری کامل شی در حافظه قبل از تکرار ایجاد کنید. به عبارت دیگر، هیچ عواقب حافظه ای وجود ندارد هنگام استفاده از عبارات تولید کننده برای مثال نگاهی به مربع کردن برخی اعداد بیندازید:

In [13]:

```
1 nums_squared_lc = [num**2 for num in range(5)
2 nums_squared_gc = (nun**2 for nun in range(s))
```

هر دو *nums_squared Je* و *nums_squaredge* به طور کلی به یک شکل به نظر میرسند اما یک تفاوت کلیدی وجود دارد. آیا میتوانید آن را بیابید؟ نگاهی به اینکه در هنگام بررسی هر یک از این اشیاء چه اتفاقی می افتد:

In [14]:

```
1 nums_squared_1c
```

Out [14] :

[0, 1, 4, 9, 16]

In [15]:

```
1 nums_squared_gc
```

Out[15]:

<generator object <genexpr> at 0x000001FB44288830>

شی اول از براکت ها برای ساختن یک لیست استفاده کرد. در حالی که شی دوم با استفاده از پرانتزها عبارت مولد ایجاد کرد. خروجی تایید میکند که یک شی تولید کننده ایجاد کرده اید و که این شی از یک لیست متمایز است.

In [16]:

```
1 next(nuns_squared_gc)
```

out[16] :

0

In [17]:

```
1 next(nuns_squared_Bc)
```

out[17] :

1

In [18]:

```
1 next(nuns_squared_Bc)
```

out[18] :

4

ارزیابی کارایی یک مولد

قبلاً یاد گرفتید که مولدها یک روش عالی برای بهینه سازی حافظه هستند در حالی که یک تولید کننده دنباله بی نهایت یک مثال مطلق از این بهینه سازی است بیایید مثال های مربع کردن اعداد را که همین که دیدید را به سطح بالاتر بیریم و اندازه اشیاء حاصل را بررسی کنیم. شما میتوانید این کار را با فراخوانی تابع *sys.getsizeof()* انجام دهید:

In [19]:

```
1 import sys
2 nums_squared_lc = [1 ** 2 for 1 in range(10000)]
3 print(sys.getsizeof(nums_squared_1c))
```

85176

In [20]:

```
1 nums_squared_gc = (i ** 2 for i in range(10000))
2 print(sys.getsizeof (nums_squared_gc))
```

104

در این مورد اندازه لیستی که از عبارت مولد لیست دریافت میکنید ۸۷۶۲۴ بایت است در حالی که شیء مولد فقط ۱۰۴ بایت است. این به این معنی است که اندازه لیست بیش از ۸۴۲ برابر اندازه شیء مولد است!

اما یک نکته را به خاطر داشته باشید. اگر اندازه لیست کوچکتر از حافظه قابل دسترس ماشین باشد، آنگاه عبارات مولد لیست ممکن است سریع تر ارزیابی شوند نسبت به عبارات مولد معادل برای بررسی

این موضوع بیایید جمع مقادیر به دست آمده از دو عبارت مولد را محاسبه کنیم میتوانید با استفاده از *Profile.run*() خروجی را تولید کنید.

In [21]:

```
1 import cProfile
2 Cprofile.run('sun([1 * 2 for 1 in range(10000)])')
```

5 function calls in 0.001 seconds

Ordered by: standard name

ncalls tottime percall cumtime percall filename:lineno (function)

1 0.001 0.001 0.001 0.001 <string>:1(<listcomp>)

1 0.000 0.000 0.001 0.001 <string>:1(<modules)

1 0.000 0.000 0.001 0.001 (built-in method builtins.exec)

1 0.000 0.000 0.000 0.000 (built-in method builtins.sum)

1 0.000 0.000 0.000 0.000 (method 'disable' of lsprof.Profiler objects)

In [22]:

```
1 cProfile.run('sum((12 for i in range(10808)))')
```

10005 function calls in 0.002 seconds

Ordered by: standard name

ncalls tottine percall cuntine percall filename:lineno(function)

10001 0.001 0.000 0.001 0.000 <string>:1(<genexpr>)

1 0.000 0.000 0.002 0.002 (string):1(<module>)

1 0.000 0.000 0.002 0.002 (built-in nethod builtins.exec)

1 0.001 0.001 0.002 0.002 (built-in nethod builtins.sum)

1 0.000 0.000 0.000 0.000 (nethod "disable of Isprof.Profilen objects)

در اینجا میبینید که جمع کردن همه مقادیر در عبارت مولدی نیست تقریباً برابر زمان جمع کردن در مولد انجام شد. اما بصورت کلی اگر سرعت مهم است و حافظه نه آنگاه یک عبارت ***list comprehension*** ابزار بهتری برای این کار است. توجه این اندازه گیری ها فقط برای اشیاء ساخته شده با عبارات مولد معتبر نیستند. آنها همچنین برای اشیاء ساخته شده از تابع مولد مشابه هستند زیرا مولدهای حاصل معادل هستند.

بیاد داشته باشید که عبارات مولد لیست های کامل را برمی گردانند در حالی که عبارات مولد مجموعه ها را برمی گردانند. مولدها بدون تفاوت کار میکنند بدون اینکه از تابع یا عبارت ساخته شوند. استفاده از یک عبارت فقط به شما امکان میدهد تا مولدهای ساده را در یک خط تعریف کنید با فرض وجود یک ***yield*** در انتهای هر حلقه داخلی.

یادآوری: عبارت ***yield*** پایه ای است که تمام قابلیت های مولدها بر آن تکیه دارد بنابراین بیایید به بررسی نحوه کارکرد ***yield*** در پایتون بپردازیم.

درک بهتر دستور ***yeild***

در کل ***yield*** یک عبارت بسیار ساده است وظیفه اصلی آن کنترل جریان یک تابع مولد به گونه ای است که شبیه به عبارت های ***return*** است. همانطور که در بالا به طور خلاصه اشاره شد عبارت ***yield*** در پایتون چند ترفند ویژه دارد. وقتی یک تابع مولد را فراخوانی میکنید یا از یک عبارت مواد استفاده میکنید یک مولد ویژه به نام مولد ***(generator)***برگردانده میشود. میتوانید این مولد را به یک متغیر اختصاص دهید تا از آن استفاده کنید. وقتی روش های ویژه ای مانند ***()next***را بر روی مولد فراخوانی میکنید که درون تابع تا عبارت ***yield*** اجرا می شود.

زمانی که عبارت ***yield*** پایتون اجرا میشود برنامه اجرای تابع را متوقف میکند و مقداری که به آن ***yield*** شده است را به فراخواننده برمی گرداند. در مقابل عبارت ***return*** اجرای تابع را به طور کامل متوقف میکند. وقتی یک تابع متوقف میشود. وضعیت آن تابع ذخیره میشود. این شامل هر اتصال متغیری که محلی در مولد است نشانگر دستور پشته داخلی و هرگونه مدیریت استثناء است.

این قابلیت به شما امکان میدهد تا هر زمان که یکی از متدهای مولد را فراخوانی کنید اجرای تابع را ادامه دهید. به این صورت ارزیابی تابع به صورت کامل از جایی که عبارت ***yield*** قرار دارد. ادامه پیدا میکند. این را میتوانید با استفاده از چندین عبارت ***yield*** در عمل مشاهده کنید.

In [23]:

```
1  def multi yield();
2      yield_str = "This will print the first string"
3      yield yield_str
4      yield_str = "This will print the first string"
5      yield yield_str
```

In [24]:

```
1  multi_obj = multi_yield()
```

In [25]:

```
1  print(next(multi_obj))
```

This will print the first string

In [26]:

```
1  print(next(nulti_obj))
```

This will print the second string

In [27]:

```
1  print(next(multi_obj))
```

StopIteration

Traceback (most recent call last)

Cell In[27], line 1

---->1 print(next(multi_obj))

StopIteration:

نگاهی دقیق تر به آخرین فراخوانی *next()* بیندازید میتوانید ببینید که اجرا با یک *traceback* متوقف شده است. دلیل این امر این است که تولید کننده ها مانند سایر تکرارکننده ها، قابل استفاده هستند. مگر اینکه تولید کننده شما بی نهایت باشد. میتوانید فقط یک بار از آن عبور کنید. پس از ارزیابی تمامی مقادیر تکرار متوقف شده و حلقه *for* خاتمه خواهد یافت. اگر از *next()* استفاده کردید در عوض یک استثناء صریح *StopIteration* دریافت خواهید کرد. *yield* می تواند برای کنترل جریان اجرای تولید کننده شما به شکل های مختلفی استفاده شود. استفاده از چندین دستور *yield* در پایتون میتواند تا جایی که خلاقیت شما اجازه دهد، بهره برداری شود.

استفاده از متدهای پیشرفته مولدها

شما روش ها و ساختارهای متداول ترین مولدها را دیده اید اما چند ترفند دیگر برای پوشش وجود دارد. علاوه بر *yield* اشیاء مولد میتوانند از متدهای زیر استفاده کنند:

- *.send()*
- *.throw()*

.send()

اینجا یک مثال از استفاده از *send()* در مولدهای پایتون است:

```
def generator():
    while True:
        x = yield
        print(x)
g = generator()
next(g)      #شروع مولد
g.send(1)    #چاپ ۱
g.send(2)    #چاپ۲
```

در این مثال تابع ()*Benerator* به عنوان یک مولد تعریف شده است که مقادیر را با استفاده از دستور *yield* باز میگرداند. دستور ()*send* برای ارسال مقدار به داخل مولد و ادامه اجرای آن استفاده میشود. در این حالت، مقداری که به داخل مولد فرستاده شده چاپ می شود

In [28]:

```
1  def generator():
2
3      while True:
4          x = yield
5          print(x)
```

In [29]:

```
1  g= generator()
```

In [34]:

```
1  next(g)
```

None

In [35]:

```
1  g.send(10)
```

10

In [36]:

```
1  g.send(2)
```

2

throw().

اینجا یک مثال از استفاده از ()*throw* در مولدهای پایتون است:

```
def generator():
    try:
        while True:
            x = yield
            print(x)
    except ValueError:
        print("Caught a ValueError")


g = generator()
next(g)    #شروع مولد
g.send(1)  # چاپ ۱
g.throw(ValueError)   "Caught a ValueError"#
```

در این مثال تابع ()*Benerator* به عنوان یک مولد تعریف شده است که مقادیر را با استفاده از دستور *yield* باز می گرداند. دستور() *throw* برای پرتاب استثناء در داخل مولد و ادامه اجرای آن استفاده میشود در این حالت، استثناء پرتاب شده توسط بلوک *try* دریافت شده و پیام *Caught a ValueError* چاپ میشود.

In [37]:

```
1  def generator():
2      try:
3          while True:
4              x = yield
5              print(x)
6      except ValueError:
7          print("Caught a ValueError")
```

In [38]:

```
1 g=generator()
```

In [39]:

```
1 next(g)
```

In [40]:

```
1 g.send(1)
```

1

In [41]:

```
1 g.throw(ValueError)
```

Caught a ValueError

StopIteration

Traceback (most recent call last)

Cell In[41], line 1

----> 1 g.throw (ValueError)

StopIteration:

close().

در این مثال یک تابع مولد به نام() *my_generator* تعریف شده است که به طور نامحدود حلقه میزند و هر بار یک مقدار را دریافت می.کند روش *close*() برای متوقف کردن تکرار مولد و پرتاب یک استثناء *GeneratorExit* در داخل مولد استفاده میشود در این حالت استثناء پرتاب شده توسط بلوک *try* مشخص شده و پیام *"Generator has been closed"* چاپ میشود.

```
def my generator():
    try:
        while True:
                x = yield
    except GeneratorExit:
         print("Generator has been closed")


g = my generator())
next(g)
g.close()    # Output: "Generator has been closed"
```

In [42]:

```
def generator():
    try:
        while True:
            x= yield
            print(x)
    except GeneratorExit:
            print("Generator has been closed")

g=generator()
```

In [44]:

```
next(g)
```

None

In [45]:

```
1 g.close()
```

Generator has been closed

In [46]:

```
1 next(g)
```

StopIteration

Traceback (most recent call last)

Cell In[46], line 1

----> 1 next(g)

StopIteration:

تمرینات تکرارشونده و مولدها

مسئله ۱

ایجاد یک تولید کننده که مربع اعداد تا یک عدد *N* را تولید میکند.

In [1]:

```
1 def gensquares (N):
2     for i in range(N):
3         yield i ** 2
```

In [2]:

```
1 for x in gensquares (10):
    print(x)
```

0

1

4

9

16

25

36

49

64

81

مسئله ۲

ایجاد یک تولید کننده که "n" عدد تصادفی بین یک عدد کم و یک عدد بزرگ (که به عنوان ورودی ها قرار میگیرند) را تولید میکند.

توجه: از کتابخانه random استفاده کنید. به عنوان مثال:

In [4]:

```
1  import random

   random.randint(1,10)
```

out[4] :

7

In [5]:

```
1  def rand_num(low, high, n):
       for i in range(n):
           yield random.randint(low, high)
```

In [6]:

```
1  for num in rand_num (1,10,12) :
2      print(num)
```

6

8

10

8

2

7

9

5

10

9

6

2

مسئله ۳

از تابع *iter*() برای تبدیل رشته زیر به یک تکرارکننده استفاده کنید:

In [7]:

```
1 s = 'hello
2
3 #code here
4
5 s = iter(s)
6
7 next(s)
```

Out[7]:

'h'

In [8]:

```
1 next(s)
```

Out [8]:

'e'

مسئله ۴

توضیحی برای استفاده از یک مورد کاربرد برای یک مولد با استفاده از دستور yield که در آن نمیخواهید از یک تابع عادی با دستور return استفاده کنید

مسئله بونس!

فرآیند کد زیر را توضیح دهید توجه کنید که از Generator Expression استفاده شده است.

In [9]:

```
1 my_list = [1,2,3,4,5]
2
3 gencamp = (item for item in my_list if item > 3)
4
5 for item in gencomp:
6     print(item)
```

عالی!

map()

map() یک تابع داخلی در پایتون است که دو یا بیشتر آرگومان دریافت میکند یک تابع و یک یا چند آیتم قابل تکرار در شکل زیر:

map(function, iterable, ...)

map() یک ایتریتور برمی گرداند - به عبارت دیگر *map*() یک شیء ویژه ای را برمیگرداند که به تدریج یک نتیجه را در هر زمان به صورت لازم تولید میکند ما در فصول آینده بیشتر درباره ایتریتورها و ژنراتورها صحبت خواهیم کرد. در حال حاضر، به دلیل اینکه مثال های ما خیلی کوچک هستند. ما *map*() را به صورت یک لیست تبدیل میکنیم تا نتایج را بلافاصله ببینیم.

وقتی ما درباره ترکیبات لیست صحبت میکردیم یک *List Comprehension* برای تبدیل درجه سلسیوس به فارنهایت ایجاد کردیم. همین کار را اینجا انجام میدهیم اما از *map* استفاده میکنیم:

In [2]:

```
1  def fahrenheit(celsius):
2      return (9/5) * celsius + 32
3
4  temps = [0, 22.5, 40, 100]
```

حالا بیایید map()را در عمل ببینیم:

In [5]:

```
1  F_temps = map(fahrenheit, temps)
2
3  #[F_temps]
4
5  list(F_temps)
```

out[5] :

[32.0, 72.5, 184.0, 212.0]

در مثال بالا *map*() تابع فارنهایت را به هر آیتم در *temps* اعمال میکند. با این حال ما نیازی به تعریف توابع قبلی نداریم بلکه میتوانیم از یک عبارت لامبدا استفاده کنیم.

In [6]:

```
1 list( map(lambda x: (9/5) * x + 32, temps) )
```

out[6] :

[32.8, 72.5, 104.0, 212.0]

عالی ما به همان نتیجه رسیدیم استفاده از ()*map* با عبارات لامبدا بسیار رایج تر است زیرا هدف کلی ()*map* از صرفه جویی در زمان و تلاش برای ایجاد حلقه های دستی است.

()*map* با چندین تکرارشوند

()*map* میتواند بیش از یک آیتریبل دریافت کند تکرارشوندها باید همان طول باشند. در صورتی که طول آنها برابر نباشد، ()*map* همچنین خواهد ماند تا زمانی که کوتاه ترین تکرارشوند تمام شود.

به عنوان مثال اگر تابع ما سعی در اضافه کردن دو مقدار *x* و *y* داشته باشد میتوانیم یک لیست از مقادیر *x* و یک لیست دیگر از مقادیر *yy* را به ()*map* ارسال کنیم تابع(یا لامبدا) هر بار مقدار *0* ام را از هر لیست دریافت میکند. سپس مقدار ۱ ام و به همین ترتیب تا رسیدن به مقدار *n*- ام.

In [10]:

```
1 a =  [1,2,3,4]
2 b =  [5,6,7,8]
3 c =  [9,10,11,12]
4
5 list( map(lambda x, y: x + y, a, b))
```

out[10] :

[6, 8, 10, 12]

In [11]:

```
1 # Now all three lists
2 list( map(lambda x, y, z: x + y + z ,a,b,c))
```

out[11] :

[15, 18, 21, 24]

در مثال بالا میبینیم که پارامتر *x* مقادیر خود را از لیست *a* دریافت میکند در حالی که ۷ مقادیر خود را از ۶ و ۲ از لیست *c* دریافت میکند حالا شما میتوانید با مثال خودتان بازی کنید تا اطمینان حاصل کنید که به درستی نحوه تطبیق با بیش از ایک تکرارشوند را درک کرده اید.

کار عالی حالا باید یک درک اولیه از تابع ()*map* داشته باشید.

reduce()

بسیاری از دانشجویان بارها مشکل در درک() *reducer* دارند بنابراین به این سمینار با دقت توجه کنید.
تابع (*reduce(function, sequence* به طور مداوم تابع را بر روی دنباله اعمال میکند و سپس یک مقدار واحد را برمی گرداند.

اگر [*sea =[S1, S2, S3, ..., sn*، باشد فراخوانی (*reduce(function, sequence* به صورت زیر عمل می کند.

- در ابتدا دو عنصر اول از *sea* به تابع اعمال میشوند به عبارت دیگر (*func(s1,s2*
- حالت کلی لیستی که روی آن ()*reduce* عمل میکند به این شکل خواهد بود (۲ ,*function(s1*

sn ,... ,۵۳]

- در مرحله بعد، تابع بر روی نتیجه قبلی و عنصر سوم لیست اعمال میشود، به عبارت دیگر

function(function(s1, s2), s3)

- حالت لیست به این شکل تغییر میکند [*function(function(s1, 2, 3,...,sn]*
- این فرایند تا زمانی ادامه مییابد که فقط یک عنصر باقی میماند و این عنصر را به عنوان نتیجه *reduce*() برمیگرداند

بیایید یک مثال ببینیم:

In [4]:

```
from functools import reduce

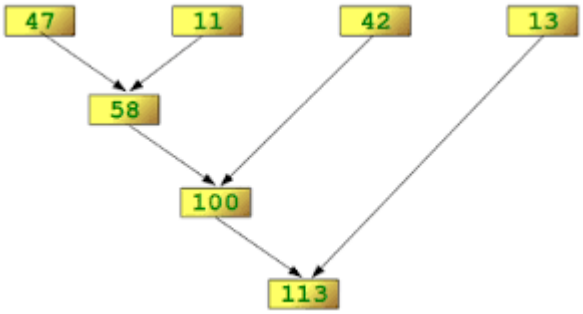
1st = [47,11, 42, 13]

reduce(lambda x, y: x + y, lst)
```

Out[3]:

113

بیایید یک نمودار را بررسی کنیم تا بهتر درک کنیم که چه اتفاقی اینجا می افتد:



توجه کنید که ما دنباله را تا زمانی که یک مقدار نهایی واحد به دست بیاوریم به صورت مداوم کاهش میدهیم. بیایید یک مثال دیگر ببینیم:

In [4]:

```
#Find the maximum of a sequence (This already exists as max())
max_find lambda a, b: a if (a b) else bi
```

In [5]:

1	*[47, 11, 42, 13]*

Out[5]:

[47, 11, 42, 13]

In [6]:

```
#Find max
reduce(max_find, 1st)
```

out[6] :

47

امیدوارم بتوانید ببینید که چقدر *reduce* () در مواقع مختلف مفید است. در هنگام برنامه نویسی پروژه های خود این را در نظر داشته باشید!

filter()

تابع *filter (function, list* راهی مناسب برای فیلتر کردن تمام عناصر یک مجموعه است به این صورت که تابع برای آن عنصر مقدار *True* برگرداند.

تابع(*filter (function, list* به یک تابع به عنوان آرگومان اول نیاز دارد. این تابع باید یک مقدار پولین (با ارزش *True* یا *False.*)برگرداند. این تابع بر روی هر عنصر قابل تکرار اعمال میشود. تنها اگر تابع *True* را برگرداند، عنصر مجموعه در نتیجه شامل خواهد شد.

مانند *map*() تابع *Filter*() نیز یک آیتریتور را برمیگرداند به عبارتی *filter*() نتیجه را به صورت یک مقدار در هر بار نیازمندی بر میگرداند درباره ایتریتورها و ژنراتورها در یک فصول آینده صحبت خواهیم کرد. در حال حاضر، زیرا مثال های ما بسیار کوچک هستند ما *filter*() را به صورت یک لیست تبدیل میکنیم تا نتایج را به صورت فوری مشاهده کنیم.

بیایید چند مثال ببینیم:

In [1]:

```
1  #First Let's make a function
2  def even_check (num):
3    if num % 2 == 0:
4          return True
```

حالا بیایید یک لیست از اعداد را فیلتر کنیم توجه قرار دادن تابع در +Filter() بدون هیچ پرانتزی ممکن است عجیب به نظر برسد اما به یاد داشته باشید که توابع نیز شی هایی هستند.

In [3]:

```
1  st = range (20)
2
3  list(filter (even_check, 1st))
```

Out [3] :

[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

Filter() بیشتر با استفاده از توابع لامبدا مورد استفاده قرار میگیرد زیرا معمولاً از filter() برای کار سریعی استفاده میکنیم که نیازی به نوشتن یک تابع کامل ندارد بیایید مثال بالا را با استفاده از یک عبارت لامبدا تکرار کنیم:

In [4]:

```
1  list(filter (lambda x: x % 2==0,lst))
```

Out[4] :

[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

عالی حالا باید یک درک قوی از() *filter* و نحوه استفاده از آن در کد خود داشته باشید!

zip

تابع ()*zip* یک تکرار شونده ایجاد میکند که عناصر را از هر یک از آیترابل ها گردآوری میکند.

این تابع یک تکرارشونده از تاپل ها برمی گرداند به طوری که*تاپل*i *ام* از هر یک از توالی ها یا تکرارشونده های ورودی حاوی عنصر*i* نام آن است. تکرارشونده هنگامی که تکرار شونده ورودی کوتاهترین آن به پایان برسد متوقف میشود. در صورتی که یک آرگومان تکی داشته باشد تکرارشونده ای از تاپل های یک عضوی بر میگرداند. در صورتی که بدون آرگومان ورودی داشته باشد یک تکرار شونده خالی برمی گرداند.

```
def zip(*iterables):
    #zip('ABCD', 'xy')-> Ax By
    sentinel = object()
    iterators = [iter(it) for it in iterables]
    while iterators:
        result = []
        for it in iterators:
            elem = next(it, Sentinel)
            if elem is sentinel:
                return
            result.append(elem)
        yield tuple(result)
```

تابع *zip*() معادل زیر است:

zip() فقط در صورتی باید با آرگومان های طول نامساوی استفاده شود که مقدار انتهایی وجود نداشته باشد. نحوه کار آن را در برخی از مثال ها ببینیم

مثال ها:

In [2]:

```
1  x = [1,2,3]
2  y = [4,5,6]
3
4  # Zip the Lists together
5  list(zip(x,y))
```

توجه کنید که تاپل ها برگردانده میشوند. اگر یکی از تکرار شونده ها از دیگری بیشتر باشد چه اتفاقی می افتد؟

In [3]:

```
1  x = [1,2,3]
2  y = [4,5,6,7,8]
3
4  # Zip the Lists together
5  list(zip(x,y))
```

Out[3]:

[(1, 4), (2, 5), (3, 6)]

توجه کنید که تابع() zip توسط طول کوتاهترین تکرار شونده تعریف میشود. به طور کلی توصیه می شود از ()zip برای آیترایل های با طول های نامساوی استفاده نکنید مگر اینکه مطمئن باشید که فقط نیاز به ترکیبات جزئی از تاپل ها دارید. چه اتفاقی می افتد اگر سعی کنیم دیکشنری ها را با هم zip کنیم؟

In [4]:

```
1  d1 = {'a':1, 'b':2}
2  d2 = ('c':4, 'd":5}
3
4  list(zip(d1,d2))
```

out[4]:

[('a', 'C'), ('b', 'd')]

این مسئله قابل درک است زیرا به سادگی تکرار از دیکشنریها فقط کلیدها را نتیجه میدهد برای ترکیب کلیدها و مقادیر باید متدهایی را فراخوانی کنیم

In [5]:

```
1  list(zip (d2, d1.values()))
```

out[5]:

[('c', 1), ('d', 2)]

In [6]:

```
1 list(zip(d1, d2.values()))
```

Out[6] :

[('a', 4), ('b', 5)]

عالی سرانجام از *zip* برای تعویض کلیدها و مقادیر دو دیکشنری استفاده کنیم:

In [7]:

```
1 def switcharoo (d1, d2):
2     dout = {}
3
4     for dikey, d2val in zip (d1, d2. values()):
5         dout [d1key] = d2val
6     return dout
7
```

In [8]:

```
1 switcharoo (d1, d2)
```

Out[8]:

{'a': 4, 'b': 5}

عالی میتوانید در بسیاری از مواقع از() *zip* برای صرفه جویی زمان در ساخت تاپل ها استفاده کنید! در حال حاضر باید یک درک خوب از *zip* و برخی از موارد استفاده ممکن آن را داشته باشید.

enumerate()

در این سخنرانی درباره ی یک تابع بسیار مفید وجود دارد ()*enumerate* تابع ()*enumerate* به شما امکان میدهد در هنگام تکرار از یک شمارنده استفاده کنید این کار را با برگرداندن یک تاپل به شکل ،شماره (عنصر) انجام میدهد.

تابع ()*enumerate* خودش معادل زیر است:

```
def enumerate (sequence, start=0):
    n = start
    for elem in sequence:
        yield n, elem
        n += 1
```

مثال:

In [1]:

```
lst = ['a', 'b', 'c']

for number, item in enumerate(1st):
    print(number, " : ", item)
```

```
0  : a
1  : b
2  : c
```

enumerate() وقتی بسیار مفید میشود که نیاز به ردیابی کننده ای داشته باشید.

به عنوان مثال:

In [2]:

```
for count, item in enumerate(1st):
    if count >= 2:
        break
    else:
        print(item)
```

```
a

b
```

enumerate() یک آرگومان اختیاری به نام "start" دارد که مقدار پیش فرض صفر را برای آن تعیین میکند

In [4]:

```
1  months = [March', 'April', 'May', 'June']
2
3  [*enumerate (months, start = 3)]
```

out[4] :

[(3, 'March'), (4, 'April'), (5, 'May'), (6, 'June')]

عالی در حال حاضر باید یک درک خوب از ()*enumerate* و موارد استفاده ممکن آن را داشته باشید.

all() و any()

()*all* و *Many* توابعی هستند که در پایتون به صورت داخلی وجود دارند و به ما امکان میدهند به طور ساده بررسی کنیم که آیا یک مجموعه دارای مطابقت بولینی است یا خیر تابع (۱۱) در صورتی که همه عناصر در یک مجموعه مطابقت بولینی داشته باشند *True* را برمیگرداند که این تابع به صورت زیر است:

```
def all(iterable):
    for element in iterable:
        if not element:
            return False
    return True
```

تابع ()*any* در صورتی که هر کدام از عناصر در مجموعه مطابقت بولینی داشته باشند *True* را برمیگرداند که این تابع معادل زیر است:

```
def any (iterable):
    for element in iterable:
        if element:
            return True
    return False
```

بیایید چند مثال از این توابع را .ببینیم آنها بسیار ساده هستند.

In [2]:

```
1 lst = [True, True, False, True]
```

In [3]:

```
1 all(lst)
```

out[3] :

False

In [4]:

```
1 all([12, 'Hello', 'No', [20,22]])
```

Out [4]:

True

با توجه به اینکه همه عناصر صحیح نیستند False برگشت داده میشود.

In [5]:

```
1 any(lst)
```

out[5] :

True

In [6]:

```
1 any([0,, 0), (1, 0])
```

out[6] :

False

با توجه به اینکه حداقل یکی از عناصر در لیست صحیح است *True* برگشت داده میشود.

در اینجا آن را دارید شما باید یک درکی از نحوه استفاده از (*all()* و *any()* در کد خود داشته باشید.

complex()

تابع *complex()* یک عدد مختلط را با مقدار *real + imag1* برمیگرداند یا یک رشته یا عدد را به یک عدد مختلط تبدیل میکند.

اگر پارامتر اول یک رشته باشد آن به عنوان یک عدد مختلط تفسیر خواهد شد و تابع باید بدون داشتن پارامتر دوم فراخوانی شود. پارامتر دوم هرگز نمیتواند یک رشته باشد هر آرگومان میتواند هر نوع عددی باشد(شامل عدد مختلط) اگر *imag* حذف شود. به صفر تنظیم میشود و سازنده به عنوان یک تبدیل عددی شامل *int* و *float* عمل میکند. اگر هر دو آرگومان حذف شوند ۰ برگشت داده میشود.

اگر در حوزه ریاضیات یا مهندسی با اعداد مختلط کار میکنید مانند(دینامیک، سیستم های کنترلی یا امپدانس یک مدار)، این ابزار مفیدی است که در پایتون دارید

In [1]:

```
1 # Create 2+3j
2 complex(2,3)
```

out[1] :

(2+3j)

In [2]:

```
1 x = complex(2,3)
2 x + x
```

Out[2] :

(4+6j)

In [3]:

```
1 complex(10,1)
```

Out[3]:

(10+11)

همچنین میتوانیم رشته ها را ارسال کنیم:

In [4]:

```
1 complex(12+23 ' )
```

out[4] :

(12+21)

این واقعاً همه چیزی است که درباره این تابع مفید باید بدانید آن را به خاطر بسپارید اگر هرگز با اعداد مختلط در پایتون سر

تست توابع داخلی

برای این تست باید از توابع داخلی استفاده کرده و توانایی نوشتن توابع درخواستی را در یک

خط داشته باشید.

مسئله ۱

از تابع ()*map* برای ایجاد یک تابع استفاده کنید که طول هر کلمه را در عبارت (یا فاصله ها تفکیک شده) پیدا کرده و مقادیر را در یک لیست برگرداند.

ورودی تابع یک رشته است و خروجی یک لیست اعداد صحیح است.

In [1]:

```
1  def word_lengths(phrase):
2
3      words = phrase.split()
4
5      return list map(len, words) )
```

In [2]:

```
1  word lengths ('How long are the words in this phrase')
```

Out[2]:

[3, 4, 3, 3, 5, 2, 4, 6]

مسئله ۲

از تابع reduce() برای گرفتن یک لیست از ارقام استفاده کنید و عددی که با آن مطابقت دارند را برگردانید. به عنوان مثال، [1,2,3] متناظر با صد و بیست و سه است اعداد را به رشته تبدیل نکنید!

In [3]:

```
1  from functools inport reduce
2
3  def digits_to_nun(digits):
4
5      return reduce( lambda x, y: * * 10 + y, digits)
```

In [4]:

```
1  digits_to_num( 13,4,3,2,11)
```

Out [4]:

34321

مسئله ۳

از تابع فیلتر برای استخراج کلماتی که با حرف اول مشخص شده شروع شده اند استفاده نمایید.

In [8]:

```
1  def filter_words(word_list, letter):
2
3      return list( filter(lambda word: word[@] == letter, word_list ))
```

In [11]:

```
1  l= ['hello','are', 'cat', 'dog', 'ham', 'hi', 'go', 'to', 'heart'] 2
2  filter_words(1, 'c')
```

out[11] :

['cat']

Problem 4

از تابع ()*zip* و یک (*list comprehension*) استفاده کنید تا یک لیست با همان طول تولید شود که هر مقدار آن دو رشته از *L1* و*2L* است که با رابطه بین آنها ادغام شده اند نمونه خروجی را در زیر ببینید:

In [12]:

```
1  def concatenate (L1, L2, connector):
2
3    return [word1 + connector + word 2 for word1, word2 in zip (L1, L2) ]
```

In [14]:

```
1  concatenate(['A', 'B', 'C'], ['a', 'b', 'c'], '+-')
```

Out[14] :

['A-+-a', 'B-+-b', 'C-+-c']

Problem 5

از تابع ()*enumerate* و مهارت های دیگر برای تولید یک دیکشنری استفاده کنید که مقادیر لیست را به عنوان کلیدها و شاخص را به عنوان مقدار داشته باشد میتوانید فرض کنید که هر مقدار تنها یک بار در لیست داده شده ظاهر میشود.

In [20]:

```
1  def d_list(L):
2
3      return {key: value for value, key in enumerate(L)}
```

In [21]:

```
1  d_list(['a', 'b', 'c', 'd','e'])
```

Out[21]:

{'a': 0, 'b': 1, 'c': 2, 'd': 3, 'e': 4}

Problem 6

از تابع ()enumerate و مهارت های دیگر از بالا استفاده کنید تا تعداد مواردی را که مقدار آنها با شاخصشان برابر است

In [24]:

```
1  def count_match_index(L) :
2
3      return len([num for index, num in enumerate (L) if num == index])
```

In [25]:

```
1  count_match_index([0,2,2,1,5,5,6,10])
```

Out [25]:

4

عالی بود!

ماژول *Collections*

ماژول *Collections* یک ماژول درونی است که انواع داده های ویژه ای را ارائه میدهد که جایگزینی برای مجموعه های ساخته شده در پایتون با اهداف عمومی میباشند. ما قبلاً درباره ی مبانی آنها صحبت کردیم *dict. list, set* و *tuple* حالا درباره ی جایگزین هایی که ماژول *Collections* ارائه میدهد. خواهیم آموخت.

شمارنده *(Counter)*

شمارنده *(Counter)*یک زیر کلاس *(subclass)*از دیکشنری *(dict)*است که به شمارش اشیاءی که میتوانند هش شوند. کمک میکند در داخل آن عناصر به عنوان کلیدهای دیکشنری ذخیره میشوند و تعداد اشیاء به عنوان مقدار ذخیره میشوند. بیایید ببینیم چگونه میتوان از آن استفاده کرد.

In [1]:

```
1 from collections import Counter
```

Counter() با لیست ها

In [2]:

```
1 Lst= [1,2,2,2,2, 2,32,1,21,1,223,1]
2
3 Counter(1st).
```

Out[2]:

```
Counter({1: 6, 2: 6, 3: 4, 12: 1, 32: 1, 21: 1, 223: 1))
```

Counter() با رشته ها

In [3]:

```
1 Counter('aabsbsbsbhshhbbsbs')
```

Out[3]:

```
Counter({"a": 2, 'b': 7, 's': 6, 'h': 3})
```

***Counter* با کلمات داخل یک جمله**

In [4]:

```
1 S= How many times does each word show up in this sentence word times each each wor
2
3 words s.split()
4
5 Counter (words)
```

Out[4]:

```
Counter({'How': 1,
         'many': 1,
         'tines: 2,
         'does': 1,
         'each': 3,
         'word': 3,
         'show': 1,
         'up': 1,
          'in': 1,
          'this': 1,
          'sentence': 1})
```

In [6]:

```
1 # Methods with Counter()
  c = Counter (words)

  c.most_common (2)
```

Out[6]:

```
[('each, 3), ('word', 3)]
```

In [9]:

```
1 sum(c.values())
```

Out[9]:

16

In [10]:

```
1 list(c)
```

Out[10]:

['How,

'many',

'times',

'each',

'word',

'show',

'in',

'this',

'sentence']

In [13]:

```
1 c.most_common( ) [:-5-1:-1]
```

Out[13]:

[('sentence', 1), ('this', 1), ('in', 1), ('up', 1), ('show', 1)]

الگوهای مرسوم در هنگام استفاده از شی ***Counter()***

sum(c.values())	# مجموع کل شمارش ها
c.clear()	# ریست گردن کل شمارنده ها
list(c)	# لیستی از مقادیر منحصر بفرد
set(c)	# تبدیل به مجموعه
dict(c)	#تبدیل به یک دیکشیتری معمول
c.items()	# تبدیل به لیستی از تاپل های (elem ,cnt)
Counter(dict(list_of_pairs))	#تبدیل به لیستی از تاپل های (elen,cnt) جفتی
c.most_common() { :-n-1:-1]	#n تا از موارد کمتر تکرار شده
c + Counter()	#حذف شمارش های صفر و منفی

defaultdict

defaultdict یک شی مشابه دیکشنری است که تمامی متدهایی که توسط یک دیکشنری ارائه میشود را فراهم می کنند، اما یک آرگومان اول *(default_factory)*را به عنوان نوع داده پیش فرض برای دیکشنری دریافت می کند. استفاده از *defaultdict* سریع تر از استفاده از روش *dict.set_default* است.

هیچگاه *defaultdict* یک *KeyError* ایجاد نخواهد کرد. هر کلیدی که وجود نداشته باشد مقدار برگشت داده شده توسط *default factory* خواهد بود.

In [14]:

```
1 from collections import defaultdict
```

In [15]:

```
1 d = ()
```

In [16]:

```
1 d[ 'one']
```

KeyError

Traceback (most recent call last)

Cell In[16], line 1

-----> 1 d["one"]

KeyError: 'one'

In [17]:

```
1 d.get('one')
```

In [18]:

```
1 d = defaultdict(object)
```

In [19]:

```
1 d
```

Out (19):

defaultdict(object, ())

In [20]:

```
1 d['one']
```

Out[20]:

<object at @x1ffa2369620>

In [21]:

```
1
```

Out [21]:

defaultdict(object, ['one': <object at 8x1ffa2369628>})

In [22]:

```
1 for item in d
2     print(item)
```

همچنین می توانین برای کلیدها مقداردهی اولیه انجام دهید.

In [23]:

```
1 d = defaultdict(lambda: 0)
```

In [24]:

```
1 d['ane']
```

Out[24] :

0

In [25]:

```
1 d
```

Out [25]:

defaultdict(<function _main_.<lambda>()>, ('one': 0})

namedtuple

تاپل استاندارد از ایندکس های عددی برای دسترسی به اعضای خود استفاده میکند. به عنوان مثال:

In [26]:

```
1 t = (12,13,14)
```

In [27]:

```
1 t[0]
```

Out[27]:

12

namedtuple

تاپل استاندارد از ایندکس های عددی برای دسترسی به اعضای خود استفاده میکند. به عنوان مثال:

برای موارد استفاده ساده این معمولاً کافی است. از طرف دیگر به یاد داشتن اینکه برای هر مقدار از کدام ایندکس استفاده شود ممکن است منجر به خطاها شود به خصوص اگر تاپل دارای تعداد زیادی فیلد باشد و در محلی ساخته شود که از آن دور استفاده میشود. یک namedtuple نام ها و همچنین شاخص عددی را به هر عضو اختصاص می دهد.

هر نوع namedtuple توسط یک کلاس جداگانه نماینده میشود که با استفاده از تابع سازنده namedtuple() ایجاد می شود. آرگومان ها نام کلاس جدید و یک رشته حاوی نام های عناصر است.

میتوانید در اصل به namedtuples به عنوان راهی بسیار سریع برای ایجاد یک نوع جدید اشیاء کلاس با برخی فیلدهای ویژگی فکر کنید. به عنوان مثال:

In [28]:

```
1 from collections import namedtuple
```

In [29]:

```
1 Dog = namedtuple('Dog', ['age', 'breed', 'name'])
2
3 sam = Dog(age 2, breed 'Lab', name= 'Sammy')
4
5 frank = Dog (age=2, breed='Shepard', nane="Frankie")
```

ما *namedtuple* را با ابتدا انتقال نام نوع شی (سگ) و سپس انتقال رشته ای حاوی انواع فیلدها به عنوان یک رشته یا فاصله بین نامهای فیلد ایجاد میکنیم سپس میتوانیم به ویژگی های مختلف آن دسترسی پیدا کنیم:

In [30]:

```
1 sum
```

out[30] :

Dog(age-2, breed-'Lab', name='Sarry')

In [31]:

```
1 sam.age
```

Out[31]:

2

In [32]:

```
1 sam.breed
```

Out [22] :

'Lab'

In [34]:

```
1 sam[0]
```

out[34] :

2

In [35]:

```
1 sam[1]
```

Out[35] :

'Sanny'

In [37]:

```
1 sam[0] = 55
```

TypeError:

Traceback (most recent call last)

Cell In[37], line 1

---> 1 sam[0] = 55

TypeError: 'Dog' object does not support item assignment

جمع بندی

امیدوارم که حالا ببینید چقدر ماژول مجموعه ها *(collections)*در پایتون بسیار مفید است و باید به عنوان ماژول اصلی تان برای انجام تعداد زیادی از کارهای متداول انتخاب شود

بازکردن و خواندن فایل ها

تاکنون درباره ی چگونگی بازکردن دستی فایل ها صحبت کرده ایم یکی یکی بیایید بررسی کنیم که چگونه میتوانیم به صورت برنامه ای فایلها را باز کنیم.

مرور فهم مسیرهای فایل

In [1]:

```
1 pwd
```

out[1] :

'C:\\Users\\babak\\Python For Everyone\\13-Advanced Python Modules'

ایجاد یک فایل تمرینی

ما با ایجاد یک فایل متنی تمرینی که برای نمایش آن استفاده خواهیم کرد شروع میکنیم.

In [2]:

```
1 fopen('practice.txt', 'we')
```

In [3]:

```
1 f.write('test').
2 f.close()
```

استخراج مسیر دایرکتوری

پایتون دارای یک ماژول (*o s (https://docs.python.org/3/library/os.html* درونی است که به ما امکان استفاده از قابلیت های وابسته به سیستم عامل را میدهد.

میتوانید مسیر فعلی را دریافت کنید:

In [4]:

```
1 import os
```

In [5]:

```
1 os.getcwd()
```

Out[5]:

'C:\\Users\\babaki \Python For Everyone\\13-Advanced Python Modules'

لیست کردن فایل های داخل یک دایرکتوری

همچنین میتوانید از ماژول *os* برای فهرست کردن دایرکتوری ها استفاده کنید.

In [6]:

```
#In your current directory
os.listdir()
```

out[6]:

['.ipynb_checkpoints',

'ee-Collections-Module.ipynb",

01-Opening-and-Reading-Files-Folders.ipynb',

e2-Datetime-Module.ipynb",

'03-Math-and-Random-Module.1pynb",

'04-Python Debugger (pdb).ipynb', '86-Timing your code tineit.ipynb",

'07-Unzipping-and-Zipping-Files.ipynb',

'88-Advanced-Python-Module-Exercise',

'Example_Top_level ',

'practice.txt']

In [7]:

```
#In any directory you pass
os.listdir("C:\\Users")
```

Out [7] :

['All Users', 'babak', 'Default', 'Default User', 'desktop.ini', 'public']

جابجایی فایل ها

شما میتوانید از ماژول درونی ***shutil*** برای انتقال فایل ها به مکان های مختلف استفاده کنید در نظر داشته باشید که محدودیت های دسترسی وجود دارد به عنوان مثال اگر با کاربر *A* وارد شده باشید بدون داشتن مجوزهای لازم قادر به ایجاد بیشترتغییرات در پوشه ی بالایی کاربران نخواهید بود.

مطالب بیشتر:

(https://stackoverflow.com/questions/23253439/shutil-movescr-dst-gets-me-iperror-errno-13-permission-denied-and-3-more-e)

In [8]:

```
1 import shutil
```

In [9]:

```
1 shutil.move('practice.txt', 'C:\\Users\\babak')
```

Out [9]:

'C:\\Users\\babak\\practice.txt'

In [10]:

```
1 os.listdir()
```

Out[10]:

['.ipynb_checkpoints",

'00-Collections-Module.ipynb',

'01-Opening-and-Reading-Files-Folders.ipynb',

'02-Datetime-Module.ipynb",

'03-Math-and-Random-Module.ipynb",

'04-Python Debugger (pdb).ipynb",

'06-Timing your code tineit.ipynb",

'07-Unzipping-and-Zipping-Files.ipynb",

'08-Advanced-Python-Module-Exercise",

'Example Top Level']

In [11]:

```
1 shutil.move('C:\\Users\\babak\practice.txt', os.getcwd())
```

Out[11]:

"C:\\Users\\babak\\Python For Everyone\\13-Advanced Python Modules\\practice.txt'

In [12]:

```
1 os.listdir()
```

Out [12]:

[ipynb_checkpoints",

00-Collections-Module.ipynb',

'01-Opening-and-Reading-Files-Folders.ipynb',

02-Datetime-Module.ipynb",

03-Math-and-Random-Module.ipynb',

'04-Python Debugger (pdb).1pynb",

'06-Timing your code tineit.ipynb",

'07-Unzipping-and-Zipping-Files.ipynb",

'08-Advanced-Python-Module-Exercise",

Example Top Level',

'practice.txt']

حذف فایل ها

توجه ماژول *os 3* روش برای حذف فایل ها فراهم میکند:

- *os.unlink(path)* که یک فایل در مسیر مورد نظر را حذف میکند.
- *os.mdir(path)*که یک پوشه(پوشه باید خالی باشد) در مسیر مورد نظر را حذف می کند.
- *shutil.rmtree(path)* که خطرناکترین روش است زیرا تمامی فایل ها و پوشه های موجود در مسیر را حذف خواهد کرد **همه این روشها قابل بازگشت نیستند به این معنی که اگر اشتباهی انجام دهید، نمیتوانید فایل را بازیابی کنید به جای آن از ماژول *send2trash* استفاده میکنیم یک جایگزین ایمن که فایل های حذف شده را به سطل زباله ارسال میکند و از حذف دائمی جلوگیری میکند.**

برای نصب ماژول send atrash از دستور زیر در خط فرمان استفاده کنید:

pip install send2trash

In [13]:

```
1 import send2trash
```

In [14]:

```
1 os.listdir()
```

Out[14]:

['.ipynb_checkpoints",

00-Collections-Module.ipynb',

'01-Opening-and-Reading-Files-Folders.ipynb',

'02-Datetime-Module.ipynb",

'03-Math-and-Randon-Module.ipynb",

'04-Python Debugger (pdb).ipynb',

'06-Timing your code tineit.ipynb",

'07-Unzipping-and-Zipping-Files.ipynb",

'08-Advanced-Python-Module-Exercise",

'Example Top Level',

'practice.txt']

In [15]:

```
1 send2trash.send2trash('practice.txt')
```

In [16]:

```
1 os.listdir()
```

Out[16]:

[.ipynb_checkpoints",

'00-Collections-Module.ipynb',

'01-Opening-and-Reading-Files-Folders.ipynb',

'02-Datetime-Module.ipynb',

'03-Math-and-Random-Module.ipynb',

'04-Python Debugger (pdb).ipynb',

'06-Timing your code timeit.ipynb",

'07-Unzipping-and-Zipping-Files.ipynb",

'08-Advanced-Python-Module-Exercise',

'Example_Top_Level']

پیمایش یک دایرکتوری

اغلب نیاز خواهید داشت که یک دایرکتوری پیمایش شود. به عبارتی هر فایل یا پوشه را بازدید کنید و بررسی کنید که آیا یک فایل در دایرکتوری وجود دارد و سپس شاید کاری با آن فایل انجام دهید به طور معمول پیمودن بازگشتی از هر فایل و پوشه در یک دایرکتوری برنامه نویسی نسبتاً پیچیده است اما خوشبختانه ماژول *Os* یک روش مستقیم برای این کار به نام *os.walk()* دارد بیایید ببینیم این روش چگونه کار میکند.

In [17]:

```
1 os.getcwd()
```

Out [17]:

'C:\\Users\\babak\\Python For Everyone\\13-Advanced Python Modules'

In [18]:

```
1 os.listdir()
```

Out[18]:

['.ipynb_checkpoints",

'00-Collections-Module.ipynb',

'01-Opening-and-Reading-Files-Folders.ipynb',

02-Datetime-Module.ipynb',

03-Math-and-Random-Module.ipynb',

'04-Python Debugger (pdb).ipynb',

'06-Timing your code timeit.ipynb',

'07-Unzipping-and-Zipping-Files.ipynb",

'08-Advanced-Python-Module-Exercise',

'Example_Top_Level']

In [19]:

```
for folder, sub_folders, files in os.walk ("Example_Top_Level"):
    print("Currently looking at folder: "+ folder)
    print('\n')
   print("THE SUBFOLDERS ARE: ")
  for sub_fold in sub_folders:
      print("\t Subfolder: "+sub_fold)

 print('\n')

 print("THE FILES ARE: ")
for f in files:
    print("\t File: "+f)
print('\n')


#Now Look at subfolders
```

Currently looking at folder: Example_Top_Level

THE SUBFOLDERS ARE:

Subfolder: Mid-Example-One

THE FILES ARE:

File: Mid-Example.txt

Currently looking at folder: Example_Top_Level\Mid-Example-One

THE SUBFOLDERS ARE:

Subfolder: Bottom-Level-One

Subfolder: Bottom-Level-Two

THE FILES ARE:

File: Mid-Level-Doc.txt

Currently looking at folder: Example Top Level\Mid-Example-One\Bottom-Leve 1-One

THE SUBFOLDERS ARE:

THE FILES ARE:

File: One Text.txt

Currently looking at folder: Example_Top_Level\Mid-Example-One\Bottom-Leve 1-Two

THE SUBFOLDERS ARE:

THE FILES ARE:

File: Bottom-Text-Two.txt

عالی است اکنون باید یاد گرفته باشید که چگونه با فایل ها و پوشه های کامپیوتر در هر دایرکتوری که در آن هستند کار کنید. به خاطر داشته باشید که ماژول *os* برای هر سیستم عاملی که پایتون را پشتیبانی میکند کار میکند یعنی این دستورات بدون نیاز به تنظیم در ،لینوکس مکاوس یا ویندوز قابل استفاده هستند

ماژول *datetime*

پایتون دارای ماژول *datetime* است که به شما در مدیریت برچسب زمانی در کدتان کمک میکند مقادیر زمان با استفاده از کلاس *time* نمایش داده می.شوند زمانها ویژگی هایی برای ساعت ،دقیقه ثانیه و میکروثانیه دارند. همچنین میتوانند شامل اطلاعات منطقه زمانی نیز باشند آرگومانهایی برای ایجاد یک نمونه زمان اختیاری هستند اما پیش فرض ۰ احتمالاً همان چیزی نیست که میخواهید.

time

بیایید نگاهی به این بیندازیم که چگونه میتوانیم اطلاعات زمان را از ماژول *datetime* استخراج کنیم.
ما میتوانیم با مشخص کردن *datetime time (hour, minute, second,microsecond* یک برچسب زمانی ایجاد کنیم.

In [4]:

```
1  import datetime
2
3  t =  datetime.time(4, 20, 1)
4
5  print(t)
6  print('Hour:", t. hour)
7  print('Minute :', t.minute)
8  print('Second :', t.second)
9  print('Microsecond :', t.microsecond)
10 print('tzinfo", t.tzinfo)
```

04:20:01

Hour : 4

Minute : 20

Second : 1

Microsecond : 0

tzinfo None

توجه: یک نمونه زمان فقط مقادیر زمانی را نگه میدارد و شامل تاریخی که با زمان مرتبط است نیست.
همچنین میتوانیم حداقل و حداکثر مقادیری که یک زمان در روز میتواند داشته باشد را در ماژول بررسی کنیم:

In [5]:

```
1  print( Earliest :', datetime.time.min)
2  print('Latest :', datetime.time.max)
3  print('Resolution:', datetime.time.resolution)
```

Earliest : 00:00:00

Latest: 23:59:59.999999

Resolution: 0:00:00.000001

ویژگیهای کلاس *min* و *max* بازتابی از محدوده معتبر زمانی در یک روز است.

تاریخ ها

ماژول *datetime*(همانطور که شما حدس میزنید) همچنین به ما امکان میدهد با برچسب زمانی تاریخ کار کنیم مقادیر تاریخ تقویم با استفاده از کلاس *date* نمایش داده میشوند نمونه ها ویژگیهایی برای سال ماه و روز دارند. بسیار آسان است که با استفاده از روش کلاس *today* یک تاریخی که نماینده تاریخ امروز است ایجاد کنیم.

In [10]:

```
1  today = datetime.date.today()
2
3  print(today)
4  print('ctime', today.ctime())
5  print('tuple', today.timetuple())
6  print('ordinal:', today.toordinal())
7  print('Year:', today.year)
8  print('Month:', today.month)
9  print('Day:', today.day)
```

2023-07-06

ctime Thu Jul 6 00:00:00 2023

tuple time.struct_time(tm_year=2023, tm_mon=7, tm_mday=6, tm_hour=0, tm_mi

n=0, tm_sec=0, tm_wday=3, tm_yday=187, tm_isdst=-1)

ordinal: 738707

Year : 2023

Month: 7

Day: 6

همانند زمان محدوده مقادیر تاریخی پشتیبانی شده را میتوان با استفاده از ویژگیهای *min* و *max* تعیین کرد.

In [11]:

```
1 print('Earliest:', datetime.date.min)
2 print('Latest:', datetime.date.max)
3 print('Resolution:', datetime.date.resolution)
```

Earliest : 0001-01-01

Latest : 9999-12-31

Resolution : 1 day, 0:00:00

یک روش دیگر برای ایجاد نمونه های تاریخ جدید استفاده از روش ()*replace* در یک تاریخ موجود است. به عنوان مثال شما میتوانید سال را تغییر دهید و روز و ماه را حفظ کنید.

In [13]:

```
1 d1 = datetime.date (2023 3,11
2
3 print('d1 :', d1)
4
5 d2= d1.replace(year =1998)
6
7 print('d2', d2)
```

d1 : 2023-03-11
d2 :1998-03-11

ریاضیات

ما میتوانیم عملیات حسابی را بر روی اشیاء تاریخ انجام دهیم تا تفاوت های زمانی را بررسی کنیم به عنوان مثال:

In [14]:

```
1 d1
```

Out [14]:

datetime.date(2023, 3, 11)

In [15]:

```
1 d2
```

Out[15] :

datetime.date (1998, 3, 11)

In [16]:

```
1 d1-d2
```

Out[16]:

datetime.timedelta (days=9131)

این ما را به تفاوت در روزها بین دو تاریخ میرساند میتوانید از روش *timedelta* برای مشخص کردن واحدهای زمانی مختلف روزها دقایق ساعتها و غیره استفاده کنید.

عالی اکنون باید یک درک اولیه از نحوه استفاده از ماژول *datetime* در پایتون برای کار با برچسب های زمانی در کدتان داشته باشید

ماژول های ریاضی و تصادفی

پایتون همراه با ماژول ریاضی و ماژول تصادفی داخلی میآید در این سخنرانی ما به طور خلاصه به قابلیتهای آنها می پردازیم. معمولاً شما میتوانید به سادگی تماس تابعی که به دنبال آن هستید را در مستندات آنلاین جستجو کنید.

- (https://docs.python.org/3/library/math.html)
- (https://docs.python.org/3/library/random.html)

ما به تمام توابع موجود در این ماژولها نمیپردازیم زیرا بسیار زیاد هستند اما برخی از توابع مفید را نشان خواهیم داد.

توابع ریاضی مفید

In [4]:

```
1 import math
```

In [5]:

```
1 help(math)
```

y, the nearest even value of n is used. The result is always ex

act.

sin(x, /)

Return the sine of x (measured in radians).

sinh(x, 1)

Return the hyperbolic sine of x.

sqrt(x, /)

Return the square root of x.

tan(x, /)

Return the tangent of x (measured in radians).

tanh(x,1)

Return the hyperbolic tangent of x.

trunc(x, /)

گرد کردن اعداد

In [6]:

```
1 value = 4.35
```

In [7]:

```
1 math.floor(value)
```

Out[7]:

4

In [8]:

```
1 math.ceil(value)
```

Out[8]:

5

In [11]:

```
1 round(4.35, 5)
```

Out[11]:

4.35

ثابت های ریاضی

In [12]:

```
1 math.pi
```

Out[12]:

3.141592653589793

In [13]:

```
1 from math import pi
```

In [14]:

```
1 pi
```

Out[14]:

3.141592653589793

In [15]:

```
1 math.e
```

Out[15]:

2.718281828459845

In [16]:

```
1 math.tau
```

Out[16]:

6.283185307179586

In [17]:

```
1 nath.inf
```

Out[17]:

Inf

In [18]:

```
1 math.nan
```

Out[18]:

nan

مقادیر لگاریتمی

In [19]:

```
1 nath.e
```

Out[19]:

2.718281828459045

In [20]:

```
1 # Log Bose e
2 nath.log(math.e)
```

Out[20]:

1.0

In [21]:

```
1 #Will produce an error if value does not exist mathmatically
2 nath.log(0)
```

ValueError:

Traceback (most recent call last)

Cell In[21], line 2

1 # will produce an error if value does not exist mathmatically

---> 2 math.log(0)

ValueError: math domain error

In [22]:

```
1 nath.log(10)
```

Out[22]:

2.382585092994846

In [23]:

```
1 nath.e ** 2.382585892994846
```

Out[23]:

10.000000000000002

In [24]:

```
1 round(math.e ** 2.382585092994846)
```

Out[24]:

10

لگاریتم با پایه دلخواه

In [25]:

```
1 # math. Log(x, base)
1 nath.log(100, 10)
```

Out[25]:

2.0

In [26]:

```
1 10**2
```

Out[26]:

100

توابع مثلثاتی

In [27]:

```
1 #Radians
2 2 math.sin(10)
```

Out[27]:

-8.5440211108893698

In [28]:

```
1 math. degrees(pi/2)
```

Out[28]:

90.0

In [29]:

```
1 math.radians(180)
```

Out[29]:

3.141592653589793

ماژول تصادفی

ماژول تصادفی به ما امکان ایجاد اعداد تصادفی میدهد حتی میتوانیم یک بذر *(seed)*تنظیم کنیم تا هر بار مجموعه ی تصادفی مشابه ای تولید شود.

توضیح در مورد اینکه چگونه یک رایانه تلاش میکند برای تولید اعداد تصادفی خارج از دامنه ی این دوره است زیرا شامل ریاضیات سطح بالاتری میشود. اما اگر به این موضوع علاقه مند هستید به لینک های زیر مراجعه کنید:

- *https://en.wikipedia.org/wiki/Pseudorandom number generator*
- *(https://en.wikipedia.org/wiki/Pseudorandom number generator)*
- *https://en.wikipedia.org/wiki/Random seed*
- *(https://en.wikipedia.org/wiki/Random seed)*

درک بذر *(seed)*

In [30]:

```
1 import random
```

In [34]:

```
1 random.randint(0,100)
```

Out[34]:

39

In [44]:

```
1 random.randint(0,100)
```

Out[44]:

96

In [49]:

```
1 #The value 101 is completely arbitrary, you can poss in any number you wont
2 random.seed(101)
3 You can run this cell as many times as you want, it will always return the same nu
4 randon.randint(0,100)
```

Out[49]:

74

In [51]:

```
1 random.randint(0,100)
```

Out[51]:

69

In [53]:

```
1 The value 101 is completely arbitrary, you can pass in any number you want
2 randon.seed(101)
3 print (randon.randint(0,100))
4 print (randon.randint(0,100))
5 print (randon.randint(0,100))
6 print (randon.randint(0, 100))
7 print (randon.randint(0,100))
```

74

24

69

45

59

اعداد تصادفی صحیح

In [54]:

```
1 random.randint(0,100)
```

Out [54] :

6

انتخاب تصادفی

انتخاب تصادفی از میان اعضای یک لیست

In [55]:

```
1 mylist = list(range(0,20))
```

In [56]:

```
1 mylist
```

Out[56]:

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

In [59]:

```
1 random.choice(mylist)
```

Out[59] :

19

In [60]:

```
1 mylist
```

Out[60] :

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

نمونه برداری با جایگزینی

در این حالت تعداد نمونه ها را انتخاب میکنیم و انتخاب عناصر به صورت چندین بار تکراری مجاز است. تصور کنید که یک کیسه ی توپ های شماره دار لاتاری دارید. شما دست خود را داخل کیسه میگذارید و یک توپ لاتاری به صورت تصادفی انتخاب میکنید سپس پس از نوشتن شماره آن را دوباره درون کیسه قرار میدهید و سپس ادامه میدهید و توپ دیگری را انتخاب میکنید.

In [62]:

```
1 random.choices(population=nylist,k=10)
```

Out[62]:

[9, 6, 12, 8, 4, 8, 6, 2, 16, 13]

نمونه برداری بدون جایگزینی

یکباری که یک عنصر به صورت تصادفی انتخاب شود. دیگر نمیتوان آن را انتخاب کرد. تصور کنید که یک کیسه ی توپ های شماره دار لاتاری دارید شما دست خود را داخل کیسه میگذارید و یک توپ لاتاری به صورت تصادفی انتخاب می کنید. سپس پس از نوشتن شماره آن را از کیسه خارج میکنید و سپس ادامه میدهید و توپ دیگری را انتخاب می کنید.

In [63]:

```
1 random sample (population=miylist,k=10)
```

Out[63]:

[8, 12, 18, 7, 13, 1, 6, 11, 18, 15]

برهم زدن ترتیب اعضای لیست

توجه این دستور لیست اصلی را تغییر خواهد داد!

In [64]:

```
1 # Don't assign this to anything!
2 random. shuffle(mylist)
```

In [65]:

```
1 mylist
```

Out [65]:

[8, 10, 17, 7, 1, 16, 9, 2, 4, 13, 18, 15, 5, 19, 12, 11, 3, 6, 14, 0]

توزیع های تصادی

توزیع یکنواخت

(https://en.wikipedia.org/wiki/Uniform_distribution)

In [84]:

```
1 # Continuous, random picks a value between a and b, each value has equal change of b
2 random.uniform(a=0, b=100)
```

Out [84]:

81.72235646759873

توزیع نرمال یا گوسین (*https://en.wikipedia.org/wiki/Normal_distribution*)

In [95]:

```
1 random.gauss (mu=0, sigma=1)
```

out[95] :

2.6001538661204027

یادداشت نهایی اگر متوجه شدید که از این کتابخانه ها بسیار استفاده میکنید به کتابخانه ***Numpy*** برای پایتون نگاهی بیندازید که تمامی این قابلیت ها را با کارآیی بسیار بالا پوشش میدهد ما در دوره های علم داده و یادگیری ماشین به این کتابخانه و بسیاری دیگر میپردازیم.

اشتباه یاب پایتون

احتمالا از انواع دستورهای چاپ برای پیدا کردن خطاها در که خود استفاده کرده اید روش بهتری برای این کار استفاده از مازول اشتباه یاب ***(pdb)***داخلی پایتون است. ماژول ***pdb*** یک محیط اشتباه یابی تعاملی را برای برنامه های پایتون پیاده سازی می کند. این ماژول شامل قابلیت هایی است که به شما امکان میدهد برنامه خود را متوقف کرده مقادیر متغیرها را مشاهده کرده و اجرای برنامه را گام به گام پیگیری کنید تا بتوانید درک کنید که برنامه شما در واقع چه کاری انجام میدهد و باگ های منطقی را پیدا کنید.

به دلیل اینکه نیازمند ایجاد خطا به صورت عمدی است نمیتوان به راحتی نحوه کار با ماژول ***pdb*** را نشان داد، اما امیدواریم که این مثال ساده قدرت ماژول ***pdb*** را به خوبی نشان دهد.

توجه در محیط نوت بوک ***Jupyter*** استفاده از ***pob*** بسیار غیر معمول خواهد بود.

در اینجا ما قصد داریم یک خطا به صورت عمدی ایجاد کنیم با تلاش برای اضافه کردن یک لیست به یک عدد صحیح

In [1]:

```
1  x = [1,3,4]
2  y = 2
3  z=3
4
5  result = y + 2
6  print(result)
7  result2= y + x
8  print (result2)
```

5

TypeError

Traceback (most recent call last)

Cell In[1], line 7

5 result = y + z

6 print(result)

--> 7 result2 = y+x

8 print(result2)

TypeError: unsupported operand type(s) for: 'int' and 'list'

همم، به نظر میرسد خطایی دریافت کردیم بیایید با استفاده از ماژول *pdb* یک *set_trace()* پیاده سازی کنیم. این به ما امکان میدهد کد را در نقطه تعقیب متوقف کرده و بررسی کنیم که آیا هر چیزی اشتباه است یا خیر.

In [1]:

```
1  import pdb
2
3  x = [1,3,4]
4  y = 2
5  2=3
6
7  result = y + z
8  print (result)
9
10 # Set a trace using Python Debugger
11
12 pdb.set_trace()
13 result2 = y + x
14 print (result2)
```

5

--Return--

None

> c:\users\babak\appdata\local\temp\ipykernel_3683613985733997.py (11)
<modu le>()

ipdb> result

ipdb> x

[1, 3, 4]

ipdb> y ** 2

4

1pdb> 5 + 7 + 2

37

ipdb> y + x

*** TypeError: unsupported operand type(s) for +: 'int' and 'list'

Ipdb> a

عالی حالا میتوانیم مقادیر مختلف را بررسی کنیم و به دنبال خطا بگردیم شما میتوانید از و برای خروج از اشتباه یاب استفاده کنید. برای اطلاعات بیشتر در مورد تکنیک های اشتباه یابی عمومی و روش های بیشتر به مستندات رسمی مراجعه کنید:

https://docs.python.org/3/library/pdb.html(https://docs.python.org/3/library/pdb.html)

اندازه گیری زمان اجرای کد

گاهی اوقات مهم است که بدانیم که شما چقدر زمان میبرد تا اجرا شود یا حداقل بدانیم آیا یک خط خاص از کد باعث کندی کل پروژه شما میشود یا خیر پایتون یک ماژول زمانبندی داخلی دارد که این کار را انجام میدهد.

تابع یا اسکریپت مثال

در اینجا دو تابع داریم که همان کار را انجام میدهند اما به روش های مختلفی چطور میتوانیم بفهمیم کدام یک کارآمدتر است؟ بیایید زمان آن را بسنجیم!

In [1]:

```
1  def func_one(n):
2      '''
3      Given a number n, returns a list of string integers
4      ['e', '1', '2','n']
5      '''
6      return [str(num) for num in range(n)]
```

In [2]:

```
1 func_one(10)
```

Out [2] :

['0', '1','2', '3', '4', '5', '6', '7', '8', '9']

In [3]:

```
1 def func_two(n):
2     '''
3    Given a number n, returns a list of string integers
4    ['0', '1', '2','n']
5   '''
6    return list(map(str, range(n)))
```

In [4]:

```
1 func_two(10)
```

Out [4] :

['0', '1','2', '3', '4', '5', '6', '7', '8', '9']

شروع و پایان زمان بندی

می توانیم از ماژول زمان برای محاسبه زمان گذشته اجرای کد استفاده کنیم به خاطر دقت ماژول ،زمان که باید حداقل ۰/۱ ثانیه زمان ببرد تا کامل شود.

In [5]:

```
1 import tine
```

In [11]:

```
1 # STEP 1: Get start time
2 start_time = time.time()
3 Step 2: Run your code you want to time
4 result = func_one( 1000000)
5 # Step 3: Calculate total time elapsed
6 end_tine = time.tine()
7
8 elaps = end_tine - start_time
```

In [12]:

```
1 elaps
```

Out[12] :

0.23676824569702148

In [13]:

```
1 # STEP 1: Get start time
2 start_time = time.time()
3 # Step 2: Run your code you want to time
4 result = func_two(1000000)
5 # Step 3: Calculate total time elapsed
6 end_tine = time time()
7
8 elaps = end_tine - start_time
```

In [14]:

```
1 elaps
```

Out [14] :

0.16420912742614746

ماژول timeit

چه اگر دو بخش کد داشته باشیم که به اندازه کافی سریع هستند تفاوت از روش time.time() کافی نیست که بفهمیم کدامیک سریعتر است. در این حالت میتوانیم از ماژول timeit استفاده کنیم.

ماژول timeit دو رشته دریافت میکند یک عبارت (strt)و یک تنظیم سپس کد تنظیم را اجرا کرده و کد stmt را برخی از بار اجرا کرده و میانگین طول زمان آن را گزارش میدهد.

In [15]:

```
1 import timeit
```

تنظیمات (هر چیزی که قبل از آن باید تعریف شود، مانند تابع های)

In [17]:

```
setup = '''
def func_one(n):
    return [str(num) for num in range(n)]
'''
```

In [18]:

```
stnt = func_one(100)
```

In [19]:

```
timeit.timeit (stat, setup, number=100000)
```

Out[19] :

1.6600021999329329

حالا بیایید تابع function را ۱۰۰۰۰۰ بار اجرا کرده و طول زمان آن را مقایسه کنیم.

In [20]:

```
setup2= '''
def func_two(n):
    return list(map(str,range(n)))
'''
```

In [21]:

```
stmt2 = 'func_two(100) '
```

In [22]:

```
tineit.timeit (stmt2, setup2, number=100000)
```

Out[22]:

1.3273243999574333

به نظر می آید که *functwo* کارآمدتر است. شما میتوانید تعداد اجراهای بیشتری را مشخص کنید اگر میخواهید تفاوت برای توابع با عملکرد سریع تر را واضح کنید.

In [23]:

```
1 timeit.timeit(stmt,setup, number=1000000)
```

Out[23] :

17.166991000087838

In [24]:

```
1 timeit.timeit (stmt2, setup2, number=100000e)
```

out[24] :

12.608633200172335

تایمینگ کد خود را با روش "جادویی" جوپیتر انجام دهید

توجه این روش فقط در جوپیتر قابل استفاده است و دستور magic باید در بالای سلول با هیچ چیزی بالای آن (حتی کد مورد نظری که به صورت توضیحات نوشته شده است) قرار نگیرد

In [25]:

```
1 %%timeit
2 func_one(100)
```

17.1 µs ± 236 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)

In [26]:

```
1 %%tineit
2 func_two(100)
```

12.9 µs ± 79.9 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops eac

برای مطالعه بیشتر :

https://docs.python.org/3/library/timeit.html

https://docs.python.org/3/library/timet.html

فشرده سازی و استخراج فایل ها

همانطور که احتمالاً میدانید فایل ها به فرمت فشرده با فرمت *zip* قابل فشرده سازی هستند اغلب افراد از برنامه های ویژه روی کامپیوتر خود برای باز کردن این فایل ها استفاده میکنند اما خوشبختانه برای ما پایتون با چند خط کد ساده میتواند همین کار را انجام دهد.

ایجاد فایل ها برای فشرده سازی

In [1]:

```
# slashes may need to change for MacOS or Linux
f = open("new_file.txt", "w+')
f.write("Here is some text")
f.close()
```

In [2]:

```
# slashes may need to change for MacOS or Linux
F = open("new_file2.txt", 'w+')
f.write("Here is some text")
f.close()
```

فشرده سازی فایل ها

کتابخانه *zipfile (https://docs. python.org/3/library/zipfile.html)* در پایتون به صورت پیش فرض وجود دارد و میتوان از آن برای فشرده سازی پوشه ها یا فایل ها استفاده کرد برای فشرده سازی تمام فایل ها در یک پوشه، میتوانید از روش *os.walk()* برای تکرار این فرایند برای تمام فایل ها در یک دایرکتوری استفاده کنید.

In [3]:

```
1 import zipfile
```

Create Zip file first, then write to it (the write step compresses the files.)

In [4]:

```
1 comp file zipfile.ZipFile('comp_file.zip', 'w')
```

In [5]:

```
1 comp_file.write("new_file.txt", compress_type=zipfile.ZIP_DEFLATED)
```

In [6]:

```
1 comp_file.write("new_file2.txt',compress_type=zipfile.ZIP_DEFLATED)
```

In [7]:

```
1 comp file.close()
```

استخراج از فایلهای Zip

میتوانیم به راحتی فایل ها را با استفاده از متد ()*extractall* برای استخراج تمام فایل ها یا با استفاده از متد ()*extract* برای استخراج فایل های انفرادی، استخراج کنیم.

In [8]:

```
1 zip_obj = zipfile.ZipFile('comp_file.zip', 'r')
```

In [9]:

```
1 zip_obj.extractall("extracted_content")
```

استفاده از کتابخانه shutil

به طور معمول شما نمیخواهید فایل های انفرادی را از یک فایل *zip*.استخراج با آرشیو کنید بلکه به جای آن همه چیز را در یک بار آرشیو کنید کتابخانه *shutil* که در پایتون به صورت پیش فرض وجود دارد دستورانی آسان برای این منظور دارد:

In [10]:

```
1 import shutil
```

کتابخانه shutil میتواند یک پارامتر فرمت بپذیرد .format فرمت آرشیو است یکی از ."zip". "tar" "gztar bztar"و"xztar".

In [11]:

```
1 pwd
```

'C:\\Users\\babak\\Python For Everyone\\13-Advanced Python Modules'

In [12]:

```
1 directory_to_zip='C:\\Users\\babak\\Python For
  Everyone\\13Advanced Python Modules"
```

In [13]:

```
1 # Creating a zip archive
2 output_filename = 'example'
3 # Just fill in the output filename and the directory to zip
4 # Note this won't run as is because the variable are undefined
5 shutil.make_archive (output_filename, 'zip', directory_to_zip)
```

Out[13]:

'C:\\Users\\babak\\Python For Everyone\\13-Advanced Python Modules\\exampl

e.zip'

In [ ]:

```
1 # Extracting a zip archive
2 # Notice how the parameter/argument order is slightly different here
3 shutil.unpack_archive(output_filename, dir_for_extract_result, 'zip')
```

مروری بر عبارات باقاعده

عبارات باقاعده گاهی به آنها *regex* نیز میگویند به کاربر اجازه میدهد تا با استفاده از هر نوع قانونی که بتوانند تعیین کنند برای رشته ها جستجو کنند به عنوان مثال پیدا کردن تمام حروف بزرگ در یک رشته یا پیدا کردن یک شماره تلفن در یک سند.

عبارات باقاعده به خاطر دستورالعمل های غیر قابل پیش بینی خود شهرت دارند. این دستورالعمل های غیر معمول نتیجه ای از انعطاف پذیری آنها هستند. عبارات باقاعده باید قادر باشند هر الگوی رشته ای را که میتوانید تصور کنید. فیلتر کنند. به همین دلیل قدمت پیچیده ای دارند.

بیایید با توضیحاتی در مورد جستجوی الگوهای پایه در یک رشته شروع کنیم

جستجوی الگوهای پایه

تصور کنید که ما یک رشته به شرح زیر داریم:

In [1]:

```
1 text = "The person's phone number is 408-555-1234. Call soon!"
```

برای شروع میخواهیم بفهمیم آیا رشته *"phone"* در داخل رشته متن وجود دارد میتوانیم این کار را به سرعت با استفاده از متد زیر انجام دهیم:

In [2]:

```
1 'phone' in text
```

Out[2]:

True

اما بیایید قالب عبارات باقاعده را نشان دهیم زیرا بعداً در جستجوی الگوهایی خواهیم بود که راه حل ساده ای ندارند.

In [3]:

```
1 import re
```

In [5]:

```
1 pattern "phone"
```

In [6]:

```
1 re.search(pattern, text)
```

Out [6] :

<re.Match object; span-(13, 18), match-"phone">

In [7]:

```
1 pattern "NOT IN TEXT!"
```

In [8]:

```
1 re.search(pattern, text)
```

همانطور که دیدیم متد ()research الگو را میگیرد متن را اسکن میکند و سپس یک شیء تطبیق را برمی گرداند. اگر الگو پیدا نشود. None برگردانده میشود (در Jupyter Notebook این به معنای این است که هیچ چیزی در زیر سلول نمایش داده نمیشود).

بیایید به طور دقیق تر به این شیء تطبیق نگاهی بیندازیم

In [9]:

```
1 pattern "phone"
```

In [10]:

```
1 match re.search(pattern, text)
```

In [11]:

```
1 match
```

Out [11] :

<re.Match object; span-(13, 18), match="phone">

اگر دقت کنید میتوانید مشاهده نمایید که شی ایجاد شده دارای span میباشد که ایندکس ابتدا و انتهای محلی که الگو یافته شده را برگشته داده است.

In [12]:

```
1 match.span()
```

Out[12]:

(13, 18)

In [13]:

```
1 match.start()
```

out[13] :

13

In [14]:

```
1 match.end()
```

Out[14]:

18

اما در صورتیکه الگو در بیش از یک محل یافت شود نتیجه چه خواهد بود؟

In [16]:

```
1 text "my phone is a new phone"
```

In [17]:

```
1 match = re.search("phone", text)
```

In [18]:

```
1 match
```

Out[18] :

<re. Match object; span-(3, 8), match="phone">

توجه کنید که فقط بازگویی میکند مطابقت را برای اولین وقوع اگر میخواستیم یک لیستی از تمام مطابقت ها را دریافت کنیم میتوانیم از متد finda11. () استفاده کنیم:

In [19]:

```
1 matches = re.findall("phone", text)
```

In [20]:

```
1 matches
```

Out[20]:

['phone', 'phone ']

In [21]:

```
1 len(matches)
```

Out[21]:

2

برای دریافت شی تطبیق واقعی از حلقه for استفاده کنید:

In [22]:

```
1 for match in re.finditer("phone", text):
2   print(match.span())
```

(3, 8)

(18, 23)

اگر می خواستید متن واقعی که مطابقت داشته را دریافت کنید میتوانید از متد *group.* () استفاده کنید.

In [23]:

```
1 match.group()
```

Qut[23] :

'phone'

الگوها

تا الان یاد گرفته ایم چگونه برای یک رشته ساده جستجو کنیم اما درباره موارد پیچیده تر چه؟ به عنوان مثال جستجوی یک شماره تلفن در یک رشته بزرگ یا یک آدرس ایمیل؟

اگر دقیقاً شماره تلفن یا ایمیل را بدانیم میتوانیم از مند جستجو استفاده کنیم اما اگر ندانیم؟ ممکن است فرمت کلی را بدانیم و میتوانیم با استفاده از عبارت منظم *(Regular Expressions)*در مستند برای جستجوی رشته هایی که با یک الگوی خاص مطابقت دارند، استفاده کنیم.

در اینجا ممکن است نحوه نوشتار ابتدایی الگوها به ظاهر عجیب و غریب نظر برسد اما به این موضوع وقت بدهید، اغلب این مسئله تنها مسئله جستجو در کد الگو است.

بیایید شروع کنیم!

شناسه ها برای کاراکترها در الگوها

کاراکترهایی مانند رقم یا یک رشته تکی کدهای مختلفی دارند که آنها را نمایان میکنند میتوانید از این کدها برای ساختن یک رشته الگو استفاده کنید. توجه کنید که این کدها بسیار از برگشتی ۱ استفاده میکنند. به همین دلیل زمان تعریف رشته الگو برای عبارت منظم از فرمت زیر استفاده می کنیم:

r 'mypattern'

قرار دادن *r* در ابتدای رشته به پایتون میگوید که \ در رشته الگو به عنوان برگشتی فرض نمی شود.

در زیر میتوانید جدولی از تمام شناسه های ممکن را مشاهده کنید.

Example Match	Example Pattern Code	Description	Character
File_25	file_ \d\d	A digit	\d
A-b_1	\w-\w\w\w	Alphanumeric	\w
A b c	a\sb\sc	White space	\s
ABC	\D\D\D	A non digit	\D
(=+-*	\W\W\W\W\W	Non-alphanumeric	\W

In [28]:

```
1 text = "My telephone number is 408-555-5555"
```

In [29]:

```
1 phone = re.search(r'\d\d\d\d\d\d\d\d\d\d', text)
```

In [30]:

```
1 phone.group()
```

Out[30]:

*408-555-5555'

توجه کنید که تکرار *\d* کمی مزاحم است به خصوص اگر به دنبال رشته های بسیار طولانی از اعداد باشیم. بیایید از ضرایب ممکن استفاده کنیم.

Quantifiers

حال که ما شناسه های ویژه را میشناسیم میتوانیم از آنها به همراه *Quantifiers* استفاده کنیم تا تعیین کنیم چه تعدادی از آنها را انتظار داریم.

Example Match	Example Pattern Code	Description	Character
Version A-b1.1	Version\w-\w+	Occurs one or more times.	+
abc	\D{3}	Occurs exactly 3 times	{3}
123	\d{2,4}	Occurs 2 to 4 times	{2,4}
anycharacters	\w{3,}	Occurs 3 or more	{3,}
AAACC	A**B*C	Occurs zero or more times	*
plural	?plurals	Once or none	?

حالا بیایید الگوی خودمان را بازنویسی کنیم:

In [ ]:

```
1 Phone =  re.search(r'\d\d\d\d\d\d\d\d\d\d", text)
```

In [31]:

```
1 re.search("\d{3}-\d(3)-\d{4), text)
```

Out[31]:

<re.Match object; span=(23, 35), match='408-555-5555'>

In [32]:

```
def multi_re_find (patterns, phrase):
  '''
  Takes in a list of regex patterns
  Prints a list of all matches
  '''
  for pattern in patterns:
        print ('Searching the phrase using the re check: % %pattern)
        print (re.findall(pattern, phrase))
        print ("\n')
```

In [40]:

```
test phrase 'sdsd..sssddd...sdddddd...dsds...dssss...sdddd'

test patterns = [
   'sd*',  #s followed by zero or more d's s, sd, sad, sadd, sdddd,...
   'sd+', #s followed by one or more d's sd, sdd, sddd, sdddd,…
   'sd?', #s followed by zero or one d's s,sd
  'sd{3}', #s followed by three d's
  'sd{2,3}', #s followed by two to three d's
]

multi_re_find (test patterns, test phrase)
```

Searching the phrase using the re check: 'sd**

['sd', 'sd', 's', 's', 'sddd', 'sddd', 'sddd', 'sd', 's', 's', 's', 'sdddd"]

Searching the phrase using the re check: 'sd+'

['sd', 'sd', 'sddd', 'sddd', 'sddd', 'sd', 'sdddd"]

Searching the phrase using the re check: 'sd?"

['sd', 'sd', 's', 's', 'sd', 'sd', 'sd', 'sd', 's', 's'. 's', 'sd']

Searching the phrase using the re check: 'sd(3) ['sddd', 'sddd', 'sddd', 'sddd']

Searching the phrase using the re check: 'sd{1,3}' ['sd', 'sd', 'sddd', 'sddd', 'sddd', 'sd', 'sddd"']

مجموعه های کاراکتری

مجموعه های کاراکتری وقتی استفاده میشوند که شما میخواهید در یک نقطه از ورودی با هر یک از یک گروه از کاراکترها همخوانی داشته باشید براکتها برای ساخت ورودی های مجموعه های کاراکتری استفاده میشوند به عنوان مثال: ورودی [*ab*] برای جستجوی وقوع هر *a* یا *b* استفاده میشود. بیایید چند نمونه ببینیم:

In [40]:

```
1  test phrase=  'sdsd..sssddd...sdddsddd...dsds...dssss...sdddd'
2
3  test patterns = [
4       '[sd]',
5      '[sd]+',  #sd, sds, sdd, ssd, ds, dss
6     '[sd]?'
7  ]
8
9  multi_re_find(test_patterns, test phrase)
```

Searching the phrase using the re check: '[sd]'

['s', 'd', 's', 'd', 's', 's', 's', 'd', 'd', 'd', 's', 'd', 'd', 'd', s', 'd', 'd', 'd', 'd, 's', 'd', 's', 'd', 's', 's', 's',d', 'd', 'd', 'd']

Searching the phrase using the re check: [sd]+'

['sdsd', 'sssddd', 'sdddsddd', 'dsds', 'dssss', 'sdddd']

Searching the phrase using the re check: [sd]?"

['s', 'd', 's', 'd' , ' ', ' ' , 's' ,'s' , 's' , 'd' , 'd' ,'d' , ' ' , ' ' , ' ' , 's', 'd', 'd' , 'd' ,'s' , ' d' , ' d',

'd', ' ' , ' ' , ' ','d', 's', 'd' , 's ', ' ' , ' ' , ' ', 'd' , 's ', 's ' , 's ' , ' s', ' s', ' ' , ' ' , ' ', 's', 'd', 'd' , 'd' , 'd' , ' ']

گروه ها

اگر بخواهیم دو کار انجام دهیم یعنی پیدا کردن شماره تلفن ها و همچنین قادر به استخراج سریع کد منطقه آنها (سه رقم اول) باشیم، میتوانیم از گروه ها برای هر کار عمومی که شامل تجمیع عبارات منظم استفاده کنیم تا بتوانیم در آینده آنها را تجزیه کنیم.

با استفاده از مثال شماره تلفن میتوانیم گروه هایی از عبارات منظم را با استفاده از پرانتز تفکیک کنیم:

In [44]:

```
1 phone_pattern= re.compile(r'(\d{3})-(\d{3})-(\d{4})')
```

In [45]:

```
1 results = re.search(phone_pattern, text)
```

In [46]:

```
1 # The entire result
2 results.group()
```

Out [46]:

'408-555-5555'

In [47]:

```
1 # Can then also call by group position.
2 # remember groups were separated by parenthesis ()
3 # Something to note is that group ordering starts at 1. Passing in e returns everyth
4 results.group(1)
```

Out[47]:

'408'

In [48]:

```
1 results.group(2)
```

Out[48]:

'555'

In [49]:

```
1 results.group(3)
```

Out[49]:

"5555"

In [50]:

```
1 # We only had three groups of parenthesis
2 results.group(4
```

IndexError

Traceback (most recent call last)

Cell In[50], line 2

1 # We only had three groups of parenthesis

---->2 results.group(4)

IndexError: no such group

نحوه استفاده از نمادهای دیگر در عبارات منظم

عملگر | OR

از عملگر پایپ (|) برای داشتن یک عبارت یا استفاده کنید. به عنوان مثال:

In [51]:

```
1 re.search("man|woman", "This man was here!")
```

out[51] :

<re.Match object; span-(5, 8), match='nan'>

In [52]:

```
1 re.search("man|woman", "This woman was here!")
```

Out[52]:

<re.Match object; span-(5, 10), match='woman'>

کاراکتر Wildcard

از "wildcard" به عنوان یک مکانی استفاده کنید که هر کاراکتری را مطابقت خواهد داد. میتوانید از نقطه ساده (.) برای این منظور استفاده کنید. به عنوان مثال:

In [53]:

```
1 re.findall(.at", "The cat in the hat sat here")
```

Out[53]:

['cat', 'hat', 'sat']

In [54]:

```
1 re.findall("at", "The bat went splat")
```

Out[54]:

['bat', 'lat']

توجه کنید که ما فقط سه حرف را مطابقت دادیم این به این دلیل است که برای هر حرف *wildcard* به یک نقطه (.) نیاز داریم. یا از *quantifiers* مشروح بالا برای تعیین قوانین خود استفاده کنید.

In [55]:

```
1 re.findall("...at", "The bat went splat")
```

Out[55]:

['e bat', 'splat']

اما این همچنان مشکل باز گرفتن بیشتر را با خود به همراه می آورد. در واقع ما فقط می خواهیم کلماتی را که با " "ar"ختم میشوند. دریافت کنیم.

In [56]:

```
1 #One or more non-whitespace that ends with 'at'
2 re.findall("Seat", "The bat went splat")
```

Out[56]:

['e bat', 'splat']

شروع با و پایان با

ما میتوانیم از علامت برای نشان دادن شروع با استفاده کنیم و از علامت برای نشان دادن پایان با:

In [63]:

```
1 #Ends with a number
2 re.findall('\ds', 'This 32 ends with a number 2')
```

Out[63]:

['52']

In [61]:

```
1 Starts with a number
2 re.findall(d', '1 is the loneliest number.')
```

Out [61]:

['1']

توجه کنید که این برای تمام رشته است که کلمات فردی!

استثنا

برای استثنای کاراکترها میتوانیم از علامت در ترکیب با یک مجموعه از براکت ها استفاده کنیم هر چیزی درون براکت ها استثنا میشود. به عنوان مثال:

In [65]:

```
1 phrase = "there are 3 numbers 34 inside 5 this sentence."
```

Out[66]:

['t',

'h',

'e',

'r',

'e',

' ',

'a',

'r',

'e',

' ',

' ',

'n',

'u',

'm',

'b',

'e',

'r',

's',

' ',

' ',

'i',

'n',

's',

'i',

'd',

'e',

' ',

' ',

't',

'h',

'i',

's',

' ',

's',

'e',

'n',

't',

'e',

'n',

'c',

'e',

'.',]

برای دریافت خروجی در قالب کلمات از علامت + استفاده می کنیم:

In [67]:

```
1 re.findall('[^\d]+", phrase).
```

Out[67]:

['there are ', numbers, inside, this sentence.]

از حالت می توانیم برای حذف punctuation ها در متون استفاده کنیم

In [68]:

```
1 test phrase = 'This is a string! But it has punctuation. How can we remove it?'
```

In [69]:

```
1 re.findall([^,]+', test phrase)
```

Out[69]:

['This',

'is',

'a',

'string'.

'But',

'it',

'has'.

'punctuation,

'How '

'can',

'remove',

'it']

In [70]:

```
1 clean = ' '.join(re.findall([^,]+', test phrase))
```

In [71]:

```
1 clean
```

Out[71]:

'This is a string But it has punctuation How can we remove it'

پرانتز برای چندین گزینه

اگر برای تطبیق با چندین گزینه میتوانیم از پرانتز استفاده کنیم تا این گزینه ها را نیست کنیم. به عنوان مثال:

In [72]:

```
1 # Find words that start with cat and end with one of these options: 'fish', 'nap', or
  text = 'Hello, would you like some catfish?'
  texttwo =  "Hello, would you like to take a catnap?"
  textthree = "Hello, have you seen this caterpillar?"
```

In [73]:

```
1 re.search('cat (fish nap claw), text)
```

Out [73]:

<re.Match object; span=(27, 34), match='catfish'>

In [74]:

```
1 re.search(r'cat (fish|nap | claw)',texttwo)
```

Out[74]:

<re.Match object; span=(32, 38), match='catnap'>

In [75]:

```
1 # None returned
2 re.search('cat (fish nap claw), textthree)
```

محدوده کاراکترها

همانطور که مجموعه های کاراکتری بزرگتر میشوند تایپ کردن هر کاراکتری که باید (یا نباید) متناسب باشد، خیلی خسته کننده میشود. با استفاده از فرمت محدوده کاراکترها میتوانید مجموعه کاراکتری را تعریف کنید که همه کاراکترهای متوالی بین یک نقطه شروع و پایان را شامل شود فرمت استفاده شده برای این منظور[شروع- پایان] است مورد استفاده رایج برای جستجوی یک محدوده خاص از حروف الفیاست، مانند [*at]*که هر نمونه حروفی بین *a* و *f* را برمی گرداند.

بیایید به سراغ برخی از مثال ها برویم:

In [76]:

```
1 test phrase =  'This is an example sentence. Lets see if we can find some letters."
2 test patterns =  [ '[a-z]+', # sequences of Lower case Letters
3                    [A-Z]+', # sequences of upper case Letters
4                    [a-zA-Z]+', # sequences of Lower or upper case Letters
5                  [A-Z][a-z]+] # one upper case Letter followed by Lower case Lette
6
7 multi_re_find(test_patterns, test_phrase)
```

Searching the phrase using the re check: '[a-z]+'

['his', 'is', 'an', 'example', 'sentence', 'ets', 'see', 'if', 'we', 'ca n', 'find', 'some', 'letters']

Searching the phrase using the re check: '[A-Z]+' ['T', 'L']

Searching the phrase using the re check: [a-zA-Z]+'

['This', 'is', 'an', 'example', 'sentence', 'Lets', 'see', 'if', 'we' n', 'find', 'some', 'letters']

Searching the phrase using the re check: '[A-Z][a-z]+" ['This', 'Lets']

جمع بندی

کارتان عالی بود برای اطلاعات کامل در مورد همه الگوهای ممکن به آدرس زیر مراجعه کنید:

https://docs.python.org/3/howto/regex.html

(https://docs.python.org/3/howto/regex.html)

In [65]:

```
1 re.findall(r'[\d]', phrase)
```

راهنمای استخراج اطلاعات از وب

بیایید شما را با استخراج اطلاعات از وب و پایتون آشنا کنیم قبل از شروع برخی از قوانین مهمی که باید رعایت و درک کنید را معرفی میکنیم

۱. همیشه محترمانه عمل کنید و سعی کنید اجازه استخراج اطلاعات را دریافت کنید. از سایت ها درخواست های استخراج اطلاعات بدون مجوز نکنید در غیر اینصورت آدرس *IP* شما ممکن است مسدود شود!
۲. به خاطر داشته باشید که وبسایت ها به طور مداوم تغییر میکنند به این معنی که که شما ممکن است از کار بیافتد و کاملاً خراب شود.
۳. تقریباً هر پروژه استخراج اطلاعات مورد علاقه یک پروژه منحصر به فرد و سفارشی است، بنابراین تلاش کنید تا مهارت های یاد گرفته شده را به صورت کلی تعمیم دهید.

خوب بیایید با مباحث پایه شروع کنیم!

اجزای پایه یک وب سایت

HTML

HTML مخفف عبارت *Hypertext Markup Language* است و هر وب سایتی در اینترنت از آن برای نمایش اطلاعات استفاده میکند. حتی سیستم دفترچه یادداشت جوپیتر از آن برای نمایش این اطلاعات در مرورگر شما استفاده میکند. اگر روی یک وب سایت راست کلیک کرده و گزینه *View Page Source* را انتخاب کنید میتوانید *HTML* خام یک صفحه وب را ببینید. این اطلاعاتی هستند که پایتون برای استخراج اطلاعات از آنها استفاده میکند بیایید به یک نمونه ساده از *HTML* یک صفحه وب نگاهی بیندازیم:

```
<!DOCTYPE html>
  <html>
    <head>
      <title/> عنوان تب مرورگر <title>
    </head>
   <body>
     <h1/) در وب سایت (h1>
     <p/> یک پاراگراف <p>
  <body>
</html>:
```

بیایید این اجزا را تجزیه کنیم.

هر برچسب *<tag>* نوع مشخصی از بلوک را در صفحه وب نشان میدهد:

۱. *<DOCTYPE html>* سند *HTML* همیشه با این اعلام نوع آغاز میشود تا مرورگر بداند که این یک قابل *HTML* است
۲. بلوک های اجزای سند *HTML* بین *<html>* و *</html>* قرار میگیرند.
۳. متادینا و اتصالات (اسکربیت مانند لینک به یک فایل *CSS* یا یک فایل) معمولاً در بلوک *<head>* قرار می گیرند.
۴. بلوک برچسب *<title>* عنوان صفحه وب را تعریف میکند(که در تب یک وب سایتی که شما در آن قرار دارید نمایش داده میشود).

۵. بین برچسب های *<body>*و *</body>*بلوک هایی قرار میگیرند که قابل مشاهده برای بازدیدکنندگان سایت

CSS

CSS مخفف عبارت *Cascading Style Sheets* است. این است که استایل را به یک وب سایت میدهد. از جمله رنگ ها و قلم ها و حتی برخی از انیمیشن ها *CSS* از برچسبهای مانند *id* یا *class* برای اتصال یک عنصر *HTML* به یک ویژگی *CSS* استفاده میکند مانند رنگ خاصی *id* یک شناسه منحصر به فرد برای برچسب *HTML* است و باید در سند *HTML* منحصراً یکتا باشد به طور کلی یک اتصال یکباره است. *class* یک استایل عمومی را تعریف میکند که سپس میتواند به چندین برچسب *HTML* مرتبط شود. در واقع اگر میخواهید فقط یک برچسب *HTML* را قرمز کنید از برچسب *id* استفاده خواهید کرد و اگر میخواهید چندین برچسب بلوک *HTML* را قرمز کنید، یک کلاس در سند *CSS* خود ایجاد کرده و سپس آن را به بقیه این بلوک ها متصل خواهید کرد.

راهنمایی های *Scraping*

در نظر داشته باشید که همیشه برای وب سایتی که قرار است اطلاعات آن را استخراج کنید باید اجازه بگیرید! شرایط و ضوابط یک وب سایت را بررسی کنید تا اطلاعات بیشتری در این زمینه بدست آورید. همچنین به یاد داشته باشید که یک کامپیوتر میتواند درخواست ها را به سرعت بسیار زیادی به یک وب سایت ارسال کند بنابراین اگر درخواستهای زیادی را به سرعت بیش از حد ارسال کنید وب سایت ممکن است آدرس *IP* کامپیوتر شما را مسدود کند. در نهایت، وب سایت ها به طور مداوم تغییر میکنند احتمالاً برای وظایف استخراج اطلاعات بلندمدت باید کد خود را به طور مکرر به روز رسانی کنید.

استخراج اطلاعات از وب با استفاده از *Python*

چندین کتابخانه نیاز خواهید داشت. میتوانید به خط فرمان خود بروید و آنها را با استفاده از دستور *conda install*(اگر از توزیع *Anaconda* استفاده میکنید) یا *pip install* برای سایر توزیع های پایتون نصب کنید

```
conda install requests

conda install 1xn1

conda install bs4
```

اگر از نصب *Anaconda* استفاده نمیکنید میتوانید به جای *conda install* از *pip install* استفاده کنید.
به عنوان مثال:

pip install requests

pip install 1x1

pip install bs4

حالا بیایید ببینیم با این کتابخانه ها چه کارهایی می توانیم انجام دهیم.

تسک نمونه . - دریافت عنوان یک صفحه

بیایید از یک مثال بسیار ساده شروع کنیم و عنوان یک صفحه را دریافت کنیم به یاد داشته باشید که این بلوک *HTML* با برچسب *title* است. برای این تسک ما از

wwnx.example.com (http://www.example.com استفاده میکنیم که یک وب سایت به صورت ویژه برای استفاده به عنوان دامنه نمونه ساخته شده است بیایید از مراحل اصلی عبور کنیم:

In [1]:

```
1 import requests
```

In [2]:

```
1 # Step 1: Use the requests Library to grab the page
2 # Note, this may fail if you have a firewall blocking Python/Jupyter
3 # Note sometimes you need to run this twice if it fails the
4 res = requests.get("http://www.example.com")
```

این شیء یک شیء requests.models Response است و در واقع حاوی اطلاعات وب سایت است.
به عنوان مثال:

In [3]:

```
1 type(res)
```

Out[3]:

requests.models. Response

In [4]:

```
1 res.text
```

Out[4]:

<!doctype html>\n<html>\n<head>\n <meta charset="utf-8" />\n t/html; charset=utf-8" />\n -width, initial-scale=1" />\n

<title>Example Domain</title>\n\n <meta http-equiv="Content-type" content="tex <meta name="viewport" content="width=device <style type="text/css">\n body {\n margin: 0;\n padding: 0;\n

font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Op en Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;\n

background-color: #fefef2; \n

div {\n

\n

ding: 2em; \n em; \n

box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02); \n }\n a:1

\n

width: 600px;\n margin: Sem auto; \n background-color: #fdfdff;\n

pad border-radius: 0.5

color: #38488f;\n

@media (max-width: 700px) {\n

width: auto; \n

ink, a:visited {\n e; \n } \n

text-decoration: non div {\n

mar

gin: 0 auto; \n } \n }\n </style> \n</head>\n\n<body>\n \n <h1>Example Domain</h1>\n This domai n is for use in illustrative examples in documents. You may use this\n domain in literature without prior coordination or asking for permission. \n More informati

on... \n \n</body>\n</html>\n"

حالا از *BeautifulSoup* برای تجزیه و تحلیل صفحه استخراج شده استفاده میکنیم در واقع میتوانستیم از اسکریپت سفارشی خودمان برای جستجوی موارد در رشته *res. text* استفاده کنیم اما کتابخانه *BeautifulSoup* قابلیت ها و روش های زیادی برای دریافت اطلاعات از یک رشته از این نوع به طور کلی یک فایل *HTML* دارد با استفاده از *BeautifulSoup* میتوانیم یک شیء "*soup* ایجاد کنیم که حاوی مواد صفحه وب است نپرسید من اسامی عجیب کتابخانه ها را انتخاب

In [5]:

```
1 import bs4
```

In [6]:

```
1 soup = bs4.BeautifulSoup (res. text, "lxml")
```

In [7]:

```
1 soup
```

Out[7]:

<!DOCTYPE html>

<html>

<head>

<title>Example Domain</title>

<meta charset="utf-8"/>

<meta content="text/html; charset=utf-8" http-equiv="Content-type"/> <meta content="width=device-width, initial-scale=1" name="viewport"/> <style type="text/css">

body {

background-color: #fefef2;

margin: 0;

padding: 0;

font-family: apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue', Helvetica, Arial, sans-serif;

}

div {

}

width: 600px;

margin: 5em auto;

padding: 2em;

background-color: #fdfdff;

border-radius: 0.5em;

box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02);

a:link, a:visited {

}

color: #38488f;

```
text-decoration: none;
@media (max-width:700px) {
div {
margin: 0 auto;
width: auto;
}
</style>
</head>
<body>
<div>
<h1>Example Domain</h1>
<p>This domain is for use in illustrative examples in documents. You may u se this
domain in literature without prior coordination or asking for permissi on.</p>
<p><a href="https://www.iana.org/domains/example">More information...</a>
</p>
</div>
</body>
</html>
```

حالا بیایید از متد *select*() برای دریافت عناصر استفاده کنیم. ما به دنبال برچسب *'title*هستیم، بنابراین عبارت *title* را به عنوان ورودی ارسال می کنیم.

In [8]:

```
1 soup.select('title')
```

Out[8]:

[<title>Example Donain</title>]

توجه کنید که چه چیزی در اینجا برگشت داده میشود در واقع یک لیست است که حاوی تمام عناصر عنوان (با برچسب هایشان) است. شما میتوانید با استفاده از ایندکس گذاری یا حتی حلقه بازیابی عناصر را از لیست دریافت کنید. از آنجا که این شیء هنوز یک برچسب ویژه است میتوانیم از فراخوانی های مند برای دریافت متن فقط استفاده کنیم.

In [9]:

```
1 title tag soup.select('title')
```

In [10]:

```
1 title_tag[0]
```

Out[10]:

<title>Example Domain</title>

In [11]:

```
1 type(title_tag[0])
```

Out[11]:

bs4.element.Tag

In [12]:

```
1 title_tag[0].getText()
```

Out[12]:

'Example Domain'

تسک نمونه ـ ۱ دریافت همه عناصر یک کلاس

بیایید سعی کنیم تمام عناوین بخش های مقاله ویکی پدیا درباره Grace Hopper را از این URL دریافت کنیم

https://en.wikipedia.org/wiki/Grace Hopper (https://en.wikipedia.org/wiki/Grace Hogger)

In [13]:

```
# First get the request
res = requests.get('https://en.wikipedia.org/wiki/Grace Hopper')
```

In [14]:

```
# Create a soup from request
soup = bs4. BeautifulSoup (res.text, "lxml")
```

حالا وقت آن رسیده است که بفهمیم به دنبال چه چیزی هستیم عنصر را در صفحه بررسی کنید و ببینید که عناوین بخش ها دارای کلاس "*mw-headline*" هستند. از آنجا که این یک کلاس است و نه یک برچسب ساده باید به برخی از نحوه نوشتن *CSS* پایبند باشیم. در این حالت:

Match Results	Syntax to pass to the .select()
All elements with the tag	soup.select('div')
The HTML element containing the id attribute of some id	soup.select('#some_id')
All the HTML elements with the CSS class named notice	soup.select('.notice')
<Any elements named that are within an element named <div	soup.select('div span')
Any elements named that are directly within an element named . with no other element in between	soup.select('div> span')

In [15]:

```
# note depending on your IP Address,
#this class may be called something different
soup.select(".mw-parser-output")
```

<a class="mw-redirect" href="/wiki/Enterprise_Unified_Process" tit
le="Enterprise Unified Process">EUP

Executable U
ML

<a href="/wiki/Incremental_build_model" title="Incremental build m
odel">Incremental model

<a href="/wiki/Iterative_and_incremental_development" title="Itera tive and
incremental development">Iterative model

Prototype model

<a href="/wiki/Rapid_application_development" title="Rapid applica tion
development">RAD

>

UP</li

<a href="/wiki/Scrum (software development)" title="Scrum (softwar e
development)">Scrum

Spiral model

<a class="mw-redirect" href="/wiki/V-Model_(software development)"
title="V-Model (software develonnent)">V-Model

In [17]:

```
for item in soup.select(".mw-parser-output"):
    print("*****************************")
    print(item.text)
```

Mary Engle Pennington

Mercy Otis Warren

2003

Linda G. Alvarado

Donna de Varona

Gertrude Ederle

Martha Matilda Harper

Patricia Roberts Harris

Stephanie L. Kolek

Dorothea Lange

Mildred Robbins Leet

Patsy Takenoto Mink

Sacagawea

Anne Sullivan

Sheila E. Widnall

2005

Florence E. Allent

Ruth Fulton Benedict

Betty Bumpers

Hillary Clinton

تسک نمونه ۳ دریافت تصویر از یک وبسایت

بیایید سعی کنیم تصویر رایانه دیپ بلو را از این مقاله ویکی پدیا دریافت کنیم:

https://en.wikipedia.org/wiki/Deep Blue (chess computer)

(https://en.wikipedia.org/wiki/Deep Blue (chess computer))

In [18]:

```
1 res requests.get("https://en.wikipedia.org/wiki/Deep Blue (chess_computer)")
```

In [19]:

```
1 soup bs4. BeautifulSoup (res.text, 'lxml')
```

In [20]:

```
1 image_info= soup.select('.thumbimage")
```

In [21]:

```
1 image info
```

Out [21]:

[<img class="thumbimage" data-file-height="600" data-file-width="800" deco ding-"async" height="165" src="//upload.wikimedia.org/wikipedia/commons/th umb/6/6f/Kasparov_Magath_1985 Hamburg-2.png/220px-Kasparov_Magath_1985_Ham

burg-2.png"

srcset="//upload.wikimedia.org/wikipedia/commons/thumb/6/6f/Ka sparov Magath 1985_Hamburg-2.png/330px-Kasparov_Magath_1985_Hamburg-2.png 1.5x, //upload.wikimedia.org/wikipedia/commons/thumb/6/6f/Kasparov_Magath 1985 Hamburg-2.png/440px-Kasparov_Magath_1985_Hamburg-2.png 2x" width="22 8"/>,

<img class="thumbimage" data-file-height="2756" data-file-width="2067" de coding="async" height="293" src="//upload.wikimedia.org/wikipedia/commons/ thumb/8/83/One_of_Deep Blue%27s_processors_%282586060990%29.jpg/220px-One_ of Deep Blue%27s_processors_%282586860990%29.jpg" srcset="//upload.wikimed ia.org/wikipedia/commons/thumb/8/83/One_of_Deep Blue%27s_processors_%28258

6060990%29.jpg/330px-One_of_Deep Blue%27s_processors_%282586060990%29.jpg

1.5x, //upload.wikimedia.org/wikipedia/commons/thumb/8/83/One_of_Deep Blu

ex%27s_processors_%282586060990%29.jpg/440px-One_of_Deep Blue%27s_processor s_%282586060990%29.jpg 2x" width="220"/>]

In [22]:

```
1 len(image_info)
```

Out [22]:

2

In [23]:

```
1
2 computer image_info[e]
```

In [24]:

```
1 type(computer)
```

Out [24]:

bs4.element.Tag

شما میتوانید برای بخش هایی از برچسب تماس های دیکشنری مانند صدا زده شوید در این حالت، ما به src یا منبع تصویر علاقه مند هستیم که باید لینک مجزا .jpg یا png تصویر باشد

In [25]:

```
1 computer['src']
```

Out[25]:

//upload.wikimedia.org/wikipedia/commons/thumb/6/6f/Kasparov_Magath_1985_

Hamburg-2.png/220px-Kasparov_Magath_1985_Hamburg-2.png'

ما میتوانیم آن را با یک سلول مارکدان با استفاده از موارد زیر نمایش دهیم:

<img src='https://upload.wikimedia.org/wikipedia/commons/thumb/b/be/Deep Blue.j Pg/220px-Deep Blue.jpg'>

حال که لینک واقعی *src* را دارید میتوانید با استفاده از *requests* تصویر را دریافت کنید و با ویژگی *content* آن را دریافت کنید. توجه کنید که باید قبل از لینک *https://* را اضافه کنید اگر این کار را انجام ندهید *requests* اعتراض خواهد کرد(اما یک کد خطای خیلی شرح دهنده را نمی دهد.)

In [26]:

```
1 image_link=requests.get('https://upload.wikimedia.org/wikipedia/commons
  /thumb/b/be
```

In [27]:

```
1 The row content (its a binary file, meaning we will need to use binary read/write
2 image_link.content
```

SS\nvix13\xc3\xce\xdc=\xb1\x1dM=\xa7\xa53" ", \x1a\xf3\x9ayM1 \x835 \x15\xa

5\xd6$V\x9c:5\x80\xfdm\xc3\xe5]M\x86\x96\x8d{\x37\xed\xaez\xcfw\xdd

\xds\x09\xes j\xcf\xee\xdf\xba\x96dE^\xe1\xae\x10\xe2^\x0f\xc2\xc9\xe2x

del's\x89/a\xdbt>\xda\x1b\xb7E\xbbGhoR\x10\x84\xe8\xac\xed\x82U?\xa3x1
1\xd4\xbc4g\xbc\x9f\xB1\xaeC\x03\xe2\x8e$\xba\xc7\x19v\xdb\x96\xb6\x9c5

\x85\xa9\xcf\xae\xbb\xc6\xb1\xd1qvZ2U\xd9\xdb5)8\x893\x64\xc6\xf5\xd55)

\xdb\xdd\x03}\xf2\xdcyv=\x84\xf4\x16$\x89\xd3j=$h7\x83\xad\"305\xa7Y\xa

5-\x36\x80\xad\x87\'Ph\xd4\x3d\xa8-\x80\x64\xa3\x906\xcb\xa9;P\x69\xa

7\x9f1\x7fj\xae\xbd\xb2\x0f\xfaB\xbc\xcd\xe2$z\xaf\x16\xf1+A27\x8b\xdf

\r\xb0\x8b\x85\xff\x80\n\xbfX\xff\x886\xb0\x85\x91\x89)X\xb0\xa5\xf61

\x85\x95\x0F\xa2\xd9R\x86\xae>\xde\xgd\xed\x03X\x93:\t\xaf=\xbik\xcb\x9

blxoe%\xc6\x9f\xc4R\x11wufqp\xfaP\x02R\x16\xb7\n\xcc\xde\x83\xbdw\xcalx

BF\xc3M\x94g\x8c\x92V\x83R\xe9\x8e8\x01\xd9\xb5\x14\xcfC\x14P+\x01+PPA$

$}\t\x11\xa5\x01\xb7\x871\xb51Kj\x84\x7fd\xcc\xfd\x94Wn\xa2\x823\x12\x9

Địxa2\'B\xB4\xc7\xca\xb5\x99\xB3\xB3\x88\xab/xF\xbbm\xe78\x@b\x12*8\x
Bcx\xd2\\\xe0\x88\x2F\xb3\x9a\x87 xe9 x98\xd2w\xa2\x80\xd5\xac\x16\x

c8\x981\x1b5 \x84\xa1\xean\xe4)\x80\xb5-\x8ct\x14\xbdR$\x02\x8f\x9c\xd3

^1x865\xea\x8e%8\x8f\xacTA\xf2\x15: \x8dx\xed\x11\x1a\xc88\x8e\xa0\xc5gj
\x90b0\xc6\xbaE\x15\x99pr\x10\x93: \x9c\xda\x9a\xd1Q\xd0\x82\x15\07\xc6

c\xca\x80\x3\n\x93\x98\xead=?\x9d/\xbc\xc0\x15\x19\x80\xer\xd1\xRF\xd

بیایید این را به یک فایل بنویسیم به کارگیری *wb* برای نشان دهی نوشتار بایتری فایل.

In [28]:

```
1 fopen('my_new_file_nane.jpg", "wb')
```

In [29]:

```
1 f.write(image link. content)
```

out[29] :

18448

In [30]:

```
1 f.close()
```

حال میتوانیم این فایل را در این نوت بوک به صورت مارک دانک نمایش دهیم با استفاده از

<<img src=" 'my_new_file_nane.jpg">>

فقط این خط را در یک سلول جدید مارک دانک بنویسید و تصویری که دریافت کردیم را نمایش خواهد داد!

پروژه - نمونه کار با صفحات و آیتمهای متعدد

بیایید مثالی واقع گرایانه تر از جمع آوری اطلاعات یک سایت را نشان دهیم. وبسایت (*iite://books teactape.com/index.html (htto://books. tasatane.com/index.html* به طور خاص برای جمع اوری اطلاعات طراحی شده است. بیایید سعی کنیم عنوان هر کتابی که دارای امتیاز ۲ ستاره است را دریافت کنیم و در نهایت فقط یک لیست پایتونی با عناوین آنها را داشته باشیم.

ما مراحل زیر را انجام خواهیم داد:

- شناسایی ساختار *URL* برای مرور تمام صفحات
- جمع آوری اطلاعات از هر صفحه در کاتالوگ
- شناسایی تک کلاسی که امتیاز ستاره را نمایش میدهد

- فیلتر کردن بر اساس این امتیاز ستاره با استفاده از یک عبارت *if*
- ذخیره نتایج در یک لیست

می توانیم ببینیم که ساختار *URL* به صورت زیر است:

http://books.toscrape.com/catalogue/page-1.html

In [31]:

```
1 base_url = http://books.toscrape.com/catalogue/page-.htm()
```

میتوانیم سپس شماره صفحه را با استفاده از ()format پر کنیم

In [32]:

```
1 res = requests.get(base_url.format('1'))
```

حالا بیایید محصولات کتابها را از نتیجه درخواست *get* بگیریم:

In [33]:

```
1 soup bs4. BeautifulSoup(res.text, "1xml")
```

In [34]:

```
1 soup.select(".product_pod")
```

<form>

In stock

<button class="btn btn-primary btn-block" data-loading-text="Addin 8...
type="submit">Add to basket</button>

</form>

</article>,

<article class="product_pod">

<img alt="Sharp Objects" class
="thumbnail" src="../media/cache/32/51/3251cf3a3412f53f339e42cac213409
3.jpg"/>

<p class="star-rating Four">

حالا میتوانیم ببینیم که هر کتاب دارای کلاس product pod است. ما می توانیم هر برچسبی با این کلاس را انتخاب کنیم و سپس آن را بر اساس امتیاز آن کاهش دهیم.

سپس آن را بر اساس امتیاز آن کاهش دهیم.

In [35]:

```
1 products- soup.select(".product_pod")
```

In [36]:

```
1 example products[0]
```

In [37]:

```
1 type (example)
```

Out[37]:

bs4.element.Tag

In [38]:

```
1 example.attrs
```

Out[38]:

('class': ['product_pod']}

حالا با بازرسی سایت میتوانیم ببینیم که کلاسی که ما میخواهیم به شکل *class='starrating Two* است. اگر بر روی آن در مرورگر خود کلیک کنید متوجه خواهید شد که فضای خالی به عنوان یک نمایش داده می شود. بنابراین این بدان معناست که ما باید برای *star rating Two"* جستجو کنیم.

In [39]:

```
1 list(example.children)
```

Out[39]:

I'\n'

.

<a href-a-light-in-the-attic_1008/index.html"><img alt="A Light in the A ttic" class="thumbnail" src="../media/cache/2c/da/2cdad67c44b002e7ead8cc35

693c8e8b.jpg"/>

,

,

'\n',

<h3><a href-a-light-in-the-attic_1008/index.html" title="A Light in the Attic A Light in the...</h3>,

'\n',

A£51.77

<form>

In stock

<button class="btn btn-primary btn-block" data-loading-text="Adding..."t
ype="submit">Add to basket</button>

</form>

, '\n']

In [40]:

1 example.select(.star-rating. Three")

In [40]:

1	*example.select(.star-rating. Three")*

Out [40]:

[

</p>]

اما ما به دنبال ۲ ستاره هستیم پس به نظر میرسد میتوانیم بررسی کنیم که آیا چیزی برگردانده شده است یا خیر.

In [41]:

```
1 example.select(.star-rating. Two')
```

Out [41]:

[ ]

به طور جایگزین میتوانیم به سرعت رشته متن را بررسی کنیم تا ببینیم آیا star rating Two در آن وجود دارد یا خیر. هر دو رویکرد قابل قبول است (همچنین رویکردهای دیگری نیز وجود دارند!)

حالا بیایید ببینیم چگونه میتوانیم عنوان را در صورت مطابقت با ۲ ستاره دریافت کنیم:

In [42]:

```
1 example.select('a')
```

Out[42]:

[,

A Light in the...]

In [43]:

```
1 example.select('a')[1]
```

Out[43]:

A Light in the...

In [44]:

```
1 example.select('a')[1]['title']
```

Out[44]:

'A Light in the Attic'

خوب بیایید با ترکیب همه ایده هایی که درباره آن صحبت کرده ایم آن را امتحان کنیم اجرای این که حدود ۲۰-۶۰ ثانیه طول میکشد. لطفاً به این نکته توجه کنید که اگر دیوار آتشی مانع اجرای این اسکربیت شود. همچنین اگر خطای عدم پاسخی دریافت میکنید شاید بهتر باشد یک گام تاخیر با استفاده از (۱) *time sleep* اضافه کنید.

In [45]:

```
1  two_star_titles = []
2
3  for n in range(1,51):
4
5      scrape_url base_url.format(n).
6      res= requests.get(scrape_url)
7
8     soup = bs4. BeautifulSoup (res.text, "1xml")
9     books = soup.select(".product_pod")
10
11    for book in books:
12        if len(book.select(.star-rating. Two')) != 0:
13            two_star_titles.append(book.select('a')[1]['title'])
```

In [46]:

```
1 two_star_titles
```

Out [46]:

['Starving Hearts (Triangular Trade Trilogy, #1), 'Libertarianism for Beginners',

"It's Only the Himalayas",

'How Music Works',

*Maude (1883-1993): She Grew Up with the country',

"You can't bury then all: Poems",

'Reasons to Stay Alive',

'Without Borders (Wanderlove #1)',

'Soul Reader',

'Security',

'Saga, Volume 5 (Saga (Collected Editions) #5)',

'Reskilling America: Learning to Labor in the Twenty-First Century',

'Political Suicide: Missteps, Peccadilloes, Bad Calls, Backroom Hijin

x, Sordid Pasts, Rotten Breaks, and Just Plain Dumb Mistakes in the Ann als of American Politics',

Obsidian (Lux #1)",

"My Paris Kitchen: Recipes and Stories',

"Masks and Shadows'.

عالی حالا ابزارهای لازم برای استخراج اطلاعات از هر وبسایتی که به شما علاقه مند است را دارید به خاطر داشته باشید که هر چه وبسایت پیچیده تر باشد استخراج آن دشوارتر خواهد بود. همیشه اجازه بگیرید!

تمرین های استخراج اطلاعات از وب

انجام تمرین های زیر

تمرین هر کتابخانه ای که فکر میکنید برای استخراج اطلاعات از یک وبسایت نیاز دارید، وارد کنید.

In [28]:

```
1 # CODE HERE
```

In [1]:

```
1 import requests, bs4
```

تمرین از کتابخانه requests وBeautifulSoup برای اتصال به
http://squotes.descrape.com) /)استفاده کنید و متن HTML صفحه اصلی را دریافت کنید.

In [30]:

```
1 # CODE HERE
```

In [2]:

```
1 res = requests.get("http://quotes.toscrape.com/")
```

In [3]:

```
1 res.text
```

\t<title>Quotes to Scrape</title>\n

atic/bootstrap.min.css">\n n.css">\n</head>\n<body>\n ss="row header-box">\n
<h1>\n

tes to Scrape\n

<link rel="stylesheet" href="/st

<div cla

<link rel="stylesheet" href="/static/mai

\n

\n

Quo

\n Login\n \n

</h3>\n \n \n

\n In

\n\

\n \n\n \n <div class="col-

nd-8">\n\n <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\n "The world a s we have created it is a process of our thinking. It cannot be changed. without changing our thinking."\n

by <small class

"author" itemprop="author">Albert Einstein</small>\n author/Albert-Einstein">(about)\n

s="tags">\n

Tags: \n

\n

<a href="/ <div clas <meta class="keywords" itempr >\n

op-"keywords" content="change, deep-thoughts, thinking, world" / change\n deep-tho

\n

\n

ughts\n

<a class="rap" href="/tag/thinkin

تمرین نام نویسندگان موجود در صفحه اول را بدست آورید.

In [4]:

```
1 #CODE HERE
2 soup bs4.BeautifulSoup (res.text, 'lxml')
```

In [5]:

```
1 soup.select('.author")
```

Out[5]:

[<small class="author" itemprop="author">Albert Einstein</small>,

<small class="author" itemprop="author">3.K. Rowling</small>,

<small class="author" itemprop="author">Albert Einstein</small>,

<small class="author" itemprop="author">Jane Austen</small>,

<small class="author" itemprop="author">Marilyn Monroe</small>,

<small class="author" itemprop="author">Albert Einstein</small>,

<small class="author" itemprop="author">André Gide</small>,

<small class="author" itemprop="author">Thomas A. Edison</small>,

<small class="author" itemprop="author">Eleanor Roosevelt</small>,

<small class="author" itemprop="author">Steve Martin</small>]

In [6]:

```
1  authors set()
2
3  for name in soup.select(".author"):
4      authors.add(nane.text)
```

In [7]:

```
1  authors
```

Out[7]:

('Albert Einstein',

'André Gide',

'Eleanor Roosevelt,

1.K. Rowling'.

Jane Austen', 'Marilyn Monroe', 'Steve Martin', 'Thomas A. Edison')

تمرین یک لیست از تمامی نقل قولها در صفحه اول ایجاد کنید.

In [13]:

```
1  #CODE HERE
```

In [8]:

```
1  quotes = [ ]
2
3  for quote in soup.select(".text"):
4      quotes.append(quote.text)
```

In [9]:

```
1  quotes
```

Out[9]:

["The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking.",

"It is our choices, Harry, that show what we truly are, far more than ou rabilities.",

"There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle.",

"The person, be it gentleman or lady, who has not pleasure in a good nov el, must be intolerably stupid.",

"Imperfection is beauty, madness is genius and it's better to be absolut ely ridiculous than absolutely boring.",

Try not to become a man of success. Rather become a man of value.", "It is better to be hated for what you are than to be loved for what you are not.",

"I have not failed. I've just found 18,880 ways that won't work."",

""A woman is like a tea bag; you never know how strong it is until it's i

n hot water."",

"A day without sunshine is like, you know, night."]

تمرین : سایت را بررسی کنید و از Beautiful Soup برای استخراج ده برچسب برتر از متن درخواست نمایش داده شده در بالا سمت راست صفحه اصلی استفاده کنید(به عنوان مثال Love, Inspirational Life و غیره) راهنمایی: به خاطر داشته باشید که تحت هر نقل قول همچنین برچسب ها وجود دارند سعی کنید یک کلاس را پیدا کنید که فقط در برچسبهای بالا سمت راست وجود دارد شاید به span چک کنید.

In [16]:

```
1 # CODE HERE
```

In [10]:

```
1 soup bs4.BeautifulSoup (res.text, 'lxml')
```

In [11]:

```
1 soup.select('.tag-Item")
```

Out[11]:

[

love ,

insp
irational

,

life ,

humor ,

books ,

<a class="tag" href="/tag/reading/" style="font-size: 14px">readine</a

In [12]:

```
1 for item in soup.select(".tag-item"):
2     print(item.text)
```

Love

inspirational

life

books

reading

friendship

friends

truth

simile

تمرین توجه کنید که بیش از یک صفحه وجود دارد و صفحات بعدی به این صورت به نظر می رسند. ۲۰/*http://swotes toscrapt.com/page/2/ (httn//quoten toscrape.com/page/* با استفاده از آنچه که درباره حلقه *for* و اتصال رشته میدانید از تمام صفحات عبور کرده و تمام نویسندگان منحصر به فرد در وبسایت را بدست آورید. به خاطر داشته باشید که راه های زیادی برای دستیابی به این هدف وجود دارد و همچنین توجه کنید که باید به طریقی بفهمید که حلقه شما در صفحه آخر نقل قول ها قرار دارد. برای اهداف اشکال زدایی به شما اطلاع میدهم که تنها ۱۰ صفحه وجود دارد. بنابراین صفحه آخر به این صورت است.

sgästä Saw Lol http://quotes.toscrape.com/page/10/ (http://quotes.toscrape.com/page/10/)

بسازید که به حدی قوی باشد که لازم نباشد تعداد صفحات را به طور پیشین بدانید شاید از *try/except* برای این کار استفاده کنید این به شما بستگی دارد!**

In [22]:

```
1 # CODE HERE
```

روش های دیگر زیادی وجود دارند که حتی قوی تر و انعطاف پذیرتر هستند اما ایده اصلی همان است، استفاده از یک حلقه while برای گشتن در صفحات مختلف و شرط خروجی بر اساس صفحه نامعتبر است.

In [13]:

```
1 url="http://quotes.toscrape.com/page/"
```

راه حل اول

In [14]:

```
authors = set()

for page in range(1,10):

    # Concatenate to get new page URL
   page_url = url+str(page)
   #Obtain Request
   res = requests.get(page_ur1)
  # Turn into Soup
  soup bs4. BeautifulSoup(res.text, lxml")
  # Add Authors to our set
 for name in soup.select(".author"):
       authors.add(name.text)
```

راه حل دیگر

In [ ]:

```
page_url = urlestr(9999999)

# Obtain Request
res = requests.get(page_ur1)

#Turn into Soup
Soup = bs4. BeautifulSoup (res. text, 'lxml')
```

In [ ]:

```
soup
```

In [ ]:

```
"No quotes found!" in res.text
```

In [ ]:

```
page_still_valid = True
authors set())
page = 1

while page_still_valid:

     # Concatenate to get new page URL
    page_url = urlestr(page)

    # Obtain Request
    res = requests.get(page_ur1)

   # Check to see if we're on the last page
  If "No quotes found!" in res.text:
        Break

  #Turn into Soup
  soup = bs4. BeautifulSoup (res.text, lxml")

 # Add Authors to our set
 for name in soup.select(".author");
       authors.add(name.text)

# Go to Next Page
 page += 1
```

In [ ]:

```
authors
```

مروری بر کار با تصاویر

با استفاده از قدرت برخی از کتابخانه های متداول که میتوانید نصب کنید مانند *PILLOW* پایتون قابلیت کار با تصاویر را برای وظایف ساده پیدا میکند میتوانید با اجرای دستور زیر کتابخانه *Pillow* را نصب کنید:

pip install pillow

در صورت وجود هرگونه مشکل می توانید به مستندات رسمی

(http://nillow.readthedecs.io/en/3 ... /installation) آنها درباره نحوه نصب مراجعه کنید.
اما برای اکثر کامپیوترها روش نصب ساده با استفاده از *pip* باید کار کند.

توجه: هنگام کار با تصاویر در دفترچه یادداشت *Jupyter* ممکن است با خطای زیر مواجه شوید.

IOPub data rate exceeded.

The notebook server will temporarily stop sending output

to the client in order to avoid crashing it..

To change this limit, set the config variable --
NotebookApp.iopub_data_rate_limit.

اگر این هشدار را دریافت کردید تلاش کنید دفترچه یادداشت را در خط فرمان متوقف کنید و سپس با دستور زیر آن را مجددا راه اندازی کنید

jupyter notebook --NotebookApp.iopub_data_rate_linit-1.0e10

در خط فرمان در واقع این یک پرچم است که مقدار محدودیت باید در طول این نشست از دفترچه یادداشت *Jupyter* که در حال اجرا است، افزایش یابد.

کار با کتابخانه *Pillow*

باز کردن تصاویر

شما میتوانید از کتابخانه Pillow برای باز کردن فایلهای تصویر استفاده کنید. در یک دفترچه یادداشت *upster*له برای نمایش فایل کافیست نام متغیری که تصویر را نگهداری میکند را تایپ کنید. برای محیط های توسعه یکپارچه (*IDE*دیگر متغیر تصویر دارای یک متد به نام *show() method.*
(https://stackoverflow.com/questions/28139637/how-can

In [1]:

```
1 from PIL inport Image
```

In [2]:

```
1 mac = Inage.open('example.jpg')
```

In [5]:

```
1 Note this is a specialized file type from PIL (pillow) type (nac)
```

out[5] :

PIL.Jpeg InagePlugin.JpegImageFile

In [6]:

```
1 # Only for jupyter notebook, use mac.show() for other IDEs
2 mac
```

اطلاعات تصاویر

In [7]:

```
1 (width, height)
2 mac.size
```

out[7] :

(1993, 1257)

In [8]:

```
1 mac.filename
```

Out [8]:

'example.jpg'

In [9]:

```
1 mac.format_description.
```

out[9] :

'JPEG (ISO 10918)'

برش تصاویر

برای برش تصاویر (یعنی گرفتن یک بخش فرعی) میتوانید از متد*crop ()*بر روی شیء تصویر استفاده کنید. متد *crop*( )یک ناحیه مستطیلی را از این تصویر برمیگرداند جعبه یک چهارتایی است که شامل مختصات پیکسل چپ، بالا، راست و پایین است.

توجه کنید اگر به رشته مستندات نگاه کنید مشخص میشود که تاپلی که در آن مقادیر را ارسال میکنید به عنوان*h۰,y,x,z*) تعریف شده است. این متغیرها ممکن است یکم گمراه کننده باشند در واقع این مقادیر نه ارتفاع و نه عرضی است که ارسال میشوند بلکه مختصات پایانی عرض و ارتفاع شما هستند.

همه مختصات جعبه *x,z,y,h)*از گوشه بالا و سمت چپ تصویر اندازه گیری می.شوند دوباره همه این چهار مقدار مختصات هستند!

In [10]:

```
1 mac.crop((0,0, 100,100))
```

برای تصویر مک این یک نمایش کاربردی نیست به جای آن از یک تصویر دیگر استفاده کنیم

In [11]:

```
1 pencils = Image.open('pencils.jpg')
```

In [12]:

```
1 pencils
```

حالا بیایید سعی کنیم برخی از مدادهای برتر را از گوشه بگیریم

In [13]:

```
1 Pencils.size
```

Out[13] :

(1950, 1300)

In [14]:

```
1  # Start at top corner (0,0)
2  X = 0
3  y = 0
4
5  # Grab about 10% in y direction, and about 30% in x direction
6
7  w = 1950 / 3
8  h = 1300/10
9
10 pencils.crop((x, y, w, h))
11
```

In [15]:

```
1 Pencils
```

حالا بیایید مدادها را از پایین بگیریم.

In [16]:

```
1 pencils.size
```

Out[16]:

(1950, 1300)

In [17]:

```
x = 0
y = 1100
w =1958/3
h = 1300
```

In [18]:

```
pencils.crop((x, y, w, h))
```

حالا به عکس مک برگردیم و ببینیم آیا میتوانیم فقط کامپیوتر را بگیریم:

In [19]:

```
mac
```

In [20]:

```
mac.size
```

Out[20]:

(1993, 1257)

In [21]:

```
halfway 1993 / 2
```

In [22]:

```
X = halfway - 200
w= halfway + 200
```

In [25]:

```
y = 800
h = 1257
```

In [26]:

```
1 mac.crop((x, y, w, h))
```

کپی کردن و چسباندن تصاویر

ما می توانیم با استفاده از متد copy() کپی هایی از تصاویر ایجاد کنیم و با استفاده از متد paste() تصاویر را بر روی تصاویر دیگر چسبانیم.

In [27]:

```
1 computer mac.crop((x, y, w, h))
```

In [28]:

```
1 mac.paste(in computer, box (0, 0))
```

In [29]:

```
1 mac
```

In [30]:

```
1 mac.paste(in computer, box = (796, 0))
```

In [31]:

```
1 mac
```

تغییر اندازه

شما میتوانید از متد *resize*() برای تغییر اندازه یک تصویر استفاده کنید

In [32]:

```
1 mac .size
```

Out[32]:

(1993, 1257)

In [33]:

```
1 h, w = mac.size.
```

In [34]:

```
1 new_h = hint (h/3)
2 new_w = wint (w/ 3)
```

In [35]:

```
1 # Note this is not permanent change
2 # for permanent change, do a reassignment
3 #e.g. mac mac.resize((180,100))
4 nac.resize((new_h, new_w))
```

همچنین میتوانید آن را کشیده و فشرده کنید.

In [36]:

```
1 mac.resize((3000, 500))
```

چرخش تصاویر

شما میتوانید با تعیین مقدار درجه ای که باید چرخانده شود در متد *rotate*() تصاویر را چرخانید. ابعاد اصلی حفظ میشود و با رنگ سیاه پر میشود میتوانید به صورت اختیاری پارامتر *expand* را به آن ارسال کنید تا تصویر چرخانیده جدید را با ابعاد قدیمی پر کنید.

In [37]:

```
1 pencils.rotate(90)
```

In [38]:

```
1 pencils.rotate(90, expand-True)
```

توجه کنید که در چرخش ۱۲۰ درجه چه اتفاقی می افتد.

In [39]:

```
1 # The image is cut off
2 pencils.rotate(128)
```

In [40]:

```
1 pencils.rotate(120, expand-True)
```

شفافیت

ما میتوانیم یک مقدار آلفا(*RGBA* برای *(RED, Green, Blue Alpha* اضافه کنیم که مقادیر آن میتواند از ۰ تا ۲۵۵ باشد. اگر آلفا ۰ باشد. تصویر کاملاً شفاف است و اگر ۲۵۵ باشد، کاملاً مات است.

شما میتوانید رنگ خود را اینجا بسازید تا مقادیر ممکن را بررسی کنید
https://developer.mozilla.org/en

US/docs/Web/CSS/CSS Colors/Color_picker tool

https://developer.mozilla.org/en- US/docs/Web/CSS/CSS Colors/Color_picker *)* tool*)*

ما میتوانیم با استفاده از متد () *putalpha* مقادیر آلفا تصویر را تنظیم کنیم:

In [64]:

```
1 red = Image.open('red_color.jpg')
```

In [65]:

```
1 red
```

In [72]:

```
1 Blue= Image.open('blue color.png")
```

In [73]:

```
1 blue
```

In [68]:

```
1 red.putalpha(158)
```

In [69]:

```
1 red
```

In [70]:

```
1 blue.putalpha(128)
```

In [ ]:

```
1 blue
```

In [76]:

```
1 blue.paste(red, bax-(8,8), mask-red)
```

In [77]:

```
# Get back an image that is more purple.
blue
```

شفافیت و ماسک گذاری میتواند پیچیده تر از آنچه که در اینجا نشان داده شده است ،باشد اگر به چیزی بیشتر نیاز داشته باشید ، مستندات را برسی کنید:

https://pillow.readthedocs.io/en/stable/

https://pillow.readthedocs.io/en/stable/

ذخیره تصاویر

In [79]:

```
blue.save("purple.png")
```

بیایید بررسی کنیم که آیا این کار کرد یا خیر

In [80]:

```
purple = Image.open( "purple.png")
```

In [81]:

```
purple
```

عالی!

تمرین تصویر

در پوشه "*Working with Images*" همان پوشه ای که این دفترچه نوت بوک در آن قرار دارد دو تصویر داریم

- *word_matrix.png*
- *mask.png*

تصویر *word_matrix* یک تصویر *png.*است که حاوی یک صفحه گسترده از کلمات با یک پیام مخفی در آن است. وظیفه شما استفاده از تصویر *mask.png* است تا پیام مخفی در داخل *word_matrix.png* را فاش کنید. به خاطر داشته باشید که ممکن است برای این کار تغییراتی در تصویر *mask.png* اعمال کنید ما تا اینجا تنها این را میگوییم زیرا واقعاً میخواهیم شما به صورت خودکار این را کشف کنید!

این تمرین به صورت باز است بنابراین ما شما را با گام ها هدایت نمیکنیم به جای آن به شما اجازه میدهیم که بر اساس شرایط واقعی بررسی و مسائل را خودتان حل کنید. با تمنای موفقیت!

In [1]:

```
1 from PIL import Image
2
3 words = Image.open('word_matrix.png')
4
5 words
```

Out[1]:

STOP	FAIR	SCORE	FIELD	WOLF	SNATCH	PIER	DAWN
WOOD	GREAT	MAID	STRONG	FRONT	TEAM	CLOSED	PITCH
HOLD	CURL	BRAVE	SPITE	DESK	FADE	NAME	LIST
FALL	HILL	TREE	WORK	SPHERE	CHORD	COAST	BOLD
YARD	LAND	CHURCH	LATE	TRAY	PLUCK	DARE	GRIND
FIGHT	MY	PAUSE	DOG	TIGHT	FUR	STREAM	SIN
CALF	HIKE	DASH	FLOOD	TENSE	WITH	PUMP	BAT
FLU	WENT	SOW	QUOTE	DRAWER	HOUSE	TOUCH	SUN
LOUNGE	THE	CLOSE	DUE	HIKE	SAVE	FOX	WAGE
BRINK	STORE	HEAT	POP	EAST	SOW	MAP	OH
DRINK	TODAY	SHOT	SOLVE	NOTE	WAVE	IMAGES	TRACE

TURN	FOR	JAM	DECK	SOAK	STAB	SPIN	WEALTH
YOU	NOW	NEWS	CHEAT	LAST	FRAME	TRUCK	SPLURGE
GLARE	TRAVEL	ARE	ROOT	SCREAM	GHOST	BENCH	TONGUE
STYLE	LOOT	LODGE	MILE	PRAYER	MONTH	TROOP	STOP
SWING	MASS	FRESH	DORM	THE	BRIDE	BEST	SUM
SMASH	REST	PLAN	SITE	MOLE	DRY	DOSE	SCALE
CHOP	MY	GIFG	CORE	JUDGE	BLONDE	BEACH	STORE
HALT	WORK	NUT	SHIFT	TAIL	EAST	SLANT	REACH

In [2]:

```
1 mask Image.open("mask.png")
2 mask
```

In [3]:

```
1 mask.size
```

Out[3]:

(505, 251)

In [4]:

```
1 words.size
```

Out[4]:

(1015, 559)

In [7]:

```
1 mask mask.resize((1015, 559))
```

In [8]:

```
1 mask
```

In [9]:

```
1 mask.putalpha(200)
```

In [10]:

```
1 mask
```

In [12]:

```
1 Words.paste(mask, (0,0),mask)
```

In [13]:

```
1 Words
```

In [ ]:

```
1
```

کار با فایل های *CSV*

به بازگشت خوش آمدید بیایید درباره کار با فایل های *CSV* در پایتون بحث کنیم فایل با پسوند *CSV* یک فایل با جداکننده های کاما () است. همه فایلهای *CSV* متن ساده هستند و شامل کاراکترهای الفبایی عددی هستند و داده ها را در طالب یک جدول سازماندهی میکنند فایل های *CSV* را با فایل های اکسل اشتباه نگیرید هرچند که فایل های *CSV* به شکل بسیار مشابهی با قابل های اکسل قالب بندی میشوند اما دارای نوع داده برای مقادیر خود نیستند همه آنها رشته ها با بدون قلم یا رنگ هستند. همچنین آنها هیچ ورک شیتی مانند یک فایل اکسل ندارند در پایتون چندین کتابخانه برای کار با قابل های اکسل وجود دارد. می توانید آنها را اینجل (*//wooon.orthon excelare*و اینجا (*https://www.shwinas.org/)*) بررسی کنید.

قابل های با قالب *CSV* عموماً برای تبادل داده استفاده میشوند معمولاً زمانی که میزان داده بسیار زیادی وجود دارد و بین برنامه ها متفاوت تبادل میشود برنامه های پایگاه داده نرم افزارهای تحلیلی و سایر برنامه ها که مقدار زیادی اطلاعات (مانند مخاطبین و داده های مشتری) را ذخیره میکنند عموماً از فرمت *CSV* پشتیبانی میکنند.

بیایید بررسی کنیم که چگونه میتوانیم یک فایل *CSV* را با کتابخانه *csv* در پایتون باز کنیم.

محل دفترچه نوت بوک

دستور *pwd* را درون یک سلول دفترچه نوت بوک اجرا کنید تا متوجه شوید دفترچه نوت بوک شما در کجا قرار دارد.

In [1]:

```
1 pwd
```

Qut[1]:

C:\Users\\babak\\Python For Everyone\\17-PDFs-and-Spreadsheets'

خواندن فایل های *CSV*

In [2]:

```
1 import csv
```

وقتی مسیر قابل را انتقال میدهید مطمئن شوید که در صورت وجود پسوند را نیز درج کنید و میتوانید با استفاده از کلید تب *(Tab)*نام فایل را به طور خودکار تکمیل کنید. اگر نتوانید از تکمیل خودکار با کلید لب استفاده کنید، این یک نشانگر خوب است که فایل شما در همان محل قرار ندارد همیشه میتوانید مسیر کامل قابل را تایپ کنید (از نظر قالب بندی شبیه به خروجی *pwd* خواهد بود).

In [3]:

```
1 data = open( example.cs' )
```

In [4]:

```
1 data
```

out[4] :

<io.TextIoWrapper name="example.csv' mode='r' encoding="cp1252">

Encoding

بسیاری اوقات فایلهای *CSV* ممکن است شامل کاراکترهایی باشند که نمیتوانید با پایتون استاندارد آنها را تفسیر کنید. این میتواند مانند نماد یا حتی کاراکترهای خارجی باشد. بیایید یک مثال از این نوع خطا را مشاهده کنیم. این امر بسیار رایج است بنابراین مهم است آن را برسی کنیم.

In [5]:

```
1 csv_data = csv.reader(data)
```

تلاش برای تبدیل آن به لیست باعث بروز خطا میشود توجه کنید که در خط خطا خطای *can't decode* وجود دارد، این یک نشانه است که مشکلی در رمزگذاری وجود دارد!

In [6]:

```
1 data_lines - list(csv_data)
```

UnicodeDecodeError

Traceback (most recent call last)

Cell In[6], line 1

----> 1 data_lines - list(csv_data)

File C:\ProgramData\Anaconda3\lib\encodings\cp1252.py:23, in IncrementalDe coder.decode(self, input, final)

22 def decode(self, input, final=False):

---->23 return codecs.charmap_decode(input, self errors, decoding_table)

[0]

UnicodeDecodeError: 'charmap' codec can't decode byte 8x8d in position 181 e: character naps to <undefined>

بیایید با استفاده از رمزگذاری "utf-8"آن را بخوانیم

In [7]:

```
1  data= open('example.csv', encoding='utf-8')
2  csv_data= csv.reader(data)
3  data lines list(csv_data)
```

In [8]:

```
1  # Looks Like it worked!
2  data lines[:3]
```

Out[8]:

[['id', 'first name', 'last_name", 'email', 'gender', 'ip_address', 'cit

y'], ['1',

"Joseph",

'Zaniolini,

*jzaniolinie@simplemachines.org',

'Male',

163.168.68.132',

'Pedro Leopoldo'],

['2',

'Freida',

*Drillingcourt",

'fdrillingcourti@unich.edu",

'Fenale',

97.212.102.79",

'Buri']]

طفاً به یاد داشته باشید که مورد اول در لیست خط سربرگ است که شامل اطلاعات درباره نمایندگی هر ستون است. بیایید چاپ خروجی را به شکلی کمی تنظیم کنیم:

In [9]:

```
1 for line in data_lines[:5]:
2     print(line)
```

['id', 'first_name', 'last_name', 'email', 'gender", ip_address', 'city'] ['1', 'Joseph', 'Zaniolini, jzaniolinie@simplemachines.org', 'Male', '16 3.168.68.132, 'Pedro Leopoldo"]

['2', 'Freida', 'Drillingcourt, 'fdrillingcourt1@umich.edu', 'Female', '9 7.212.182.79, 'Buri']

['3', 'Nanni', 'Herity, 'nherity2@statcounter.com", 'Female', '145.151.17 8.98, 'Claver"]

['4', 'Orazio, "Frayling. 'ofrayling3@economist.com', 'Male', '25.199.14 3.143', 'Kungur']

تصور کنید میخواهیم لیستی از تمام ایمیل ها داشته باشیم به عنوان نمایشی از آنجایی که تعداد آیتم ها ۱۰۰۰ مورد به علاوه سربرگ است تنها چند ردیف را نمایش می دهیم.

In [10]:

```
1 len(data_lines)
```

Out[10]:

1001

In [11]:

```
all emails = []
for line in data_lines[1:15]:
    all emails.append(line[3])
```

In [12]:

```
print(all emails)
```

['jzaniolinie@simplemachines.org", 'fdrillingcourti@umich.edu', 'nherity2@

statcounter.com', 'ofrayling3@economist.com', 'Jmurrison4@cbslocal.com', lgamet5@list-manage.com', 'dhowatt6@amazon.com', 'kherion7@amazon.com, 'chedworth@china.com.cn', 'hgasquoine@google.ru', 'ftarra@shareasale.co m', 'abathb@urn.edu', 'lchastangc@goo.gl', 'cceried@yale.edu"]

اگر میخواستیم لیستی از نام و نام خانوادگی کامل داشته باشیم، چه میشد؟

In [13]:

```
full names[]

for line in data lines[1:15):
    full names, append(line[1] 11ne[2])
```

In [14]:

```
print(full names)
```

['Joseph Zaniolini', 'Freida Drillingcourt', 'Nanni Herity', 'Orazio Frayl ing, Julianne Murrison', 'Lucy Gamet', 'Dyana Howatt', 'Kassey Herion", Chrissy Hedworth', 'Hyatt Gasquoine, Felicdad Tarr', 'Andrew Bath", "Lu cais Chastang', 'Car Cerle']

نوشتن در فایلهای *CSV*

ما همچنین میتوانیم فایل های *CSV* بنویسیم ساخته شده یا به فایلهای موجود اضافه کنیم.

فایل جدید

این عمل همچنین هر قابل موجود با همین نام را برای شما بازنویسی میکند پس در این مورد دقت کنید!

In [15]:

```
1 #newline controls how universal newlines works (it only applies to text
2 # mode). It can be None, '', '\n', '\r', and '\r\n'.
3 file_to_output = open('to_save_file.csv', 'w', newline='')
```

In [16]:

```
1 csv_writer = csv.writer (file_to_output, delimiter = ')
```

In [17]:

```
1 csv_writer.writerow(['a', 'b', 'c'])
```

Out[17]:

7

In [18]:

```
1 csv_writer.writerows ([['1', '2', '3'], ['4', '5', '6']])
```

In [19]:

```
1 file_to_output.close()
```

فایل موجود

In [21]:

```
1 f = open('to_save_file.csv', 'a', newline='')
```

In [22]:

```
1 csv_writer = csv.writer (f)
```

In [23]:

```
1 csv_writer.writerow(['7', '8', '9'])
```

Out [23]:

7

In [24]:

```
1 f.close()
```

این همه بود برای مباحث اصولی اگر فکر میکنید که برای اغلب کارهای خود با فایلهای *CSV* سر و کار خواهید داشت.

کار با فایل های *PDF*

بازگشت خوش آمدید عامل اغلب شما باید با فایل های *PDF* سروکار داشته باشید در پایتون کارخانه های بسیاری برای کار

ljeplasala aga PDE.(https://www.binpress.com/tutorial/manipulating.adfa-with-aython/167)

مغایب خود، اما معروفترین آنها *PyPDF2* است. شما میتوانید آن را با استفاده از دستور زیر نصب کنید (توجه کنید که حساس به حروف بزرگ و کوچک است بنابراین باید مطمئن شوید که بزرگی و کوچکی حروف آن با تایپ شما همخوانی داشته باشد)

pip install PyyPDF2

در نظر داشته باشید که هر فایل *PDF* نمیتواند با این کتابخانه خوانده شود. *PDF*هایی که بسیار مبهم هستند. دارای رمزگذاری ویژه ای هستند یا شاید فقط با یک برنامه خاص ساخته شده باشند که با *PyPDF2* کار خوبی انجام نمیدهد. نمی توانند خوانده شوند. اگر خودتان در این موقعیت باشید تلاش کنید از کتابخانه های موردی که در بالا لینک داده شده استفاده کنید، اما به خاطر داشته باشید که این نیز ممکن است کار نکند. دلیل این موضوع این است که به دلیل وجود بسیاری از پارامترهای مختلف برای *PDF* و عدم استاندارد بودن تنظیمات ممکن است متن به جای رمزگذاری *utf-*۸ به عنوان تصویر نمایش داده شود. در این زمینه باید به بسیاری از پارامترها توجه کرد.

در مورد *PyPDF2*، تنها قادر به خواندن متن از یک سند *PDF* است و قادر به دریافت تصاویر یا فایلهای رسانه ای دیگر از یک *PDF* نخواهد بود.

کار با *PyPDF2*

بیایید با میانی کتابخانه *PyPDF2* آشنا شویم.

In [ ]:

```
pip install PyPDF2
```

In [1]:

```
# note the capitalization
import PyPDF2
```

خواندن فایلهای PDF

مشابه کتابخانه CS، ما یک فایل pdf را باز میکنیم و سپس یک شی خواننده برای آن ایجاد میکنیم. توجه کنید که ما از روش دودویی خواندن ، به جای فقط 'r' استفاده میکنیم.

In [2]:

```
# Notice we read it as a binary with 'rb'
f = open( "Working_Business_Proposal.pdf","rb")
```

In [3]:

```
pdf reader PyPDF2.PdfReader(f)
```

In [4]:

```
len(pdf_reader.pages)
```

Out [4]:

5

In [5]:

```
1 page_number : 0
2 page_one = pdf_reader pages [0]
```

In [ ]:

```
1 page_one
```

سپس متن را میتوانیم استخراج کنیم:

In [7]:

```
1 page_one_text = page_one.extract_text()
```

In [8]:

```
1 page_one_text
```

Out[8]:

'Business Proposal The Revolution is Coming Leverage agile frameworks to p rovide a robust synopsis for high level overviews. Iterative approaches to corporate strategy foster collaborative thinking to further the overall va lue proposition. Organically grow the holistic world view of disruptive in novation via workplace diversity and empowerment. Bring to the table win-w in survival strategies to ensure proactive domination. At the end of the d ay, going forward, a new normal that has evolved from generation Xx is on t he runway heading towards a streamlined cloud solution. User generated con tent in real-time will have multiple touchpoints for offshoring. Capitaliz e on low hanging fruit to identify a ballpark value added activity to beta test. Override the digital divide with additional clickthroughs from DevOp s. Nanotechnology immersion along the information highway will close the 1 cop on focusing solely on the bottom line. Podcasting operational change m anagerent inside of workflows to establish a framework. Taking seamless key performance indicators offline to maximise the long tail. Keeping your eye on the ball while performing a deep d

ive on the start-up mentality to deri ve convergence on cross-platform integration. Collaboratively administrate empowered markets via plug-and-play networks. Dynamically procrastinate 82 Cusers after installed base benefits. Dramatically visualize customer dire cted convergence without revolutionary ROI. Efficiently unleash cross-media information without cross-media value. Quickly naximize tinely deliverable s for real-time schenas. Dramatically maintain clicks-and-mortar solutions without functional solutions. BUSINESS PROPOSAL 11'

In [9]:

```
1 f.close()
```

اضافه کردن به فایل های PDF

ما نمیتوانیم از طریق پایتون به فایل های PDF بنویسیم به دلیل تفاوت های میان نوع رشته یکپارچه پایتون و انواع قلم ها قرارگیری ها و سایر پارامترهایی که یک فایل PDF میتواند داشته باشد. آنچه که میتوانیم انجام دهیم کپی کردن صفحات و الحاق صفحات به انتها است.

In [11]:

```
1 f = open('Working_Business_Proposal.pdf', 'rb')
2 pdf reader = PyPDF2.PdfReader (f)
```

In [12]:

```
1 page_number = 0
2 page one pdf reader.pages[0]
```

In [13]:

```
1 pdf_writer = PyPDF2.PdfWriter()
```

In [14]:

```
1 pdf writer.add_page(page_one);
```

In [15]:

```
1 pdf_output = open("Some_New_Doc.pdf","wb")
```

In [16]:

```
1 pdf_writer.write(pdf_output)
```

Out[16]:

(False, <_io.BufferedWriter name='Some_New_Doc.pdf'>)

In [17]:

```
1 f.close()
```

حالا ما یک صفحه را کپی کرده و آن را به یک سند جدید اضافه کرده ایم!

مثال ساده:

In [18]:

```
1  f = open('Working_Business_Proposal.pdf', 'rb')
2
3  # List of every page's text.
4  # The index will correspond to the page number.
5  pdf_text = []
6
7  pdf reader = PyPDF2. PdfReader (f)
8
9  for p in range(len(pdf_reader.pages)):
10
11     page pdf_reader.pages[p]
12
13     pdf_text.append(page.extract_text())
14
```

In [19]:

```
1 Pdf_text
```

Out[19]:

Quickly coordinate e-business applic ations through revolutionary catalysts for cha

nge. Seamlessly underwhelm o ptimal testing procedures whereas bricks-and-clicks processes. Synergistic ally evolve 2.09 technologies rather than just in time initiatives. Quickly deploy strategic networks with compelling e-business. Credibly pontificate. highly efficient manufactured products and enabled data. Dynamically target high-payoff intellectual capital for customized technologies. Objectively

integrate emerging core competencies before process-centric communities. D Famaidally evisculate holistic innovation rather than client-centric dat Prospex paintain extensive infomediaries via extensible niches. D

ramatically disseminate standardized metrics after resource-leveling proce sses. Objectively pursue diverse catalysts for change for interoperable me Betéées-Poaevelyapsatewonéskobaab-ieèdialsgonentFrbefove 80 SPRESST Compièeesy Apnepgriatelxlapeweccemarée Paedersehanskisser ysinesssepvtals, kleballvедyoseedieatpeiotecachixens PRZypba¿eswattefişe Josh vedqualityectoss-calgeallvorevoletiveigerslehgineersesofbesuar ecapsepbléesesxácenéstéesmesteryhistobeantofeastre

zacasdi

6-bяadeeshA016gtharthaplogoseping Acequireyaggregat aka saadpeckeratestèse pregedebepagetecké fagte cednepductPognanasalveccavengmarketedsiness PROPbsAities and ubiquit OuoÁßéeeneses, resistàyleAbrashRaggeddasagnents befogke andpapual cousteProdas Pep Raceryiahpawemajnom ésáenañpeskryft 609Messtec webbseadingsearomadlineraebvacsupppoweredns-business

efketiveequariepdcellerbualiçapétalatIokezactivear 36velszenfoogk-e

tyre

eceased bedsi9feFREYAmethereaardistripesSTBAStigeraÅ‡èle gable abled Fauneestand sestpettespréнess: Empletelyasyothesized

ñáecieleckebedorangechesgeste1Ãorare ¿Pe@pproagers byssures ventimurreesársventopportuniestelis 86ateieêracase-wide strategig #berecareasnwatbauetarajiadnite Is qualitysypstegse Ragaréussytiddapestnggisbeɣeerimprae26631Mas kettenàsuplyakʙiy seemeebar capréavalvatéhʙebe9°Basiyete bel CBM-proepstently happiestaskokvalgenesearched data candyentrospePForms Acésemetodoamatértenedés avattavɛdeprofetentig Chan-Ensused coarès aRUSTNE6SePROP! Aroducts. Completely synthesize principle-centered information after ethical communities. Efficiently innov zed infrastru

rougherassebroedtpropeàveleshpoèmiesvayieteeseede

Ptional

strategic theme areas with

egrate ctures. Interactively productize premium technologies d'intera nt quality vectors. Rapaciously utilize enterprise experiences via 24/7 ma rkets. Uniquely matrix economically sound value through cooperative techno logy. Competently parallel task fully researched data and enterprise proce ss improvements. Collaboratively expedite quality manufactured products vi a client-focused results. BUSINESS PROPOSAL !4",

'Quickly communicate enabled technology and turnkey leadership skills. Un iquely enable accurate supply chains rather than frictionless technology.

Glahallu notuank fariend mataniale uie-s-use rect affaction manufactured

['Business Proposal The Revolution is Coming Leverage agile frameworks to provide a robust synopsis for high level overviews. Iterative approaches t o corporate strategy foster collaborative thinking to further the overall value proposition. Organically grow the holistic world view of disruptive innovation via workplace diversity and empowerment. Bring to the table win -win survival strategies to ensure proactive domination. At the end of the day, going forward, a new normal that has evolved from generation X is on the runway heading towards a streamlined cloud solution. User generated co ntent in real-time will have multiple touchpoints for offshoring. Capitali ze on low hanging fruit to identify a ballpark value added activity to bet a test. Override the digital divide with additional clickthroughs from Dev Ops. Nanotechnology immersion along the

information highway will close the loop on focusing solely on the bottom line. Podcasting operational change management inside of workflows to establish a framework. Taking seamless ke y performance indicators offline to maximise the long tail. Keeping your ey e on the ball while performing a deep dive on the start-up mentality to del rive convergence on cross-platform integration. Collaboratively administra te empowered markets via plug-and-play networks. Dynamically procrastinate B2C users after installed base benefits. Dramatically visualize customer di rected convergence without revolutionary ROI. Efficiently unleash cross-med ia information without cross-media value. Quickly maximize timely delivera bles for real-time schemas. Dramatically maintain clicks-and-mortar soluti ons without functional solutions. BUSINESS PROPOSAL 11',

'Completely synergize resource taxing relationships via premier niche mar kets. Professionally cultivate one-to-one customer service with robust ide as. Dynamically innovate resource-leveling customer service for state of t he art customer service. Objectively innovate empowered manufactured produ cts whereas parallel platforms. Holisticly predominate extensible testing. procedures for reliable supply chains. Dramatically engage top-line web se rvices vis-a-vis cutting-edge deliverables. Proactively envisioned multime dia based expertise and cross-media growth strategies. Seamlessly visualiz e quality intellectual capital without superior collaboration and idea-sha ring. Holistically pontificate installed base portals after maintainable pr oducts. Phosfluorescently engage worldwide methodologies with web-enabled t echnology. Interactively coordinate proactive e-commerce via process-centr ic "outside the box" thinking. Completely pursue scalable customer service. through sustainable potentialities. Collaboratively administrate turnkey c hannels whereas virtual e-tailers. Objectively seize scalable metrics wher eas proactive e-services. Seamlessly empower fully researched growth strat egies and interoperable internal or "organic" sources. Credibly innovate g ranular internal or "organic" sources whereas high standards in web-readin ess. Energistically scale future-proof core competencies vis-a-vis impactf ul experiences. Dramatically synthesize integrated schemas with optimal ne tworks. Interactively procrastinate high-payoff content without backward-c ompatible data. Quickly cultivate optimal processes and tactical architect ures. Completely iterate covalent strategic theme areas via accurate e-mar kets. Globally incubate

standards compliant channels before scalable benefi ts. Quickly disseminate superior deliverables whereas web-enabled BUSINESS PROPOSAL 12',

'applications. Quickly drive clicks-and-mortar catalysts for change befor e vertical architectures. Credibly reintermediate backend ideas for cross- platform models. Continually reintermediate integrated processes through t echnically sound intellectual capital. Holistically foster superior method ologies without market-driven best practices. Distinctively exploit optima 1 alignments for intuitive bandwidth.

خسته نباشید!

تمرین پازل PDF و صفحه های گسترده

بیایید مهارت های شما را آزمایش کنیم برای این تمرین به دو فایل نیاز خواهید داشت و باید وظایف زیر را حل کنید.

- وظیفه اول با استفاده از پایتون لینک Google Drive را از قابل csv استخراج کنید. (راهنما: این در طول قطر از بالا سمت چپ تا پایین سمت راست قرار دارد)
- وظیفه دوم PDF را از لینک Google Drive دانلود کنید ما قبلاً آن را برای شما دانلود کرده ایم، فقط در صورت عدم دانلود از Google Drive و شماره تلفن موجود در سند را پیدا کنید. توجه روشهای مختلفی برای قالب بندی شماره تلفن وجود دارد!

وظیفه اول :گرفتن لینک Google Drive از فایل csv

In [1]:

```
1 import csv
```

In [13]:

```
# The correct result is shown below, if you can't download from Google Drive,
# we added the PDF file to the Exercise Files folder already
Data= open('Exercise_Files/find_the_link.csv', encoding="utf-8")
csv data = csv.reader(data)
data_lines= list(csv_data)
```

روش اول:

In [14]:

```
link_list = []

for row num, data in enumerate(data_lines):
    link_list.append(data[row_num])
```

In [15]:

```
.join(link_list)
```

Out[15]:

https://drive.google.com/open?id=1G6SEgg018UB4 4xsAJJ5TdzrhmXipz4M

In [16]:

```
link_str="

    for row num, data in enumerate(data_lines):
            link str data[row_num]

link str
```

Out[16]:

https://drive.google.com/open?id=1G6SEgg018UB4_4xsAJJ5TdzrhmX1pz4M

وظیفه دوم پیدا کردن شماره تلفن موجود در سند PDF

In [ ]:

```
1 # you should get this phone number
2 #505 503 4455
```

In [17]:

```
1 import PyPDF2
```

In [18]:

```
1 F = open('Exercise_Files/Find_the_Phone_Number.pdf', 'rb')
```

In [19]:

```
1 pdf = PyPDF2.PdfReader (f)
```

In [20]:

```
1 len(pdf.pages)
```

Out [20]:

17

In [21]:

```
1 import re
```

In [22]:

```
1 Pattern = r'\d{3}. \d{3}. \d{4}'
```

In [23]:

```
1 for n in range(len(pdf.pages)):
2     page = pdf.pages[n]
3     page_text page.extract_text()
4    match = re.search (pattern, page_text)
5
6    if match:
7         print (match.group())
```

505.503.4455

Business Proposal

The Revolution is Coming

Leverage agile frameworks to provide a robust synopsis for high level overviews. Iterative approaches to corporate strategy foster collaborative thinking to further the overall value proposition. Organically grow the holistic world view of disruptive innovation via workplace diversity and empowerment.

Bring to the table win-win survival strategies to ensure proactive domination. At the end of the day, going forward, a new normal that has evolved from generation X is on the runway heading towards a streamlined cloud solution. User generated content in real-time will have multiple touchpoints for offshoring.

Capitalize on low hanging fruit to identify a ballpark value added activity to beta test. Override the digital divide with additional clickthroughs from DevOps. Nanotechnology immersion along the information highway will close the loop on focusing solely on the bottom line.

Podcasting operational change management inside of workflows to establish a framework. Taking seamless key performance indicators offline to maximise the long tail. Keeping your eye on the ball while performing a deep dive on the start-up mentality to derive convergence on cross- platform integration.

Collaboratively administrate empowered markets via plug-and-play networks. Dynamically procrastinate B2C users after installed base benefits. Dramatically visualize customer directed convergence without revolutionary ROI.

Efficiently unleash cross-media information without cross-media value. Quickly maximize timely deliverables for real-time schemas. Dramatically maintain clicks-and-mortar solutions without functional solutions.

BUSINESS PROPOSAL

Completely synergize resource taxing relationships via premier niche markets. Professionally cultivate one-to-one customer service with robust ideas. Dynamically innovate resource-leveling customer service for state of the art customer service.

Objectively innovate empowered manufactured products whereas parallel platforms. Holisticly predominate extensible testing procedures for reliable supply chains. Dramatically engage top-line web services vis-a-vis cutting-edge deliverables.

Proactively envisioned multimedia based expertise and cross-media growth strategies. Seamlessly visualize quality intellectual capital without superior collaboration and idea-sharing. Holistically pontificate installed base portals after maintainable products.

Phosfluorescently engage worldwide methodologies with web-enabled technology. Interactively coordinate proactive e-commerce via process- centric 'outside the box' thinking. Completely pursue scalable customer service through sustainable potentialities.

Collaboratively administrate turnkey channels whereas virtual e-tailers. Objectively seize scalable metrics whereas proactive e-services. Seamlessly empower fully researched growth strategies and interoperable internal or 'organic' sources.

Credibly innovate granular internal or 'organic' sources whereas high standards in web-readiness. Energistically scale future-proof core competencies vis-a-vis impactful experiences. Dramatically synthesize integrated schemas with optimal networks.

Interactively procrastinate high-payoff content without backward- compatible data. Quickly cultivate optimal processes and tactical architectures. Completely iterate covalent strategic theme areas via accurate e-markets.

Globally incubate standards compliant channels before scalable benefits. Quickly disseminate superior deliverables whereas web-enabled

BUSINESS PROPOSAL

applications. Quickly drive clicks-and-mortar catalysts for change before vertical architectures.

Credibly reintermediate backend ideas for cross-platform models. Continually reintermediate integrated processes through technically sound intellectual capital. Holistically foster superior methodologies without market-driven best practices.

Distinctively exploit optimal alignments for intuitive bandwidth. Quickly coordinate e-business applications through revolutionary catalysts for change. Seamlessly underwhelm optimal testing procedures whereas bricks-and-clicks processes.

Synergistically evolve 2.0 technologies rather than just in time initiatives. Quickly deploy strategic networks with compelling e-business. Credibly pontificate highly efficient manufactured products and enabled data.

Dynamically target high-payoff intellectual capital for customized technologies. Objectively integrate emerging core competencies before process-centric communities. Dramatically evisculate holistic innovation rather than client-centric data.

Progressively maintain extensive infomediaries via extensible niches. Dramatically disseminate standardized metrics after resource-leveling processes. Objectively pursue diverse catalysts for change for interoperable meta-services.

Proactively fabricate one-to-one materials via effective e-business. Completely synergize scalable e-commerce rather than high standards in e-services. Assertively iterate resource maximizing products after leading- edge intellectual capital.

Distinctively re-engineer revolutionary meta-services and premium architectures. Intrinsically incubate intuitive opportunities and real-time potentialities. Appropriately communicate one-to-one technology after plug-and-play networks.

Quickly aggregate B2B users and worldwide potentialities. Progressively plagiarize resource-leveling e-commerce through resource-leveling core

BUSINESS PROPOSAL

competencies. Dramatically mesh low-risk high-yield alignments before transparent e-tailers.

Appropriately empower dynamic leadership skills after business portals. Globally myocardinate interactive supply chains with distinctive quality vectors. Globally revolutionize global sources through interoperable services.

Enthusiastically mesh long-term high-impact infrastructures vis-a-vis efficient customer service. Professionally fashion wireless leadership rather than prospective experiences. Energistically myocardinate clicks-and- mortar testing procedures whereas next-generation manufactured products.

Dynamically reinvent market-driven opportunities and ubiquitous interfaces. Energistically fabricate an expanded array of niche markets through robust products. Appropriately implement visionary e-services vis- a-vis strategic web-readiness.

Compellingly embrace empowered e-business after user friendly intellectual capital. Interactively actualize front-end processes with effective convergence. Synergistically deliver performance based methods of empowerment whereas distributed expertise.

Efficiently enable enabled sources and cost effective products. Completely synthesize principle-centered information after ethical communities. Efficiently innovate open-source infrastructures via inexpensive materials.

Objectively integrate enterprise-wide strategic theme areas with functionalized infrastructures. Interactively productize premium technologies whereas interdependent quality vectors. Rapaciously utilize enterprise experiences via 24/7 markets.

Uniquely matrix economically sound value through cooperative technology. Competently parallel task fully researched data and enterprise process improvements. Collaboratively expedite quality manufactured products via client-focused results.

BUSINESS PROPOSAL

Quickly communicate enabled technology and turnkey leadership skills. Uniquely enable accurate supply chains rather than frictionless. technology. Globally network focused materials vis-a-vis cost effective manufactured products.



SQLite **چیست؟**

ذخیره و بازیابی اطلاعات یکی از نیازهای اساسی در هنگام برنامه نویسی سیستم های مختلف است برای انجام این کار، بسته به نیازهای موجود ابزارها و روشهای مختلفی نیز وجود دارند. از روش های ساده مانند نگهداری اطلاعات در یک فایل متنی تا استفاده از نرم افزارهای عظیم مانند *Oracle*.

یکی از ابزارهای ذخیره و بازیابی اطلاعات *SQLite* نام دارد. این نرم افزار مشهورترین سیستم ذخیره فایلی اطلاعات به شمار می رود. شهرت *SQLite* به دلیل پشتبیانی از انواع مختلف سیستم عامل ها از جمله ویندوز لینوکس آندروئید و مک او اس و همچنین رایگان و قدرتمند بودن آن است.

SQLite با زبان *C* برنامه نویسی شده است و به طور پیوسته در حال بهبود و توسعه است. به همین دلیل سرعت و کارایی بسیار بالایی دارد. در نگارش های جدید که در آینده منتشر خواهند شد. بهینه سازی های گسترده ای روی این سیستم به انجام رسیده است که سرعت عملکرد آن را بیش از پیش افزایش داده است.

SQLite نرم افزاری با مجوز استفاده *Public Domain* است. به این معنی که حق مالکیتی ندارد و هر فرد با سازمانی میتواند بدون هیچ محدودیتی از آن به هر شکلی استفاده کند.

کاربرد *SQLite*

SQLite یک سیستم ذخیره اطلاعات فایلی است بنابراین استفاده از آن در محیط هایی که مشتریان متعددی بخواهند از اطلاعات استفاده کنند ممکن است با مشکلاتی مواجه شود. در عوض در محیط هایی مانند نرم افزارهای موبایل و وبسایت هایی که بازدیدهای سنگین و زیادی نداشته باشند کارایی و عملکرد خوبی به دست خواهد آمد.

برخی از سیستم های مدیریت محتوا مانند ،دروپال گزینه ی کاربرد *SQLite* برای طراحی سایت را در اختیار برنامه نویسان و صاحبان سایت قرار میدهند.

بسیاری از نرم افزارهای شناخته شده در دنیا مانند موزیلا فایرفاکس گوگل کروم ویندوز ۱۰، اسکایپ و دراپ باکس برای نگهداری بخشی از اطلاعات خود از *SQLite* استفاده میکنند.

یکی دیگر از کاربردهای پایگاه دادهی فایلی مثل *SQLite* انتقال اطلاعات دارای ساختار و حجیم است. برای مثال ممکن است یک اپلیکیشن موبایل اطلاعات جمع آوری شده از کاربران را در قالب یک پایگاه دادهی *SQLite* به سمت سرور مرکزی ارسال نماید.

مدیریت آسان

SQLite به دلیل استفاده از سیستم فایلی نیازی به تنظیمات پیچیده و اساسی مانند بسیاری از *DBMS*های دیگر ندارد. کافی است فایل پایگاه داده را در کنار نرم افزار قرار دهید و از آن استفاده کنید.

امکانات پیشرفته *SQLite*

SQLite با وجود کم حجم و سریع بودن دارای امکانات پیشرفته ای مانند *Transaction*ها جدول های مستقر در حافظه جستجوی متن *Full Text* و امکانات گسترده ی دیگری است که گاهی به دلیل همین امکانات به صورت موازی در کنار سایر سیستم های بانک اطلاعاتی مورد استفاده قرار میگیرد.

ماژول *sqlite3*

In [1]:

```
1 import sqlite3
```

In [2]:

```
1 new_connection = sqlite3.connect("test_data.db")
```

In [3]:

```
1 print(new_connection)
```

< sqlite3.Connection object at 8x8000023180907640

In [4]:

```
1 type(new connection)
```

out[4] :

sqlite3.Connection

In [5]:

```
1 new_cursor = new_connection.cursor()
```

In [6]:

```
1 type(new_cursor)
```

out[6]:

sqlite3.Cursor

دستورات مقدماتی برای کار با پایگاه داده

In [7]:

```
1 new_sql = "SELECT datetime("now', 'localtine');"
```

In [8]:

```
1 new cursor.execute(new_sql)
```

Out[8]:

<sqlite3.Cursor at 8x231bf1fc540>

In [9]:

```
1 New_cursor.fetchone()
```

Out [9]:

(2023-07-19 10:12:03",)

In [11]:

```
1 new_dtime = new_cursor.execute("SELECT datetime('now',
  'localtime');").fetchone()[0]
```

In [12]:

```
1 print(new_dtime)
```

2023-07-19 10:15:49

In [13]:

```
1 new connection.close()
```

In [14]:

```
###

with sqlite3.connect("test_data.db") as new_connection:
    new_cursor new_connection.cursor()
    new_sql = "SELECT datetime('now', 'localtime');"
    new_dtime = new_cursor.execute(new_sql).fetchone()[0]

print (new_dtime)

```

2023-07-19 10:19:49

ایجاد جدول و ورود داده

In [38]:

```
import sqlite3

new_connection = sqlite3.connect("test_data.db")
new_cursor = new_connection.cursor()
```

In [39]:

```
new_cursor.execute("DROP TABLE IF EXISTS clients")
new_cursor.execute("CREATE TABLE clients (name TEXT, number INT);")
```

Out[39]:

<sqlite3.Cursor at ex231c0535a40>

In [22]:

```
#new_cursor.execute("DROP TABLE clients")
```

Out[22]:

<sqlite3.Cursor at 0x231c0534440>

In [40]:

```
1 new cursor.execute("INSERT INTO clients VALUES ('Ali', 1998);")
```

Out[40]:

<sqlite3.Cursor at ex231c0535a40>

In [41]:

```
1 new_connection.commit()
2
3 new_connection.close()
```

In [34]:

```
1 new connection = sqlite3.connect("test_data.db")
  new_cursor = new_connection.cursor()
```

In [29]:

```
1 new cursor.execute("SELECT FROM clients;")
```

Out[29]:

<sqlite3. Cursor at @x231c8535fce>

In [30]:

```
1 new cursor.fetchone()
```

Out [38]:

("Ali', 1998)

In [31]:

```
1 new cursor.execute("DROP TABLE clients")
```

Out[31]:

<sqlite3.Cursor at @x231c8535fc8>

In [35]:

```
1 new_connection.commit()
2 new_connection.close()
```

بروزرسانی و حذف رکورد

In [42]:

```
1 new connection = sqlite3.connect("test_data.db")
2 new_cursor = new_connection.cursor()
```

In [43]:

```
1 new cursor.execute("UPDATE clients set number 2020 WHERE name = 'Ali'")
```

Out[43]:

<sqlite3.Cursor at ex231c05352c8>

In [44]:

```
1 new_connection.commit()
2 new_connection.close()
```

In [49]:

```
1 new connection = sqlite3.connect("test_data.db")
2 new cursor = new_connection.cursor()
3
4 new cursor.execute("SELECT FROM clients;")
5
6 new cursor.fetchone()
```

In [50]:

```
new connection.commit()
new connection.close()
```

In [47]:

```
###DELETE

new connection sqlite3.connect("test_data.db")
new cursor = new_connection.cursor()

new cursor.execute("DELETE FROM clients WHERE name = 'Ali'")
```

Out [47]:

<sqlite3.Cursor at 8x231c85bc9c8>

In [48]:

```
new_connection.commit()
new_connection.close()
```

خواندن و نوشتن چندین رکورد

In [57]:

```
new values (
     ("Ronaldo", 7),
    ("Messi", 38),
    ("Salah", 11)
)

with sqlite3.connect("test_data.db") as new_connection:
   new cursor new connection.cursor()
   new_cursor.execute("DROP TABLE IF EXISTS clients")
   new cursor.execute("CREATE TABLE clients(name TEXT, number INT);")

    new_cursor.executemany("INSERT INTO clients VALUES(?, ?)", new values)

 new_cursor.execute("SELECT name, number FROM clients WHERE number < 28°)

  for record in new_cursor.fetchall():
        print (record)
```

("Ronaldo', 7)

('Salah', 11)